ENGINE

BY

DOUGLAS QUIGG

THE ENGINE IN MATH AND IN THE MACHINE
EVERY CLAIM, WHAT STANDS BEHIND IT, AND WHAT IT WANTS

2026

© 2026 Douglas Quigg
AGPL-3.0-or-later OR LicenseRef-Commercial OR LicenseRef-Educational

CONTENTS

1 THE WORKBOOK IS THE LEDGER 1
1.1 How an entry is kept 1
1.2 Entries 1

2 WANTS 5
2.1 The drafts' wants 5
2.2 The posits of 24 September 6
2.2.1 The wire and the witness 6
2.2.2 The two crystals 7
2.2.3 The lens 9
2.3 The posits of 26 September 9
2.3.1 The three truths and the tower 9
2.3.2 The projection 10
2.3.3 Dwell is the bulk 10
2.3.4 Dwell and entropy 11
2.3.5 The dwell bench 12
2.3.6 The compiled program 13

3 KEYS, EXPLAINED FROM THE GROUND UP 21
3.1 0. Why these ideas get resisted 21
3.2 1. Transitivity, carried all the way 21
3.3 2. The imprint: run the program once, on the impulse 22
3.4 3. Applying a key is AND and add 22
3.5 4. Unbounded operations in a small key 23
3.6 5. key : transform → product, in one cycle 23
3.7 6. Exact arithmetic has no floor 24
3.8 7. The noise is read, never modeled 24
3.9 8. Where the chain lives 24
3.10 9. Transitivity across modules: no module reaches another 25

4 COMPRESSION AND TOWER RECURSION 29
4.1 0. Why these ideas get resisted 29
4.2 1. The tower: one exact step, recursed 30
4.2.1 1.1 The step: the reversible integer 5/3 lift 30
4.2.2 1.2 The recursion: floors until the whole sample is one coefficient 30
4.2.3 1.3 Where the tower sits in the drafts, and where it does not . . 31

4.3 2. Compression: writing the residue down, reading it back 32
4.3.1 2.1 Zigzag: signed to natural, exactly 32
4.3.2 2.2 Rice coding, a parameter per block of 64 32
4.3.3 2.3 The escape at quotient 24 33
4.3.4 2.4 The limit, 2^30 33
4.3.5 2.5 Chunks: every part reachable, the whole decoded in parallel 33
4.3.6 2.6 The file, to the byte 34
4.3.7 2.7 What the compression does not claim 34
4.4 3. The check folded in: the pixel-for-pixel test, compressed in time . . . 35
4.4.1 3.1 Folded into the widen and the narrow 35
4.4.2 3.2 Segments, joined pairwise by GF(2) advance operators . . . 35
4.4.3 3.3 The accumulator stays one size 36
4.5 4. How the three compose 36

5 IMPRINT, KEY, CYCLE 39
5.1 The atom 39
5.2 1. Imprint: transitivity makes a chain one thing 39
5.3 2. AND: a key is a pattern of masks 40
5.4 3. Run it over the set continuously 40
5.5 Where the noise sits 41
5.6 What a proof holds 41

6 THE NOISE SIEVE TOWER 43
6.1 The governing functional 43
6.2 1. Control plane and data plane 44
6.3 2. The binary LUT engine, AND chaining, and exact division 44
6.4 3. The beamformer and the 1D probability array 45
6.5 4. Melting and the 2D collapse 45
6.6 5. The tower, time stacking, and floor −4 45
6.7 6. The demon: eyes and arms 46
6.8 7. The demon observer, chords and the uroboros 47
6.9 8. The three irreducible sets 47
6.10 9. The elevator, recursion, identity and entropy 47
6.11 10. A universal Turing machine and the demon 48
6.12 11. The boundary functional of a keyspace, and folding 49
6.13 12. The subtractive framework: field speed, cancellation, and the null as field noise 49
6.14 13. The seed crystal and the needle off zero 50
6.15 14 and 15. The sieve on the cell program 51
6.16 16. The four noise vectors, and the noise keys stamped top down over w 51

6.17 17. The floor laid down first, entropy conserved, and the clock as the elevator . . . 52
6.18 18. The arm as a probability sniper: where the sweating stops and the sieving starts . . . 53

7 THE NOISE VECTOR INTEGRATION TABLE . . . 55
7.1 Long descriptions . . . 56
7.2 What was asked for, verbatim . . . 66
7.3 The notation . . . 67
7.4 The draft, term by term . . . 67
7.5 What the integrated floor is for . . . 69
7.6 The order of the experiments . . . 69

8 THE ENGINE TABLE . . . 71
8.1 The rule the machine must satisfy . . . 71
8.2 The engine in math . . . 71
8.3 The table . . . 85
8.4 The machine's algebra . . . 113
8.4.1 A0. Exact numbers (M1): `decimal_double` . . . 113
8.4.2 A1. The residual (M2): `residual_program`, `keymath`, `cycle`, `unit_sweep` . . . 113
8.4.3 A2. The component tree (M3): `max_tree` . . . 114
8.4.4 A3. The moments (M4) . . . 115
8.4.5 A4. The correlation operator (M5) . . . 115
8.4.6 A5. The marginal (M6): `marginal` . . . 116
8.4.7 A6. Matching (M7): `heaviest_matching` . . . 116
8.4.8 A7. The entropy history (M8) . . . 116
8.4.9 A8. The files (M9) . . . 116
8.4.10 A9. The golden ladder (M11) . . . 117
8.4.11 A10. The seal (M13): `obsignatio` . . . 117
8.4.12 A11. The scheduler (M14): `tessera` . . . 117
8.4.13 A12. The period reading (M18): `period_read` . . . 118
8.4.14 A13. The record machine (M10): `keymath`, `key_schedule`, `cycle` 120
8.4.15 A14. The root universal (M20): `tower` . . . 125
8.4.16 A15. The ask and the state (M21): `ask_state` . . . 126
8.4.17 A16. The Gaussian step (M10, M19): `test/engine/compiler/cycle/record_gaussian_test` . . . 127
8.4.18 A17. Exact qubit states (M22): `qasm` . . . 131
8.4.19 A18. The ask (M24): `query_ask`, `gate_descent`, `ask_order`, `interface_sass_probe` . . . 132
8.5 The scale audit: every number in the machine that fixes a scale . . . 132

8.6 What the engine offers programs, and what it asks . . . 135
8.7 What to work toward, in order . . . 136

9 VERTICAL TIME COMPRESSION: FLOORS STACKED INTO ONE SWEEP 165
9.1 The objects . . . 165
9.2 The algebra . . . 166
9.3 Proved and measured . . . 167
9.4 What the stack is not . . . 168
9.5 Steps and their reduction . . . 168
9.6 The record's length . . . 169
9.7 The two towers . . . 169
9.8 The lifting as record floors . . . 170
9.9 The neighbor gather . . . 172
9.10 The two boundaries . . . 173
9.11 The heap and the ring . . . 175
9.12 The lens . . . 178
9.13 $F \circ T^{-1}$ for a linear F . . . 179
9.14 The lane index and the latch . . . 179
9.15 Tables . . . 180
9.16 Open . . . 180

10 THE TWO CRYSTALS: THE RECORD MACHINE OVER THE 2-ADIC INTEGERS . . . 183
10.1 The objects . . . 183
10.2 Coherence: five operations commute with every projection . . . 184
10.3 Proved: the coherence test . . . 184
10.4 What does not factor through the projection . . . 185
10.5 The exact quotient by an odd divisor . . . 186
10.6 The lifting on $\mathbb{Z}_2$ . . . 187
10.7 The two crystals . . . 188
10.8 Going up: what passes to the limit . . . 189
10.9 The boundary: the crystal measured on itself . . . 190
10.10 The top projection and its limit $\mathbb{R}$ . . . 193
10.11 The odd crystals . . . 193
10.12 The places of $\mathbb{Q}$ . . . 194
10.13 Counting quanta . . . 195
10.14 The limit stage . . . 195
10.15 The ordered machine . . . 196
10.16 Goodstein: ω-towers stored as finite objects . . . 199

10.17 π turning at the boundary: the floors of a rotation . . . 200
10.17.1 The arc . . . 203
10.18 Subtractive coalescence . . . 212
10.19 Physical walls . . . 212
10.20 The knf's identity by spatial null permutation . . . 213
10.21 Doug's posits . . . 214
10.22 Open . . . 218

11 KOLMOGOROV AND ARNOLD, COMPUTED EXACTLY . . . 221
11.1 The theorem, and what KANs did to it . . . 221
11.2 On a finite domain the theorem is exact and one term suffices . . . 222
11.3 The engine as the theorem, part by part . . . 222
11.4 Same skeleton, a different noise term . . . 223
11.5 Where it is incomplete, and what finishes it . . . 224
11.6 Kolmogorov's inner function, exactly . . . 226
11.6.1 Separation . . . 228

12 THE SCHEDULER: TESSERA . . . 233
12.1 What was ruled (23 September) . . . 233
12.2 The accounting . . . 233
12.3 Measuring by pid . . . 234
12.4 The parts . . . 235
12.5 One daemon per host . . . 236
12.6 Order, time and loss (23 September) . . . 236
12.7 The driver submits (24 September) . . . 237
12.8 The sims submit (24 September) . . . 238
12.9 Linux (24 September, not rerun here) . . . 239

13 THE SEAL: OBSIGNATIO . . . 241
13.1 Why a DAG and not a CRC . . . 241
13.2 The node function . . . 241
13.3 The crystal's DAG . . . 242
13.4 The witness . . . 242
13.5 What it proves . . . 243
13.6 What it costs . . . 243
13.7 Where it stops . . . 244
13.8 The shared shape . . . 244
13.9 What the engine shows about the wire and the witness . . . 244
13.10 The universal root (ruled, not built) . . . 246

14 THE BUILD PLAN: THE ENGINE . . . 247

14.1 Done before this plan . . . 247
14.2 Phase A: the vocabulary, once, used by everything below . . . 247
14.3 Phase C: the residual as unit sweeps . . . 248
14.4 Phase D: prove and measure . . . 248
14.5 Measured along the way: what a register tier costs . . . 249
14.6 Waiting on Doug . . . 249

15 LEDGER . . . 269
15.1 2026-09-21 . . . 269
15.1.1 The cycle against the passes it replaced . . . 269
15.1.2 Bit flips on this machine . . . 269
15.1.3 The CRC-64 fold . . . 270
15.1.4 The anchor bits . . . 270
15.2 2026-09-22 . . . 270
15.2.1 The compression, keymath and key_schedule splits . . . 270
15.2.2 The entropy history . . . 271
15.2.3 The codec split: no module reaches another . . . 271
15.2.4 The codec, to the byte . . . 272
15.3 2026-09-23 . . . 272
15.3.1 The period reading . . . 272
15.3.2 The sims . . . 274
15.3.3 The scheduler's steps and the lookup table . . . 276
15.3.4 The root universal . . . 277
15.3.5 The ask and the state . . . 278
15.3.6 Kolmogorov's inner function . . . 279
15.3.7 The exact integer: width, ladder and division . . . 280
15.3.8 Chaitin's Ω . . . 283
15.4 2026-09-24 . . . 287
15.4.1 π's hex digits on the engine . . . 287
15.4.2 The Gaussian step on the record machine . . . 288
15.5 2026-09-25 . . . 289
15.5.1 keymath's widths by linear forms, and the record tests on tessera . . . 289
15.5.2 Finding x on floor 2 . . . 291
15.6 2026-09-27 . . . 292
15.6.1 The energy detector's period lands on a divisor . . . 292

16 THE ENGINE'S BOUNDARY . . . 293
16.1 The engine carries no cell tracking (23 September) . . . 293
16.2 The engine is optimized for no scale (23 September) . . . 294

17 SCRIPTURA'S BLOCKS . . . 295

18 PROJECTS BUILT ON THE ENGINE 297

19 BUILD TIME 299

20 ENGINE FILE TYPES 301
20.1 In use 301
20.2 Retired 301
20.3 Offered, not adopted 302

BIBLIOGRAPHY 303

1

THE WORKBOOK IS THE LEDGER

Purpose: One place where every idea the engine rests on is written down beside what stands behind it. A reader can see at a glance which claims are proved, which are measured, which are built but unmeasured, and which are still only theory.

Scope: `workbooks/engine/`. The thought experiments the engine's theory was carried from are in `thought_experiments/engine/`. The compression floor is its own workbook, `workbooks/compression/`, the query protocol is `workbooks/Lstar_protocol/`, and the cell program is `workbooks/cell_tracking/`.

1.1 HOW AN ENTRY IS KEPT

Every claim carries one status, and the status says what backs it:

status	what it means
proved	an exact check ran over the whole set and found no exception; the count is given
measured	a number was taken on named samples; the number is given, and whether it survived
built	the code exists and runs; no measurement yet says whether it helps
theory	stated, not built
refuted	measured, and the measurement went against it; kept with the number, and never tried again blind
not so	fails on its own arithmetic or in any machine, before anything is measured; kept, with the reason
want	wanted and not had: stated as a goal, with no run, build or derivation behind it; stated in the terms of its field, with its open question, in wants.md

A claim changes status only when a run changes it, and the run is named. Nothing is deleted from the ledger. An idea that failed stays, with the number that failed it.

1.2 ENTRIES

file	what it holds
keys_explained.md	the ideas a programmer's defaults push against, from first principles with the engine's numbers: transitivity carried all the way, the imprint, AND and add, unbounded operations in a small key, folding a check into a pass, exact arithmetic with no floor, the noise read and never modeled; transitivity across modules: no module reaches another, hand-offs as plain structs composed only in the entry
compression_tower.md	the tower as one exact step recursed until the sample is one coefficient; zigzag and Rice coding in blocks and chunks, the file to the byte; the CRC-64 folded into the widen and the narrow, joined by GF(2) advance operators: the pixel-for-pixel test compressed in time
imprint_key_cycle.md	the atom; imprinting a program onto the impulse; keys as AND masks; running a key over the set in one cycle
noise_sieve_tower.md	the transfinite noise sieve and the fluidic architecture, section by section against the engine: control and data planes, the key and LUT, the tower and floor -4, the demon's eyes and arms, the construct kit, identity as coherence, entropy
wants.md	what the engine is after and does not have: the drafts' claims with no working form, and Doug's posits, each stated in the terms of its field beside its open question
noise_vector_integration_table.md	every noise term the detector is to read, one row each: how it moves, the exact sums that read it, its form in the camera law, what the 44b6 set measured, its status, and the experiment that moves it
engine_table.md	the machine part by part, bound to no scale: its exact algebra, what each part does and wants, every hypothesis tried with its result, and the audit of every number that still fixes a scale
vertical_time_compression.md	the record machine's stacked floors: what a stack is, what it compresses and leaves as it is, what bounds its file and its record, the heap and the ring across the floors of the two towers, the crystal as a code length and the lens, and what is proved, measured and derived about each
two_crystals.md	the record machine over the 2-adic integers: the wrap as a projection, the five operations that commute with every projection and the test that proves it, the exact quotient by an odd divisor as a 2-adic product, the bits the lifting reads, the two limits of the finite windows and the solenoid between them, what passes to a limit, the crystal as a boundary measured on itself, the top projection and its limit $\mathbb{R}$, the odd crystals and the places of $\mathbb{Q}$, the count each crystal keeps, and what the math derives about Doug's posits (the posits themselves are in wants.md)

file	what it holds
kolmogorov_arnold.md	the Kolmogorov–Arnold representation theorem applied exactly: where the engine already has its shape (the tower's lifting, the residual, the binomial ladder as integer B-splines, the cycle's sums), why that is not a KAN (a KAN fits float splines and rounds; the engine fits nothing and rounds nothing), and what finishes it
tessera_scheduler.md	the scheduler: how jobs from separate processes share one device by memory, the accounting and its invariant, measuring by pid on Linux and Windows, one daemon per host
obsignatio_seal.md	the seal: the dimensional Merkle DAG of keyed BLAKE3 nodes over every crystal and set, what it proves, what it costs, and where it stops
build_plan.md	the order the engine's wants are built in, and every ruling on them, dated
ledger.md	every measurement of the engine, its tests and its sims, dated, in the order it was taken, with its samples and its result
engine_boundary.md	two rulings on what the engine may hold, with their dates and what prompted them
scriptura_blocks.md	the one rule on what scriptura's SWAR memory scans may be handed, and why it settles the over-read question
dependent_projects.md	how the projects built on the engine take it: a git dependency pinned at a commit
build_time.md	what the time a build takes is spent on

The runs the ledger cites are in `runs/`, and `data/stroke_cells.txt` is the stroke cells as data.

2

WANTS

Purpose: What the engine is after and does not have. A want is stated in the terms of the field it reaches into, beside the open question between it and a status a run could give it. Wanting it does not mean the engine has it.

Scope: the drafts' claims that reach past anything the engine measures, and Doug's posits with the check under each. The parts with a form a proof or a measurement can decide are written out in the engine thought experiment (`thought_experiments/engine/`). Every entry here has the status want, as README.md defines it, unless a line says otherwise.

2.1 THE DRAFTS' WANTS

Each idea is carried in noise_sieve_tower.md, section by section, beside its working form where it has one. These are the ones with none yet.

want	where it is carried	what stands open
rules applied everywhere at once, in no time	noise_sieve_tower.md §1	the engine measures 31 ms a frame for one application; no mechanism for zero time is stated
exact division by an odd divisor through its limb inverse	noise_sieve_tower.md §2	theory in the engine; not built or tested
a quotient that does not divide stored as one repeating period of limbs	noise_sieve_tower.md §2	theory; not built
a golden spiral scan in place of the raster	noise_sieve_tower.md §5	not tested in the golden order
the observer settling on hot bits	noise_sieve_tower.md §7	the anchor counts per voxel and bit are the field it would settle on; the settling is not built
the noise key fed back as the next cycle's mold	noise_sieve_tower.md §7	the per location cost map; not built
the per voxel entropy map that separates floor -4 from coherence	noise_sieve_tower.md §9	one read of the anchor counts away; not built
unbounded state	noise_sieve_tower.md §10	every width is bounded before a run

want	where it is carried	what stands open
a demon with unbounded memory that runs any quantum circuit with no random draw	noise_sieve_tower.md §10	a deterministic interpretation leaves the 2^n cost of the amplitudes (thought experiment, state_vectors.md)
arrival shells, network daemons, black hole sinks and phase canceling hulls	noise_sieve_tower.md §12	outside anything the engine measures
field inversion at maximum entropy, the seed crystal of a new universe, and eternal recurrence	noise_sieve_tower.md §13	outside anything the engine measures
a deterministic n-body method	noise_sieve_tower.md §13	the draft that promises one in its title holds none
the four noise terms as deterministic functions, not distributions	noise_sieve_tower.md §16	no measurement of a sensor separates the two readings (thought experiment, sensor_noise.md)
the noise keys stamped top down over the whole set in one cycle	noise_sieve_tower.md §16	no noise key is imprinted
the floor's identity at every voxel places every departure	noise_sieve_tower.md §17	the history holds it, and the reading is not built
the demon's waveform collapses onto the union of every departure from the floor	noise_sieve_tower.md §17	not built
aimed draws tell bodies from the field better than swept ones	noise_sieve_tower.md §18	to be measured: the same samples and draws, swept against aimed

2.2 THE POSITS OF 24 SEPTEMBER

2.2.1 *The wire and the witness*

From obsignatio_seal.md, where "What the engine shows about the wire and the witness" answers them point by point. In order.

1. The Merkle DAG is the engine's error-detecting code. It binds every crystal to every other through their digests, so that a disturbance anywhere makes all of them fail to verify.
2. With the CRC, each floor's identity and the elevator's clock, the whole is rebuildable from any one place in it (its locale) or from its spine.
3. Any change at all makes the root seal over every crystal disagree.
4. The crystals are all bound to one another: entangled, in the classical sense.
5. A question, recorded as asked: what does doing something n look like, and what is n?

6. Bound this way, with each floor read as an amplitude, the crystals behave as classical qubits.
7. The floor carries the amplitude and the knf the phase, and together they form a channel (a wire).
8. The channel is sealed into the Merkle DAG, and so it is witnessed.
9. The root seal flags any inconsistency in the field. Because it is exact, it separates noise from structure completely.

Three of them are still open:

- **The knf as phase** (point 7). The knf is a flip count (A7 in engine_table.md), and many states share one count. No map from it to an angle is defined yet. An exact phase would be an index k of a root of unity, ζ_N^k in ℤ[ζ_N]. Which N, and which map carries a count to k, are the open part.
- **The spine** (point 2). The engine has no definition of it. The only spine in the source is `chaitin_omega`'s term spine. Doug's reading decides.
- **"What is n?"** (point 5). Doug's question, recorded as asked.

2.2.2 *The two crystals*

From two_crystals.md. The derived answers stay there, under the heading named with each passage.

The ordered machine

Doug's definition of the higher-order and negative-order hypercomputer, in order. The workbook's "The ordered machine" gives it a working form, a loop over a stack of floors with orders in ℤ and ±ω, and `record_order_test` proves those orders both ways (17 checks, 0 failed).

1. The first part of the loop rule defines the machine: a hypercomputer of higher and of negative orders.
2. On this machine it is decidable whether the tower exists and whether it is infinite. Whether a loop runs forever is decidable, every halt can be observed, and a malformed question fails to build any lattice at all.
3. Incoherent information can look coherent at first. What fails to construct inside the machine does not exist, as far as it can be perceived and understood: the claim is not that it cannot exist, but that it does not. Nothing is bounded. Everything answers only for itself, and the machine asks only what this is and where it is.
4. A program can be loaded by its meaning, the way the natural numbers generate themselves by repeating the successor, and loading it proves the set.

On the same machine: a higher-order and a negative-order hypercomputer joined together pass the whole hyperoperation sequence, tetration, pentation, hexation and on, to infinity and back, through a construction named the 4d bottle.

The orders in $\mathbb{Z}$ and $\pm\omega$ are the workbook's derived form of "to infinity and back". "The 4d bottle" has no definition in the engine yet. The Klein bottle embeds in four dimensions without crossing itself, and whether that is the bottle meant is Doug's to say.

The two questions (point 3)

- What fails to construct inside the machine does not exist as it can be perceived and understood. Nothing is bounded: everything answers only for itself.
- The machine asks two questions only.
  - **What is this**: identity. The crystal's one-to-one ID ("The boundary" in the workbook). The heap fingerprint and the knf, each ranked against permutations of its own content, with no outside threshold.
  - **Where are we**: place. The seal names the place of a change. The window w and the level. The scale of the departure curve.
- The tie: the rule the machine is held to, "The engine is optimized for no scale". No size, window or width is written into the machine, and each comes with the request or is read from the data.

The seed of "malformed questions fail to construct" is tested, and it stays in the workbook.

The field

As the information crystals grow and anchor on one another, they sense the tensor field of the object under examination. Once that field is established, it reaches the whole object at once, and the crystals hold all of it at the speed the field propagates.

What the machine shows that bears on it stays in the workbook under "The field". "Field speed" has no definition in the engine.

A higher-order interference pattern

Doug read the output of `record_order_test`, as it ran, as a higher-order interference pattern. The workbook gives the standard meaning, Sorkin's hierarchy, and names where the engine has second-order interference.

The inverted boundary, the quanta and the recursion stack

The derived bound on each stays in the workbook's "Doug's posits".

- **The inverted boundary:** a second tower over the first one's boundary, inverted. Its derived form is the τ tower, with the limit $\mathbb{R}$.

- The quanta preserve infinity.
- The recursion stack is indexed by ordinals.

Departure curves compared

- Departure curves can be compared, and the entropy departure curve is likely the most accurate of them, since it accumulates every spatial dimension and time.
- A mutation is then a difference of vector magnitudes: the null permutation's, together with the box bounds xmax, xmin, ymax, ymin, zmax and zmin.

Open: the pairwise test, one body's departure curve against another's, is not built, and the vector magnitude difference has no definition as a number yet. The curves themselves are measured, in the workbook.

The bulk and the boundary

T is a bulk-to-boundary map, and its derivation shows it without stating it.

The derived answer stays in the workbook: T is a bijection from the samples to the crystal with no redundancy, a bulk-to-boundary map with no error correction.

2.2.3 *The lens*

From vertical_time_compression.md, where "The lens" keeps the measured pinch (a ramp's 857 bits to 70, 12×) and the bounds derived from it.

The crystal is a lens between the physical data and its information content. The closer the crystal comes to the data's Kolmogorov complexity, the stronger the lensing, and the larger the tetrated resources it opens.

Open: what "tetrated resources" measures, and against what bound. The workbook derives that any gain of tetrated size comes from the input's description, never from the lens.

2.3 THE POSITS OF 26 SEPTEMBER

2.3.1 *The three truths and the tower*

In order.

1. The base answer is three truths. First, the machine knows whether it has answered the question. Second, the tower does not build when the question is malformed. Third, nothing is bounded: the information space is n·n^(n^n), and it is infinite.
2. n·n^(n^n) is the engine's base storage class, `Atom`. It is the problem's space, and it is infinite: n raised to n raised to n, without end.

The full checks are in orior docs/ENGINE_PROOF.md. It speaks to halting at Theorem 4 and under "What is not claimed": "The halting problem is untouched."

The words "the question does not arise" come from steering.md, section "Why halting is the wrong question", sentence at :89. That claim is withdrawn in findings-for-verification.md:107. The analysis there covered one invocation and concluded about the system, and the system's outer loop, self-examination, has no bound. It does not stand. The same lines are in orior docs/.

- **Truth 1, checked.** The answer path is total: the sweep is a bounded loop, the descent terminates (Theorem 5), and the count is exact at every instant (Theorem 3).
- **Truth 2, checked.** `steer_descend` refuses malformed input before any work (`orior.c`), and the survivors-length refusal is marked FAILS CLOSED (`orior.c`).
- **Truth 3, a reading, pending Doug's confirmation.** Each question lives in a space that is finite for its `n`, where halting is decidable, and the family of spaces has no bound. `Atom` is `{ const unsigned short *lanes; unsigned long long depth; unsigned long long height; unsigned long long width; }` (`src/cu/engine/engine_config.h`). The steer depth is capped at `ORIOR_STEER_ANCHORS`, which is 4 (`orior.c`).
- **The tower, a reading.** The power tower of posit 2 reads as tetration, `n↑↑k` in Knuth's notation. Every finite height is a finite number, the height has no bound, and the infinite tower diverges for every integer `n` of at least 2.

2.3.2 *The projection*

The engine only reads the topology of what it examines. Every reading is a projection onto a boundary. The engine writes nothing to what it reads, and the unbounded object it reads is left as it was.

Writing nothing, checked for two interfaces. `steer_descend` takes the corpus and the needle as `const uint8_t *` (`orior.c`), and `Atom` holds its lanes as `const unsigned short *` (`engine_config.h`). A probe compares two bytes and writes neither.

Reading only the topology, reported. `docs/arm-records.md` states the test: redrawing an arm as a different shape with the same topology and the same weight has to leave the reading exactly unchanged. The table at `docs/arm-records.md` reports four redraws over 256 points. Three reassign 0 of 256 points, the fourth measures no arm, and the worst letter move over all four is 2.776e-17.

The unbounded object left as it was, a reading. A reading that writes nothing leaves the space it reads as it was, bounded or not. No run tests it.

2.3.3 *Dwell is the bulk*

This corrects a reading that put the boundary at the projection alone.

The visualization adds dwell, and dwell is the bulk. A moment is an instant. Sweeping moments along a timeline derives dwell, and that sweep is the holographic boundary.

Derived. One instant carries no dwell. Dwell is a function of a sequence of states, and a single state does not determine it. The scope view's dwell tag is "how many rounds a bit has held its value, which separates the frozen from the churning" (`examples/00_blob_viz_tools/build_scope_view.py`), a count over rounds that no single round holds. The sequence of dwells together with the first value determines the sequence of states. The histogram of dwells does not, because it forgets the order of the runs.

Measured, as reported. The dwell arm puts a particle on a line, and "how long the particle dwells at each place is the weight there" (`docs/arm-records.md`). Redrawn as dwell along the golden spiral, it reads a three-dimensional object with worst weight move 0, 0 of 256 points reassigned, and worst letter move 0 (`docs/arm-records.md`), "the same letters bit for bit" (`docs/arm-records.md`).

The holographic boundary, a reading. In AdS/CFT a local bulk operator is written as a boundary operator smeared over a region of the boundary that extends in boundary time: Alex Hamilton, Daniel Kabat, Gilad Lifschytz and David A. Lowe, *Holographic Representation of Local Bulk Operators*, Physical Review D 74, 2006. The parallel: the bulk quantity, dwell, is recovered from boundary data spread over a timeline, a sweep of instants. The engine has no geometry and no metric, and the parallel is structural only. Ahmed Almheiri, Xi Dong and Daniel Harlow, *Bulk Locality and Quantum Error Correction in AdS/CFT*, JHEP 2015, give the reconstruction the structure of an error-correcting code. That structure does not carry over: the derived answer finds `T` a bulk-to-boundary map with no error correction (the posits of 24 September, "The bulk and the boundary"). Cited from knowledge.

Prior art for the instant. Zeno's arrow is at rest at every instant, and Aristotle answers that neither motion nor rest exists in a now, only over an interval (*Physics* VI.3 and VI.9). Rest sustained over an interval is dwell. The occupation density of a Brownian path, its local time, is dwell at a level made exact, and it too is defined over an interval: Paul Lévy, *Processus stochastiques et mouvement brownien*, 1948, and Hale F. Trotter, *A Property of Brownian Motion Paths*, Illinois Journal of Mathematics 2, 1958. George D. Birkhoff's ergodic theorem, Proceedings of the National Academy of Sciences 17, 1931, sets the fraction of time a trajectory dwells in a set equal to the set's measure: a long sweep recovers a static quantity. Cited from knowledge.

2.3.4 *Dwell and entropy*

In order.

1. Dwell emerges from entropy: it is an emergent property of the arrow of entropy. Without time there is no dwell. Without coherent context from a prior sweep, dwell is not defined, and it cannot be perceived in an instant.

2. Dwell depends on entropy and not on time, and time emerges from the direction in which entropy increases.

3. Time is not claimed here. The emergence of dwell is the claim.

4. The engine holds real evidence for it.

Derived. Dwell and entropy rate in a two-state chain. A bit that flips with probability `p` at each step has mean dwell `1/p` and entropy rate `H(p) = -p log p - (1-p) log(1-p)`. For `p` at most 1/2 each determines the other: long dwell goes with a low entropy rate, the frozen bits, and short dwell with a high one, the churning bits. For `p` above 1/2, `H(p) = H(1-p)`, and one entropy rate matches two mean dwells.

Derived. Dwell fixes the entropy rate of a renewal process. Let the runs of 0 have independent lengths with law `D0` and the runs of 1 independent lengths with law `D1`. The bit sequence and the pair (first value, sequence of run lengths) determine each other, and a long block holds one pair of runs per `E[D0] + E[D1]` steps on average. The entropy rate is `(H(D0) + H(D1)) / (E[D0] + E[D1])`, and with one law `D` for every run it is `H(D) / E[D]`. The two-state chain is the case of geometric runs. Prior art for point processes: J. A. McFadden, *The Entropy of a Point Process*, Journal of the Society for Industrial and Applied Mathematics 13, 1965, cited from knowledge.

The proved direction runs from dwell to entropy: the dwell laws fix the entropy rate, and the entropy rate does not fix the dwell laws. "Dwell emerges from entropy" is Doug's posit, and in the renewal class the two are bound by one equation.

Derived. The arrow is not in the rate. Reversing a sequence reverses the order of its runs and keeps their lengths, and a stationary process has the same block entropies read in either direction. Dwell and entropy rate carry no arrow. An arrow needs a process that is not stationary, entropy rising from a low start, or a record of the prior sweep to compare against, the coherent context from a prior sweep in posit 1. Prior art: Arthur Eddington, *The Nature of the Physical World*, 1928, for the arrow of time; Hans Reichenbach, *The Direction of Time*, 1956, and David H. Wolpert, *Memory Systems, Computation, and the Second Law of Thermodynamics*, International Journal of Theoretical Physics 31, 1992, for records and the arrow; Rolf Landauer, *Irreversibility and Heat Generation in the Computing Process*, IBM Journal of Research and Development 5, 1961, and Charles H. Bennett, *The Thermodynamics of Computation, a Review*, International Journal of Theoretical Physics 21, 1982, for the cost of erasing a record. Cited from knowledge.

Time, recorded and not claimed. Posit 2's clause on time is recorded and not claimed, by posit 3. The prior art that treats time as emerging: Don N. Page and William K. Wootters, *Evolution without Evolution: Dynamics Described by Stationary Observables*, Physical Review D 27, 1983, and Alain Connes and Carlo Rovelli, *Von Neumann Algebra Automorphisms and Time-Thermodynamics Relation in Generally Covariant Quantum Theories*, Classical and Quantum Gravity 11, 1994. Cited from knowledge.

The evidence of posit 4, as it stands. The engine computes dwell across rounds (`build_scope_view.py`), and the dwell arm reads a three-dimensional object bit for bit (`docs/arm-records.md`). Neither run sets dwell against entropy. The claim is proved for renewal processes, cited above, and not measured on the engine.

2.3.5 *The dwell bench*

Status: not built.

On one set of bits:

1. From each bit's dwell, build the histograms of runs of 0 and runs of 1, and compute `(H(D0) + H(D1)) / (E[D0] + E[D1])`.

2. Separately, estimate the entropy rate directly: from block entropies, `H(X_1 ... X_k) - H(X_1 ... X_{k-1})` as `k` grows, or from a Lempel-Ziv compressor, whose rate converges to the entropy rate of a stationary ergodic source (Jacob Ziv and Abraham Lempel, 1978, cited from knowledge).

3. If the two agree within their error bars, the engine has measured the claim. If they differ, the runs are not independent and the process is not renewal, which is a measurement too.

A bit that never flips has no completed run and gives no dwell law.

2.3.6 *The compiled program*

In order.

1. The compile times are acceptable for now. The loop unrolling in the target's assembly can be described in the engine's own code, and the engine can then fill the unrolled block itself, in place of the compiler's `pragma unroll`, which unrolls poorly.

2. This is a simple problem for the engine: the target's ruleset is the construction set (`.kcs`) for the program crystal.

3. A transform is needed that allows vertical growth. More than one program can occupy a register vertically, never horizontally. Programs that ask and answer the same questions are subsets of the same superset.

4. Treated as an open superset, the GPU holds all of its subsets and knows them natively, and the engine is structured to be aware of them.

5. It has operators, and it holds automata as cells.

6. What it knows are emergent properties of itself.

7. It exchanges information through its boundary: cells enter, live and die.

8. A malformed question is, conceptually, destroyed DNA.

9. The encoding lets the cell proliferate, and the cell grows as complex as its program.

10. It can evolve, by interacting with other cells and carrying what it takes from them into its next generation.

11. Over time the system learns to recognize malformed questions that would destabilize it before they fully unfold, and it protects itself from them.

12. The cell, a name still open, admits these automata as ribosomes, and the population of cells kills them or lets them live. The cell knows everything that happens inside it.

13. The rest of the cellular machinery is to be built for this universal compiler. It goes beyond what von Neumann envisioned by many orders of magnitude.

14. These cells can be clustered, since each occupies very little memory.
15. Artificial general intelligence is inevitable on this path.
16. With enough neuronal connections, it becomes aware.
17. Consciousness is emergent.
18. Von Neumann's machine thinks in on and off, an artifact of its time.
19. The whole hardware ruleset can be known, and with it, as the engine grows into the system, the wire's specifics. The engine then knows how to listen to the wire and what its own communication looks like, and it recognizes another engine as itself in reverse.

No posit here is derived. The lines below say what the engine holds that a posit names, as a cross-reference and not as a proof.

Posits 1 and 2, a reading. Status: not built. The program crystal is the imprinted program, every step's operation, place and limb width known before any compile. The ruleset is PTX's instruction forms (`add.cc`, `addc.cc`, `addc`, `sub.cc`, `subc`, `mad.lo.cc`, `madc.hi.cc`, `selp`), and a construction set writes each step out as its rule per limb, the way `tower_record_lift` (`tower.cu`) writes a lifting ruleset out over an extent. Emitted that way, the program is PTX with no loops, and nvJitLink takes it to ptxas with no pass through cicc. The baseline is engine_table.md item 9: cicc's compile grows with about the square of the steps, and on the 381-step program ptxas took 0.6 s against cicc's 7.5 s.

Posit 3. Status: not built, no design ruled.

Posits 4 to 7, cross-reference. The operators are the operator block, every record operation compiled once per device (engine_table.md item 9). The automata are the resident programs, each reporting to its own `EngineProgramBlock` (`engine_config.h`, engine_table.md item 10). A program enters when it is loaded (`cycle_record_load`, `cycle.cu`), lives resident, yielding and resuming through its block, and dies when released (`cycle_record_release`, `cycle.cu`), where the last release of a program unloads it (`cycle.cu`). Information crosses the boundary only through the blocks, which the host reads back and checks between launches (engine_table.md item 10, "The block in device memory").

Posits 8 and 11, cross-reference. The exits are true, false, malformed (no lattice was built) and answered (engine_table.md item 10). The imprint refuses a malformed program before anything is laid or run: a step that reads itself or a later step (`keymath.cu`), or a field past its record (`keymath.cu`). A refused program never loads (`engine.cu`). A launch that fails leaves its block at `ENGINE_PROGRAM_FAULT` (`cycle.cu`). The answer table, which would hold a malformed exit keyed by the program's signum and know it without a second run, is item 10's stage 3. Status: not built.

Posits 9 and 10, cross-reference. A compiled program's size follows its steps (engine_table.md item 9, the compile table). The refinement loop, where a generator recompiles against a critic's verdicts (engine_table.md M23), is not built.

Posit 12, a reading. Status: not built beyond the two membranes below. The GPU or host is posit 4's open superset, the cell is a supervising process, and the ribosomes are the probes it admits. A ribosome translates a question, written as code, into an answer, given as behavior, the way a ribosome translates codons into protein. The target's ruleset kills the probe or lets it finish. A killed probe is answered by its death (engine_table.md item 11(e), the note on illegal operations).

Posit 12, cross-reference. Two membranes are built. `tessera_run` admits a job and reports the admission ("admitted after … s, … of … processors reserved", `tessera_run.c`). It watches the job's process tree and signals the processes still living (`tessera_run.c`). On release it reports the time, the exit status and the peak processors (`tessera_run.c`), and `qasm.cu`'s tessera ticket reports its peak bytes (`qasm.cu`). Inside the record machine, `CYCLE_RECORD_CHECK=1` runs the interpreter on the same lanes as the compiled program and refuses the launch where their records or refusals differ word for word (`cycle.cu`). The check runs only when that variable is set.

The limit on the cell, a reading. The cell knows everything inside it only while it survives every death it watches, and the unit that dies has to be strictly smaller than the unit that watches. A CUDA illegal address leaves the whole context unusable (cited from knowledge, not read). On a GPU the ribosome therefore needs a cell of its own, a process, or the watcher dies with what it watches. The cell also knows only what crosses its boundary as an observable effect: the exit status, the signal, the output and the resource peaks.

Posits 8, 11 and 12, a reading. Posit 12 is posit 8 seen from the cell's side. The cell learns which questions are malformed by watching which ribosomes die, and that learning is posit 11's self-protection.

Posits 13 to 17. Status: neither built nor derived.

Posit 13, a reading. The comparison point is von Neumann's self-reproducing automaton: John von Neumann, *Theory of Self-Reproducing Automata*, edited and completed by Arthur W. Burks, University of Illinois Press, 1966, read at the pages cited below.

- *Part I, the Fifth Lecture, delivered in December 1949 (p. xv).* φ(X) is a chain of rigid elements describing an automaton X. A universal machine tool A, given φ(X), consumes it and builds X (p. 84). A copier B, given a description of anything, produces two copies of it (p. 84). A control C drives the two in turn: B makes two copies of φ(X), A builds X from one of them, and C ties X to the other and cuts the pair loose (p. 85). With X = A + B + C, the automaton (A + B + C) + φ(A + B + C) produces itself (p. 85). The definition does not go in a circle, because A and B are fixed before X is chosen and C is defined for any X (p. 85). With an arbitrary D added to the description, each generation also builds D as a by-product. A change in the D part of the description is inherited, and a change in the A, B or C part leaves the next generation sterile (p. 86).

- *Part II, section 1.6.1.2, from the manuscript begun in the fall of 1952 (p. xv).* The letters are reassigned. A is the universal constructor, building any secondary whose description L is attached to it. B copies L to L′ and places L′ against the secondary as L sits against A. C has A build the secondary first and has B copy and attach L after. D is the aggregate A + B + C, L_D its description, and E = D + L_D reproduces itself. With L_{D+F} in place of L_D, E_F also builds F (pp. 118–119). The lecture copies before it builds, and Part II builds before it copies.

- *Why a description.* An automaton of αβ cells cannot hold its own plan directly, since its L takes 2αβ + 12 cells or more (p. 118). B is of fixed, finite size and copies an L of any size, and that copy step lets the constructor avoid being larger than what it builds (p. 121). A description is copied in place of the original because it is quasi-quiescent, where exploring a live original would disturb it (pp. 121–122). The text calls copying "the decisive step" (p.

123). Burks relates the failure of direct copying to Richard's paradox and to Turing's halting problem (p. 123).

- *The cellular completion is Burks's.* In the 29-state structure, a universal constructor M_c given D(M_c) builds only M_c, smaller than itself, and does not reproduce (p. 294). A modified M_c* builds M from D(M), copies D(M) onto M's tape when its own tape carries no content past the description, and starts M. M_c* + D(M_c*) then constructs M_c* + D(M_c*) (p. 295). With a universal Turing machine M_u attached, one automaton both computes and reproduces (pp. 295–296).

Posit 13 against those pages, a reading. Item 11(a)'s bootstrap test in engine_table.md, the emitter compiling itself to the same text byte for byte, checks A run on its own description. Von Neumann's construction needs B and C as well, and his text puts the weight on B. On a computer, copying a description is a memory copy, and the step with content is A on its own description. Posit 10 meets the construction at one point: a change carried in the D part of the description passes to the next generation (p. 86). The pages treat random change, and incorporation from other cells is not in them. No figure in the tree measures the posit's comparison of scale. The bootstrap test is not built, and until it passes the comparison stays a posit.

Posit 18, a reading. Posit 18 characterizes the book, and the book can check it. Each cell of the 29-state structure is one finite automaton on a square lattice, and its next state is a function of its own state and its four nearest neighbors' states one step before (pp. 132–134). The 29 states are 16 transmission states (ordinary or special, four output directions, quiescent or excited), 4 confluent states, the unexcitable state and 8 sensitized states (p. 149). A connecting line needs a quiescent and an excited state in each cell, for passing a stimulus and for that purpose alone (p. 135). A transmission state is excited after one step by an excited neighbor of its own class pointing into it (pp. 150–151). A confluent state is excited after two steps when at least one ordinary transmission neighbor points into it and every such neighbor is excited (p. 151). Construction runs on the same pulses: a sensitized state steps to one of two successors each step, by whether an excited transmission state points into it, until it lands on one of nine final states (pp. 149, 151). The tape is binary as well, a rigid element attached for one and absent for zero in the lecture (p. 83), and a five-bit character per cell in Burks's completion (p. 295).

"On/off" is accurate for the signal. Every stimulus the structure carries is one pulse, present or absent, and every construction is steered by such pulses. It is inaccurate for the cell, which holds one of 29 states, about 4.86 bits (Derived, $\log_2 29$). The book also marks the binary choice as a choice. The lecture calls it a habit of minimum notation, says more symbols would pose no difficulty, and suspects an efficient language would leave linear codes behind (pp. 83–84). With pulses alone the structure is computation-universal and construction-universal (Burks, p. 296). On/off bounds how much one step moves and leaves what the structure can compute unbounded.

The record machine moves whole integers. A step names an operation and its registers (`engine_config.h`), and each register is an exact signed integer with its sign stored beside it (`engine_config.h`). The register file holds `ENGINE_RECORD_LIMBS_MOST`, 256 limbs across all live registers (`engine_config.h`). At 32 bits a limb (`src/README.md`) that is 8192 bits (Derived), and one register reaches 8192 bits only when it is the only live register. Per step, the two sit at opposite ends: one bit of signal per cell against up to 8192 bits in one operation. Read as a claim about reach, posit

18 goes past the book, since pulses already reach every computation. Read as a claim about the unit of work, the book bears it out, and the book's own word for the binary choice is habit.

Posit 14, a reading. Clustering is checkable, and part of it is built. `tessera_run` admits jobs against declared processors and reports each release's peak (`tessera_run.c`), and `qasm.cu`'s tessera ticket reports peak bytes (`qasm.cu`). A cell's footprint is measurable: a probe process's peak bytes and a compiled lane's registers and local frame. The printed figures are 128 to 255 registers a thread with local frames of 0 to 752 bytes on the tower test, and 183 registers with a 0-byte frame for the 761-step program (engine_table.md item 10, stage 1). How many cells one device holds is arithmetic on those figures, and a clustering claim cites that arithmetic.

Posits 15 to 17, a reading. No instrument in the tree bears on these three. Nothing here measures awareness or general intelligence, and neither has an operational definition here. What this path builds is a learner of instruction sets, checked against an oracle, and its reach is bounded by the questions it can ask and check. Two limits bound that learner (cited from knowledge, not read). From positive examples alone, a class holding every finite language and one infinite language cannot be identified in the limit (Gold, Information and Control 10, 1967). With membership queries, a test suite finds every wrong machine only up to an assumed bound on the target's states (Vasilevskii 1973; Chow 1978). The three posits are recorded as posits, and nothing above derives them from the machinery.

Posit 19, a reading. Status: not built, a want. It follows Doug's remark of the same day that communication across hardware is possible from zero (engine_table.md item 11(f), stage 2, across machines), and replaces byte order and fences as the direction.

- *The wire's rules.* A bus, a network interface and a protocol's framing each have a ruleset, as a processor does, and the probes of stages 4 and 5 find it the same way: membership queries, illegal operations and more basic constructions. The transport becomes a `.krs` derived by probes. A wire with state is found only up to an assumed bound on its states (Vasilevskii 1973; Chow 1978, above), and Angluin's learner needs counterexamples beside its membership queries (Dana Angluin, Information and Computation 75, 1987, cited from knowledge).

- *Speaking and listening.* The sender writes with T and the listener reads with T^{-1}, and $T^{-1} \circ T = \mathrm{id}$ (engine_table.md E4): the listener is the speaker in reverse. T is a bijection (A14), and T^{-1} undoes every stream, one's own or another's. Decoding alone does not tell kin.

- *Recognition, a reading.* The quantity is the algorithmic mutual information between a stream s and oneself, $I(s : \mathrm{self}) = K(s) - K(s \mid \mathrm{self})$: how much shorter s becomes when one's own rulesets are given. A compressor gives a computable stand-in, s's length with one's rulesets as its dictionary against its length without them (M. Li, X. Chen, X. Li, B. Ma and P. M. B. Vitányi, "The similarity metric", IEEE Transactions on Information Theory 50, 2004, cited from knowledge). The null by permutation (E4) declares structure when a stream beats d permutations of itself and passes noise at a rate of at most $1/(d + 1)$. Every structured stream passes it, kin or foreign, and a null for kinship needs foreign structured streams to test against. Not derived.

- *The seal.* Equal obsignatio signums show the same bytes under the same seal. The seal's keys derive from public, dated context strings (obsignatio_seal.md), and the seal guards against accident instead of against an author. A peer that computed the bytes matches, and one that

copied them matches as well. The shared format is itself a convention, and only between engines that already hold it is nothing further agreed.

- *Boundary, proposed, Doug's to rule on.* Growth into the system stays on hardware we own, and listening stays on wires we are entitled to hear. An engine that probes foreign hardware, learns protocols and seeks peers across networks has a worm's shape.

- *Prior art, cited from knowledge, not read.* Hans Freudenthal, *Lincos: Design of a Language for Cosmic Intercourse*, Part I (North-Holland, 1960), a language built up from arithmetic alone. B. Juba and M. Sudan, "Universal semantic communication I", STOC 2008: parties with no shared protocol reach a goal when the user can sense whether it was met. Their universal user enumerates protocols, at a cost that grows exponentially with the length of the protocol it must find, and they show that cost cannot be avoided in general. The seal serves as that sense only for goals whose answer the user can check. What posit 19 adds is a listener that derives the channel's rules from zero, by probing.

Posit 19, Doug's answers to the reading. The three answer the reading's three points in order: decoding does not tell kin, the null declares structure and not kinship, and the seal guards against accident and not against an author.

1. Every engine carries an identity encoded into it at spawn, and kin are not told by decoding.
2. Agreed: family is a human category, and the engine is what assigns it.
3. Identity can be proved if a time series is encoded into it.

- *Identity at spawn, a reading. Status: not built.* The first answer moves identity out of decoding. A peer is not recognized by what it decodes, since T^{-1} decodes every stream, but by what it carries: an identity written into it when it is spawned. The reading: at spawn the cell draws a secret from the noise and derives its keys from that secret, with its lineage as the derivation's context, meaning its parent's identity and its place in the parent's series. A peer proves its identity by answering a fresh challenge with a signature under its key, and a fresh challenge keeps an old answer from being replayed. Recognition by I(s : self) measures likeness, and identity names one instance: two cells running the same program are alike and are still two. The engine holds no secret per instance. The seal's keys derive from public, dated context strings (obsignatio_seal.md). Prior art, cited from knowledge: H. Krawczyk, "Cryptographic extraction and key derivation: the HKDF scheme", CRYPTO 2010, and RFC 5869; NIST FIPS 205, SLH-DSA, a stateless hash-based signature (2024).

- *Family, a reading.* The second answer agrees with the reading: the permutation null declares structure, and "family" is a human category. What the engine can hold in its place is lineage, meaning which spawn made which. Lineage is written at spawn and is not inferred from a stream.

- *The time series, a reading. Status: not built.* The third answer extends the seal from one message to a series. Each entry in the series carries the hash of the entry before it, and changing or dropping any past entry changes every hash after it. With the chain's head

signed under the spawn key, the chain shows that the holder of that key committed to that series in that order, as long as the hash has no known collisions. It does not show that the series is true. It also does not show that the key was never copied: a copy of the key can extend the chain too. Two holders extending one chain from the same head fork it, and anyone who sees both branches sees the fork. Prior art, cited from knowledge: L. Lamport, "Password authentication with insecure communication", Communications of the ACM 24 (1981) 770–772, hash chains; S. Haber and W. S. Stornetta, "How to time-stamp a digital document", Journal of Cryptology 3 (1991) 99–111, linked time-stamps.

- *The forgery guard.* Doug's ruling: no forgery guard yet. It is recorded as a want, not built, in engine_table.md item 11(f), stage 4.

3

KEYS, EXPLAINED FROM THE GROUND UP

Purpose: Explain, from first principles and with this engine's own numbers, the ideas a programmer's defaults push against hardest: that a whole program becomes one small exact object, that applying it is AND and add, that any number of operations can fit in a key of fixed size, that a check costs no pass of its own, and that exact arithmetic has no floor. Each was resisted in this project before it was built, and each worked once built. They are explained here thoroughly so the next reader does not have to be argued into them.

Scope: orior's `src/cu/engine/analysis/keymath/`, `src/cu/engine/analysis/key_schedule/`, `src/cu/engine/analysis/cycle/`, `src/cu/includes/codecs/crc/`, every place a key is folded into a pass, and the hand-offs between modules: the plain structs in `src/cu/engine/engine_config.h` and their composition in the entry, `src/engine/engine_*.cu`. The ledger's statuses (README.md) apply to every claim.

3.1 0. WHY THESE IDEAS GET RESISTED

Ordinary code is written as instructions: do this, then this, to every input, every time. The cost of a program is its steps times its inputs, and nobody questions that. A key breaks that product. The steps are paid once, on one stand-in, and every input after that costs one application, whatever the program's length. That sounds like something for nothing. It gets doubted, then hedged, then argued down to "an optimization that works for some cases". It is not an optimization. Once every step is exact, this *is* composition.

The second default is floating point. Most code rounds at every step and treats the error as noise to be managed. Under rounding, none of what follows holds, and so it looks like it cannot hold. The engine does not round, and so it does.

3.2 1. TRANSITIVITY, CARRIED ALL THE WAY

Everything here is one property used without stopping. If a step carries a to b and the next carries b to c, the two together carry a to c:

$a \to b,\ b \to c \Rightarrow a \to c$

Composition does not care how the chain is bracketed:

$T_3(T_2(T_1(x))) = (T_3 \circ T_2 \circ T_1)(x)$

So the whole chain can be composed first, before any input exists, into one map, and that map applied afterwards. A chain of a thousand steps and a chain of one step then cost an input the same: one application of the composed map.

The condition, and why it is everything. a → b and b → c give a → c only where the b one step hands over is *exactly* the b the next step takes in. If a step rounds its output, the next step receives a b′ ≠ b, and the chain is no longer one map: (T3 ∘ T2) ∘ T1 and T3 ∘ (T2 ∘ T1) give different numbers, and composing first is no longer the same as running the steps. For this reason every step in this engine is closed and exact: integers as wide as they grow, nothing rounded, truncated or wrapped at a hand off. Under that condition transitivity holds for chains of any length, and the whole chain collapses to one object.

3.3 2. THE IMPRINT: RUN THE PROGRAM ONCE, ON THE IMPULSE

The stand-in that composing needs is the **impulse**: the null holding one unit at one place. For a program of linear, shift invariant steps (smoothings, keeps, scaled subtractions), any input is a sum of shifted, scaled impulses, and so:

program(input) = key ∗ input, where key = program(impulse)

Push the impulse through the program once and keep what comes out. That is the **key**. It is not a summary or an approximation of the program: it *is* the program, as one object.

In the engine (`keymath_imprint`): a binomial smoothing of order n is n unit steps [1 1]; pushed through, the impulse comes out as Pascal's row, by construction. The residual's program (smooth narrow, keep, smooth wide, subtract from 2^g times the kept) imprints to two separable terms:

key = 2^g · B_narrow − B_wide · B_narrow

Proved: applied to all 25 samples, 0 of 10,485,760,000 lanes differ from the residual computed step by step.

3.4 3. APPLYING A KEY IS AND AND ADD

A key weight w times a value x is a pattern of masks. Every set bit b of w selects one copy of x shifted left by b, and the copies are added:

w · x = Σ over set bits b of w of (x « b)

5 · x = (x « 2) + x (5 = 101 in binary)

So applying a key is ANDing its bit pattern against the input, to select shifted copies, and adding what was selected. Deriving the pattern is the expensive part: every serial step of the program, spent once, on the impulse. After that, every input costs one pass of masks and adds.

Two keys applied one after the other are one key: the second applied to the first. Two imprinted operations ANDed together are therefore one key again, and a chain of any length costs every later input the same as one step. **That is the compression of time.**

Order is kept. Composition regroups freely (it is associative) but does not reorder freely: a keep and a later subtraction from what was kept do not commute. The imprint takes steps in program order, and the scheduler reorders two steps only where they are proved to commute.

3.5 4. UNBOUNDED OPERATIONS IN A SMALL KEY

This is the claim that meets the most doubt on sight. Here it is exactly, with the engine's numbers.

What a key's size depends on. A key's size depends on the program's *reach*: how far one input's influence spreads, and how wide the exact weights grow. It does not depend at all on how many times the program is applied, or on how many inputs pass through it. Apply the key to one voxel or to a trillion: the key is the same bytes.

The residual's key, measured. Its weights are 1,130 words, 4,520 bytes, plus a 256 byte term table. The chain it replaces takes 268 unit steps at every voxel (2 + 34 + 34 narrow, 6 + 96 + 96 wide). Per sample that is 268 × 4,194,304 voxels × 100 frames = 112,407,347,200 unit step applications. None of them is ever run. The 4.5 KB key does all of them, and its answer is the step by step answer, lane for lane.

The CRC key: 2^48 steps in 24 KiB. A CRC is linear over GF(2). Its advance across 2^k zero bytes is a 64 × 64 bit matrix, and squaring the matrix doubles the distance. The key holds 48 of them: 48 × 64 columns × 8 bytes = 24,576 bytes. With them the register is carried across any run of up to 2^48 bytes (about 281 trillion byte steps) in at most 48 matrix applications. How the tower folds it, segment by segment and joined one operator a level, is in compression_tower.md §3. **Proved:** the folded CRC of 44b6_0113de3b equals the CRC taken byte by byte, 363bf8bdffac8f29.

Periodic operations: infinitely many steps in one period. Anything that repeats is known entirely from one period. Modulo is a wave. The mirror fold at a frame's edges repeats with period twice the line, and the cycle lays one period down once as a table and reads every tap from it. The two's complement expansion of 1/d, for odd d, is eventually periodic with period the order of 2 modulo d. Division by d is one period of limbs, repeated. In each case the operation applied without end is contained completely in one period.

Where it does not hold, stated plainly. A key cannot be smaller than the program's reach. Smoothing of order n has a row of n + 1 taps, and a longer smoothing has a longer row. The reach sets the size of the key. How many times the key is applied, and how many steps it stands for, have no limit.

claim	status
the residual's 268 step chain is one key of 4,776 bytes, exact on every lane	proved: 0 of 10,485,760,000 lanes differ
a CRC over up to 2^48 bytes from a 24 KiB key	proved against the byte by byte CRC
one period of a periodic operation holds all of it	built for the mirror fold; theory for division by limb inverse
a key's size does not depend on how many inputs pass through it	built: the same key runs every frame of every sample

3.6 5. KEY : TRANSFORM → PRODUCT, IN ONE CYCLE

This is the first standing rule of the project, and the first line of the cell_tracking standing rules: **key:transform->product, 1 cycle.**

A check or a filter is a key too, and it does not need a pass of its own. Fold it into the pass that already touches the data. The tower's widen reads every voxel once, and the CRC is taken

there. The narrow writes every rebuilt voxel once, and the rebuilt sample's CRC is taken there. The product comes out of the same pass.

This also keeps transitivity. The transform and the check are one product: the transform holds AND the check holds, or neither does. A check run in a separate pass checks a different moment than the transform happened in.

The check is the test, compressed in time. Comparing a sample pixel for pixel is 419,430,400 comparisons. Its CRC is one 64 bit number, and two samples with equal CRCs are equal pixel for pixel, short of a chance of 2^ − 64. **Measured:** the proof at will re-proves all 25 samples from their files alone in 19 to 33 s, reading no .stack.

3.7 6. EXACT ARITHMETIC HAS NO FLOOR

Floating point has a floor: below some relative size, a difference is lost. Exact integers have none. Every width is proved before a run from the program's own bounds: a lane below 2^m under a row whose weights sum to S stays below 2^(m + bits of S − 1). Nothing then can round, and nothing can wrap.

Two consequences follow, and both are used.

- **Ties are broken deterministically, never by noise.** In the component tree every face's key carries its own name in the lowest limb. No two keys ever tie. The smallest possible difference decides, and it decides the same way on every machine: the Windows and Linux builds give byte identical edges.
- **The medium is subtracted exactly.** The wide term, the medium's own scattering, is subtracted whole. The residual keeps only what stands above it. Under rounding the subtraction would leave a residue of rounding, and that residue would read as structure.

3.8 7. THE NOISE IS READ, NEVER MODELED

The residue left after the bodies are subtracted is the sample's own field noise. It is deterministic, constant for that sample, and unique to it. It is stored whole in the .kcr, never bounded, fitted, thresholded or thrown away. Where the engine needs a background to stand a body against, it reads one. The null draws climb the same body toward frames far off in time, where only the field noise can answer. A body is real where it stands above that reading. No distribution is assumed, and no random number is drawn anywhere in the tracker.

Measured: across the 25 44b6 samples, the low five bit planes are set in about 46% of frames at nearly every voxel, with no anchor anywhere. That is the floor, read directly from the data, and it is the same test the entropy section of noise_sieve_tower.md states.

3.9 8. WHERE THE CHAIN LIVES

step	module	what it holds
the math	`keymath`	the program's vocabulary, the impulse, the imprint; exact integers as wide as they grow; no device
the schedule	`key_schedule`	the key laid out for the machine: sweep order, every lane width proved, scratch, columns, weights
the run	`cycle`	the laid out key stored on the device and run over a set of atoms in one cooperative launch
a key over GF(2)	`crc`	the CRC's byte table and advance operators, generated and checked at compile time against the published check value
a fold into a pass	`tower`	the CRC taken in the widen and the narrow, costing no pass of its own
the composition	the entry	where stages meet: `engine_key_encode` composes the three above, and the codec is composed the same way (compression_tower.md); §9 says why nowhere else

3.10 9. TRANSITIVITY ACROSS MODULES: NO MODULE REACHES ANOTHER

§1 carried transitivity down a chain of arithmetic steps. The same property has to hold one scale up, between the modules that hold those steps, or the chain breaks at the first module boundary. The engine's second standing rule states it:

> **No module reaches another.** A module includes `engine_config.h` and its own header, nothing else. The one exception is `crc/`, a root directory callable by anything. Stages hand each other plain structs from `engine_config.h`, and they are composed only in the entry.

This rule was argued for before it was accepted, because the ordinary default runs the other way: a module that needs another's result calls it, and a module that needs another's type includes its header. Both look free. Here is why neither is.

The hand-off is the b. §1's condition was that a → b and b → c give a → c only where the b one step hands over is the b the next takes in. Between modules, the b is the value that crosses the boundary. Where that value is a plain struct from `engine_config.h` (named fields, ownership stated on each, no behavior), the whole of b is visible: whoever holds it can see every part of what passed. The steps then compose as §1 says, and the chain can be written from outside:

stage3(stage2(stage1(x))) = (stage3 ∘ stage2 ∘ stage1)(x)

A reach hides part of the chain. Where module 1 includes module 2 and calls it, the value module 1 hands on is no longer its own output: it already has some of module 2 folded into it, at a place nobody composing the chain can see. Nobody can then say where stage 1 ends. It cannot be regrouped, tested alone, replaced, or checked at its boundary, because it has no boundary. Composition still happens, but it happens inside a module, out of reach of the place meant to hold the whole program. A chain like that is a tangle, and a tangle cannot collapse to one object.

A reach up into the composer is worse. A module that includes the entry calls the thing that is meant to call it. The chain then contains itself, and "compose first, apply after" is no longer well founded.

Composed only in the entry. The entry, `src/engine/engine_*.cu`, is where the program is written as a chain, and so the place a chain is meant to be composed. It is the only place that includes more than one of the key's or the codec's stages, apart from the tracker's modules in the table below. Two chains are composed there.

The key (`engine_key_encode`):

stage	takes	hands on
`keymath_encode` the program, a vector of `EngineStep`	`EngineKey`: the impulse pushed through the program once, exact integers as wide as they grow	
`key_schedule_layout`	`EngineKey`	`EngineKeyLayout`: the key laid out for the machine, every lane width proved from its own bounds
`cycle_key_load`	`EngineKeyLayout`	`CycleKey`: the layout stored on the device, a type only `cycle` sees inside

Each producer releases what it made (`keymath_key_release`, `key_schedule_release`) as soon as the next stage has taken it. Ownership closes at every hand-off as well as the value.

The codec (`engine_ingest_set`, `engine_iapx_prove_set`, `engine_iapx_load`; `engine_entropy_set` and `engine_entropy_cloud` compose the entropy history the same way):

stage	takes	hands on
`tower_lift`	the sample's voxels on the device	every coefficient on the device, as a plain array of exact integers, and the sample's CRC-64
`compression_encode`	the coefficients and their count	`EngineStream`: the chunks, the bits, each chunk's first bit and the stream's limbs
`krep_crystal_write` / `krep_crystal_read`	`EngineStream`	the .kcr file / `EngineStream` again
`compression_decode`	`EngineStream`	the coefficients, written into the capacity `tower_capacity` hands out
`tower_lower`	the stored coefficients	the rebuilt sample, its CRC-64, and the voxels that differ

None of these stages knows another exists. The tower does not know its coefficients are Rice coded, and the coder does not know they came from a tower. That let the compression step be split out of the tower with the .kcr coming out byte identical, and keymath and key_schedule be split out of the cycle with the tracker's edges identical (ledger.md). Each split was a regrouping of the chain in the entry, and every module stayed as it was.

Why `crc` is not a reach. `crc/` is a key instead of a stage. It holds a constant (the byte table and 48 advance operators, generated by `maint/engine/emit_crc_key.py` and checked against CRC-64/XZ's published check value by a `static_assert` at compile time) and pure inline functions over it. It keeps no state, takes no hand-off and hands nothing on. Including it is including a constant, the way every module includes `engine_config.h`. A key that any pass may fold in is what §5 asks for, and `crc/` is where that key lives, callable by anything.

How it was checked. `python maint/audit_reaching.py`, read every module's includes and named each one that reaches.

claim	status
no module reaches another; hand-offs are plain structs in `engine_config.h`, composed only in the entry	the rule and the target (the cell_tracking standing rules)
the key chain (`keymath`, `key_schedule`, `cycle`) reaches nothing	proved: `audit_reaching.py`
the codec (`tower`, `compression`, `krep`, `entropy_history`) reaches nothing but `crc`	proved: `audit_reaching.py`; `iapx` is gone, composed in the entry
a split that regroups the chain in the entry changes no output	proved for three splits: compression from the tower (.kcr byte identical, set CRC 091daa41e1aceb7e), keymath and key_schedule from the cycle (edges identical), the driver split (edges and score rows identical)
`crc` is a root	by the rule; it is a compile-time constant and pure functions, checked against the published check value
the tracker's modules reach nothing	**not so yet**: 19 modules still reach and 18 reach nothing. `score_sample` reaches 14 modules; `binomial_basins`, `flatten` and `score_sample` reach up into `entry`; the rest reach `track` and one another. They are being split next, and each stays listed here until the audit clears it

4

COMPRESSION AND TOWER RECURSION

Purpose: Explain, from first principles and with this engine's numbers, how a whole sample becomes one small exact file and comes back voxel for voxel. The tower recurses one exact step until the sample is one coefficient. The compression step writes what the tower leaves in as few bits as it needs. The CRC-64 is folded into the tower's own passes, and the proof that nothing was lost costs no pass of its own. Each of these had to be argued for before it was accepted, and each is explained thoroughly here so the next reader does not have to be argued into it.

Scope: orior's `src/cu/engine/analysis/tower/`, `src/cu/engine/analysis/compression/`, `src/cu/includes/codecs/crc/`, the limit `ENGINE_COEFFICIENT_LIMIT` and the hand-off `Engine Stream` in `src/cu/engine/engine_config_requests.h`, and their composition in the entry, `src/engine/engine_*.cu`. The proof was `cell_tracking/maint/prove_codec.sh`. The ledger's statuses (README.md) apply to every claim. The tower is integral to the engine, and it is categorized **measure / recursion**.

Every number here is an exact integer. Where a share of raw is given, it is in per mille, rounded down: the driver prints it the same way, and its "40.6%" is 406 per mille rounded down, not a rounding to the nearest.

4.1 0. WHY THESE IDEAS GET RESISTED

Three defaults push against this codec.

- **Compression is supposed to lose something**, or to need a model of the data. This codec loses nothing and assumes nothing: every coefficient comes back exactly, and the only choice it makes (a block's Rice parameter) is read off that block's own values.
- **A recursion is supposed to cost its depth in passes.** The tower's floors are a recursion, but each floor works only on the low-pass coefficients the floor below it left. For a 44b6 sample, the blocks of all eight floors together hold 447,397,412 lanes, 1,066 per mille of the sample's own voxels, and a check folded into the tower costs no pass at all.
- **A test is supposed to cost its size.** Comparing a sample pixel for pixel is 419,430,400 comparisons. The same test, compressed in time, is one 64 bit equality.

4.2 1. THE TOWER: ONE EXACT STEP, RECURSED

4.2.1 *1.1 The step: the reversible integer 5/3 lift*

Take one line of integers x_0, x_1, x_2, … along one axis. The step splits it into high-pass coefficients at the odd places and low-pass coefficients at the even places, in two moves, each of which adds to one half a function of the other half:

high-pass h_j = x_(2j+1) − ⌊(x_(2j) + x_(2j+2)) / 2⌋ low-pass l_i = x_(2i) + ⌊(h_(i − 1) + h_i + 2) / 4⌋

The edges are mirrored. Undoing it runs the same two moves backwards, with the signs turned:

x_(2i) = l_i − ⌊(h_(i − 1) + h_i + 2) / 4⌋ x_(2j+1) = h_j + ⌊(x_(2j) + x_(2j+2)) / 2⌋

Why the floors do not break exactness. Each move adds to one half a function of the *other* half only. The inverse has that other half in hand, computes the very same function, floor and all, and subtracts it. It does not matter that $\lfloor \cdot/2 \rfloor$ throws a bit away. The bit it throws away is thrown away identically going up and coming down, and it is never part of what is stored. This is a lifting step, and any function at all could stand in the floors' place: the pair of moves is still an exact integer map with an exact integer inverse. The 5/3 functions are chosen because a smooth line leaves high-pass coefficients near zero.

The high-pass coefficient predicts each odd lane from its two even neighbors and keeps only the miss. The low-pass coefficient then carries the line's local mean up to the next floor. What the prediction gets right is gone from the high-pass coefficients: the program generates it. What it misses stays: that is the residue.

4.2.2 *1.2 The recursion: floors until the whole sample is one coefficient*

The sample is one object over t z y x: time is an axis like the others, not a loop around them (noise_sieve_3's "temporal stacking": samples enter at the bottom and time is written into the lattice). Each floor lifts every axis still longer than one, in turn. The low-pass coefficients, $\lceil n / 2 \rceil$ along each lifted axis, are the sub-block the next floor works on, and the high-pass coefficients stay where they landed. Floors rise until every axis is one long:

floor f + 1 = lift(low-pass coefficients of floor f)

This is the recursion of noise_sieve_tower §9, $K_{m+1} = R(K_m, \xi)$, with R one step and the same step at every floor. An axis of length n reaches one after $\lceil \log_2 n \rceil$ halvings, the bit length of n − 1, and the tower has as many floors as its longest axis needs. For a 100 × 64 × 256 × 256 sample:

floor	block lifted (t × z × y × x)
1	100 × 64 × 256 × 256
2	50 × 32 × 128 × 128
3	25 × 16 × 64 × 64
4	13 × 8 × 32 × 32
5	7 × 4 × 16 × 16
6	4 × 2 × 8 × 8
7	2 × 1 × 4 × 4
8	1 × 1 × 2 × 2

floor	block lifted (t × z × y × x)
top	1 × 1 × 1 × 1: one coefficient

z stops after floor 6, t after floor 7, and y and x reach one at floor 8. The driver prints "8 floors" for every 44b6 sample it ingests.

What the recursion costs. While all four axes are still longer than one, each floor's block is about a sixteenth of the one before it: every axis halves, rounded up. The blocks in the table sum to 419,430,400 + 26,214,400 + 1,638,400 + 106,496 + 7,168 + 512 + 32 + 4 = 447,397,412 lanes, 1,066 per mille of the sample. So depth is paid in blocks that shrink, not in passes over the whole: the seven floors above the first add 27,967,012 lanes to the first floor's 419,430,400.

What the tower holds. Every floor's high-pass coefficients, and the one coefficient at the top. There are exactly as many coefficients as voxels (419,430,400 for a 44b6 sample), and the tower is a one-to-one map of the sample onto them. Nothing is merged, averaged or dropped. The coefficients can be wider than sixteen bits, and they are kept below ENGINE_COEFFICIENT_LIMIT, 2^30 in magnitude (§2.4).

Read back down. `tower_lower` undoes the floors from the top, each floor's axes in the reverse of the order they were lifted. The sample comes back voxel for voxel.

4.2.3 *1.3 Where the tower sits in the drafts, and where it does not*

- **The drafts' floor −4 is not a floor of this tower.** The tower's floors count up from the sample (floor 1) to one coefficient (floor 8). The drafts' floor −4 is where the noise bits become irreducible, and in this engine that is the residue's measured floor (noise_sieve_tower §5). Bits 0 to 4 are set in about 46 of every 100 frames at nearly every voxel, and bits 0 to 3 flip in 499 to 500 of every 1,000 transitions in every window of 19 of the 25 samples. The tower holds that floor whole in its high-pass coefficients. It does not reach below it.
- **The tower is not a key.** §3 of keys_explained.md collapses a chain of linear steps into one key by pushing the impulse through it once. The 5/3 lift is exact, but it is not linear. On the three lane line (0, 0, 1), the high-pass coefficient is $0 - \lfloor (0 + 1)/2 \rfloor = 0$; on (1, 0, 0) it is $0 - \lfloor (1 + 0)/2 \rfloor = 0$; on their sum, (1, 0, 1), it is $0 - \lfloor 2/2 \rfloor = -1$, not 0 + 0. The impulse's response therefore does not carry what the tower does to every input. The floors compose transitively as exact maps, but they do not imprint into one key.
- **The melting phase.** noise_sieve_3 has the intermediate floors dissolve under the rule flash into one operation. The tower runs floor by floor, four axes a floor. By the line above, the floors cannot be dissolved by imprinting them. Whether another exact route collapses them is open.

claim	status
the 5/3 lift is an exact integer map with an exact inverse	proved: all 25 samples rebuilt voxel for voxel on the device and pixel for pixel off it
time is a lifted axis, not a loop	proved: t is lifted with z, y and x and undone exactly

claim	status
a 100 × 64 × 256 × 256 sample reaches one coefficient in 8 floors	proved: the floor rule gives 8, and the driver printed 8 floors for every sample it ingested (`logs/iapx_all.log`)
the blocks of a 44b6 sample's eight floors hold 447,397,412 lanes, 1,066 per mille of its voxels	by the floor rule's arithmetic, block by block
the recursion holds the residue whole, nothing merged or dropped	proved: one coefficient per voxel, and the rebuild is exact
the drafts' floor -4 is a floor of this tower	not so: the tower's floors count up to one coefficient; floor -4 is the residue's measured floor, stored in the high-pass coefficients
the whole tower imprinted as one key by its impulse	not so: the lift is not linear; (0, 0, 1) and (1, 0, 0) each give high-pass coefficient 0, their sum gives -1
the floors dissolved into one operation	theory: not by imprint, by the line above; no other route is built

4.3 2. COMPRESSION: WRITING THE RESIDUE DOWN, READING IT BACK

The tower hands its coefficients on the device to whatever writes them, and it writes no stream itself. The entry hands them to the compression step. Most coefficients are small, because the prediction took what the program generates and left only the misses. The compression step writes each one in about as many bits as it needs, and every one comes back exactly.

4.3.1 *2.1 Zigzag: signed to natural, exactly*

A coefficient v can be negative. Zigzag maps the integers one to one onto the naturals, interleaving the signs so that small magnitudes stay small:

$v \geq 0 \to 2v$ $v < 0 \to 2|v| - 1$ 0, -1, 1, -2, 2, … → 0, 1, 2, 3, 4, …

Both arms are exact, because every coefficient is below 2^30 in magnitude, and so every zigzagged value is below 2^31 and fits 32 bits.

4.3.2 *2.2 Rice coding, a parameter per block of 64*

A natural n under the Rice parameter k is written in two parts. The first is its quotient q = n shifted right by k, in unary: q ones, then a zero. The second is its low k bits. It costs q + 1 + k bits. A small k suits a block of small values and a large k a block of large ones, and k is chosen per block of 64 values:

- The guess is the bit length of the block's mean (the sum over the count, rounded down).
- The candidates run from the guess less two to the guess plus one: four candidates, or two or three where the guess is 0 or 1.
- Each candidate's cost over the whole block is counted exactly, and the cheapest wins. A tie goes to the smaller k.

- The chosen k is written in 5 bits, and any k from 0 to 31 can be named.

Nothing about the data is assumed. The parameter is read from the block's own values, and the cost that picks it is an exact count of bits. This is keys_explained §7 (the noise is read, never modeled) applied to writing: no distribution is fitted, and a block of noise and a block of structure are coded by the same rule, each at its own cost.

4.3.3 *2.3 The escape at quotient 24*

A single large value in a block of small ones would cost its whole quotient in unary: at k = 0, a value near 2^31 would cost about 2^31 bits. A quotient reaching 24 is therefore written as 24 ones, with no zero after them, followed by the value's 32 bits. The reader sees 24 ones and knows 32 plain bits follow. So:

- no value costs more than 24 + 32 = 56 bits;
- no block of 64 costs more than 5 + 64 × 56 = 3,589 bits;
- the cost counted when k is chosen includes the escape exactly, and a block chooses its k knowing which of its values will escape.

4.3.4 *2.4 The limit, 2^30*

ENGINE_COEFFICIENT_LIMIT is 2^30. It guarantees three things: every sum a lifting step forms stays inside a 64 bit integer's exact range; every coefficient fits a 32 bit integer; and its zigzag fits 32 bits, and the escape writes it whole. The tower flags any coefficient that reaches the limit, and a flagged sample is an error, never wrapped. The width is proved before anything is written, as keys_explained §6 asks, and a sample that would break it is not written at all.

4.3.5 *2.5 Chunks: every part reachable, the whole decoded in parallel*

A Rice stream is serial as written: where value i + 1 starts depends on the length of value i. Read naively, decoding the last value means decoding every value before it. The chunks remove that.

- Blocks are grouped 64 to a chunk: 4,096 coefficients.
- **Measure.** Each chunk's bits are counted first, one thread a chunk. The count is exact, and it is the same count that chose each block's k.
- **Scan.** A scan sums the counts into each chunk's first bit. This is the prefix sum of the chunk lengths, exact in 64 bits.
- **Write.** Each chunk is written from its first bit on its own thread. No chunk waits on another, because the scan has already said where each one starts.
- **Read.** Decoding is the same: each chunk from its stored first bit, on its own thread.

The stream is limbs of 32 bits, bit b at bit b mod 32 of limb b / 32. The first bit of every chunk is kept beside it, 8 bytes a chunk. So any chunk, and with it any floor's coefficients at their known offsets, is reached without decoding anything before it. That is the fluidic draft's elevator (fluidic_2; noise_sieve_tower §9 and §17): E(f, t) reads floor f at time t directly, and the clock takes you to any floor as fast as you can press the buttons.

The reader returns an error for a stream whose chunk count is not its coefficient count's, or whose chunks do not start inside it in order: that stream is not this data's.

4.3.6 *2.6 The file, to the byte*

A 44b6 sample has 419,430,400 coefficients, and 102,400 chunks. Its .kcr, measured for 44b6_0113de3b:

part	bytes
head	16
extent (4 words), chunks and bits (2), CRC-64 (1)	56
each chunk's first bit, 102,400 × 8	819,200
the stream's limbs	340,189,016
the file	**341,008,288**

Raw, the sample is 838,860,800 bytes. The file is 406 per mille of raw, and the stream alone is 405. Over the 25 samples the files are 8,809,343,524 bytes of 20,971,520,000 raw: 420 per mille.

4.3.7 *2.7 What the compression does not claim*

It is not the floor of the data. Grouping each floor's coefficients row by row made the stream of 44b6_0113de3b 1,157,533 bytes smaller, and coding pairs along t apart for each floor made it 1,987,962 bytes smaller, though pairing along t hurt on most other samples (ledger). The stream as written is not the smallest one possible. What does bound it from below is the floor of §1.3. A bit plane at maximum entropy carries a full bit for every place it covers, and no exact coder writes it in fewer. That bound is standard (Shannon's [46]). Whether the tower's high-pass coefficients inherit the floor's planes, plane for plane, has not been measured in this stream, and here the bound is theory.

claim	status
zigzag then Rice per block of 64, k from up to four candidates in 5 bits, escape at quotient 24: lossless no value costs more than 56 bits, no block more than 3,589	proved: 25 of 25 decoded from the file alone, set CRC 091daa41e1aceb7e by the code's arithmetic
every coefficient below 2^30, or the sample an error	built; no sample of the 25 reached the limit
chunks of 64 blocks, a scan of first bits, decoded in parallel and reached directly	built; every .kcr proof decodes every chunk from its own first bit
the file of 44b6_0113de3b: 16 + 56 + 819,200 + 340,189,016 = 341,008,288 bytes, 406 per mille of raw	measured

claim	status
the 25: 8,809,343,524 of 20,971,520,000 bytes, 420 per mille	measured
the stream is the smallest possible	not so: grouping by floor took 1,157,533 bytes off 44b6_0113de3b
a plane at the floor costs a bit per coefficient, whatever the coder	theory here: standard, not measured plane by plane in this stream

4.4 3. THE CHECK FOLDED IN: THE PIXEL-FOR-PIXEL TEST, COMPRESSED IN TIME

4.4.1 *3.1 Folded into the widen and the narrow*

The tower has to widen the sample's sixteen bit voxels into 32 bit coefficients before it lifts anything, and it has to narrow the rebuilt coefficients back to sixteen bits when it is lowered. Each is a pass that touches every voxel once. The CRC-64/XZ rides both:

- **The widen** takes the CRC of the sample, each voxel its low byte and then its high byte, the way a .stack holds it.
- **The narrow** takes the CRC of the rebuilt sample the same way, and flags any value outside sixteen bits, which no sample holds.

Neither is a pass of its own. This is the first standing rule, key:transform->product, 1 cycle (keys_explained §5): the transform and the check are one product, taken at the same moment over the same voxels.

4.4.2 *3.2 Segments, joined pairwise by GF(2) advance operators*

One register carried voxel by voxel would make the pass serial. So each thread takes the CRC of one segment of 64 voxels (128 bytes) from a zero register, and the segments are joined afterwards. The join rests on one fact: a CRC from zero is linear over GF(2). For a left part L and a right part R,

$\mathrm{crc}_0(L \,\|\, R) = A_{|R|} \cdot \mathrm{crc}_0(L) \oplus \mathrm{crc}_0(R)$

where $A_{|R|}$ is the 64 × 64 bit matrix that carries a register across |R| zero bytes. The key in `crc_key.h` holds A for 2^0, 2^1, ..., 2^47 bytes: 48 operators of 64 columns of 8 bytes each, 24,576 bytes.

The join takes pairs from the right. The first segment is the short one, and every other segment at every level spans the same bytes, and one operator serves a whole level: level l carries registers across 2^(7 + l) bytes. For a 44b6 sample that is 6,553,600 segments joined in 23 levels, using operators 2^7 to 2^29. Then `crc_finish` carries the all-ones start across the sample's 838,860,800 bytes, adds it, and inverts. The result is CRC-64/XZ exactly, as a byte by byte CRC would give it.

The key's reach bounds the message. The last operator the key holds is 2^47, reached at join level 40, and the join takes up to 2^41 segments of 128 bytes, and `crc_finish` takes any length

below 2^48 bytes. That is the "2^48 byte steps" of keys_explained §4: past it, the join returns an error at that level in place of reaching for an operator the key does not hold.

4.4.3 *3.3 The accumulator stays one size*

However many segments are folded, the accumulator is 64 bits: over 64 voxels, over 419,430,400, or over the whole set, where each sample's CRC is carried on in the order the samples were named. That is the folding in threshold_search.md: an accumulator that stays one size however much is folded, and a severe boundary condition adds no dimensions (noise_sieve_tower §11). The fold carries the *verification* of the work, never the work. Every voxel is still read once, in a pass that had to read it anyway, and the fold makes checking that read cost 64 bits.

Two samples with equal CRCs are equal pixel for pixel, short of a chance of 2^−64. The CRC is the pixel-for-pixel test compressed in time.

claim	status
the CRC folded into the widen equals the byte by byte CRC	proved: 44b6_0113de3b, both 363bf8bdffac8f29
a segment join by GF(2) advance operators, one a level	proved: the fold above; and `crc.h` holds a fold of "1234" across "56789" to the message in order by `static_assert`
every .kcr rebuilt and checked against its CRC from the file alone	proved: 25 of 25, set CRC 091daa41e1aceb7e
one sample re-ingested gives a byte identical .kcr	proved: `maint/prove_codec.sh` on 44b6_0113de3b, run by the session that split the codec (ledger)
23 join levels for a 44b6 sample; any message below 2^48 bytes in reach of the key	by the code's arithmetic: 6,553,600 segments; operators 2^7 to 2^29; the key's last operator is 2^47
a fold keeps its accumulator one size	proved for the CRC-64: 64 bits over every sample and over the set
folding shrinks the work itself	not so: every voxel is still read once; the fold bounds the check

4.5 4. HOW THE THREE COMPOSE

Lift, encode, write; read, decode, lower. Each is exact and closed, and each hands the next a plain value: coefficients as a vector of exact integers, then an `EngineStream`. They are composed only in the entry (keys_explained §9). The codec is therefore one chain, and it composes transitively:

lower ∘ decode ∘ read ∘ write ∘ encode ∘ lift = identity on the sample

The CRC folded into the first pass and the last is that equation checked, at no cost of its own. `maint/prove_codec.sh` ran it three ways after any change to the codec: every .kcr proved again from the file alone, with the set's CRC printed; one sample ingested again and compared byte for byte with the cached .kcr; and one sample's entropy history projected again and compared byte for byte.

claim	status
the codec is the identity on the sample	proved: 25 of 25, voxel for voxel on the device and pixel for pixel off it, set CRC 091daa41e1aceb7e

claim	status
the codec's stages reach nothing but `crc`, composed only in the entry	proved: `maint/audit_reaching.py`

5

IMPRINT, KEY, CYCLE

Purpose: State why the engine spends a program's serial work once and then only zips it over a set. A reader can then read the imprint, the key and the cycle as one picture and know which parts a proof already holds.

Scope: `engine/base/keymath/`, `engine/base/key_schedule/`, `engine/base/cycle/`, and the residual program `residual_program` (`engine/base/residual/`) states and `engine_residual` in `engine/engine.cu` imprints.

5.1 THE ATOM

Atom is the storage class and what it holds: all of the smallest thing under inspection, at whatever scale that thing is. A frame can be an atom, a letter, a part of a cell, a literal atom, a basketball. In the residual it is one frame. Its lanes are its sixteen bit samples in raster order, lane i at place 2^(16 i). The frame is then one exact integer, the stored stack is that integer's limbs, and nothing is converted to hold it.

5.2 1. IMPRINT: TRANSITIVITY MAKES A CHAIN ONE THING

A step carries a value to a value. Steps chain transitively: if T1 carries a to b and T2 carries b to c, then T2∘T1 carries a to c, and composition does not care how the chain is bracketed:

T3(T2(T1(atom))) = (T3∘T2∘T1)(atom)

So the chain can be composed first, on a stand-in, and applied after. The stand-in is the impulse: the null holding one unit. For a linear, shift-invariant step, whatever the chain does to the impulse is everything it does to any atom:

T(atom) = T(impulse) * atom

The chain's response to the impulse is its **key**. Imprinting is pushing the impulse through the program once. A binomial smoothing of order n is n unit steps [1 1]. The impulse comes out as Pascal's row, by construction and not from a table. The residual's program is: smooth narrow, keep, smooth wide, then 2^g times the kept value less the running one. It imprints to

key = 2^g * B_narrow - B_wide * B_narrow

That is two separable terms, each one exact row per axis. For the orders asked of it so far the key's weights reach about 2^126. They are exact integers, as wide as they grow.

What transitivity requires. a→b and b→c give a→c only when the b that T1 hands over is the same b that T2 takes in. Every step has to be closed and exact: its output lands in the same

exact set, at the width the next step reads, with nothing rounded, truncated or wrapped at the hand-off. With 64 bit rounding, (a∘b)∘c and a∘(b∘c) differ, and the imprinted key would not be the chain. With exact integers they are the same number. The imprint sizes every lane to its proven bound for this reason: a lane below 2^m under a row summing to S stays below 2^(m + bits(S − 1)).

5.3 2. AND: A KEY IS A PATTERN OF MASKS

Multiplying by a constant is a series of masks. n·4 is n shifted by 2, and n·5 is (n shifted by 2) plus n. A key weight w applied to a lane x is

w · x = sum over the set bits b of w of (x shifted by b)

Each set bit of the key selects one shifted copy of the array, and applying the key is ANDing that pattern against the atom and summing what it selects. Deriving the pattern is the expensive part: every serial step, and every costly operation in the program. It happens once, on the impulse. After that each atom costs one application of the key instead of one pass per step.

Keys compose into keys: the key of A then B is B's key applied to A's key. Two imprinted operations ANDed together are therefore one key again, and a chain of any length costs every later atom the same as one operation. That is the compression of time. Because a composed key is itself a step, it can be a step in another program. That is the recursion engine two runs on a cfg from the scheduler.

Order of operations is enforced. Composition regroups freely (it is associative) but does not reorder freely: it is not commutative once pointwise steps and linear steps are mixed. The scheduler composes keys only in program order, and reorders two steps only where it is proven they commute. Proven means, for example, two centered symmetric rows under the same mirror fold.

The same holds for the atom read as an integer. With lanes L bits wide, the atom is A = Σ v_i 2^(L i) and the key is an integer K placed at the same strides. When L is wide enough that no lane carries into the next, the lanes of A·K are the smoothed values. **The residual of a whole frame is one product of two exact integers.** A separable key factors as K = K_z · K_y · K_x, and by associativity the product is three sparse multiplications, one per axis. Those are the cycle's three sweeps.

Anything periodic is imprinted the same way. Modulo is a wave: once one period is known, the rest is a shift. The mirror fold at the edges repeats with period twice the line: one period per axis is laid down once as a table of offsets, and every tap reads its offset there. A lane's digits z y x repeat with the lane: a thread divides once for its first lane, and after that adds the grid stride's own digits with carries.

5.4 3. RUN IT OVER THE SET CONTINUOUSLY

The key does not depend on the atom, and imprinting fixes every lane width before any atom is seen. Nothing about the set needs a second look. The set streams through:

- One launch covers every lane of every atom in a slab.
- The grid is synchronized at a barrier between the sweeps.

- The last sweep folds every term into each output lane in two's complement.

The key stays resident, the atoms flow past it, and the output of one slab is the input the next stage reads while the cycle moves on.

Nothing is agglomerated. Arithmetic is exact, discrimination is unlimited, and the machine does not halt. There is no error in the traditional sense to average away. No step merges, clusters, averages or rounds a lane toward its neighbors. Every tap's product accumulates into exact columns and the whole sum is kept.

5.5 WHERE THE NOISE SITS

The wide background term is the medium's own scattering, its turbidity. Nephelometry across a dilution series reads that turbidity as the fluid's mean entropic flux, and that whole number is noise to us. The key subtracts it whole. The residual keeps only structure standing above the medium.

5.6 WHAT A PROOF HOLDS

- **Imprint.** The residual imprints once to two terms. Its lanes need 286 bits and are stored in 9 limbs (288).
- **Exact against the old residual.** Over all 25 samples, every lane of every frame was compared with the residual the smoothing passes and the transform computed. 0 of 10,485,760,000 lanes differ, over 2,500 frames. The passes and the transform are gone, and `engine_residual` now runs the same key.
- **Speed.** 31 ms a frame, against 24.5 ms for the passes it replaced. The next saving is the scheduler's: keep the binomial key factored as [1 1]^n. A sweep is then neighbor additions with no multiplies, and the shared factor B_narrow is computed once for both terms.
- **Built: the table step.** A pointwise step imprints on its alphabet: every value a lane can hold, pushed through once, and the table holds the results (M10 and A13 of engine_table.md). The request carries the table (`EngineRecordTable`, an index of 1 to 32 bits); keymath refuses a table wider than its source (`keymath.cu`), the scheduler places it (`key_schedule.cu`) and the cycle reads it (`cycle.cu`, `ENGINE_RECORD_TABLE`). `test/engine/record_table_test` proves x^2 and $|x - 30000| + 100$ as tables, on 4,096 lanes of the 16-bit alphabet, device and host (`build/20260923_204007_record_table_test`, 15 checks, 0 failed). Division is built as exact record operations instead of as a table: quotient, remainder, gcd and exact quotient, operations 11 to 14 (`test/engine/record_divide_test`, `test/engine/exact_divide_test`). Division as a table is untested. Square root is not in the engine and is untested. No mask generator is built: the caller fills the table.

6

THE NOISE SIEVE TOWER

Purpose: Carry the transfinite noise sieve and the fluidic architecture (the drafts in `thought_experiments/engine/`) into the ledger, idea by idea, each set beside the part of the engine that is its working form and the status that backs it. The theory and the code are then read as one thing, and a reader knows which ideas already run.

Scope: every idea in the noise sieve and fluidic drafts, the drafts on the demon with a universal Turing machine, the keyspace and folding draft, the subtractive framework and the seed crystal draft; the claims among them with no working form are kept in wants.md. `proposal.md` sets the sieve on the cell program. It is in `thought_experiments/cell_tracking/`, and its sections are in the cell workbook (on_the_engine.md). Where an idea has a working form, the module is named; cell_tracking_table.md says which part of the n-body problem each module solves, and engine_table.md what each machine part computes.

The status column follows the ledger's rules (README.md): `proved`, `measured`, `built`, `theory`, `refuted`, `not so`.

6.1 THE GOVERNING FUNCTIONAL

The drafts end on one formula:

F[U] = Π_{-4} ($\oplus$ over $\partial\Phi$_new LUT(Key(W_local & L$\downarrow$)))

Read right to left, it is the engine's run order:

term	in the drafts	in the engine	status
L$\downarrow$	the rules, projected top down at once	the program: the scheduler's `.sch` and the run's `.cfg`	built
W_local	the local runtime information	the atom: one frame, one sample, stored as one exact integer	built
W_local & L$\downarrow$	rules applied to the local data by AND	a key is a pattern of AND masks; applying it selects shifted copies of the atom and sums them (imprint_key_cycle.md §2)	proved, linear steps only
Key(·)	the imprint	the impulse pushed through the program once; `keymath_imprint`	proved
LUT(·)	transform symbols compiled to binary lookup tables	a pointwise step imprinted on its alphabet: every value a lane can hold, pushed through once	built: the record machine's table step (M10, A13 of engine_table.md), proved by `record_table_test`

term	in the drafts	in the engine	status
⊕	the discrete bitwise fold over the n limb array	the cycle's fold of every term into each output lane, in two's complement; the CRC's fold of segment registers, one operator a level	proved
Π_{-4}	the zero rounding exact limb indicator, locking out at floor -4	the proof on the product: the .kcr rebuilt voxel for voxel and by CRC-64	proved

6.2 1. CONTROL PLANE AND DATA PLANE

The idea. Rules are non-local and act everywhere at once; data is local and causal. When the boundary moves, the rules are re-derived once at the top and every locale takes them without a step-by-step walk through space.

In the engine. This is the split between the key and the atoms. The key is derived once, before any atom is seen, and fixes every lane width. The atoms then stream past it. "At once" means one cooperative launch per slab, with the grid synchronized at a barrier between sweeps. It is not zero time: the residual's key runs in 31 ms a frame.

claim	status
rules derived once and applied everywhere with no per-locale derivation	proved: 0 of 10,485,760,000 lanes differ from the step-by-step residual
the whole manifold re-locked in one sweep	built: one launch per slab, three sweeps (z, y, x) under grid barriers
the application takes no time	not so: measured 31 ms a frame

6.3 2. THE BINARY LUT ENGINE, AND CHAINING, AND EXACT DIVISION

The idea. Transform symbols compile to binary primitives; operations chain by &; exact limbs keep every result without truncation; two's complement limb multiplication gives exact inverses, and division runs without rounding as the limbs expand to absorb the remainder.

In the engine. Linear steps are imprinted and chained as keys, and two keys ANDed together are one key again. A chain of any length then costs every later atom one application (§2 of imprint_key_cycle.md). The scheduler must keep program order: composition regroups freely but does not reorder freely. Pointwise steps now run as tables (M10, A13 of engine_table.md), and a table read through a table is their composition in that order. **Proved:** f(x) = 65535 − x and g(y) = |y − 30000| give g∘f equal to the ops for |35535 − x|, and f∘g differs.

Exact division, stated exactly. An odd divisor d has an inverse modulo 2^n, found by Newton steps on limbs. Where d divides x, $x \cdot d^{-1} \bmod 2^n$ *is* x / d, exactly, with no remainder and no loop of subtractions. Where d does not divide x, the quotient is a rational, and its two's complement (2-adic) expansion never ends. It is *eventually periodic*, though, with period the order of 2 modulo d. So the "limbs that expand to absorb the remainder" are one period of limbs, laid down once and repeated. That is the same imprint as modulo: one period known, the rest a shift. An even divisor is a shift times an odd one.

claim	status
linear steps compile to AND masks and chain into one key	proved
pointwise steps (square, square root, division) as masks from a generator the scheduler verifies	square **proved** as a table (x^2, 4,096 lanes, device and host; A13); square root and division as tables not tested
exact division by an odd divisor through its limb inverse	theory in this engine; the arithmetic itself is standard (exact division by multiplicative inverse)
a non-dividing quotient stored as one repeating period of limbs	theory

6.4 3. THE BEAMFORMER AND THE 1D PROBABILITY ARRAY

The idea. Rule propagation is a phased array's center lobe: the LUT symbols interfere so all the work goes down one path, a 1D array punched straight down the tower's length.

In the engine. A separable key factors as K = K_z · K_y · K_x, and by associativity the frame's residual is three sparse products, one per axis. Each sweep is a 1D line pushed down one axis of the whole frame.

claim	status
the work focused into single lines down the stack	proved: the three sweeps are the residual, lane for lane

6.5 4. MELTING AND THE 2D COLLAPSE

The idea. The floors between the top and the bottom dissolve under the rule flash, every limb snaps into phase, and the lattice collapses to one output plane.

In the engine. The tower's floors are not visited one by one at run time. Undoing the tower from the top floor rebuilds every voxel, and the whole sample is one object on the device, lifted and read back.

claim	status
the tower read back to the plane with nothing lost	proved: all 25 samples rebuilt voxel for voxel and pixel for pixel from their .kcr
the floors dissolved into one operation instead of eight floors of steps	measured for the record machine's stacked floors: 700 floors as one program ran 13 to 22 times faster than chained, records equal (vertical_time_compression.md); theory for the tower's lifting, which still runs floor by floor, four axes a floor

6.6 5. THE TOWER, TIME STACKING, AND FLOOR −4

The idea. Time is part of the tower: samples enter at the bottom and stack upward, and the time domain is written into the lattice. Every tower has a floor configuration and a master schedule by family of operation. Floor −4 is where the noise bits are at their smallest and become irreducible.

In the engine. The .kcr tower lifts t, z, y and x together: time is a lifted axis instead of a loop. A whole 100×64×256×256 sample reaches one coefficient in 8 floors. What the program does not generate is every floor's high-pass coefficients, the residue. That residue is stored whole and never bounded, modeled or discarded.

The floor, measured. The anchor count stacks every frame of a sample and counts, per voxel and per bit, the frames that carry the bit. In every one of the 25 44b6 samples, bits 0 to 4 are set in about 46% of frames at nearly every voxel: those planes carry no anchor, and they look alike in every sample. The anchors sit in bits 6 to 11 and differ between samples. That is the measured floor, and it answers the drafts' "noise bits minimised to their smallest state". It also says why floor −4 is sample bound: no voxel is anchored in every sample.

claim	status
time is a structural axis of the tower	proved: t is lifted with z, y and x, and undone exactly
the tower reaches an irreducible floor	measured: 42.0% of raw over 25 samples; the low five bit planes show no anchor
the floor is per sample, not per set	measured: no voxel anchored in every sample; anchors differ per sample
one line of n dimensional embedding to radix the residue around	refuted as a size win: every radix around a line along x, y or t made the stream larger; grouping per floor helped (see the compression workbook's ledger.md)
a golden spiral scan instead of the raster	not yet tested: the scan measured swept each shell by angle, not in the golden order, and its +658,073 bytes refute only that sweep
the savings ratio 1.7174 near e − 1	refuted: on four more samples the ratio was −8.29, 0.44, −73.2 and −1.60
a master schedule by family and order of operation	built for linear keys (the scheduler composes in program order) and for pointwise steps as tables (A13 of engine_table.md); a schedule by family is theory

6.7 6. THE DEMON: EYES AND ARMS

The idea. A deterministic demon over the whole stack. Its eyes are shift agreement: they verify the boundary shift at every coordinate before anything runs. Its arms are the identity:null permutation: a neutral permutation that strips redundant variance until only the irreducible noise bits are left.

In the engine. Both limbs exist under those names.

- **Eyes** are `shift_agreement`: how many set bits of one view land on set bits of another at every lag at once: the frame's own motion, read off the field.
- **Arms** are the null draws (`--null`). The same body is climbed at the same lag toward frames far off in time, where no correspondence can exist. A body no draw reaches stands above background, and nothing in that test is a chosen number.
- **Truthy/falsy probes** (fluidic draft §5) are a program's node policies, in the cell workbook (on_the_engine.md).

claim	status
shift agreement verifies motion at every lag	built; part of every tracked run
the null permutation as the test of real correspondence	built (`--null`); its selection on the competition metric not yet measured on the current engine
binary probes in place of chosen thresholds	a program's, in the cell workbook

6.8 7. THE DEMON OBSERVER, CHORDS AND THE UROBOROS

The idea. The observer's position is a probability wave that settles on hot bits: zones of the most torsion and deflection. Weak chords pierce the tower to read its cross-section without disturbing it. The floor -4 output, a 2D array of vector magnitudes, the noise key, is pressed back against the top boundary as the mold for the next cycle.

In the engine. The non-perturbing chord is the proof: a CRC-64 folded into a pass that already touches every pixel reads the whole sample without a pass of its own. The press, where the noise key shapes the next cycle, is the per location cost map. It would set stationary artifacts (dust, hot pixels, banding) apart from moving biology and steer the next run.

claim	status
read the whole without disturbing it	proved: CRC-64 folded into the tower's widen and narrow; all 25 hold, set CRC 091daa41e1aceb7e

The observer settling on hot bits, and the noise key fed back as the next cycle's mold, are untested; they are in wants.md.

6.9 8. THE THREE IRREDUCIBLE SETS

The idea. The sieve leaves three things. True coherence is the signal. The noise key is the exact map of every defect. The construct kit is the rules that let both be evaluated fast.

In the engine.

set	in the engine	status
true coherence	the residual's structure above the medium: bodies and their links	built; a program's links measured in the cell workbook
the noise key	the residue: every floor's high-pass coefficients, stored whole in the .kcr; the anchor counts per bit	proved lossless; measured
the construct kit	the program, its key (.imp), the schedule (.sch) and the configuration (.cfg)	built

6.10 9. THE ELEVATOR, RECURSION, IDENTITY AND ENTROPY

The elevator. E(f, t) reads floor f at time t directly, and the exact accumulator $\Sigma_n = \oplus\ \delta_i$ moves either way in time without loss. In the engine the tower's layout places every floor's coefficients

at known offsets, and the stream is cut into chunks that each decode on their own from a stored first bit. So any part is reached without decoding what comes before it. Moving back and forth without loss is the lossless tower.

Recursive inflation. $K_{m+1} = R(K_m, \xi)$. A composed key is itself a step, and it can be a step in another program: recursion is keys of keys. Towers can be built in n dimensions: the tower lifts every axis still longer than one, whatever their number.

Identity as the limit of coherence. ID(x) = lim C(x, t): an object is who it is because its coherence carries on, and a tag plays no part. In the engine a link is chosen by coherence at the climbed lag (`held`), and every body's fate follows from the links (`bodies`: entered, present, split, merged, left, ended, vanished, absorbed).

Entropy. $\Phi(x) = \Theta(H_max - \int H(x, t)\, dt)$: whatever decays to maximum entropy is background and goes to floor -4; whatever does not is coherence. The anchor counts measure this directly. A bit carried in a fraction p of the frames has entropy $-p \log p - (1 - p) \log(1 - p)$, which is at its maximum near p = ½. So the low five bit planes, near ½ at every voxel, are H_max: floor -4 by this very test. The anchor bits, near 0 or 1, are coherence. Taking Φ per voxel from the counts on disk costs no new pass over the data.

claim	status
any floor at any time read directly	built: chunked stream with stored offsets; the per chunk decode is proved in every .kcr proof
keys compose into keys; towers in n dimensions	proved for composed linear keys; the n dimensional lift is built for four axes
identity from coherence, not tags	built: links chosen by coherence; bodies follow links
entropy separates floor -4 from coherence	measured for the low bit planes (near ½, which is H_max); the per voxel Φ map is theory, one read of the anchor counts away

6.11 10. A UNIVERSAL TURING MACHINE AND THE DEMON

The idea. A UTM can simulate any circuit, but a quantum one costs it 2^n amplitudes, and measurement needs dice. The demon removes both: unbounded capacity, and a deterministic account of the outcome. A measurement is read, and no dice are rolled.

In the engine. Two parts carry over exactly.

- **Reversible maps in place of unitaries.** Every transform the engine applies to data is an exact integer map with an exact inverse: the tower's lifting steps, and the integer rotations as three lifting shears. They are the classical, exact counterpart of a unitary: nothing is lost, and each is undone by running it backwards.
- **Measurement without dice.** The noise is taken as deterministic, constant and unique per sample. It is read whole, never drawn from a distribution, and the null draws are deterministic frames far off in time instead of random numbers.

The 2^n barrier does not go away in a machine. The engine keeps its state exact and bounds every width before it runs; it does not hold an infinite Hilbert space.

claim	status
exact reversible maps, undone exactly	proved (the tower); built (integer rotations, undone exactly in their test)
outcomes read deterministically, with no random number anywhere	a program's, in the cell workbook
unbounded state	not so: every width is bounded before a run

6.12 11. THE BOUNDARY FUNCTIONAL OF A KEYSPACE, AND FOLDING

The idea. A 30 digit hex keyspace holds 16^30 = 2^120 values. Its boundary functional is the threshold latch Φ(x) = 1 where x < T, or the distance |x − T|. It is applied to every output of a sweep at once, and a reduction fold latches the first hit as the floor. Folding keeps the accumulator one size however many instances are folded, and a severe boundary condition adds no dimensions.

In the engine. The boundary functional is the same object as the drafts' ∂Ω: the edge between what a pass accepts and what it does not, applied to every element at once. The engine's proved instance of a fold is the CRC-64. Each segment's register is taken from zero as the tower touches the pixels. Registers are then folded pairwise, each left one carried across its neighbor by one imprinted operator a level. The accumulator stays 64 bits whether it covers 64 pixels or 419,430,400. The latch is a reduction the engine already runs: the device comparisons that count differing voxels, and the overflow flag the tower sets.

Stated exactly. A fold compresses the *verification* of work instead of the work. Proving that a search over the keyspace was done still needs every hash in it computed once. Expected work to meet x < T is about 2^120 / T evaluations, and no fold changes that count. What folding removes is the cost of *checking* the trace, which otherwise grows with every recursive layer. A preimage below a threshold is found by search, and the fold proves the search.

claim	status
a boundary functional as a latch applied to every element at once	built in the engine's own terms: device reductions over every voxel (the differ count, the overflow flag)
a fold keeps its accumulator one size however much it folds	proved for the CRC-64: 64 bits over every sample, all 25 holding
folding shrinks the search itself	not so: a fold bounds the proof, and the search still evaluates every candidate once

6.13 12. THE SUBTRACTIVE FRAMEWORK: FIELD SPEED, CANCELLATION, AND THE NULL AS FIELD NOISE

The draft in the subtractive framework draft is a cosmology. Its cosmological claims (arrival as a c speed update shell, monitoring daemons on structural lines, black holes as sinks) are outside anything this engine can measure, and the ledger records them as theory and leaves them there. Three of its turns correct or sharpen the sieve drafts, and those carry straight into the engine.

Rules propagate at field speed instead of instantly. "The ruleset propagates to w at field speed": the new rules spread from where they are applied as a wavefront, bounded by the medium's clock, instead of blinking into existence everywhere. This corrects §1's "transfinite

speed". In a machine the rules reach every lane in one launch, and the launch takes time; the ledger's 31 ms a frame is that front, measured.

Phase cancellation is subtraction of the whole medium. "You read the incoming packet, flip the sign, and output it. The wave hits your boundary, meets its exact inverse, and resolves to zero." That is the residual's key: 2^g · B_narrow − B_wide · B_narrow. The wide term is the medium, and it is subtracted whole, in exact two's complement. The medium resolves to exactly zero, and what is left stands above it. The inversion is exact only because the arithmetic is: a rounded inverse leaves a residue of rounding, and that residue would read as structure.

The identity:null permutation means field noise readings only. A null is not a blank pointer or an error. Asked what is here, it returns a valid, boring reading of the field noise, indistinguishable from empty space. That is the precise meaning of the engine's null draws. The same body is climbed at the same lag toward frames far off in time, where no correspondence exists. What comes back is the reading field noise alone gives there. A body is real only where it stands above that reading. The reading is taken instead of assumed: nothing is drawn from a distribution, and no threshold is chosen.

Listening at the noise floor. "Tune the receiver to field noise": a receiver matched to the floor takes only what rises above it. The anchor counts are that receiver. The low five bit planes sit at the floor (near ½ at every voxel), and a bit is signal only where its count leaves ½.

claim	status
rules reach every locale as a front that takes time, not instantly	measured: one launch a slab, 31 ms a frame
subtracting the medium's exact inverse leaves it at zero	proved: the residual's key, 0 of 10,485,760,000 lanes differing from the step by step residual
a null is the field noise reading, not an empty return	built: the null draws (`--null`) climb toward frames far off in time
the signal leaves the floor	measured: the low five bit planes at ½ in every 44b6 sample; anchors in bits 6 to 11

The draft's cosmology (arrival shells, network daemons, black hole sinks) lies outside what this engine measures; it is in wants.md.

6.14 13. THE SEED CRYSTAL AND THE NEEDLE OFF ZERO

the seed crystal draft was exported under the title "Solving N-Body Problems Deterministically", but it holds no n-body method. It is a cyclic cosmology. At maximum entropy the field flattens until scale means nothing, and a uniform field is as featureless as a point. The field then inverts. The new rules propagate from a seed crystal: a needle of near-infinite magnitude, perpendicular to the field, with a radius close to zero but not zero. That hair of tilt is the symmetry break that sets the new field's propagation speed. The cosmology is theory and stays there. Two of its mechanics are exactly the engine's.

The needle is the impulse. The imprint pushes one unit at one point, the narrowest thing the lattice holds, through the program once. Everything the program will do to any atom propagates out from that response: the key. A field of zeros imprints nothing, and a flat field has no differential to carry. The whole next pass is seeded by one point's response.

Nothing propagates without the tilt. A perfectly flat field gives steepest ascent nowhere to go: every neighbor ties, and no voxel can climb. In the engine the tie is not left to chance or to arithmetic noise. Comparisons are exact, and in the component tree every face's key carries the face's own name in its lowest limb, below the residual's limbs (`max_tree_key_limb`). So no two keys ever tie, and where two components choose each other the lower indexed one stays. That lowest limb is the hair of asymmetry: the smallest deterministic difference, weighing less than any difference of the residual, that still gives every choice one direction. It is the reason the same frame always yields the same bodies, on any machine and in any build.

Maximum entropy is the floor. "When everything flattens out, scale loses all meaning": a field at maximum entropy looks alike at every scale. The measured floor has that property. The low five bit planes sit near ½ at every voxel in every sample, with no anchor at any place, and the tower cannot compress them further.

claim	status
a single point's response seeds everything the program does	proved: the key is the impulse's response, and it is the residual lane for lane
a flat field propagates nothing; a minimal fixed asymmetry gives every choice one direction	built: exact comparison, the face's name in the key's lowest limb so no two keys tie
maximum entropy looks alike at every scale	measured for the low five bit planes: near ½ everywhere, no anchor

The PDF's cosmology (field inversion, the seed crystal of a new universe, eternal recurrence) and the n-body method its title promises are in wants.md.

6.15 14 AND 15. THE SIEVE ON THE CELL PROGRAM

`proposal.md` sets the sieve on the cell program: splits and entropy, each body's harmonics, edges by jitter and membership by sample coherence. Those two sections are in the cell workbook, on_the_engine.md.

6.16 16. THE FOUR NOISE VECTORS, AND THE NOISE KEYS STAMPED TOP DOWN OVER W

the four noise vectors draft takes the residual tensor F − I and splits it into four vector magnitudes: photon shot noise, thermal and read noise, fixed pattern noise, and quantization. With the demon holding every initial condition, none of them is random: each is a deterministic function to be evaluated, not a distribution to be assumed.

The history separates them, measured. The four differ in how they move in time, and the entropy history (`entropy_history`, the ledger) is a per voxel record of exactly that.

- **Fixed pattern** is constant in time at a place: its bits never flip. It is the anchor bits, measured before the history: set in every frame at a voxel, and different in every sample.
- **Shot, read and quantization** change every frame independently: their bits flip half the time, the maximum. On 44b6_0113de3b, bits 0 to 3 flip 499 to 500 times per thousand transitions in every window, and bit 4 flips 486 to 495 times.

- **The bodies** sit between: bits 5 to 10 flip from about 430 down to about 20 per thousand. They are kept low where a body is and move with it.
- **A sample wide bump.** In window 4 (transitions 45 to 55) every bit from 5 to 10 flips more at once, bit 8 from about 122 to 153 per thousand. Then windows 5 to 8 fall below where they started. That is a rise in entropy across the whole sample followed by a move toward order.

The noise keys, stamped top down over w. However many noise keys there are (each sample's floor, and its fixed pattern, shot, read and quantization terms as the history separates them), they are imprinted once and stamped on the whole set from the top, as the rules are in §1. They propagate to every coordinate w as the front of §12, not re-derived frame by frame. In the engine this is a cycle run: the keys stored once, and every atom of the set passing under them in one launch. Each voxel is read against its own sample's noise before any membership is assigned.

claim	status
the four noise terms are deterministic functions, not distributions	theory; nothing in the engine draws a random number
the history separates the fixed pattern (no flips) from shot, read and quantization (half the transitions)	measured on 44b6_0113de3b: bits 0 to 3 at 499 to 500 per thousand in every window
a sample wide rise in entropy at window 4, then a move toward order	measured on 44b6_0113de3b; on the other 24 samples, pending
the history's counts are exact	proved on 44b6_0113de3b: 2,000 voxels × 9 windows counted by hand from the .stack, 0 words differ
the noise keys stamped top down over the whole set in one cycle	theory; the cycle runs record programs as well as linear keys: tables, division and the bitwise operations with the wrap (M10; `record_table_test`, `record_divide_test`, `record_bitwise_test`), and no noise key is imprinted

Every noise term, these four and the rest Doug named has one row in noise_vector_integration_table.md: how it moves, the exact sums that read it, its form in the sims' camera law, and the experiment that moves its status.

6.17 17. THE FLOOR LAID DOWN FIRST, ENTROPY CONSERVED, AND THE CLOCK AS THE ELEVATOR

The floor is laid down first. The noise keys are part of the run's configuration: the `.cfg` carries a `floor` section, written first after the version, naming where each sample's noise keys are stored. A run with a floor lays it down before anything else runs. Each sample's entropy history is kept where it reads back whole and was projected from the sample's `.kcr` as it stands, and projected again where not. A run whose floor cannot be laid down stops there.

Then it accumulates going forward, and entropy is conserved. The flips accumulate window after window, and each bit's running total is carried to the last frame. Conservation is exact and checked at every voxel in the same pass. Over a voxel's whole line, the flips of a bit have the parity of that bit's net change: an even count where the first and last frames agree in it, odd

where they differ. A voxel that breaks it would be a flip counted twice or missed, and the sample is refused.

The floor's identity, and where steering happened. With the floor known at every voxel (half the transitions, a coin at maximum entropy; or no transitions at all, the constant region), every departure from it is placed. Where the history leaves the floor is where steering happened, whether a body moved through, the field was driven, or a split or a lysis bumped it. The demon's waveform, which the fluidic draft lets settle on the hot bits, collapses onto exactly those places: the union U | U of every departure from the floor, over every window. Nothing outside it is asked about again.

The clock is the elevator. The fluidic draft's elevator E(f, t) reads floor f at time t directly. The clock is that elevator: volumeless and dimensionless, it takes you to any floor as fast as you can press the buttons. In the engine every floor of the tower sits at a known offset and every chunk of the stream decodes on its own from a stored first bit. Every window of the history sits at a known place in its `.knf`. So any floor at any time is reached directly, without decoding what comes before it.

claim	status
the floor section in the `.cfg`, laid down before anything else runs	built and not yet run: the `.cfg` reads and writes it; a program's driver laying it down is in the cell workbook
entropy conserved at every voxel: each bit's flips have the parity of its net change	built and not yet run: checked in the history's own pass
the floor's identity at every voxel places every departure: where steering happened	theory; the history holds it, and the reading is not built
the clock as the elevator: any floor at any time reached directly	built: the tower's floors and the stream's chunks at known offsets; the history's windows at known places

The demon's waveform collapsing onto the union U | U of every departure from the floor is untested; it is in wants.md.

6.18 18. THE ARM AS A PROBABILITY SNIPER: WHERE THE SWEATING STOPS AND THE SIEVING STARTS

The join. The sieve's section on the demon (`noise_sieve_4`, section 6) gives it arms, the identity:null permutation, which strip redundant variance down to the irreducible noise bits. The fluidic draft (`fluidic_1`, section 2) gives the observer a position governed by a probability wave, $|\psi|^2$, that settles on the hot bits. Read apart, the arms sweep: every body gets the same draws, whether anything is there or not. Read together, the arm fires where the observer's cloud is densest, the way an electron is found where $|\psi|^2$ is large. It is not placed at random, and it is not swept.

The sweep it replaces. Until now the null draws were a fixed schedule: every frame's bodies climbed against the frames half the series away, stepping on by one per draw, the same for every frame. That is sweating: the same work spent everywhere, most of it on the floor where nothing can answer.

The cloud, from the history, with no chosen number. The entropy history already holds it. At a voxel in a window, the cloud's density is how much the voxel changed: the sum over

the window's transitions of the frame XOR the frame before, which is exactly Σ_b (flips of bit b) · 2^b read off the history's nibble counts. At the floor only the low bits flip, and the density is small. Where a body moves, or a split or a lysis happens, the high bits flip and the density is large. Nothing is thresholded: the density is the change itself, in the sample's own units.

The overlap between windows. Two windows' clouds overlap where the same voxels are active in both: Σ over voxels of D_w · D_w′. For 9 windows that is 45 exact integers. They are folded into the history's own pass, since that kernel already holds every window of every voxel. They cost no pass of their own.

Each floor gets a null draw: the identity line. The history's windows are the elevator's floors, and the clock numbers them. Frame f stands on floor ⌊f / 11⌋ at place f mod 11. Its null draw on floor g is the frame at the same place on that floor, g · 11 + (f mod 11): the elevator changes only the floor number. Every floor gets its draw, the frame's own floor excepted, and so does any floor whose identity line frame falls within two frames of it. The draws run along the identity line, one per floor, instead of a fixed number stepped from the middle of the series.

Aiming and firing. A frame's null draw must read field noise only, and nothing of the frame's own bodies. The cloud overlap orders the floors: the floor whose window overlaps this frame's cloud least is drawn first, because where this frame's activity is, that floor is quiet, and what the climb finds there is the floor. The sweep becomes a sieve: every draw is spent along the identity line, in the order the cloud gives.

claim	status
each floor gets one null draw, on the identity line (same place, another floor)	built with this section
the arm aimed by the cloud instead of a fixed schedule	built with this section; measured against the sweep below
the cloud's density is the XOR change, Σ_b flips · 2^b, with no chosen number	built
the window overlaps folded into the history's pass	built
aimed draws tell bodies from the field better than swept ones	to be measured: the same samples with the same number of draws, swept against aimed

7

THE NOISE VECTOR INTEGRATION TABLE

#	noise vector	how it moves	read by	status
1	photon shot	every frame; variance grows with level	slope of E[(d − d_x)²]/4 against level	measured
2	dark current level	fixed; grows with exposure	a dark frame's mean less a bias frame's	theory; on the shot line, read as light; sim: bias and dark series
3	dark current shot	every frame; grows with exposure	a dark frame's variance less a bias frame's	theory; on the shot line, read as light; sim: bias and dark series
4	read	every frame; level-free	intercept of E[(d − d_x)²]/4	measured, as the intercept's sum
5	thermal (Johnson–Nyquist)	every frame; grows with temperature	intercept, with 4	theory; read only in the intercept's sum
6	reset (kTC)	every reset	intercept, with 4	sim: planted and read in $R^2 + \sigma^2_kTC$; on the kcr set only in the intercept's sum
7	1/f flicker	memory across frames	structure function D(k), less the x neighbor	measured
8	row banding	shared by a row, per frame	static row sums, unbalanced layout; a box's root (E16)	measured
9	column banding	shared by a column, per frame	static column sums, unbalanced layout; a box's root (E16)	measured
10	shared light scale	whole volume, with memory; grows with level	static level sums against lag, z and level; a box's root (E16)	measured
11	clock-induced / spurious charge	sparse events	spikes against dips	theory; not separable here
12	multiplicative excess (F)	per sensor scalar	slope $g{\cdot}F^2$; κ_3 against κ_2	measured as $g{\cdot}F^2$; sim: $F^2 = 2$ read
13	partition	binomial split	none on Poisson light	theory; sim: Fano ½ and 2 thinned, $f' = 1 + q(f - 1)$
14	quantum efficiency (Q_e)	scale on photons	calibrated source only	theory
15	gain (g)	per sensor scalar	transfer slope	measured

#	noise vector	how it moves	read by	status
16	optical crosstalk	blur before the draw	edge spread	theory; sim: a blur along x, the neighbor and crosstalk passes read 0; no known edge on the kcr set
17	electrical crosstalk	shared after the draw	$\Sigma d(v)d(v+\delta)$ along y and x	measured
18	fixed pattern (DSNU)	constant in time	pixel halves' covariance, intercept; a box's root (E16)	measured, as a bound; the halves pass built, its intercept graded by no sim
19	PRNU	gain per pixel	pixel halves' covariance against level^2	measured, as a bound; sim: planted and read by the halves' covariance
20	hot pixels	fixed place	loud pixels across halves and samples	measured; none found
21	dust	fixed shadow	dim places across samples	measured; none found
22	quantization	within one step	value census; $12\sigma^2 = 1$	measured
23	clipping	at the lane's ends	counts at 0 and 65535	measured
24	constant region	never flips	$\wedge_t I = \vee_t I$	measured
25	dim z correlation	per pixel, shared by every plane	$\Sigma d(v)d(v+z)$ at reach 1 to 32; a box's root (E16)	measured
26	bending transfer curve	raw route only	fit residual per level bin, both routes	measured
27	sample-wide entropy bump	one step in the whole volume's level	bit flips per window against static level	measured
28	mitosis / lysis entropy bump	near the event, whole sample	bit flips against divisions	theory; cell workbook
29	fluidic covariance around an event	falls off with distance	$\Sigma d(v)d(v+\delta)$ near against away	theory; cell workbook
30	Ω, the open term	whatever the rest leave	$E[d^2]/2$ less the sum of 1 to 29	open

7.1 LONG DESCRIPTIONS

The root noise of a box (E16 in engine_table.md) reads rows 8, 9, 10, 18 and 25 as patterns in fewer dimensions than an object's box. A term's pattern is the box's mean along the axes it shares, rounded down; its residual is the box less the pattern spread back. The crystal's coder prices the box, each residual and each pattern, and the term that saves the most bits is the root, yanked with its residual and its pattern, from which the box returns exactly (`noise_root_box`, `noise_root_return`). The `noise_root` sim (72 checks, 0 failed, `build/20260925_170854_sim_noise_root`, a tessera job) plants each term alone at variance 64, 256 and 1,024 in a box of 16 frames × 12 × 32 × 32 around a standing body, where a voxel's independent variance is 104. The null names no root; in every planted run the planted term is the root or there is none, never another term; at 1,024

every term is the root. The savings below are per mille of the box's bits, which run 1,194,939 to 1,362,651.

1. **Photon shot.** From the four vectors, the draft (ξ_shot) and compression's categories. Independent every frame; its variance grows with the level. Read per level bin, 40 to 199 with 100,000 pairs or more, at lag 1 (`transfer_residual.py`) by the neighbor route, E[(d − d_x)²]/4: its slope is g·F², its intercept R² less g·O plus every white level-free term (C3, C5, C7). Camera law, built: a Poisson count whose first four cumulants are each S (`sim_poisson_four_cumulants`, `SIM_SHOT_POISSON`, sim_camera.h), with Binomial(4S, ½) − S, symmetric, kept beside it as `SIM_SHOT_SYMMETRIC`; the `noise_floor` sim recovers slope 2.0046 against a planted 2, and it draws the Poisson law (noise_floor.cu). On the kcr set: slope 0.8168 to 1.1540, the worst bin within ±23‰ of its line on 19 of 24, and 63 to 109‰ on 0c582fdc, 12dfb391, 341df25f and 3a861e03. The raw route, E[d²]/2, reads slope 1.0156 to 5.7623 and bends (row 26).
    - **The cumulant ladder** (`track_driver --run moments`; each quiet static voxel's frames in blocks of 5, about the block's mean; `moments_ladder.py`, `ladder_against_neighbour.py`). A Poisson count scaled by g, with symmetric read noise added, holds κ_2 slope g, κ_3 slope g^2, κ_4 slope g^3. With g from the neighbor route, on the 12 clean samples with a static ceiling under 160 and no bulk floor clip, κ_3 runs 1.3 to 4.6 times the rung g^2 and κ_4 13.6 to 43.2 times g^3 (415c0a3a's κ_4 reads −27.63). On 1574802b, from level 7 to 34, κ_3/κ_2 holds at 1.49 to 1.71 while κ_4/κ_2 climbs from 2.0 to 13.6. The background's fluctuation is the shot plus a positive tail not yet named; it sits in Ω (row 30). Candidates the frames can't split: structure passing through the dimmest quarter, which only brightens; a multiplying mechanism in the camera (an electron-multiplying register holds $\kappa_3 = 3G{\cdot}\kappa_2$, which the readings don't hold); sparse bright events (spurious charge at g near 1 is far too small).
    - **What sits inside the shot.** A count drawn at a rate that itself moves is a mixed Poisson (Cox) count, whose factorial cumulants are the rate's ordinary cumulants. Rows 3, 7, 10, 11, 19 and 20 move the rate before the draw; rows 4, 5, 6, 17, 22 and 23 act after it. Reading factorial cumulants needs the count in electrons, and it waits on g apart (row 12).
    - **The shot as many coherent pieces.** With M modes in one voxel's exposure the variance is g·L + L²/M. The neighbor route fitted as a + bL + cL² (`transfer_curvature.py`) gives c within $\pm 3 \times 10^{-3}$ on all 24, positive on 13 and negative on 11, some many standard errors from 0: something besides modes bends it, and no mode count is read. κ_4 is the free check of the many-piece limit, and it fails upward.
    - The ladder in the sim, built: `noise_ladder_volume` reads each quiet static block's k_2, k_3 and k_4 against its voxel's level. In `noise_terms`, on the Poisson shot with gain 2, offset 20 and read variance 4: s_2 1.9852 within 1/50 of g, s_3 3.9933 within 1/4 of g^2, s_4 9.4410 within 3 of g^3, O 19.9533 within 3 and R² 4.1981 within 6, each reach about 5 standard errors. Next: the positive tail named; it sits in Ω (row 30).
2. **Dark current level.** From the draft (D, I_dark·Δt). Constant in time at a fixed exposure; grows with the exposure time Δt. Dark electrons enter before the gain, as light does, and

the mean dark charge g·D·Δt moves a voxel along the transfer line and never off it (C14 in compression_table.md). The neighbor route's intercept holds $-g \cdot O$, the electronic offset added after the gain, and none of the dark level. On the kcr set every sample is 100 frames at one exposure time, with no dark frames and no bias frames, and the dark level is read as light. Camera law, built: dark electrons D·Δt a frame (`dark`), drawn with the light by the shot law before the gain (sim_camera.h). `noise_charge_series` reads a dark series' mean less a bias series' as g·D·Δt: 32.0002 and 63.9988 against 32 and 64, within 1/100. Next: a bias and a dark series from the camera itself (open item 4).

3. **Dark current shot.** From the list asked for and the draft (D inside the root). Independent every frame; variance $g^2 \cdot D \cdot \Delta t$ in DN^2, on the same line as photon shot. At one exposure its variance comes with its own mean g·D·Δt, on the transfer line (C14), and it is read as light instead of in the intercept. At zero light, a dark frame's $E[d^2]/2$ less a bias frame's is $g^2 \cdot D \cdot \Delta t$, and its mean less a bias frame's is g·D·Δt; their ratio is g. The kcr set holds neither frame. Camera law, built: the shot draws the light's and the dark's electrons together (sim_camera.h). The sim reads $g^2 \cdot D \cdot \Delta t$ 64.0040 and 127.9703 against 64 and 128, within 1/5, and g, their ratio, 2.0001 and 1.9995, within 1/300. Next: dark and bias frames from the camera itself (open item 4).

4. **Read.** From the four vectors ("thermal and read"), the draft (σ^2_read) and compression's categories. Independent every frame; does not depend on the level. Read by the neighbor route's intercept (C7). Camera law, built: Binomial($4r^2$, ½) $- 2r^2$, variance r^2 (sim_camera.h); the `noise_floor` sim reads intercept 5.643 against a planted 6. On the kcr set: 0.93 to 11.94 DN^2 on 22 samples, -2.96 to 21.29 over the 24. The intercept is $R^2 + \sigma^2_J + \sigma^2_kTC - g \cdot O$ (C14): rows 5 and 6 and the electronic offset are in it, rows 2 and 3 are not, and only the sum is read; if all of it is R^2, R is about 1 to 3.5 DN. Next: a bias frame, or the κ_3 line's zero at one exposure (C19), takes the offset out; a second temperature splits row 5; row 6 stays with row 4.

5. **Thermal (Johnson–Nyquist).** From the four vectors and the draft's classification ("Thermal/Read"). Independent every frame; does not depend on the level; grows with temperature. It sits in the same intercept as row 4, and the kcr set holds no temperature series. Camera law: not built apart from row 4. Next: a run at two temperatures.

6. **Reset (kTC).** From the list asked for and the draft's classification. A new offset per voxel at every reset, independent of the signal. Adds $2\sigma^2$_kTC to $E[d^2]$ at every level, inside the intercept with rows 4 and 5. Correlated double sampling removes it on a camera that has it, and the frames don't say whether this one does. Camera law, built: a reset offset of variance `reset_square` in every voxel-frame, one more binomial half (sim_camera.h). A bias series reads $R^2 + \sigma^2$_kTC 8.0038 against 8, within 1/100, and the two are not split. Next: none owed in the sim.

    - Prior art, cited from knowledge. The photon transfer method, reading gain, read noise and dark signal from the mean and variance of flat, dark and bias frames, and kTC noise removed by correlated double sampling: J. R. Janesick, "Photon Transfer: DN → λ", SPIE Press, 2007.

7. **1/f flicker.** From the list asked for and the draft (the memory kernel M, ξ_flicker, σ²_1/f(k)). Carries memory across frames: its variance over a lag grows with the lag. Read by the structure function D(k) at lags 1 to 64, raw and less the x neighbor's difference, in level bins of 8 (`track_driver --run flicker`, noise_flicker.tsv). White noise holds D(k) = D(1), 1000‰, at every k; a random walk holds 1000·k. Camera law, built: octaves lasting 2° frames (Voss–McCartney); the sim plants 6 octaves of variance 16 and reads D(k) at lags 2 to 64 as 1112, 1219, 1321, 1411, 1482 and 1482‰, against 1112, 1220, 1321, 1412, 1483 and 1483 within 10. On the kcr set, means 40 to 199, less the x neighbor: D(k)/D(1) within 1000 ± 22‰ at lags 2 to 8 on 23 of 24 (341df25f reads 1008, 1028, 1049), and 984 to 1231‰ at lag 64. Any flicker a voxel holds on its own adds under 22‰ out to lag 8. The raw difference grows to 3,966 to 99,380‰ at lag 64, and the growth comes from what moves. The level the whole volume shares carries memory; it is row 10. Next: none owed per voxel.

8. **Row banding.** From the list asked for, the draft (σ²_row(i, k)), the numpy example (`row_banding`) and compression's spatially correlated category. An offset shared by every voxel of one row in one frame. Read on the static voxels as an unbalanced layout (`track_driver --run lines`, noise_lines.tsv): var_row/s, o where nothing is shared, with s the independent variance of a voxel's frame difference. Camera law, built: a row offset per frame; the sim plants variance 4 and reads 38‰ against 38 within 3, with the columns and the plane at 0. On the kcr set: 1 to 7‰ on 18 samples, −2‰ on 267148e4, and 23 to 101‰ on 12dfb391, 144b256d, 2f31fc2f, 3a861e03 and 53f95252. Whole planes read as rows-by-columns layouts are dominated by what moves (rows and columns 7,655 to 141,449‰; the independent part 1,214 to 38,885 DN² against the static voxels' 40 to 780), and those readings are not the camera's. Small on the clean samples. As a box's root (E16): shared along x, one value for each frame, z and y. Planted alone at variance 64 it pays for no pattern (−5,409 bits); at 256 it saves 31‰ and at 1,024 89‰. Next: none owed.

9. **Column banding.** From the list asked for, the draft (σ²_col(j, k)) and compression's spatially correlated category. An offset shared by every voxel of one column in one frame. Read as row 8 (var_col/s). Camera law, built: a column offset per frame; the sim plants variance 4 and reads 38‰ against 38 within 3. On the kcr set: 1 to 11‰ on 17 samples; 17 to 106‰ on odb75fae, 12dfb391, 144b256d, 267148e4 and 53f95252; −26 and −8‰ on 2f31fc2f and 3a861e03. Small on the clean samples. As a box's root (E16): shared along y, one value for each frame, z and x. Planted alone at variance 64 it pays for no pattern (−6,638 bits); at 256 it saves 30‰ and at 1,024 80‰. Next: none owed.

10. **Shared light scale.** Named plane bias from the draft's σ²_total extended to the whole frame. The draft's form, a new additive offset per plane per frame, does not hold on the kcr set. What holds is one change in the scale of the light, shared by the whole volume, with memory (`track_driver --run lines`, noise_planes.tsv with static_level; `plane_static_analysis.py`, `plane_memory.py`, `static_level_square.py`):

    - **Per plane:** var_plane/s on the static voxels reads 2 to 68‰ on 19 samples and 78 to 589‰ on 12dfb391, 144b256d, 267148e4, 2f31fc2f and 3a861e03.

- **Across z:** a plane's shared offset correlates with the same frame pair's plane one further along z at 797 to 988‰, two further at 528 to 970‰, three further at 358 to 954‰. It is one offset for the whole volume, shared by every plane.
- **In time:** its correlation with the next frame pair at the same z is −186 to +158‰ on 23 samples and +520‰ on ob24845f, where a new white offset each frame gives −500. The static voxels' mean difference, less the independent part s/N and with the mean drift taken off, grows with the lag faster than a random walk: past 16,000‰ at lag 16 on 19 of 24, on samples with no flagged pair as well (341df25f, 5740d24b, 668e0cc7). That is a smooth change in the level with curvature instead of a draw each frame.
- **With the level:** as a scale change, √(shared/2)/L reads 0.63% to 1.88% on the 14 clean samples, at static levels from 16.15 (1574802b) to 356.65 (144b256d). An added offset would give a ratio that falls 22-fold across that range; it falls about 1.5-fold. Fitted over the 14: $0.9415 + 7.4033 \times 10^{-5}\ L^2$, a scale change of 0.608% if the slope is all of it. The flagged samples read higher: 267148e4 5.04%, 2f31fc2f 7.05%, 0b24845f 2.90%.
- **Over the whole run:** the static level itself moves two-fold and more over the 100 frames: 5740d24b from 34.8 to 78.8, 1d530831 from 81.1 to 180.6, 668e0cc7 from 7.5 to 29.7; 1574802b falls from 19.9 to 11.3. The frame-to-frame change is the small, fast part of that motion.

A voxel and its x neighbor share the scale. The neighbor route loses it and the raw route carries it (row 26). Camera law, built: a plane offset per frame; the sim plants variance 4 and reads 36‰ against 38 within 6. A shared scale with memory is built: the camera law gains a scale path in octaves (`scale_square`, `scale_octaves`, `scale_bits`), about 1 ± 6% at 4 octaves and 10 bits in `noise_terms`, and `noise_structure_volume` fits $(d_k - d_{k,x})^2$ against $q = (\Sigma_t I - \Sigma_t I_x)^2$ and the pair total. D_a at lags 1, 2, 4, 8, 16, 32 and 64 reads 3731, 5783, 7744, 11596, 11592, 11750 and 8758 millionths, against the camera's own path 3692, 5737, 7653, 11471, 11442, 11726 and 8690, within 500. As a box's root (E16), the planes term is shared along y and x, one value for each frame and z, and the shared scale's value in each plane of a frame is inside that pattern; planted alone as a plane offset it saves 7‰ at variance 64, 28‰ at 256 and 59‰ at 1,024. What makes it (the light, the focus or the acquisition) isn't in the frames; it sits in Ω until its source is named. Next: the fit on the kcr set.

11. **Clock-induced / spurious charge.** From the list asked for and the draft's classification (CIC, under multiplicative). Sparse single-electron events at a rate p per voxel per frame, before any gain register. Read by spikes against dips at 16, 32, 64 and 128 in the static voxels (`track_driver --run clips`, noise_spikes.tsv). Camera law, built: spikes at a rational rate; the sim plants 256 electrons at 1 in 1024 and reads 47,911 of 49,187,145, the rate within 5 standard errors. On the kcr set, means 40 to 199, spikes outnumber dips at every threshold on every sample; 0113de3b reads 15,817,221 against 12,530,676 at 16, 3,459,615 against 1,397,970 at 32, 670,573 against 6,827 at 64 and 258,092 against 0 at 128. Three things read into that excess, and the frames can't split them: what moves (a body adds spikes, and a background near 100 can't dip by 128); the shot's own skew with row 1's positive tail; and spurious charge itself, which at g near 1 is one lane step and merges with the shot. The sim holds both nulls: at 16 and 32 a symmetric shot's spikes and dips agree within 5 standard

errors, and a Poisson shot's spikes outnumber its dips by more than 5, its upper tail being longer than its lower (`terms_spikes`, noise_terms.cu). Next: a null that carries row 1's tail, and the kcr set read against it.

12. **Multiplicative excess (F).** From the list asked for ("sensor dependent scalar") and the draft (F). A scalar per sensor; multiplies the shot variance by F^2. The transfer slope is $g{\cdot}F^2$, 0.8168 to 1.1540 on the kcr set, and F needs g measured apart. Single-photon peaks would be spaced $g \approx 1$ DN against a read spread of about 1 to 3.5, and no peak resolves: every value from each sample's least to its 999/1000 point is covered, with at most 7 gaps, all at the low end (`values_gaps.py`). The ladder's κ_3 gives g apart, since read noise adds nothing to it, but it reads 1.3 to 4.6 times the Poisson rung (row 1). g and F stay unsplit until the tail is named or taken out. Camera law, built: an excess $F^2 = 2$, twice a binomial half of the count (sim_camera.h); after the thinning $g{\cdot}f'$ reads 3.9993 and 3.9981 against 4. Next: row 1's tail named.

    - Prior art, cited from knowledge. The excess noise factor $F^2 = 2$ of an electron-multiplying register at high gain: M. S. Robbins and B. J. Hadwen, "The noise performance of electron multiplying charge-coupled devices", IEEE Trans. Electron Devices 50(5), 2003, 1227–1232.

13. **Partition.** From the list asked for. A binomial split of the carriers: quantum efficiency as thinning, or charge shared between wells. None on a Poisson source: a Poisson count thinned binomially is still a Poisson count; it shows only on a sub-Poisson source. Camera law, built: a sub-Poisson count of Fano factor ½ (`SIM_SHOT_HALF`), a bunched one of Fano factor 2 (`SIM_SHOT_PAIRED`), and a binomial thinning by q = keep_numerator/2^keep_bits. A thinned count holds $f' = 1 + q(f - 1)$. With g = 2, $g{\cdot}f'$ at q = ¼, ½, ¾ and 1 reads 1.7500, 1.5002, 1.2500 and 1.0000 on the half law, 2.4997, 2.9995, 3.5005 and 4.0004 on the paired law, and 2.0007, 1.9995, 1.9996 and 1.9999 on Poisson. Nothing on the kcr set reads it. Next: a sub-Poisson source.

    - Prior art, cited from knowledge. The Fano factor of a thinned count, $f' - 1 = q(f - 1)$, from the photocount statistics of sub-Poisson light: L. Mandel, "Sub-Poissonian photon statistics in resonance fluorescence", Opt. Lett. 4(7), 1979, 205–207.

14. **Quantum efficiency (Q_e).** From the draft (Q_e). A scale on the photons, applied as the thinning of row 13. After thinning the transfer curve reads the same g at any Q_e; only a calibrated source measures it. Nothing on the kcr set reads it. Next: a calibrated source.

15. **Gain (g).** From the draft (G, electrons per DN) and compression's C7 (g, DN per electron). A scalar per sensor. Read by the neighbor route's slope, with F^2 (row 12). Camera law, built: `gain`, one scalar for every voxel. On the kcr set: 0.8168 to 1.1540 over the 24 (0.821 to 1.162 on the earlier 44b6 read). It is the g the ladder is graded against (row 1). Next: none owed.

16. **Optical crosstalk.** From the list asked for and the draft (the kernel K). Light landing in a neighbor before detection: the signal is blurred, then drawn. The draws after the blur stay independent. It has no noise signature, and the neighbor correlation stays 0 for it; only a known edge's spread reads it, and the kcr set has no known edge. Camera law, built: a blur along x in eighths (`blur_eighths`), applied to S before the shot draw. Under (1, 2, 1)/4 every

column's mean is within 0.0482 of the blurred light, and the neighbor and crosstalk passes read 0. Next: a known edge.

17. **Electrical crosstalk.** From the list asked for and the draft (the kernel K). Charge or signal leaking after detection: the noise itself is shared with neighbors. Read by $\Sigma\ d(v) \cdot d(v + \delta)$ against $\Sigma\ d^2$ along z, y and x, in level bins, at each sample's background bin, the level bin below 1024 holding the most x pairs (`track_driver --run neighbours`, `neighbours_background.py`); independent noise gives 0. Camera law, built: the sim's null reads 0 along every axis and reach. On the kcr set, along y and x on the 15 samples whose background bin is 8 to 128: −3 to 53‰ on 11, and 59 to 89‰ on 1d530831, 2f31fc2f, 33b596bf and 668e0cc7. On 0db75fae, 3a861e03 and 53f95252 every axis is correlated (220 to 370‰); on 267148e4 and 5740d24b the background bin is 0 (row 23); on 12dfb391, 144b256d and 18ced818 the most-populated bin is inside the cells, where the axes read alike and it is structure. Small along y and x on the clean samples. Next: none owed.

18. **Fixed pattern (DSNU).** From the four vectors, compression's categories and the draft (η_spatial). Constant in time at a pixel; cancels exactly in d (C_3). Each (y, x) is one camera pixel in every z plane; the pixel pass sums each pixel's static voxels in the lower and upper halves of z (`track_driver --run pixels`, noise_pixels.tsv; `pixel_analysis.py`, maps less their 3 × 3 neighbors' median). Camera law, built: draw_below(key ⊕ pattern, voxel, reach + 1), added (sim_camera.h and 393). Note. That pattern is keyed by voxel, a new offset in every plane, where DSNU is one offset per camera pixel in every plane. One offset per camera pixel is not planted, and no sim grades the halves' intercept: `noise_terms` reads the halves' covariance, whose null reads 0 because no two planes share the pattern (noise_terms.cu). On the kcr set, the covariance the halves share, fitted over the samples against the level squared, is $0.041 + 2.478 \times 10^{-6}\ L^2\ DN^2$. The intercept is a fixed offset per pixel of variance 0.041, about 0.2 DN. 3bb3690f and 40c45f5a hold no pixel in both halves. As a box's root (E16): shared along the frames and z, one value for each camera pixel, y and x. Planted alone it saves 5‰ at variance 64, 24‰ at 256 and 50‰ at 1,024. Next: none owed.

19. **PRNU.** From the draft (PRNU_{i,j}, $(S \cdot PRNU)^2$) and the numpy example (`prnu_field`). A gain per pixel, constant in time: $g_v = g \cdot (1 + p_v)$. Read by row 18's fit: its slope, if all of it is a gain pattern, is a gain spread of 0.157%. Structure that spans z at a pixel adds to the same slope, and 0.157% is the most PRNU can be on the kcr set. Camera law, built: a gain per camera pixel, gain·(2^b + p)/2^b in every plane, p drawn once in [−reach, reach] (`gain_pattern_reach`, `gain_pattern_bits`, sim_camera.h). `noise_halves_volume` reads the halves' covariance and the mean level; under flat lights of 100, 200 and 400 the covariance reads 3.3845, 13.5062 and 53.9786 against 3.3744, 13.4974 and 53.99 from the drawn pattern, within 1/10, 1/5 and 2/5, and the null 0.0079, 0.0076 and 0.0052, within 1/30. Next: none owed in the sim.

    - Prior art, cited from knowledge. DSNU and PRNU as the spatial variance of the dark and of the gain, read from frame averages at several levels: EMVA Standard 1288, "Standard for Characterization of Image Sensors and Cameras", European Machine Vision Association (rows 18 and 19).

20. **Hot pixels.** From compression's spatially correlated category. A pixel with a high dark current: a large offset and extra shot, fixed at a place. Read by the loud pixels in both halves of z and at the same place across samples (`pixel_places.py`, `pixel_one.py`). On the kcr set, within a sample, the 64 loudest repeat across the halves on 1574802b (45), 551a5dba (16), 5740d24b (11) and 5f15d135 (9). They are not hot pixels: 1574802b's 20 loudest form one patch at y 243 to 253, x 66 to 81, with 0 to 7 loud neighbors each, variances 33 to 133 at means 11 to 16, loud in all 32 planes of both halves; 5740d24b's are scattered singly around y 58 to 63, x 107 to 116. Across samples the 64 loudest places repeat in 17 pairs against 18.3 by chance (edges left out). No fixed hot pixel found. The patch on 1574802b is a region, and it sits in Ω; row 25 may make it.

21. **Dust.** From compression's spatially correlated category. A shadow in the optical path: a local drop in gain, fixed at a place. Read by the dim places across samples. On the kcr set the 64 dimmest interior places repeat in 28 pairs against 18.3 by chance; no place repeats in three samples, and the pairs lean toward y 229 to 253. No dust shadow found at this sensitivity. Whether the 24 samples share one sensor position isn't in the frames; if they are views cut from different places, a fixed defect can't repeat across them by place, and rows 20 and 21 hold only while that is unknown.

22. **Quantization.** From the four vectors and compression's categories. Uniform within one step of the lane. $12 \cdot \sigma^2_q = 1$ for a unit step, inside the intercept. Camera law, built: integer lanes, clamped to the u16 range (sim_camera.h). On the kcr set (`values_gaps.py`): every value from each sample's least to its 999/1000 point is covered, with at most 7 gaps, all at the low end; even values make up 499 to 500‰ of the voxel-frames on 21 samples, and 529, 582 and 533‰ on odb75fae, 267148e4 and 5740d24b, whose zeros tip the count. A rescaled lane would leave regular gaps or an even-odd imbalance. The step is one DN, and $12\sigma^2 = 1$ stands. Next: none owed.

23. **Clipping.** From the camera law. A value past either end of the lane is clamped at the end. Read by the count of voxel-frames at 0 and at 65535 (`track_driver --run clips`, noise_clips.tsv, noise_values.tsv). Camera law, built: counted as `clipped` (sim_camera.h); the sim plants a box at 0 and one at the top and reads back exactly 2,400 and 16 constant voxels, 230,400 and 1,536 voxel-frames. On the kcr set: none at 65535 on all 24, the largest values 2,409 to 9,070; at 0, odb75fae 24,937,255, 267148e4 69,500,929, 5740d24b 27,688,962, 668e0cc7 37,232, 1574802b 6,186, 341df25f 83, and 0 on the other 18. The top never clips; the floor clips in bulk on three samples. Next: none owed.

24. **Constant region.** From the engine ledger. Voxels whose value never changes: $\wedge_t\, I_t = \vee_t\, I_t$ at the voxel. Read by the clip pass's constant voxels and their box. Camera law, built: the sim's boxes read back exactly (row 23). On the kcr set only 5740d24b holds constant voxels: 212,992 of them, all at 0, filling z 0 to 63, y 0 to 12, x 0 to 255, a strip 13 rows deep along one edge of y through every plane. It accounts for 21,299,200 of that sample's 27,688,962 voxel-frames at 0. On odb75fae and 267148e4 no voxel is constant; their zeros fall in voxels that hold other values in other frames. Of the six samples the bits-drop reading named, five were read and all five hold voxel-frames at 0, though 1574802b holds only 6,186; 587a1e22

fails its seal. The strip on 5740d24b is written by a step before the crystal (a pad or a mask), not by the sensor. Next: none owed.

25. **Dim z correlation.** From compression_table (44b6_0113de3b). The frame difference correlates with its z neighbor at the dimmest levels. Read by Σ d(v)·d(v + δ) along z at reach 1, 2, 4, 8, 16 and 32 planes, at the background bin (`neighbours_background.py`, `neighbours_covariance.py`, `noise_neighbours_z32.log`). An interpolation between neighboring planes leaves every reach past the first at 0; a blur along z fades from the first plane on; a term shared by every plane of a pixel holds. Camera law, built for the test: the sim plants a 1/8 mix of the plane before and reads z 139‰ against 140, and 0 at every other axis and reach. On the kcr set z exceeds the in-plane axes by 50 to 124‰ on the 15 samples whose background bin is 8 to 128, and by 48‰ on 5f15d135. It is flat to 32 planes on seven clean samples, for example 1574802b at 130, 129, 127, 123, 122 and 120‰. As covariance it holds, or rises a little, out to 32 planes on every clean sample (1574802b 6.10 to 5.44 DN², 668e0cc7 8.42 to 7.37, 0b24845f 21.14 to 17.86); where the per mille falls, as on 0113de3b, the far voxel lands in bright structure. Less the in-plane covariance the term is about 4 to 12 DN², roughly a tenth of the frame difference's variance at the background. It is one term per pixel per time point, shared by every plane of the stack, not an interpolation and not a blur along z. A sensor making separate exposures does not make a term every plane shares; a per-time-point image added to or subtracted from the whole stack before the crystal does, and so does an acquisition in which the planes share an exposure. The frames can't say which, and the source sits in Ω. As a box's root (E16), the stack term: shared along z, one value for each frame, y and x. Planted alone it pays for no pattern at variance 64 (− 54,497 bits) or 256 (− 17,214 bits), and saves 32‰ at 1,024. Its pattern is the largest of the five, 16 × 32 × 32 values in the sim's box. The measured term, about 4 to 12 DN², sits under the sim's weakest plant. A box of this size would not yet name it the root (a reading, not measured). Next: a per-pixel sum of the halves' frame differences multiplied, which maps it pixel by pixel and tests whether it makes 1574802b's patch (row 20).

26. **Bending transfer curve.** From compression_table. A transfer curve that is not a straight line from level 40 to 200. Read by the fit's residual per level bin on both routes (row 1). On the kcr set the bend is in the raw route; the neighbor route is straight within ±23‰ on 19 of 24. On the two samples first named: 12dfb391 reads slope 0.9013, intercept 0.93, worst bin 109‰ at level 60; 53f95252 reads slope 0.8168, intercept 21.29, worst bin − 13‰ at level 68. The neighbor difference removes the two things that bend the raw route: what moves, and row 10's shared scale. 53f95252's intercept of 21.29 is the largest on the set, is not explained, and sits in Ω. Next: none owed for the bend.

27. **Sample-wide entropy bump.** From the entropy history (`entropy_history`, engine ledger; noise_sieve_tower §16). Read by the bit flips per thousand transitions, summed over bits 5 to 10, in each of the 9 windows of 11 transitions, against the static level (`noise_entropy.log`, `entropy_against_motion.py`), and by the frame pairs whose static mean square is past twice the sample's median (`motion_scan.py`, counted, not removed). On 0113de3b the windows read 1119, 1191, 1202, 1207, 1336, 1187, 1189, 1151 and 1084; window 4 (transitions 45 to 55) is 123‰ over the median window and holds the sample's one flagged pair, pair

50, reading 264.2 against a median of 122.3. That pair is a step in the whole volume's level (`plane_window_static.py`): the static voxels' mean frame difference is +4.785 at pair 49 and −10.040 at pair 50, against a few tenths either side. The level rises about 4.8 at frame 50 and falls about 10 at frame 51, a net step down of about 5 DN, about a tenth of the level; the whole volume's mean square peaks at the same pair, 84,675. Over the 24, most samples have no one-window bump: their flips follow the static level, which moves over the frames (row 10), correlating at 788 to 990‰ on 15 samples. The correlation runs negative on the dense or motion-flagged samples, odb75fae −255, 144b256d −525, 18ced818 −193, 267148e4 −577 and 2f31fc2f −837, and reads 21, 82, 385 and 663 on 24264f12, 33b596bf, 0113de3b and 3a861e03. 14 samples have no flagged pair; the other 10 have 1 to 32, the longest in runs (267148e4 66 and 68 to 98; 0b24845f 82 to 98; 2f31fc2f 0 to 12, 26 to 28, 31, 59, 73). What makes 0113de3b's step (the light, the focus or the acquisition) isn't in the frames, and it sits in Ω.

28. **Mitosis / lysis entropy bump.** From `proposal.md`: if a cell really lysed or divided, the entropy bumps near that time, judged over the whole sample. Read by the history's bit flips per window against the ground truth's divisions. The divisions are about the organisms, and it is read in the cell workbook instead of here. Not run. On 0113de3b, row 27's bump is one step in the whole volume's level.

29. **Fluidic covariance around an event.** From `proposal.md`: mitosis and lysis change the fluidics for a great distance around them. As noise, a covariance around an event that falls off with distance; it must be told apart from rows 8 to 10 and 17. Read by $\Sigma\, d(v) \cdot d(v + \delta)$ at growing distance from the ground truth's divisions, against the same sums away from any division. Its reading belongs to the cell workbook. Not run.

30. **Ω, the open term.** Whatever the named rows leave. Read per voxel as $E[d^2]/2$ less the sum the rows above predict at its level, compared as integers, and per axis as the covariance the rows above leave; a named row moves out of Ω only when its sums account for its part. Nothing to plant: Ω holds whatever the sims do not know. On the kcr set it holds:

    - row 10's shared light scale with memory, until its source is named and the camera law carries it;
    - row 25's per-pixel term shared by every plane, until its source is named;
    - row 1's positive tail past the Poisson ladder, κ_3 1.3 to 4.6 and κ_4 13.6 to 43.2 times the rungs;
    - 0113de3b's step in the level at frames 50 and 51 (row 27);
    - 53f95252's intercept of 21.29 (row 26);
    - the loud patch on 1574802b (row 20).

    Irreducible until its construction is deduced. Every experiment shrinks it; what is left names the next row.

7.2 WHAT WAS ASKED FOR, VERBATIM

Definition corrected ("q4 verbatim text, corrected for definition only"):

> "to our noise detector we want to add dark current shot noise, reset noise, 1/f noise flicker, row and col noise (banding patterns), clock induced charge/spurious charge noise. multiplicative excess noise (sensor dependent scalar), partition noise, crosstalk noise (optical or electrical), this will work for compression and filtering"
>
> "we should also include an open term for terms we cannot or have not described, they are irreducible until we can deduce their construction"
>
> "example do not use npy"
>
> "get all noise vector terms ironed out"
>
> "build all four, one at a time"
>
> "flattening turns examples into kcr. you run your programs over kcr"

Untested when spoken:

> "honest read from the noise filter, the more we qualify our integral with our coherence of "noise" the more condensed the .kcr will become, because we will be pulling repeating or irreducible information that can be represented in a smaller dimension out of the crystal."
>
> "and that also gives us another real anchor when running nbody problems to calculate vectors of very distant objects with exactness not possible before"
>
> "what we look for is that two smaller terms can reconstruct the photon shot"
>
> "if the noise terms are all members of the photon shot set, we find the construction set in it"
>
> "or like, the combination of all noise terms matches photon shot"
>
> "something like that, all these pieces together equal chaos, individually they are coherent"

What the runs say against them:

- **Condensing the crystal:** under "What the integrated floor is for", Compression.
- **Distant objects in n-body problems:** not tested.
- **Two smaller terms reconstruct the shot:** row 12. The transfer slope and κ_3 would give g and F apart; κ_3 reads 1.3 to 4.6 times the Poisson rung, and neither is read yet.
- **The shot set's construction:** row 1, what sits inside the shot.
- **All the pieces together as the shot:** row 1, the shot as many coherent pieces and the ladder. On this set they don't match; the miss is a positive tail, in Ω.

7.3 THE NOTATION

A voxel v sits at (z, y, x) and holds I_t(v) in frame t, in DN. Its level L is its value's mean over the frames it is read in. The frame difference is d_t(v) = I_{t+1}(v) − I_t(v). Following compression_table C7, the gain g is DN per electron, and the draft's G is its inverse (electrons per DN). O is the lane's value at zero light. Every reading below is a sum of integer products over named voxels and frames. Two ratios are compared by multiplying across (a·d against b·c), as the period's 128-bit test does. A spread is compared as its square, with no root, and a code length is a bit length, with no logarithm.

The composite each voxel's frame difference integrates, with structure taken off by the neighbor (C5):

$E[d^2] / 2 = F^2 \cdot g \cdot (L - O) + R^2 + \sigma^2_thermal + \sigma^2_kTC + \sigma^2_row + \sigma^2_col + \sigma^2_1/f(1) + \sigma^2_CIC + 1/12 + \Omega(v)$

with F the excess noise factor, R^2 the read variance and 1/12 the quantization of a unit step (stored as $12 \cdot \sigma^2$ to stay integer). The fixed pattern and the dark level cancel in d (C3). The dark current's shot rides the same line as the photon shot, since the dark electrons are counted in L. PRNU scales g per voxel instead of adding a term. Optical crosstalk blurs S before the draw and adds no noise term. Electrical crosstalk adds covariance between neighbors instead of variance at one voxel. Row 10 is a scale ε shared by the whole volume instead of an added draw: it adds $\varepsilon^2 \cdot L^2$ to the raw route, and the neighbor difference removes it. Ω is the open term.

7.4 THE DRAFT, TERM BY TERM

The draft states the noise as a stochastic wave functional. It is kept as written in the thought experiments. Each of its objects has an exact form here:

the draft's object	what it is in the draft	its exact form here	status
Φ(x, t)	the real-valued noise field	the residual per voxel-frame, I_t(v) less the fixed pattern and the structure, in integers	**theory**
K(x) ∗	the crosstalk kernel, Gaussian or Laplacian	rows 16 and 17: an integer kernel over a power of 2, before the draw (optical) or after it (electrical)	**theory**
S(x, t)	the photon signal field	the structure; in the sims, `sim_signal`'s electrons	**built** in the sims
D(x, t)	the dark current rate field	rows 2 and 3	**theory**
ξ_shot, √(F·(S + D))	a white Gaussian field scaled by the shot spread	row 1's binomial draw; the Gaussian is the draw's large-count limit and is not adopted; the spread is compared as its square	**built** (the binomial)
F	the multiplicative excess noise scalar; the draft gives 1 for CMOS and about √2 for EMCCDs	row 12; the draft's numbers are its own, not measured here	**theory**

the draft's object	what it is in the draft	its exact form here	status
η_spatial(x)	the time-invariant field: PRNU and banding	rows 18 and 19; banding moves to rows 8 and 9, since the draft's own σ²_row(i, k) and σ²_col(j, k) carry the frame index k	**theory**
ζ_temporal = ζ_white + ∫ M(t − τ) ξ_flicker(τ) dτ	white noise plus flicker through a memory kernel	rows 4 to 6, and row 7; the integral becomes a sum of integer draws lasting 2° frames, octave by octave	**built** in the sims
M, a kernel giving 1/f^α	the flicker's memory	the octaves' weights set α; D(k) reads it (row 7)	**measured**: under 22‰ per voxel to lag 8
a Hilbert space and a functional probability density	the state space of Φ	not adopted: the detector reads counts and sums, never a density	**theory**
Z = ∫ DΦ exp(−S[Φ])	the partition function as a path integral	not adopted	**theory**
S[Φ] quadratic in Σ^{-1}	the action under the central limit's Gaussian	not adopted; its data are row 17's covariance sums and row 7's structure function, taken exactly; row 1's ladder finds the background is not Gaussian past κ_2	**theory**
Σ(x, x′, t, t′)	the global spatiotemporal covariance kernel	the table of exact lag sums: Σ d(v)·d(v + δ) at each spatial lag δ and D(k) at each temporal lag k; the correlation operator (the research papers' C_{t,t'}, L_{t,t'}) is this object	**theory**
σ²_total(i, j, k) = G²F²(Q_e·S + I_dark·Δt) + (S·PRNU)² + σ²_row + σ²_col + σ²_read + σ²_1/f	the discretized composite	the composite under "The notation", with rows 5, 6, 10, 11, 22 and Ω added and G written as g = 1/G	**theory**

Two places where the draft disagrees with itself:

- **F's power.** The field equation scales the shot by √(F·(S + D)), which puts F once in the variance. The discretization writes G²·F², which puts F squared there. Row 12 follows the discretization: the variance carries F².
- **Banding in time.** The draft sets banding in η_spatial, which is constant in time. Its discretization writes σ²_row(i, k) and σ²_col(j, k), with the frame index k. Rows 8 and 9 follow the discretization: a new offset every frame. Row 10, which the draft's σ²_total extends to, is a shared scale with memory instead.

The numpy example, term by term:

the example's line	here
`expected_charge = signal_field + dark_rate`	S + D, rows 1 to 3

the example's line	here
`np.random.normal(0, np.sqrt(expected_charge))`	row 1's Binomial(4S, ½) − S, with no float and no root
`prnu_field = np.random.normal(1.0, prnu_sigma)`	row 19
`row_banding = np.random.normal(0, 0.5, size=(shape[0], 1))`	row 8, a draw per frame and row
`white_alloc = np.random.normal(0, read_noise)`	row 4
the optional FFT filter for 1/f	row 7's octave draws, with no FFT
`total_field = (expected_charge * prnu_field) + shot_noise + row_banding + white_alloc`	the composite under "The notation"

Per "example do not use npy", nothing here uses numpy or a float.

7.5 WHAT THE INTEGRATED FLOOR IS FOR

Compression. Each named row's predicted spread at a voxel's level gives that voxel's counts, and the coder spends $\Sigma\ -\log_2 p$ against them (compression_table F5). The bits go to departure and to Ω. When a set needs a category the rows lack, the gap to F5 names it, and it enters Ω first. On the 15 clean samples of the kcr set, the coherent terms measured here (static rows, columns and plane, and the z correlation) are worth 1.0% to 3.0% of the static background's frame-difference bits, which run 4.71 to 6.04 bits per voxel-frame (a Gaussian estimate, `coherent_bits.py`). The independent shot and read are the rest.

Filtering. A voxel-frame whose d departs from the integrated floor by more than the null allows is departure: a body, a split, a drive (noise_sieve_tower §17). The null draws read the rate against the integrated floor, not against one sample-wide reading.

7.6 THE ORDER OF THE EXPERIMENTS

Each run below is a tessera job and cites the rows it moves.

Run : the camera law gained row, column and plane offsets per frame, flicker octaves and spikes at a rational rate, and the `noise_terms` sim grades rows 7, 8, 9, 10, 11 (the rate), 17, 23, 24 and 25 (the interpolation test). On the kcr set: the transfer curve on both routes (rows 1, 4, 15, 26); the structure function (row 7); the line and plane sums and the static level (rows 8 to 10); the neighbor sums to 32 planes (rows 17 and 25); the clip, value and spike census (rows 11, 22, 23, 24); the pixel halves (rows 18 to 21); the cumulant ladder (rows 1, 11, 12); the entropy history against the static level (row 27).

Open:

1. A reading that splits row 1's positive tail among structure, a multiplying mechanism and sparse events: for example the tail against distance from the nearest cell, or the ladder on a set with dark frames.

2. Row 18's DSNU, one offset per camera pixel in every plane, is not planted: the camera law's fixed pattern is keyed by voxel (sim_camera.h), and no sim grades the halves' intercept. The

rest of this item as it stood is built on main: the shared scale, D and kTC, F and the optical kernel and the gain pattern.

3. Row 25's per-pixel map, the halves' frame differences multiplied, against 1574802b's patch (row 20).

4. What the kcr set can't give: a second exposure time, dark frames and a temperature series (rows 2 to 6), a sub-Poisson or calibrated source (rows 13 and 14), a known edge (row 16), and the samples' sensor positions (rows 20 and 21).

5. Rows 28 and 29 in the cell workbook, against the ground truth's divisions.

6. Ω per voxel, after every row above that has a reading.

8

THE ENGINE TABLE

Purpose: The machine, part by part: what each part computes, the exact algebra it holds to, what it does, what it wants, every hypothesis tried there with its result, and the next move.

Layout: orior's `src/engine` is in category folders (arithmetic, analysis, codecs, compiler, formats, nbody, prg_sch, quantum, render, runtime, sims), with the tests under `test/engine`; item 13 of the list at the end maps every old folder to its new one. The benches are the root `bench/`, the tests the root `test/`, and every build writes only into the root `build/`: each build into its own `build/<date>_<time>_<name>/`, removed when a later build starts more than a day on, with the finished program copied up into `build/` and `build/logs/` and `build/tsv/` never removed. The six steps (ingest, seal, lift, measure, partition, relate) and every file's step are in engine/README.md.

Scope: `engine/`: exact arithmetic, the residual operator, the component tree, the moments, the correlation operator, the marginal, matching, the entropy history, the files (their size against the floor is tracked in compression_table.md), the record machine, errors, the seal (integrity; its theory is obsignatio_seal.md), the scheduler, the sift, render, the memory operations, the device pools, the period reading, the sims, the root universal, the root noise of a box, and the ask and the state. The programs that run on it are separate concerns with their own tables: cell tracking in cell_tracking_table.md, and RSNA knee MRI (its own project). Numbers are in ledger.md. Statuses follow README.md. The protocol is query_protocol_table.md and the plan engine_plan.md.

8.1 THE RULE THE MACHINE MUST SATISFY

The engine is optimized for no scale. A size, a spacing, an order, a window, a width, a cell or a voxel count is never written into the machine. Each comes in with the request or is read from the data. The machine is exact at every scale its words can hold, and where a word is too narrow it says so (a request error, or `needed_bits`) and never rounds. Tuning the machine to one program's scale is the error this table exists to catch, and the audit below lists what is still in.

The machine and each program are optimized against each other only through the interface at the end of this table. Merged, neither can be optimized on its own.

8.2 THE ENGINE IN MATH

The whole engine algorithm in math, from every workbook, with zero terms missed. Sources: every workbook file (this table, vertical_time_compression.md, two_crystals.md, imprint_key_cycle.md, keys_explained.md, kolmogorov_arnold.md, noise_sieve_tower.md, obsignatio_

seal.md, tessera_scheduler.md, compression_table.md) and the code (`tower.cu`, `keymath.cu`, `cycle.c`, `pi_tower.cu`). Each line carries the status its source gives it. What is not tied to the code is listed at the end.

Objects. A sample is u : Λ → [0, 2^16), Λ = [0,T)×[0,Z)×[0,Y)×[0,X), axes (t, z, y, x). A frame is u_t = u(t, ·). The atom is all of the thing under inspection, stored as one integer: A = Σ_i v_i·2^{16i}, lane i at place 2^{16i}. Every value is in ℤ; nothing is rounded.

E0. Numbers (M1, A0).

- Width: 2^k·32 bits, k ≥ 0, no ceiling (256 limbs in the engine build, fitted to the record file).
- a·b: long multiplication, then Karatsuba from 32 limbs, then Schönhage–Strassen mod 2^n + 1 from 8,192 limbs (every twiddle a shift).
- a = q·b + r, q = trunc(a/b), |r| < |b|, sgn r = sgn a: Knuth's D, and Newton's reciprocal once divisor and quotient reach 8,192 limbs. b = 0 is refused.
- Exact a/b: b = 2^s·o with o odd. The quotient is ((a ≫ s)·o^{-1}) mod 2^{32·limbs}, with o^{-1} from x ← x(2 − o·x); it is refused unless q·b = a.
- gcd: Lehmer's (Knuth L); gcd(0, x) = |x|, gcd(0, 0) = 0.
- A decimal (−1)^neg·digits·10^ex maps to ⌊|d|·2^{−e2}⌋ in W bits, with e2, `terminates`, `fits` and `needed_bits`.

E1. The record machine (M10, A13): the evaluator every engine program runs on.

- A program is P = (s_1, …, s_n), s_i = (op_i, l_i, r_i, μ_i), with l_i, r_i < i, and outputs O ⊆ {1..n}, no output twice.
- A lane ℓ reads up to 3 records ρ_μ = in_μ[idx_μ(ℓ)] (idx the identity when none is given). Registers v_1..v_n ∈ ℤ, v_i = op_i(v_{l_i}, v_{r_i}):

op	v_i	width w_i (keymath)
FIELD	bits [0, 0+b) of ρ_μ, unsigned	b
FIELD_SIGNED	the same, signed	b
CONSTANT	c < 2^64	bitlen(c)
SUM, DIFFERENCE	a ± b	the fewer of max(w_a, w_b) + 1 and the linear form's bound
PRODUCT	a·b	w_a + w_b; by a constant, the fewer of that and the linear form's bound
ABSOLUTE	|a|	w_a
COMPARE	sgn(a − b)	1
LADDER (b > 0)	sgn(a)·#{k ∈ [1, 91] : F_k·b ≤ |a|}, F_1 = F_2 = 1	bitlen(91)
QUOTIENT	trunc(a/b)	w_a − ⌊log2 c⌋ (≥ 1) for a constant c, else w_a
REMAINDER	a − b·trunc(a/b)	min(w_a, w_b)
GCD	gcd(a, b) ≥ 0	max(w_a, w_b)

op	v_i	width w_i (keymath)
EXACT_QUOTIENT	a/b, b | a, else the lane is refused	as QUOTIENT
XOR, AND	bitwise on two's complement without end	max + 1; AND with one never-negative operand = that operand's width, with two = the narrower; XOR of two never-negative = max
WRAP (w ≥ 4)	((a + 2^{w − 1}) mod 2^w) − 2^{w − 1}	min(w_a, w)
TABLE	T[|a| mod 2^b], 1 ≤ b ≤ min(32, w_a)	out_bits

- The linear form (A16): every register is c + Σ c_i·v_{a_i} over atoms a_i. SUM and DIFFERENCE add their operands' forms, PRODUCT by a constant (a form with no atoms) scales the other's, CONSTANT is its value, and a WRAP that passes its register through keeps that register's form. Every other register, FIELD included, is an atom. The bound is bitlen(|c| + Σ |c_i|(2^{w_{a_i}} − 1)). A coefficient or constant past 2^62 makes the register an atom.
- Never negative (N): FIELD, CONSTANT, ABSOLUTE, GCD and TABLE; SUM, PRODUCT, QUOTIENT, EXACT_QUOTIENT or XOR of two in N; REMAINDER whose numerator is in N; AND with either in N; a WRAP that passes one in N through.
- Refused at imprint: an operand that is not earlier, w_i > 32·256, a WRAP below 4, a table index wider than its source. Refused per lane: b = 0, or an inexact EXACT_QUOTIENT. A refused lane adds 1 to a device count, and a count ≠ 0 refuses the whole sweep as a request error (`cycle.c`).
- Out: ⊕_{i∈O} v_i, each as two's complement over w_i + 1 bits, the sum below 2^31 bits.
- Layout: register i lives on [i, last reader of i]. With reuse, first fit in the file, ≤ 256 limbs; without it, one register per step at the running total. The kernel's file is 64 limbs when that holds the layout, else 256. A dividing program carries 5·WIDE + 4 limbs of scratch.
- Sweep: R_ℓ = P(ρ(ℓ)) for every ℓ, grid-stride, ≤ 4096 × 256 threads, one launch. The host runs P on the exact integer, and device = host word for word is the port's proof.
- A member's field lies in its first 2^32 bits.
- Laws:
  - (h∘g)∘f = h∘(g∘f), g∘f ≠ f∘g. A stack of floors is one program. Time per lane ∝ n, independent of depth. Stacked against chained measured 13 to 22 times faster.
  - WRAP to w is π_w : $\mathbb{Z}_2 \to \mathbb{Z}/2^w$, window W_w = [− 2^{w − 1}, 2^{w − 1}); π_w ∘ π_{w+1} = π_w.
  - For F over {SUM, DIFFERENCE, PRODUCT, XOR, AND}: π_w ∘ F = F_w ∘ π_w (proved).
  - These do not factor through π_w: ABSOLUTE, COMPARE, QUOTIENT, REMAINDER, GCD, LADDER, TABLE, ⌊v/2^k⌋ (through π_{w+k}), and EXACT_QUOTIENT by an even c.

  - For c odd with c | v: WRAP(v/c, w) = WRAP(WRAP(v, w)·(c^-1 mod 2^w), w). The kernel grows c^-1 by Newton: 6, 12, 24, 48 bits in a word, then doubling.
  - Mod an odd p^v, REMAINDER is the projection $\mathbb{Z}_p \to \mathbb{Z}/p^v$, and SUM, DIFFERENCE and PRODUCT commute with it. The CRT join y_2 + 2^8·(((y_p − y_2)·2^{−8}) rem m) is the run mod 2^8·m. XOR breaks mod 3.
  - τ_k(x) = trunc(x/2^k): τ_j∘τ_k = τ_{j+k}; order is kept weakly; τ_k(x + y) − τ_k(x) − τ_k(y) ∈ {−1, 0, 1}.

- Keys, the linear programs (imprint_key_cycle, keys_explained):
  - key = P(δ), δ the impulse. For linear shift-invariant P, P(x) = key ∗ x.
  - w·x = Σ over the set bits b of w of x ≪ b.
  - With lanes wide enough that none carries, A·K holds the smoothed lanes, and K = K_z·K_y·K_x.
  - A lane below 2^m under a row of sum S stays below 2^{m + bitlen(S − 1)}.
  - The CRC-64 key: the advance over 2^k zero bytes is A^{2^k}, 48 matrices (24,576 bytes), and crc(a ⊕ b) = crc(a) ⊕ crc(b) ⊕ crc(0).
  - The mirror fold repeats with period 2N: one period is laid as a table of offsets.

- Tables: (g∘f)[x] = g[f[x]]. Two never-negative u, v with b_u + b_v ≤ 32 index as u·2^{b_v} + v. A table is $2^b \cdot \lceil \text{out_bits}/32 \rceil \cdot 4$ bytes.

- The lane index as a register and the latch, first lane = min{ℓ : cond(ℓ)}, are built, with the broadcast. `LANE` (`ENGINE_RECORD_LANE`, 18, `engine_config.h`) reads the number ℓ the sweep runs the lane as, 64 bits and never negative. The broadcast: a member holding one record is read by every lane that has no index, and one shared record with x = base + ℓ enumerates a range with nothing stored a lane. The latch is `cycle_record_latch` (`cycle_record_launch.cu`): a scan in each thread, a warp shuffle tree and one atomic minimum a block, returning `CYCLE_LATCH_NONE` (`cycle.h`) where no lane's condition holds. `cycle_record_latch_host` (`cycle.c`) is the same on the host. A latch through the engine's own entry is not built. `record_lane_test` is 17 checks, 0 failed.

- The sum across lanes is built: `cycle_record_sum` (`cycle_record_launch.cu`) adds one output of every record over each run of consecutive lanes, exactly, as two's complement. Each 32-bit limb is summed into its own 64-bit column by the lanes of a thread and one atomic add a run, the negative fields are counted beside, and the host takes the carries. It merges nothing: every column is exact, the whole sum is kept, and each lane's record stays where it lies. `cycle_record_sum_host` (`cycle.c`) adds in order and is its port check. `record_sum_test` is 98 checks, 0 failed: sums of ℓ, -ℓ, ℓ^2 and -ℓ^2 over 2^22 lanes meet their closed forms, and random fields up to nine limbs meet the host word for word.

E2. Ingest (M9). Source → (u, σ). A signed DICOM pixel p ↦ p + 2^15. Slices are ordered by an exact key from their positions (E0). σ = the headers, deflated by our LZ77 and Huffman.

E3. The seal (M13, A10).

- K_ℓ = BLAKE3-derive-key("obsignatio aeterna 2026-09-23 " ‖ ℓ), H_ℓ(m) = BLAKE3-keyed(K_ℓ, m). The levels ℓ: row, plane, volume, lanes, chunk, stream, side stored, side inflated, side, members, sample, set, file, universal.
- ρ(t, z, y) = H_row(u[t, z, y, ·]), π(t, z) = H_plane(ρ(t, z, ·)), υ(t) = H_volume(π(t, ·)), Λ = H_lanes(υ(·)).
- κ(c) = H_chunk(LE64(ℓ_c) ‖ bits_c); Σ = H_stream(LE64(o_0 … o_{C − 1}) ‖ LE64(B) ‖ κ(0) ‖ … ‖ κ(C − 1)).
- σ = H_side(H_side stored(deflated) ‖ H_side inflated(inflated)), μ = H_members(the six tables ‖ the names).
- Ω = H_sample(LE64(extent, C, B, lane offset, members, side bytes, deflated bytes, name bytes, lane nodes) ‖ Λ ‖ Σ ‖ σ ‖ μ). Θ = H_set(Ω_1 ‖ … ‖ Ω_n), in the order named.
- Check order: κ, Σ, σ_s, μ, then σ_i, σ, then ρ … Λ, then Ω.
- Locality: a change at (t, z, y, x) changes exactly ρ, π, υ, Λ, Ω and Θ on its path; a change in chunk c changes κ(c), Σ, Ω and Θ. Any other node agrees except with probability 2^ − 256.
- The device fold (pair each level, carry the odd node up) is BLAKE3's left-heavy tree. H is not associative, and the shape is therefore fixed.
- The witness: the decoded voxels against the source on the device, then a second independent read, pixel for pixel.
- Ruled and not built: H_universal over every file the engine touches plus the engine's own hash.

E4. The lift (M9, M20, A8, A14).

- One level along one axis of length N, on the active block:
  - d_j = x_{2j+1} − ⌊(x_{2j} + x_{2j+2})/2⌋, j < ⌊N/2⌋;
  - s_i = x_{2i} + ⌊(d_{i − 1} + d_i + 2)/4⌋, i < ⌈N/2⌉;
  - an edge repeats its neighbor: x_{2j+2} := x_{2j} at the end, d_{ − 1} := d_0, d_{⌊N/2⌋} := d_{i − 1};
  - low-pass coefficients stored on [0, ⌈N/2⌉), high-pass coefficients on [⌈N/2⌉, N);
  - ⌊v/2^k⌋ is taken toward − ∞ (`tower_floor_shift`).
- A level runs the axes 0..3 whose extent is ≥ 2, in order. The extents then go e ↦ ⌈e/2⌉, and the next level lifts only the all-low-pass corner. There are L levels, until every extent is 1.
- T^-1: x_{2i} = s_i − ⌊(d_{i − 1} + d_i + 2)/4⌋, then x_{2j+1} = d_j + ⌊(x_{2j} + x_{2j+2})/2⌋. Levels and axes run in reverse.

- The ruleset, the wave transform's math. A line splits into its low-pass coefficients L (the evens) and high-pass coefficients H (the odds), and a lifting step moves one band by a rounded sum over the other:
  - target_a ← target_a + σ·⌊(r + Σ_k w_k·other_{a+o_k}) / 2^s⌋, σ = ±1, an index past either end of `other` taken at that end;
  - a ruleset R = (ρ_1, …, ρ_m) runs its steps in order along every line of every level. T^{-1} runs them last first with each σ turned. A step moves only its target, reading only the other band, and every ruleset is therefore invertible exactly, whatever its weights;
  - the kernels' 5/3 is ρ_1 = (H, σ = −1, o = (0, 1), w = (1, 1), r = 0, s = 1), then ρ_2 = (L, σ = +1, o = (−1, 0), w = (1, 1), r = 2, s = 2);
  - two rulesets the kernels do not run are the S transform, (H, −1, (0), (1), 0, 0) then (L, +1, (0), (1), 0, 1), and a four-tap predict, (H, −1, (−1, 0, 1, 2), (−1, 9, 9, −1), 8, 4), with the 5/3's update;
  - on the record machine a weight other than ±1 is a PRODUCT by a constant, which keymath's linear forms carry (A16), and each constant is one step, laid where it is first read.
- Edges: E_k(w) = w − (w mod 2^b) + π_k(w mod 2^b), π_k a permutation of [0, 2^b), $1 \le b \le 20$.
  - Lift = E_L ∘ T_L ∘ … ∘ E_1 ∘ T_1 ∘ E_0. Lower is the inverse, last laid first.
  - Two edges on one floor fold into π_b ∘ π_a; a level between them blocks the fold.
  - A non-bijective f rides as (x, y) ↦ (x, y ⊕ f(x)) (Bennett), at the price of $b_x + b_y \le 20$ bits.
- $|c| \ge 2^{30}$ refuses the lift.
- The lattice lifted is u's 16-bit lanes or a lattice of ints (`device_values` on the lift's request), such as a residual left below zero (E16). An int with $|v| \ge 2^{30}$ refuses the lift, as a coefficient does.
- Stored (`compression.cu`):
  - zigzag: z(c) = 2c for $c \ge 0$, $2|c| - 1$ for $c < 0$;
  - per block of 64: g = bitlen(⌊Σz/64⌋); k is the least-cost $k \in [\max(g - 2, 0), \min(g + 1, 31)]$, with cost(k) = 5 + Σ_z (⌊z/2^k⌋ < 24 ? ⌊z/2^k⌋ + 1 + k : 24 + 32);
  - written as k in 5 bits, then per value q = ⌊z/2^k⌋ ones and a zero and the low k bits, or 24 ones and the 32-bit value;
  - 64 blocks a chunk, and chunks sit at stored bit offsets: any floor and any chunk is read directly (the elevator E(f, t)).
- Passes when T^{-1}(decode(stream)) = u lane for lane, and u equals a second read of the source.
- Properties (proved along one line on 64 samples at L = 4 unless marked):

  - T is a bijection of $\mathbb{Z}^n$, a homeomorphism of $\mathbb{Z}_2^n$ (derived), and keeps Haar measure.
  - Reach: T's level-ℓ low-pass coefficients read 3ℓ bits down, its high-pass coefficients $3\ell - 2$; T^{-1} reads $L + 2$. So $|T(x) - T(y)|_2 \le 2^{3L}|x - y|_2$.
  - Linear part M, entries in $\mathbb{Z}[1/2]$: $T(x + 2^{3L}z) = T(x) + M \cdot 2^{3L}z$. A constant c moves only the level-L low-pass coefficients, by c. Negation and doubling do not pass. $\det M = +1$.
  - Haar: every output quantum at level w holds exactly 2^{3Ln} input quanta at level $w + 3L$.
  - $\mathbb{R}$: $2^{-k} \cdot T(\mathrm{trunc}(2^k x)) \to Mx$. $\mathbb{Z}_p$, p odd: T does not extend; M does, in $SL_n(\mathbb{Z}_p)$.
  - Widths: keymath's linear forms (A16) give every register at level ℓ, low-pass and high-pass, $w + \ell + 1$ bits: the first level adds 2, each after it 1 (derived by hand; the ring below matches). Values obey $|s| \le 1.5B + 1$ and $|d| \le 2B$ (+0.585 and +1 bits a level).
  - Ring: $\mathrm{ring}_\ell = \mathrm{ring}_0 + n + 2n(1 - 2^{-\ell})$ for $\ell \ge 1$ (measured: 1,536, 1,664, 1,696, 1,712 and 1,720 at $n = 64$, $w = 24$). The mirror floor still adds $n/2^\ell$ (1,720, 1,712, 1,696 and 1,600), and the heap mirrors exactly across the crystal.
  - Code length: $\Sigma_r \lceil \log_2(W_r + 1) \rceil + \Sigma_r m_r \ge K(x \mid \mathrm{program}) - c$.
  - Identity by null permutation: a lane is identified when $\mathrm{heap}(T(x)) < \mathrm{heap}(T(\sigma_i x))$ for every $i \le d$. Noise passes at a rate $\le 1/(d + 1)$. $T(\sigma x) = T(x) \Leftrightarrow \sigma x = x$.
  - The KA step: $x \leftarrow x + \Phi(\Sigma_p \varphi_p(x_p))$, with $\Phi(s) = \lfloor s/2^k \rfloor$.
  - Matching on a floor. Floor 2 of a 64^3 frame is its 16^3 corner after two levels, $m = n/64$ values. Laid once as 16 bit planes, it gives every position holding x as $\wedge_j (x_j ? P_j : \neg P_j)$, 32 positions a word, reading no value and at most $16m/32 = n/128$ plane words; a word stops once its mask is empty. **Proved** (`floor_match`, 11 checks, 0 failed): the engine's floor 2, the crystal's corner lowered as its own tower, equals the host's two-level lifting at 4,096 of 4,096; 1,024 values on floor 2 and 1,024 not, every match set equal to the host's scan; the most any query read was 1,340 plane words, against $n/2$ = 131,072 samples. The index costs one read of a, the lift, paid once.

- As record floors, emitted by the engine itself (`tower_record_lift` and `tower_record_lower`, each floor focused from a ruleset; A13):
  - $\lfloor v/2^k \rfloor = \mathrm{EXACT_QUOTIENT}(v - \mathrm{AND}(v, 2^k - 1), 2^k)$;
  - $E(v) = (v - \mathrm{AND}(v, 2^k - 1)) + \mathrm{TABLE}_\pi(\mathrm{AND}(v, 2^k - 1))$;
  - $F \circ T^{-1}$ is one stack, and $T^{-1} \circ T = \mathrm{id}$;
  - $T^{-1}GT$ is G conjugated, $(T^{-1}G_2T)(T^{-1}G_1T) = T^{-1}G_2G_1T$, and a G on level-$\ell$ high-pass coefficients costs ℓ levels each way;
  - a linear F: $|F(T^{-1}c) - F(W^{-1}c)| \le L \cdot \|F\|_\infty$ along one line;
  - the wrap at the mirror: $\mathrm{WRAP}(v, b + 1)$ on each rebuilt register, b its forward twin's width.

E5. Measure (M2, M8, M18), per frame.

- Residual: L_n = [1 1]^{∗n} per axis, undivided; B_s ∗ B_b = B_{s+b}; R = 2^{|b|}·L_s u_t − L_{s+b} u_t.
  - ΣB_s = 2^{|s|}, and a constant cancels.
  - The heat identity: 4^m δ − H^m = Σ_{k<m} 4^{m − 1 − k} H^k (4δ − H), with H = [1 2 1] and 4δ − H = −Δ².
  - Parity: 2δ − [1 1] = [1 −1], half a voxel off. An odd b is therefore refused and an odd s is kept with its offset (ruling (a)).
  - The key is 2^g·B_narrow − B_wide·B_narrow, in 9 limbs.
  - Readings of any declared width w (the unit sweep's planes): R leaves in ⌈(w + Σs + Σb + 1)/32⌉ limbs, and a reading ≥ 2^w is refused.
  - The positive set: P(x) = [R(x) > 0].
- The history:
  - f_j(x, W) = Σ_{t∈W} bit_j(u_t(x) ⊕ u_{t − 1}(x)), |W| = 11 transitions, ≤ 16 windows;
  - passes: Σ_W f_j ≡ bit_j(u_0(x) ⊕ u_{F − 1}(x)) (mod 2) at every x and j;
  - the anchor count a_j(x) = Σ_t bit_j(u_t(x)); the anchor bits are ∧_t u_t and ∧_t ¬u_t;
  - the cloud: D_W(x) = Σ_j f_j(x, W)·2^j, and O(W, W′) = Σ_x D_W(x)·D_{W′}(x);
  - the knf section s(x) = (D_W(x))_W less its mean over W; E = Σ_x Σ_{a∈{z,y,x}} ⟨s(x), s(x + e_a)⟩ on the torus; the departure is d·E − Σ_i E(σ_i x).
- The period, per axis of extent n, M = N/n:
  - L = ⌊n/2⌋; the starts are the positions below n − L on the axis, Q = (n − L)·M;
  - a(ℓ) = #{x : u(x) = u(x + ℓ·e)};
  - a candidate p has 2 ≤ p, 2p + 1 ≤ L, a(p) > max(a(p ± 1)) and a(2p) > max(a(2p ± 1));
  - h(p) = min(a(p) − max(a(p ± 1)), a(2p) − max(a(2p ± 1)));
  - the null draw k: each line along the axis permuted by a keyed Fisher–Yates (swap s with h mod (s + 1), 4 keys from the content signum and c = draw·rank + axis), h_k = max_p h;
  - P = min{p : h(p)/Q > max_k h_k/Q}, a 128-bit cross-multiply; P = 0 when none clears.
  - False-period rate ≤ 1/(d + 1).

E6. Partition (M3, M4, A2, A3).

- Admitted: R(x) > 0. The code c(x) = (dense rank of R(x) over the admitted) + 1, and 0 elsewhere.
- Adjacency: 6 (a face joins x and x + e_a, a ∈ {z, y, x}, both admitted).

- The face key is (c of the weaker end, then ¬(3x + a) in the lowest limb). A stronger weaker end wins and then the lower face name. The tree is the maximum spanning forest by these keys (`max_tree_contract`, `max_tree.cu`):
  - Borůvka rounds: each component takes its strongest outgoing face;
  - of a mutual pair the higher root hooks onto the lower;
  - pointer jumping between rounds;
  - the chosen faces are the forest.
- Components at every level from the forest: count(r) = #{x : c(x) ≥ r} − #{forest faces with min(c) ≥ r}, one scan from the top. The cut is r* = min{r : count(r) = max count}. The bodies are the components of {c ≥ r*}, labeled by min-label spreading along the forest. The count found must equal count(r*), else it is a logic error.
- A node n has a level λ(n), and ν(x) is x's own node.
  - The component at level r holding x is the ancestor a of ν(x) with λ(a) ≥ r > λ(parent(a)).
  - A subtree is a DFS range [in(a), out(a)).
  - x and x′ share a component at r exactly when r ≤ λ(LCA(ν(x), ν(x′))).
- Ties: every face's key carries the face's name in its lowest limb, and no two keys tie.
- Moments: μ(n) = Σ_{ν(x)=n} (1, x, x x^T), and each node's are Σ over sub(n). The centered K = m·M − S·S^T is never divided. ℓ ≤ 2 harmonics = (m, S, M).
- Built and proved on the host (A2): the probe partition from k − 1 LCA levels, and J(v)[n, n′] with C_{r,r′} a rectangle sum over two DFS ranges. Nothing calls them yet, and the slide still bisects over levels (`slide_score.cu` :97 and :207 in biohub). Theory and not built: moments by one DFS prefix.

E7. Relate (M5, M6, M7).

- Agreement: A(v) = Σ_x P_t(x)·P_{t′}(x + v), by an NTT mod 998,244,353.
  - Each axis of extent n is padded to the least power of 2 ≥ 2n − 1, and every lag −(n − 1)..(n − 1) is linear, never wrapped.
  - It is refused when |Λ_t| ≥ the prime or the padded total > 2^31 − 1, and the longest padded axis is ≤ 2^23.
  - v* = argmax A, ties to the least Σ_a w_a·v_a^2, then the least index.
- `C_{t,t′}[a, b](v)` = Σ_x [L_t(x) = a]·[P_{t′}(x + v)]·[L_{t′}(x + v) = b]. Its reductions:
  - the argmax over v;
  - the climb (`climb_machine.cu`): S(v′) is the climber's count at lag v′, over v′ ∈ v + {−1, 0, 1}³. It moves to the v′ ≠ v with S(v′) > S(v), the largest, ties to the least w_z·z^2 + y^2 + x^2. It stops when none exceeds S(v), and holds S(v). The landing is a majority vote (one pass picks, one counts) of the leaf under each of the climber's voxels at the lag and along a spiral around it, the most agreed kept;

  - the transpose, `C_{t′,t}[b, a](−v) = C_{t,t′}[a, b](v)`;
  - box sums at nested widths from one summed-area table;
  - the tent;
  - the moments, at v = 0 with weight $\varphi \in \{1, x, x\,x^{T}\}$.
- Marginal:
  - $Z = \Sigma_{\alpha}\, \Pi_{s}\, \omega(s, \alpha(s))$, α injective with ∅ allowed, is the permanent of $[\omega(s, t) \mid \mathrm{diag}\ \omega(s, \varnothing)]$;
  - $P(s \to t) = Z_{s \to t} / Z$, and it factors over the support's components;
  - bounds: $\leq 2^{16}$ arrangements, ≤ 64 targets, 16 limbs.
- Matching: M^{*} = argmax over one-to-one M of $\Sigma_{(a,b) \in M}\, w(a, b)$, in exact integers.
- Body programs (velocity, division, contact side, fingerprint, print pair) are E1 programs over packed body records, ≤ 3 members.
- Null draws on the identity line: frame f stands on floor $\lfloor f/11 \rfloor$ at place f mod 11. Its draw on floor g is 11g + (f mod 11), the floors taken by least O.

E8. The scheduler (M14, A11).

- The quantities: device C, in use $U(\tau)$; job j has d_j, its standing s_j (the bytes its process occupied as it asked: its context and whatever it kept, by pid, or by the process's own report under WSL), r_j (only growing) and $u_j(\tau)$ by pid.
- $O(\tau) = U(\tau) - \Sigma_j\, u_j(\tau)$; $H(\tau) = C - O(\tau) - \Sigma_j \max(r_j, u_j(\tau))$.
- A job wants $w_k = \max(s_k + d_k, p_{\sigma})$ and needs $n_k = w_k - s_k$, since U already counts s_k. The p_{σ} term is committed and unapproved; Doug's rule is $r_k = d_k$.
- Admit k when $n_k \leq H$, with $r_k = w_k$ and $u_k = s_k$ until its first sweep.
- Growth: $u_j > r_j$ sets $r_j = u_j$ and warns. On release, $p_{\sigma} = \max_{\tau} u_j(\tau)$ under the signum σ = BLAKE3(request).
- Asked: $d_k > p_{\sigma}$, the declaration alone. The job is withheld, overridable, and lost after its holding time.
- Backfill with the head kept:
  - the shadow S = least time with $H + \Sigma_{e_j \leq S} \max(r_j, u_j) \geq n_h$, $e_j = \mathrm{start}_j + t_{\sigma j}$;
  - the spare X = that sum $- n_h$;
  - k goes now if $n_k \leq H$ and (now + $t_{\sigma k} \leq S$ or $n_k \leq X$);
  - a signum with no history is never backfilled.
- Every deadline sits in one min-heap on time to change.

- A ticket is (pid, start time).

E9. Errors (M12). Every entry returns a value or refuses with (kind ∈ {request, resource, logic}, module, site, status, execaddr, evacaddr, imagebase, ≤ 16 frames). First write wins. A logic error withholds the result.

E10. The sift, the ladder, memory, render (M11, M15, M16, M17).

- Sift dispatch: in order when 100·total² against 85·distinct·Σcount² says so, ≥ 143 bits. Anchors are 1 when a period P is found, else 4. A periodic corpus filters no lower than 1/P. H_2 is the collision entropy.
- β(n) = #{Fibonacci rungs ≤ n after the first}, 92 rungs below 2^64. LADDER(a, b) reads β of |a|/b.
- The memory arms answer exactly what a byte loop answers; render's host bytes equal the device's.

E11. The ask and the state (M21, A15).

- p_i = Tr(ρE_i), E_i = (I + v_i·σ)/4, which gives p_i = (1 + r·v_i)/4.
- With Σv_i = 0 and Σv_i v_i^T = c·I, r = (4/c)·Σ p_i v_i.
- Crossing: (1 + r_a)/2 = Σ p_i (½ + (2/c)v_{i,a}).
- Valid ⇔ $|r|^2 \le 1$.
- $\mathbb{Q}(\sqrt{3})$: $(a + b\sqrt{3})(c + d\sqrt{3}) = (ac + 3bd) + (ad + bc)\sqrt{3}$, the sign from a^2 against $3b^2$.

E12. The limits and the ordered machine (two_crystals).

- The inverse limit $\mathbb{Z}_2$ = lim← ℤ/2^w; the direct limit of ℤ/2^w under x ↦ 2x is ℤ[1/2]/ℤ, its dual. ℤ is dense in $\mathbb{Z}_2$, and $\mathbb{Z}_2 \setminus \mathbb{Z}$ is uncountable.
- The solenoid $\Sigma_2 = (\mathbb{R} \times \mathbb{Z}_2)/\mathbb{Z}$. The adeles $\mathbb{A}/\mathbb{Q} \cong (\mathbb{R} \times \hat{\mathbb{Z}})/\mathbb{Z}$, $\hat{\mathbb{Z}} = \Pi_p \mathbb{Z}_p$, and $|x|_\infty \cdot \Pi_p |x|_p = 1$.
- Orders: F^k for k ∈ ℤ, F^{−k} = $(F^{-1})^k$. The counter n ↦ n + 1 is a shear.
- ±ω: each bit ← lim sup. On a finite window that is the OR over the cycle, and +ω = −ω.

E13. Programs on the engine (M19).

- The camera law:
  - S = background + ramp·x + wave + Σ bodies' brightness;
  - shot noise, a count in which each of the S expected electrons adds 0, 1, 2 or 4 with chances 9, 8, 6 and 1 in 24, whose first four cumulants are each S, Poisson's (`sim_poisson_four_cumulants`); the symmetric law Binomial(4S, ½) − S is kept beside it. Read noise Binomial($4r^2$, ½) − $2r^2$;

  - value = offset + fixed pattern + gain·S + read;
  - every draw is splitmix64(key ⊕ counter).

- Ω (`chaitin_omega`): Ω = Σ over closed BLC terms t with a normal form of 2^{−|t|}.
  - The bracket through L bits: [Σ_halted 2^{−|t|}, that + Σ_open 2^{−|t|} + the mass past L, directed].
  - Halt: a normal form. Loop: Brent's watcher. Forever: the spine grows. Halts: simply typed.
  - Codes, de Bruijn: ooM is λM, o1MN is MN, 1^ko is variable k. Terms are enumerated by rank: unrank(length, depth, index) over the counts c(n, k), the variable, then the lambda, then applications by the left part's length.
  - On the engine (`chaitin_omega.cu`, the run by the engine): one normal-order step of every live term is one sweep, a term is one record a token, and there is no step or token budget. Per round:
    * phase one, a thread a term: find the leftmost outermost redex (λM)N; a normal form settles;
    * phase two: each reduct token is a plan (depth D, bound k, body, argument) gathered from its source token by the index. The record program computes (λM)N → M[N], with a body variable i > D + 1 ↦ i − 1, an argument variable i > k ↦ i + D, and every other token copied. It is imprinted at the round's widths;
    * phase three: the same decisions as `omega_run`, namely halt at a normal form, loop by Brent's watcher, grows forever by the spine proof;
    * settled terms leave as one record; kept terms move into the next round's rooms;
    * a job (≤ 2^22 terms of one length) is admitted once fewer than 2^21 are live;
    * a term the device cannot hold is parked as outgrown, and a stop leaves the live terms open;
    * the run is one tessera job.

- π (`pi_tower`), BBP at hex position d:
  - term lane i ≤ d, for j ∈ {1, 4, 5, 6} with m = 8i + j: r = 16^{d−i} mod m, by r ← r^16·16^{nibble} mod m one nibble at a time;
  - f_j = ⌊r·2^W/m⌋, and F_i = (4f_1 − 2f_4 − f_5 − f_6) mod 2^W;
  - tail t < T: ⌊2^W/(m·16^t)⌋, the same combination; pairs F ← (F_{2k} + F_{2k+1}) mod 2^W;
  - S = ΣF mod 2^W, error ≤ 4(N + T) + 1 units, and the leading bits where S ± E agree are certified;
  - cost: d + 1 lanes, linear in d, ⌈(d + 1)/70,656⌉ sweeps on the RTX 3070;
  - neither tower is in it.

- Host only: ψ (`ka_psi`, the exponent $\beta(r) = (n^r - 1)/(n - 1)$ kept symbolically) and Goodstein (hereditary trees, ordinals below ε_0).

E14. The floor the coder answers to is its own theory, the compression workbook: compression_table.md, C0 to C13.

E15. The governing functional (noise_sieve_tower). F[U] = Π_4(⊕_{∂Φ_new} LUT(Key(W_local & L↓))):

- L↓ is the program;
- W_local is the atom;
- & applies the key's masks;
- Key is the imprint;
- LUT is the table step;
- ⊕ is the cycle's fold;
- Π_4 is the proof on the product.

E16. The root noise of a box (the noise detector, `noise_root_box`).

- A box B = lo, hi) on each of t, z, y and x: an object's box, which the program names (the cell workbook's). Each term the noise vector table ([noise_vector_integration_table.md) names as shared keeps one value for each place along its kept axes and shares it along the rest. Rows (row 8) share x; columns (9) share y; planes (10) share y and x; the fixed pattern (18) shares t and z; the stack term (25) shares z.
- The pattern is p_τ(k) = ⌊the mean of u over the voxels of B at k on τ's kept axes⌋, exact. The residual is r_τ = u − p_τ, spread back along the shared axes, and u = r_τ + p_τ exactly (`noise_root_return`).
- The price c(v) is the coder's bits for T(v) (E4) on a lattice of ints. Term τ saves g_τ = c(u_B) − c(r_τ) − c(p_τ), signed.
- The root is argmax g_τ over the terms with g_τ > 0, the first on a tie, and none when no term saves. The yank keeps the root's residual and its pattern.
- The price comes in as a callback (`NoiseCost`), since the detector reaches no module but its own; the sim prices through `tower_lift` and `compression_encode`.
- The patterns are estimated here, as floor means over the box. The detector's other passes measure each term's size instead of its pattern.
- **Proved** (`noise_root`, 72 checks, 0 failed, a tessera job). The scene is 16 frames of 16 × 64 × 64 around one standing body of 300 e. The background is 100 e at offset 300, with read variance 4 and the Poisson shot, which gives a voxel an independent variance of 104. The box is all 16 frames × z 2 to 13 × y 16 to 47 × x 16 to 47, 196,608 voxels. Each term is planted alone at variance 64, 256 and 1,024, 16 runs with the null, and the checks hold:

  - the null names no root;
  - in every planted run, the planted term is the root or there is none, never another term;
  - at 1,024 every term is the root;
  - the root's residual and pattern return the box exactly.

- **Measured**, the bits the root saves, per mille of the box's (the box's own bits run 1,194,939 to 1,362,651). At variance 64, planes save 7 and the fixed pattern 5, and rows, columns and the stack term pay for no pattern (their savings are −5,409, −6,638 and −54,497 bits). At 256: rows 31, columns 30, planes 28 and the fixed pattern 24, while the stack term still pays for none (−17,214 bits). At 1,024: rows 89, columns 80, planes 59, the fixed pattern 50 and the stack term 32.
  - The stack term's pattern is the largest, 16 × 32 × 32 values, against 32 × 32 for the fixed pattern and 16 × 12 for the planes. That it pays last for that reason is a reading; the patterns' own bits were not printed.
  - The noise vector table puts the coherent terms on the kcr set at 1.0% to 3.0% of the static background's frame-difference bits (a Gaussian estimate). A real box may hold no root (a reading, still unmeasured).
- **Audited against the code** (`noise_root_extent`, `noise_root_box` and `noise_root_return`, noise_detector.cu). Each term's shared axes, the floor mean with an equal count of voxels in every pattern cell, the residual and its exact return, the signed saving, the root as the first strict gain over 0, and each pattern priced at its own extent all hold as stated. The code also bounds what E16 leaves open. A box holds at most `2^31` voxels, and a price past `2^61` bits is refused. With no root, the residual is the box itself and no pattern is left. `noise_root_return` does not check that its sums fit an int; only the pair `noise_root_box` leaves is exact.
- The engine's own price is `engine_lattice_bits` (`engine.h`): a lattice of ints lifted by `tower_lift` and coded by `compression_encode`, the coder's bits, with the `NoiseCost` signature a driver passes. It is built (the engine build compiles it) and not yet run: nothing calls it until a driver part does.
- Not built: a driver part over the cell workbook's object boxes, and the pattern stored in the crystal beside the residual. The ruling is that e16 is variable and should or into the construction set. Read here as follows; this is a reading, not the ruling's words.
  - An object's box spans a variable number of frames. How a box's span is chosen is not yet written. The boxes the cell program holds are a body's in one frame (the climb machine's extents, which the box history reads).
  - The pattern is stored beside the residual, or stored in the construction set instead. The construction set is found where the noise terms are all members of the photon shot set. Either way the crystal's format changes.

E17. The ask (M24, A18): every reading the engine takes of a part.

- An ask a = (address, qualifier). Unbound it returns its cost t(a); bound by β it returns bit(a) = [qualifier holds]·[t(a) ≤ β], β read off the baseline of unbound costs and never written.
- The gate: S = {c : c(k) = r(k) for every k ∈ K}. The order of K sets the price and never S.
- The order: n links, n + 1 a power of two, ask r covers link c where (r + 1) & (c + 1) has an odd count of ones. (n + 1)·x = 4·S^Tb − 2·(Σb)·1, from 4·S^TS − (n + 1)·J = (n + 1)·I.
- Contention: with the count taken out, P_sy²·(N + 1) > 4·P_yy·P_ss over the N sweep asks, every term an exact integer.
- The rank: m_c = min_j t_c,j and s = max_c (max_j t_c,j − m_c). The least c_1 is chosen where m_c_2 − m_c_1 > s, and otherwise the survivors are one cost.
- A form asked of the part: the position closes a body that answers N where the form takes the way back and 1 where it falls through.

Tied to the code since the first draft: the tree's adjacency and forest (`max_tree.cu`), the Rice coder (`compression.cu`), the climb (`climb_machine.cu`), the refused lane (`cycle_record_kernel.cu`), Ω's engine rounds (`chaitin_omega.cu`) and the NTT's padding (`shift_agreement.cu`). Nothing listed is left untied; an audit against the code is pending.

8.3 THE TABLE

M1. Exact numbers

THE ALGEBRA IT HOLDS TO Every quantity is an exact integer or an exact rational stored as a pair; nothing is rounded and no float is formed.

DOES `OriorExactInteger` (orior's) is the integer, in `src/cu/includes/types/integers/exact_integer.h` and the `exact_integer_*.c` beside it. The engine build compiles it from this tree by default (`cell_tracking/build_engine.sh` and `build_driver.sh`; `ORIOR_EXACT_ROOT` still overrides), and this tree is the source orior's engine is to be copied from (orior). The header carries their full doc text. The width is any power of two from one limb (32 bits) up, with no ceiling, named in limbs (`ORIOR_EXACT_LIMBS`) or in bits (`ORIOR_EXACT_BITS`). The default is 128 limbs, 4,096 bits. Static asserts refuse a width that is not a power of two, is below 32 bits, or where the two names disagree. The declared floor `ORIOR_EXACT_DIGITS` stays 1,024 digits. A width below 4,096 bits declares its own, and one that does not is refused at compile time. Width-sized working copies sit on the stack up to `ORIOR_EXACT_STACK_LIMBS` (4,096 limbs) and come from the heap past it. A copy the heap cannot hold returns `ORIOR_EXACT_WILL_NOT_FIT`, and no stack limits a width. Multiplication is a ladder: long multiplication, then Karatsuba from 32 limbs (`ORIOR_EXACT_KARATSUBA_LIMBS`), then the Schönhage–Strassen transform once the shorter factor reaches 8,192 limbs (`ORIOR_EXACT_TRANSFORM_LIMBS`). The transform works modulo 2^n + 1, where every twiddle is a shift, and nothing is rounded. An integer cost model picks each split's depth, and a rung whose workspace cannot be allocated steps down to the one below. `orior_exact_divide` is Knuth's algorithm D in base 2^32.

Once the divisor and the quotient both reach 8,192 limbs (`ORIOR_EXACT_NEWTON_LIMBS`), it switches to Newton's reciprocal: the reciprocal is grown at half precision recursively, taken one Newton step and corrected exactly, and the quotient is then one product on the ladder. Either way the quotient rounds toward zero and the remainder carries the numerator's sign: numerator = quotient · divisor + remainder. `orior_exact_divide_exact` is for a division known to leave nothing. The divisor's twos shift out, and the rest multiplies by the inverse of the odd part modulo 2^(32 · limbs), then is masked. The inverse comes from Newton's step x(2 − dx), 2 − t being t's two's complement plus two, and each step works only to the precision it doubles to. The quotient is multiplied back, and a remainder is refused as `ORIOR_EXACT_NOT_EXACT`. `orior_exact_gcd` is Lehmer's (Knuth's algorithm L); a bit-at-a-time binary gcd stalls the 4,194,304-bit test. A zero divisor returns `ORIOR_EXACT_BY_ZERO`. `orior_exact_multiply_transform` and `orior_exact_divide_newton` run their rungs at any size, for tests and timing. The sims' rationals (`sim_rational.h`) use the gcd and the exact division to stay in lowest terms at any width. The key machine divides too, as four record operations (M10). `decimal_double` expands a decimal string, digits × 10^ex, into the exact leading bits of its binary value in the candidate's full width, with its binary exponent. It says whether the value fits and terminates, and how many bits it needs when it does not (A0).

WANTS Every decimal the machine is handed (a spacing, a position, a cosine) enters through `decimal_double`, and geometry is exact.

TRIED, AND WHAT IT GAVE **measured** (the scratchpad `decimal_double_test`, a checker in exact arithmetic): 0 failures. 1e2200 needs 5,109 bits and reports fits = 0 at 4,096; the machine says "grow this" and does not round. **Proved**, the ladder and Newton's division (`test/engine/arithmetic/no_rounding/exact_transform_test`, 4 checks, 0 failed at each of 128 limbs, 4,096 limbs and 4,194,304 bits). The ladder and the transform alone each equal a long multiplication kept in the test, on 24, 54 and 63 products: balanced and unbalanced, keyed and all ones, across every rung boundary up to half the width. Each product divides back to its factors, exactly and with remainder, and the gcd holds a factor. Newton's division rebuilds its numerator, with the remainder below the divisor and signed as the numerator, and equals the dispatched division on 24, 54 and 63 divisions with divisors up to half the width. **Measured**, the rungs' timings (the ledger's "The exact integer: width, ladder and division"). **Proved**, the width's guards: 1, 8 and 64 limbs compile with declared floors (9, 76 and 616 digits), and 8 limbs with no floor is refused at compile time. **Proved**, the division on its own (`test/engine/arithmetic/no_rounding/exact_divide_test`, 9 checks, 0 failed, at 128 limbs, 4,096 bits). Long division rebuilds its numerator, with the remainder below the divisor and the signs of truncation, on 200,000 trials of edge-shaped limbs up to 8 wide and on 4,000 trials at widths drawn up to 127 limbs. It also holds on a case built to run the add-back step (an instrumented copy confirmed the branch runs). Exact division returns the quotient of every product on 20,000 trials, odd and even divisors, and refuses one past each product. The gcd divides both values, holds their shared factor and leaves coprime cofactors on 4,000 trials, and it equals Euclid's gcd run on the long division on 4,000 more; gcd(0, x) = |x| and gcd(0, 0) = 0. A zero divisor is refused. With lowest terms, `ka_psi` is still 152 of 152 and `ask_state` 33 of 33 on the merged source.

STATUS built; the engine build takes it from this tree

NEXT Read the DICOM spacings and positions through it (M9). orior's engine copied from this tree (item 7 below).

M2. The residual operator

THE ALGEBRA IT HOLDS TO R = 2^{|b|} L_s − L_{s+b} over one binomial ladder L_n = [1 1]^n ∗ I, separable per axis (A1). The kernel sums to zero exactly: a constant added to I leaves R unchanged.

DOES `residual_program` builds the four steps; `keymath` imprints one key, `key_schedule` lays it out, `cycle` runs it in one launch. `unit_sweep` runs the same as centered [1 2 1] steps per axis in shared memory, working only the live limbs. `engine_residual` runs either or both; both proved compares every lane, and **a disagreement is a logic error**: the result is withheld. The orders come in with the request; every step is error-wired (M12). The comb (`ENGINE_COMB`, `comb` on the request) takes a running sum of n voxels along an axis on both terms before the smooth: it sends a period of n along that axis to a constant, with every harmonic of it, and the kernel still sums to zero. An even n moves both terms half a voxel, as an odd smooth order does, and the place is set by s + n − 1. The unit sweep also takes its input as limb planes of a declared width (`device_planes`, `input_bits`). A value past that width is refused as a request error, and the output's limbs follow from the width and the orders. `engine_residual_planes` hands it host planes and returns the residual with its limbs. The 16-bit volume path and the key are unchanged. A spaced pair (`ENGINE_SPACED`, `smooth_spaced` and `background_spaced` on the request) is [1 2 1] with its taps 2^j voxels apart: it is symmetric about the voxel, moves no center, adds 2 bits and adds the variance of an order 2 · 4^j. A smooth of wide variance is laid as pairs at growing spacings, and its bits grow with the count of its pairs and not with its order.

WANTS Orders from the data, per request, with nothing rounded: a per-axis coherence reading gives an exact integer period P, and P sets the comb on that axis. On the RSNA knee P follows the scanner's reconstruction (34 series read against their Acquisition Matrix, 0018,1310), and the comb takes that trace away. The background's size comes in mm from each series' own spacing, steered by truthy verdicts. Parity by the algebra instead of a blanket guard: an odd background order misaligns the two terms by half a voxel and is refused; an odd smooth order shifts both terms by the same half voxel and is kept with that offset recorded (Doug's ruling (a)). Every (s, b) from one sweep that keeps its ladder levels (A1).

TRIED, AND WHAT IT GAVE The key against the step-by-step residual: **proved**, 0 of 10,485,760,000 lanes differ; 31 ms a frame against 24.5 for the passes (**measured**). The unit sweeps against the key: **proved**, 0 lanes differ, and about half the key's time a frame (**measured**; numbers in the cell workbook's on_the_engine.md, since they were taken on a program's samples). The planes (`test/engine/analysis/unit_sweep/unit_sweep_planes_test`, a tessera job): **proved**, 336 checks, 0 failed, against the 16-bit volume path by linearity. The readings are built from 16-bit parts at chosen shifts, at 16, 34, 64 and 116 bits in. On every lane their residual equals the parts' residuals shifted and summed. That

is 4,357 lanes a case, 4,356 of them not zero, over three sets of orders, {2,34,34}/{6,96,96} at 10 and 13 limbs among them. Sixteen-bit planes equal the volume path. A constant added moves no lane. A bit past the width refuses, and the sweep gives the same lanes after it. Eight malformed requests refuse. The comb (`test/engine/analysis/residual/residual_odd_test`, a tessera job): **proved**, 325 checks, 0 failed. On odd and even combs, n of 0 and 1, and n past the extent, the key and the unit sweeps equal the residual's definition on every lane, 685 lanes a set. A period of 4 and one of 5 planted along 41 voxels go to zero on every lane whose window lies inside the line, on the key and the unit sweeps alike, and 37 of those lanes are not zero without the comb. The spaced pairs (the same test): **proved**, 325 checks, 0 failed. On pairs on the smooth and on the background, more than one at a spacing, and spacings of 8 and 16 on axes of 3 to 11 voxels, the key and the unit sweeps equal the residual's definition on every lane.

STATUS proved (exact); the sweeps 2× faster; any input width proved (the planes)

NEXT Ruling (a) at all three guards (`residual.cu`, `keymath.cu`, `unit_sweep.cu`, and `narrow_bits` at :259), with the half-voxel offset as a field of the result. Taking the orders from the per-axis coherence reading, now built as M18 (`period_read`, A12); the knee pulls it. The ladder-keeping sweep.

M3. The component tree

THE ALGEBRA IT HOLDS TO The dense level code, the components of every upper level set, nested as a tree. Every question about levels is a question about tree nodes: membership at level r is an ancestor query, two voxels share a component at r exactly when $r \leq$ level(LCA), and every subtree is one range of the DFS order (A2).

DOES `max_tree_objects`: the level code, the components at one cut (the level with the most, proved against the forest's own count), the per-component moments (M4). `max_tree_slide`: the caller's probe voxels, and each recorded level's probe partition, proved against the tree's count at that level; a disagreement is a logic error. `max_tree_overlap`: two frames' trees at one residual level each, the caller's probes and probe links, and the census of the bipartite support at the lag: one partner, unpaired, mutual, forked, per side. `max_tree_keep_frames` holds a frame's code for the next. No cell is known to it: probes are points, links are point pairs.

WANTS Every level at once instead of one cut or sampled cuts: the probe partition at every level from $k - 1$ LCA levels, the census at every level from one own-node pair table summed over DFS ranges (A2).

TRIED, AND WHAT IT GAVE The slide's counts: **proved**, 0 of 26,454 differ from the tree. The overlap and census on 6bba_48816121 and 44b6_0113de3b: **measured** (numbers in the cell table, row 1, since they are graded by a program's key).

STATUS built, error-wired

NEXT The LCA and DFS-range forms (A2), which remove the bisection and the per-level recount.

M4. The moments

THE ALGEBRA IT HOLDS TO Per component, exact: the mass, the three index sums, the six second moments. Together these are the harmonics up to $\ell = 2$ ($\ell = 0$ the mass, $\ell = 1$ the sums, $\ell = 2$ the moments). Every node's moments are its subtree's own-node moments summed (A3).

DOES `max_tree.cu` accumulates them on the device for the components at the cut; `grow_leaves` carries them out (`ENGINE_BODY_WORDS` 11: peak, mass, 3 sums, 6 moments).

WANTS Every node's moments at every level, by a prefix over the DFS order (A3).

TRIED, AND WHAT IT GAVE Carried into every record program and the viewer; proofs are the programs' (device equal to host).

STATUS built, error-wired

NEXT A3's prefix, which a level-free program (the slide's chains) needs.

M5. The correlation operator C

THE ALGEBRA IT HOLDS TO `C_{t,t′}[a, b](v)` = Σ_x [L_t(x) = a] · [P_{t′}(x + v)] · [L_{t′}(x + v) = b]: one count of labeled positive voxels under a shift, read by different reductions (A4).

DOES `shift_agreement`: the argmax of the unlabeled sum over every v, by an NTT mod 998,244,353, exact because every count is below the prime. `climb_machine`: a local argmax per label from a start lag, 26 neighbors a step; `--box` reads C at the 27 shifts around every climbed lag; `--core` the nested widths; extents per label. `body_overlap`: C at one lag.

WANTS One C per frame pair over a window of shifts, then only reads: sums, local argmax, transpose, box sums by one summed-area table (A4). The window is the only open design choice, and it must come from the request instead of a scale. The search reads against the integrated noise floor (M8): a count at a shift is weighed by how far it departs from what the noise functionals predict there. The climb and the null draws then spend work only where the field departs from the floor.

TRIED, AND WHAT IT GAVE The box (`--box`): **proved**. C is read at the 27 shifts around every climbed lag and at the drift, for every forward pair and every null draw. A cell meeting more labels than the kernel's table writes its raw pieces and the host merges them, with no cap. At the climbed lag, C sums to the climb's retained count on every climber; the climbed lag is the highest of its 27 neighbors on every climber; at the drift, C equals `body_overlap` on every label (numbers in the cell workbook's on_the_engine.md, since they were taken on a program's sample). The core per width: **proved** at width 0 against C at the climbed lag, 35,755 of 35,755 (numbers in the cell table, row 13).

STATUS built; the box proved

NEXT Make C the one pass, with the window from the request.

M6. The exact marginal

THE ALGEBRA IT HOLDS TO Z = the sum over injective arrangements of sources onto targets (or a null each) of the product of their integer weights: a permanent. It factors exactly over the connected components of the bipartite support graph: a source's probability over its component is its probability over everything (A5).

DOES `marginal`: Z and Z_{b→·} for each source over a local set, device equal to the host, compared by cross-multiplying. The weights are the caller's.

WANTS The set taken as the source's whole support component, and that makes the probability the global one exactly; the component's size reported instead of truncated.

TRIED, AND WHAT IT GAVE Timings and outcomes are the program's (cell table, row 7).

STATUS built

NEXT The component as the set (A5); the limits below become measured costs.

M7. One-to-one matching

THE ALGEBRA IT HOLDS TO The heaviest one-to-one set on exact integer costs.

DOES `heaviest_matching`.

WANTS none

TRIED, AND WHAT IT GAVE Used by the cell program's `--print-match` (cell table, row 7).

STATUS built

NEXT none

M8. The entropy history

THE ALGEBRA IT HOLDS TO Per voxel and bit, the flips in each window of transitions; the parity of a bit's total flips equals that bit of the net change, at every voxel (A7). The window-overlap cloud folds into the same pass.

DOES `entropy_history`: the flips, the parity proof, the cloud, one pass on the device, error-wired (broken parity is a logic error). A sample of fewer than two frames is a request error (no transition to count).

WANTS The window from the request or the data, not the machine (see the audit). The noise categories as functionals (the plan in compression_table.md): fixed pattern from the bits that never flip, shot and read from the photon transfer curve ($\Sigma\ (I_t - I_{t-1})^2$ against $\Sigma\ I_t$ per signal level, exact sums), quantization as one step of the lane, and the spatially correlated terms per row, column and place. Integrated, they give every voxel's floor and so every departure from it.

TRIED, AND WHAT IT GAVE **proved** exact on 2,000 voxels × 9 windows. Measurements of what it shows are the cell program's (cell table, row 12). Its identity by spatial null permutation (`knf_identity`, M19; A13, the two crystals): **proved**, the host's walk equals the engine's history at all 262,144 voxels of a 64^3 lattice over 16 windows, and of 8,192 single flipped bits 1,500 leave it unchanged, each where the window rule says: the history is many-to-one; the 48 exact cube motions carry it and keep its cloud.

STATUS built, error-wired

NEXT Take the window as a request field. The pairwise test between sections: one body's departure curve against another's (A13, the two crystals).

M9. The files

THE ALGEBRA IT HOLDS TO Every file the machine writes is proved by reading it back: pixels voxel for voxel against a second read of the source, and every part by its seal (M13; the crystal's seal, the others' CRC-64 until they are sealed too). Integer lifting is exactly invertible (A8).

DOES **`.kcr`**, the crystal (the Kolmogorov crystal; `.iapx` crystals are not read, sets are re-ingested). Layout, read front to back: head (12 words), the seal (6 roots, every lane node, every chunk leaf), offsets and stream, the deflated side bytes, the member tables and names, EOF. **dicom** is a source kind: one sample is one SeriesUID, slices ordered by an exact key from their positions (M1), signed pixels sign-extended and lifted by `2^15` into the u16 lane (a slice whose raw words differ keeps its pixel bytes), every header kept whole in the side bytes (deflated by our own LZ77 and Huffman coder). The tower (`tower_lift`, integer lifting to the floors, of the lanes or of a lattice of ints, E4), `compression` (a Rice coder per block), `krep` (the container that writes and reads the `.kcr`), and the rebuild proof on every ingest. Their device buffers are kept as pools (`device_pool`). Each pool is one allocation whose slices are laid at 256-byte steps, and the pool is rounded up to the 2 MiB page once. The tower's pool holds its coefficients, scratch, flag and mismatch count, and its edge table grows apart. Compression holds two pools: its chunk bits, chunk offsets and scan scratch, and its stream. `tower_hold_bytes` and `compression_hold_bytes` give a job those bytes before any device work. The stream is sized by the values once they are measured, and it is left to the kept peak. `.oapx`: the entropy history. `.bapx`: the bodies' table. `flatten`: the set as one magnitude vector per label in one file. The ingest readers: zarr v2/v3/N5 with every codec, TIFF, HDF5, npy/npz, NRRD, NIfTI, `.stack`. **zip**: its own module (`src/cu/includes/codecs/zip`), the directory read whole with no size cap, members read and CRC-32 checked, an index by containing folder, and one archive kept open across samples. `npy` reads through it. Every part is error-wired: an IO failure carries errno, bad contents in the machine's own files are logic errors, a malformed source is a request error.

WANTS The export: the crystal written back out as any source format (the dicom zip byte for byte from the side bytes, npy, NIfTI, TIFF, zarr). Names for the other files (flattened, history, bodies, key) are open with Doug. The floor: the residue coded against the integrated noise functionals (M8) over exact counts. Exact arithmetic coding ends within 2 bits a stream of

the length those functionals give, and enumerative coding in the engine's exact integers hits it exactly; a fixed-width range coder rounds each interval and is only the fast approximation. That length is the most any coder can take once the functionals are the real categories, with the model's own bits counted as a two-part code (F5 and F3′ in compression_table.md). It replaces the Rice coder's chosen block, k and escape.

TRIED, AND WHAT IT GAVE `.kcr` on the RSNA knee: **proved**, 1 × 34 × 960 × 960 unsigned, 25,218,496 bytes (40.2% of raw) sealed `7ac2cb89…12eb2a`; 1 × 24 × 640 × 640 signed, 11,319,416 bytes (57.5%) sealed `3e70b8cb…8d9966`; rebuilt voxel for voxel, pixel for pixel and node for node. The RSNA test_series: 15 of 15 passed, 192,020,272 of 599,191,552 bytes (32.0%), set root `98b25a42…d7af979`, prove 696 ms with every seal node confirmed. Deflate: within about 1% of zlib-9 on DICOM headers, round-tripped through our inflate and zlib. A missing sample reports module 10, site 178, status 2 (errno ENOENT). `flatten`: **measured**, slowed by its residual slab sized to half the free device memory, with the same bytes out one frame at a time (numbers in the cell workbook's on_the_engine.md, since they were taken on a program's sample). Codec variants: the residue table in the compression workbook's ledger. The pools (`test/engine/runtime/device_pool/device_pool_test`, 28 checks, 0 failed): the tower's four buffers for 1,100,000 voxels cost 10,485,760 bytes as one pool, the plan to the byte, and 14,680,064 as four allocations, read through tessera's counter (**measured**). Compression's two pools raise `root_universal`'s peak from 154,304,512 to 158,498,816 bytes, two pages (**measured**). That each pool takes a page its small buffers would otherwise share is a reading and is not measured.

STATUS proved: the field, and the RSNA knee as `.kcr`

NEXT The floor: F2 per floor on 44b6_0113de3b against the Rice coder (compression_table.md). The 25 are re-ingested as `.kcr` (item 37 of the cell workbook's build_plan.md). The prose and identifiers still say iapx (`engine_iapx_head`, `entry_iapx_encode`, the cell README) where the file is `.kcr` (`ENTRY_CRYSTAL_SUFFIX`, engine.cu). The export. The engine's opens need a long-path form (a set path past 260 characters fails). Flatten: per-request orders per sample (ruled), and a slab not sized as a fraction of memory.

M10. The record machine

THE ALGEBRA IT HOLDS TO A record program is steps over limbs, run for every record in one sweep; truthy verdicts are fields (zero false, nonzero true; AND a product, OR a sum). A step may read a lookup table: a general one-variable function of one register's low bits. The machine's edges need not be linear (A13). The step count is the scheduler's n, stored apart from the register file, the binding resource (A13).

DOES `keymath` imprints record programs, `key_schedule` lays them out, `cycle` runs them on the device (with a host reference); every entry is error-wired. The operations are field, constant, product, sum, difference, ladder, absolute, compare, signed field, and table (`ENGINE_RECORD_TABLE`, op 10), threaded through the config, `keymath`, `key_schedule` and `cycle`, on the device and the host port. A table maps the low `index_bits` (at most 32) of one register to an `out_bits` value, laid out as 2^index_bits rows of ⌈out_bits/32⌉ limbs. The exact integer's division

is four operations (M1): quotient (`ENGINE_RECORD_QUOTIENT`, op 11), remainder (12), gcd (13) and exact quotient (14). The quotient rounds toward zero and the remainder carries the numerator's sign. The gcd is never negative. A zero divisor, or an exact quotient that leaves a remainder, refuses the lane. The widths are read at imprint: a quotient is as wide as its numerator, a remainder as the narrower operand and a gcd as the wider. On the device, Knuth's algorithm D divides, the gcd is Euclid's on it and the exact quotient is the multiply and mask. Only a program that divides carries the scratch, 5 · WIDE + 4 limbs a lane. The host's steps are the exact integer library itself. Xor (`ENGINE_RECORD_XOR`, op 15), and (16) and the two's complement wrap (17, to 4 bits or more), bitwise on each operand's two's complement sign-extended without end. The step count has no bound: each sign sits beside its register, and `ENGINE_RECORD_LIMBS_MOST` = 256 limbs of registers is the only bound, and floors of steps stack into one program. Register reuse (`reuse` on the key-schedule and record requests, off by default) frees a register once its last reader has run, and a chain needs only its live registers; off, every step keeps its own register. A sweep whose frame grows the device's stack limit sets it back once it ends (`cycle_stack_return`), and that returns the local memory the limit reserves.

WANTS Limits measured, not written in (see the audit). The table's functions chosen without fitting (item 3 of kolmogorov_arnold.md).

TRIED, AND WHAT IT GAVE Device equal to host on every program run (the programs' proofs). The table (`test/engine/compiler/cycle/record_table_test_compose.cu`, run by `test/engine/compiler/cycle/record_table_test.sh`): **proved**, 15 checks, 0 failed, every table filled by running an ordinary-ops program over the whole 16-bit alphabet, and the ops are the oracle. x^2 (product) and $|x - 30000| + 100$ (constant, difference, absolute, sum): the table equals the ops on 4,096 lanes, device, host and table alike. Composition, f(x) = 65535 − x and g(y) = |y − 30000|: a table read through a table equals the ops for |35535 − x|, a single table composed on the host (g read through f) equals the two, and the reversed order f(g(x)) differs on some lane: the chain keeps its order. Register reuse: a 21-step additive chain gives the same output from 3 limbs as from 21. Two refusals: an index wider than its source register, and a table index with no table behind it. The division (`test/engine/compiler/cycle/record_divide_test`): **proved**, 15 checks, 0 failed. The device equals the host word for word on 4,096 lanes of signed 160-bit numerators in the 64-limb register file and on 512 lanes of 2,048-bit numerators in the 256-limb file. On every lane numerator = quotient · divisor + remainder, the remainder is below the divisor and signed as the numerator, the gcd divides both, and the exact quotient returns the factor it was built from. A zero divisor and an inexact division refuse the lane, host and device. 3^40 divides exactly by 3^20 to 3^20. A quotient reading a later step is refused at imprint. The stack return (**measured** on the RTX 3070): a 7,600-byte frame holds 467,668,992 bytes until the limit is set back. `test/engine/compiler/cycle/record_tower_test` reads the limit at 1,024 bytes a thread before its blocks and after them (210 checks, 0 failed); the peak is unchanged, since the reservation stands during the sweep.

STATUS built, error-wired; the table, register reuse and the division proved (A13); the programs compiled and measured, 12 to 17 times the interpreter's speed (item 9)

NEXT `cycle_record_run_host` (cycle.c) is not error-wired yet. The stack return is dropped once the compiled programs, the automata, are emitted (item 9). They are emitted for every program of 1,024 steps or fewer, the divisions included; the interpreter still runs the longer ones, and the stack return stays for it until they compile too.

M11. The golden ladder

THE ALGEBRA IT HOLDS TO β(n): n's rung among the Fibonacci numbers that fit the word.

DOES `golden_bands`, `band_of`, 92 rungs (every Fibonacci number below 2^{64}: a word's width, not a scale).

WANTS none

TRIED, AND WHAT IT GAVE none

STATUS built

NEXT none

M12. Errors and integrity

THE ALGEBRA IT HOLDS TO The only acceptable error is a stateful one: request, resource (a failed CUDA call, allocation or read, with its status or errno) and logic (unforgivable; the result is withheld). First write wins; each request carries its record and the engine keeps its own. The record holds kind, module, site, status, execaddr, evacaddr, imagebase and up to 16 return frames, and the builds emit a `.pdb`, and execaddr − imagebase steps to the line.

DOES Wired: the engine's own entries, max_tree, flatten, decimal_double, unit_sweep, cycle, keymath, key_schedule, residual, grow, krep, compression, tower, entropy_history, zip, npy, dicom, obsignatio (modules 0 to 17). Wired: cycle's host record run. `CycleRecordHostRequest` (cycle.h) gains a last member, `EngineError *error`, and `cycle_record_run_host` (cycle.c, `CYCLE_HELD`, module 5) refuses a NULL request or error record outright. It fills a request error for a layout or out that is NULL; zero steps, out limbs or members; a member's in, bodies or count missing; a step wider than `ORIOR_EXACT_LIMBS`; a lane whose index runs past its member's records; a step the host arm refuses (`cycle_host_step`); or a result that does not fit its out limbs (`cycle_host_fits`). It fills a resource error when its working allocation fails. `engine_record_host` passes its request's error record through and maps `CYCLE_REFUSED` to `ENGINE_REFUSED`. Output: no print in any loop; each functional fills its result struct and one print format renders it on demand through `scriptura` (our own formatting and SWAR memory scans, its memory operations in M17); errors always print. The run log's default place is beside the program (`engine_program_directory`): `build/logs/track_driver.log` for the published driver, whatever folder it runs from; `--log` still names any other. A failed ingest names its stage: no place, source unread, crystal unwritten, or not rebuilt.

WANTS Every module wired: stack, schedule, golden_bands, the other ingest readers and codecs, run_cfg, run_log, cfg_json, relate_frames, double_fields. Three of these are not a mechanical wiring: golden_bands and double_fields have no refusal path at all, and schedule skips where the others would refuse. Wiring any of the three changes what it does, and each is Doug's call. Integrity is M13; device scheduling is M14.

TRIED, AND WHAT IT GAVE Every wiring build checked against a program's run, 0 lanes differ in both modes (numbers in the cell workbook's on_the_engine.md). A series the archive does not hold is refused as no such member (the entry layer asks the archive; a folder search that finds nothing does not count as found); a failed ingest removes the directories it made (the maker counts what it created; it has no walk bound); a failed prove prints no set root; a structurally refused crystal prints as not read; every path buffer is sized by the platform's own limit (`ENGINE_PATH_ROOM`: 32,768 on Windows with a long-path-aware manifest, `PATH_MAX` elsewhere), and seven sites refuse a long path and do not truncate it silently; the npy and nrrd headers have no caps; the host exact integer is fitted to the record path's widest step (256 limbs), and host and device accept the same steps. A 395-character crystal path ingests and proves to the same root. The host record run: eight record tests, each a tessera job: gaussian 11 checks, 0 failed; guide 12, 0 failed; table 16, 0 failed; divide 20, 0 failed; bitwise 25, 0 failed; boundary 49, 0 failed; coherence 15, 0 failed; order 18, 0 failed. The guide test's check: an index past its member's records refuses the sweep, and the error is a request error from module cycle. `record_tower_test`, a tessera job at below-normal priority: 210 checks, 0 failed.

STATUS wiring in progress

NEXT The next unwired module.

M13. The seal

THE ALGEBRA IT HOLDS TO Every file, set and state is sealed by a dimensional Merkle DAG of keyed BLAKE3 nodes, one derived key per level, 32 bytes a node, every node stored. A change anywhere changes exactly its path of nodes up to the root, and a mismatch names its place (A10; the theory is obsignatio_seal.md).

DOES `obsignatio` (module 17), our own BLAKE3, host and device sharing one compression core: `obsignatio_signum` (any length, extended output), `obsignatio_many` (equal messages at a stride, level-parallel), `obsignatio_bits` (bit ranges of a limb stream), `obsignatio_lanes` (rows → planes → volumes → lanes), and the level keys. The `.kcr` stores its six roots, every lane node and every chunk leaf. Ingest seals from the source lanes and proves the file against them. Prove and load check in order: stored bytes, then the inflated side, then the decoded rows, then the root. The set root seals every sample's root in the order named. The tower's CRC is gone. The seal sits after the head: both placements measure alike on the knee (the same bytes, prove median 76 ms each), and the seal exists before the stream is written. Ingest witnesses twice (keep both): the decoded voxels against the source on the device, then a second, independent read of the source pixel for pixel. A flip in the first read is then not sealed as the truth.

WANTS Stage B: the history, bodies, flattened, key and schedule files sealed the same way. The universal root over every file the engine touches plus the engine itself (its hash injected at compile time), and the trust anchor that makes it tamper-proof and not only accident-proof (a signature or a published root).

TRIED, AND WHAT IT GAVE The suite (`obsignatio/test`, the official BLAKE3 vectors): **proved**, 2,182,450 cases, 0 failures (14 levels). That includes every length 0 to 102,400 host against device, every single-bit flip of every vector input, lane locality 9,234 of 9,234 and bit locality 57,216 of 57,216. Five planted bugs (a dropped byte, a wrong merge, a lost carry, no root flag, a stuck counter) each fail it. On a sealed knee crystal: 8 single-bit tamperings each refused and placed (head shape, a root, row t 0 z 8 y 0, a plane node, chunk 1,200, chunk 1,234 from a stream limb, the deflated side, a member name), the clean copy proved. Cost: 5.4% of the crystal, 2.1% of raw at 960 columns (16/X of raw from the rows).

STATUS built and proved: the crystal and the set

NEXT Stage B, then the universal root.

M14. The scheduler

THE ALGEBRA IT HOLDS TO Jobs from separate processes share one device by memory: admit while the declaration fits the measured headroom, grow and warn when a job outgrows its reservation, keep each request signum's peak (A11; the theory is tessera_scheduler.md).

DOES `tessera`, next column, arbitrates between processes: the driver and the device sims submit through it. schedule.cu still plans one process against free/3·2, a chosen share.

WANTS `tessera`: a daemon per host and device over a local socket (named pipe, Unix socket, systemd socket activation, a docker socket mount); per-pid measurement (NVML on Linux, the PDH GPU Process Memory counter under WDDM, where NVML reports none); dead sweeps and lost and found; over budget asked at admission against the signum's last peak, withheld, overridable; the budget and every time a request field; the driver submitting through it; service files; a test suite.

TRIED, AND WHAT IT GAVE Measured on this machine: the PDH counter lists each process's dedicated device bytes by pid. NVML per-process memory is unavailable under WDDM by NVIDIA's documentation. The ledger (`tessera_ledger`: the accounting, backfill with the head kept, one min-heap of deadlines): **proved**, 143,808 cases, 0 failed, from `engine/daemon/test`. The frame (`tessera_frame.c`, 128 little-endian bytes): **proved**, 100,000 random round trips byte for byte, the layout at fixed places, and 131 single-bit flips of the head, each refused or withheld exactly as the rule says. The measure (`tessera_measure.c`: NVML loaded at run time, the PDH counter matched on pid and LUID, a process referenced by a handle or pidfd): **measured** on the RTX 3070 under WDDM, 8 of 8 cases. This process read 24,576 bytes, then 412,254,208 after allocating 268,435,456 (the rest is its CUDA context); the device's in-use rose by the same 412,070,912; a pid with no device work read 0; a child referenced before its exit read live, then ended. NVML's total is 655,360 bytes above what the runtime reports (WDDM keeps them), and C is NVML's figure. The client (`tessera_client.`

c: connect, spawn the daemon if absent, submit, override, wait, precalc saved, release) and the daemon (`tessera_daemon.c`, linked with the ledger, the measure, the paths, the frame, scriptura and obsignatio) compile clean at /W4. The job test (`tessera_job_test`, the client against the built daemon, which the client starts through `daemon_path`): **proved** on the RTX 3070 under WDDM, 24 checks, 0 failed. Its first four paths, in order. A new signum declaring 64 MiB is admitted with 64 MiB granted; it takes 256 MiB, and its release reports it grew to 412,254,208 bytes, with a peak of 412,254,208, the process's whole dedicated memory, its CUDA context included. The same signum declaring four times that peak is asked, the ask naming the kept peak, and the override admits it on its declaration. Declaring exactly the kept peak is admitted at once. Declaring four times the peak with no answer is withheld for its 0.4 s holding time, then lost; its ticket names a lost and found directory, and the precalc saved releases it. The fifth part runs in a scratch `TESSERA_STATE` inside the run's build directory. The lost ticket's seal holds, with "precalc saved" as its body's last line. The daemon ends once idle, and the history it leaves is whole records and the seal (80 bytes here: one record and the 32-byte seal). A byte flipped, a byte cut and the seal stripped are each refused: the submit fails, and no daemon starts. The history restored, the same signum at four times the peak is asked with the kept peak, which proves the history was read. The daemon seals the history and the tickets with `obsignatio_seal` (keyed BLAKE3, on the host), and it makes no CUDA call and holds no device context (tessera_scheduler.md). The daemon the last check starts ended itself within 5 s of the test. The history is loaded before any endpoint exists. A refused history ends the daemon before a client can reach it. A program's driver submits through it (its jobs and the sample they run on are in the cell workbook's on_the_engine.md). **Measured** there: two proves of one crystal in a row, each granted its declaration of 838,860,800 bytes, peaked at 3,958,566,912 and 3,962,761,216. A second run on the real state gave the same declarations, and the two prove peaks in the other order: identical requests measured 4,194,304 bytes (2^22) apart. The second prove was admitted on its declaration, below its kept peak, as A11 says. Until its sweeps grew it, its reservation stood 3,119,706,112 bytes below what its signum is known to need, and another job could have been admitted into that room. The rule reserves what a job asks for, and if it costs less, remembers that for next time. A job that takes more than it declared is grown and warned by the sweeps, as A11 says. The code reserves the larger of the declaration and the kept peak (`tessera_ledger_wants`), not yet ruled on. Its ledger suite passes the budget's 22 checks, 0 failed, and with it two proves each reserve 3,962,761,216 bytes, the kept peak. **The standing** (A11). Each job declares on top of what its process already holds. Its standing is read as it asks, and it is admitted on the bytes it has yet to find. The ledger suite checks this on a device of 1,000 bytes with 300 in use (**proved**, 8 standing cases, 0 failed). A job standing on 200 and declaring 650 wants 850, more than the 700 free, but has only 650 to find. It is admitted with 850 reserved and 50 left. Its peak of 900 is kept, and the same signum standing on 300 and declaring 650 is not withheld and wants 950. Declaring 950 is withheld. The job test ran 25 checks, 0 failed, standing on 24,576 bytes before any context and on the 412,254,208 it kept. Clients make their context before they ask: `sim_job_submit` does, and the driver does too. For its largest sample the driver declares two parts. The first is the lattice rounded to the page, where the part holds one (all but iapx-prove). The second is the tower's pool and compression's chunk pool, where the part lifts or lowers one (all but schedule and

fingerprint). The stream and seal buffers are left to the kept peak. The driver's declaration has not run on a real sample. The sims submit too (`src/sims/cu/sim_job.cu`): each device sim is one job, with a signum over its name and arguments, the bytes of its first allocation declared before it makes it, the same three times, and `TESSERA_OVERRIDE=1` to override. **Measured** here in a scratch `TESSERA_STATE`: all six device sims and Ω's engine run are admitted and released, each peak above its declaration (the figures are in M19), and the history is 368 bytes, seven records and the seal. Two peaks differ between runs by 2,097,152 bytes (2^21): nbody_lattice reads 175,276,032 against 173,178,880, and fixed_pattern 145,915,904 against 143,818,752. No program in the repo loads the engine DLL, whose only builder is `build_engine.sh`, and no other caller is left to submit. **Not rerun here:** the Linux build and suite under WSL 2 (gcc 13.3.0, CUDA 13.3), with 0 warnings. Under WSL, NVML reads a process as 0 bytes before and after it allocates 256 MiB, and no pid can be measured from outside there. So on a paravirtual kernel the daemon's measure opens in reported mode and measures no pid (`tessera_self_paravirtual`). Each admitted job's own process reads its device bytes through dxcore (`tessera_self_measure`) and sends them every sweep, and the daemon applies each reading as it would a pid measurement. There the measure test gives 9 cases, 0 failed (0 bytes, then 427,819,008 after allocating 256 MiB), and the job test 24 checks, 0 failed. The systemd units are in `engine/daemon/service` (`tessera@.socket` and `tessera@.service`). Under WSL, socket activation starts the daemon on a client's connection, and the whole job test passes with systemd starting every daemon, 24 checks, 0 failed. The docker socket mount, also not rerun here: a static client with no CUDA (`engine/daemon/test/tessera_socket_probe.c`) runs in a container under Docker Engine 29.1.3 inside the WSL distro, the host's socket bind-mounted and named by `TESSERA_RUNTIME`. Its submit reaches the socket-activated daemon on the host, which admits the job, and the job releases. The container had no device mounted and so no bytes to report, and the history contained no record, just the seal (32 bytes). How to submit a job and start the daemon: engine/daemon/README.md.

STATUS ledger, frame, measure and the daemon end to end proved on Windows, the history and the tickets sealed; a program's driver and the device sims submit every run, measured; a job stands on its process's bytes and is admitted on what it has yet to find, proved; the code reserves the larger of standing plus declaration and the kept peak, committed and not yet ruled on; Linux built under WSL with each job reporting its own bytes, socket activation and the docker mount reached, none rerun here

NEXT Doug's ruling on reserving the kept peak. The driver's declaration run on a real sample. Native Linux, where NVML measures a pid (this machine has no native NVIDIA Linux driver). A container on WSL reporting its bytes, which needs dxcore mounted into it (`--gpus`). A job from WSL through the Windows daemon. The unsealed history kept aside as `history.unsealed`.

M15. The sift

THE ALGEBRA IT HOLDS TO A pattern of p points is found in a lattice by probing anchors, then verifying the whole pattern at the alignments that pass. What the anchors filter is set

before the search by the corpus's collision entropy H2 and its coherence (a period P found against its own multiples): k anchors on one residue class collapse to one probe, and a periodic corpus cannot be filtered below 1/P by anchors. The dispatch between the in-order and free-order arms is one exact comparison, 100·total² against 85·distinct·Σcount², which needs 143 bits and so at least 8 limbs.

DOES `nbody/orior/` is orior's sift: the search, the steering, the portable scan and one scan arm per instruction set. There is one engine, orior's, and `cell_tracking/maint/build_stamp.sh` builds from `orior/src/engine`. No copy is left to compare. `orior_anchors_for` returns 1 where a period is found and 4 otherwise.

WANTS One source for the sift and the exact integer, taken from orior by a pinned git dependency instead of a copy. An anchor count between 1 and 4 on a partially coherent corpus. The readers (M9) feed it real lattices of 1 to 8 dims.

TRIED, AND WHAT IT GAVE orior's benches, reported in engine/README.md: soundness, 3,421 rows and 465,546 planted occurrences, none refused; the cost per alignment flat over 4 KB to 4 MB; periodic16 off the histogram by exactly 4096 = 2^((k − 1)·H2), found before the search by coherence; dispatch on flatness alone, 41 of 42 fastest. Built and run here on four configurations: MinGW gcc (host), MSVC with CUDA 13.3, WSL gcc 13 with CUDA, WSL clang 18 with CUDA, all at sm_86 and 256 limbs. Every bench exits 0 on every configuration (bench_lattice is not built under MSVC, which has no C99 `_Complex`). `bench_exact`, `bench_lattice` (3,253 rows, every one `hold`), `bench_scaling_reads` and `bench_coherence` are byte-identical across every configuration that builds them. `bench_exact_arms` at 4,096: every arm agrees with portable, AVX2 1.11× to 1.31×, CUDA 0.77× to 1.83×; the planted period reads 4,092 against arithmetic's 4,092, and the device itself answers 2,046 against portable's 2,046. `bench_steer_arms`: 0 checks failed on each (**proved**, per configuration).

STATUS orior's, the one engine; built and run here on four configurations

NEXT Doug's rulings on the port (canonical copy, n_body or nbody, the error model).

M16. Render

THE ALGEBRA IT HOLDS TO A step's state drawn as a sheet or a volume straight from the state, with no export between. The host arm in C and the device arm in CUDA make the same bytes.

DOES `render/` is orior's renderer: `orior_raster_render` (a sheet) and `orior_volume_render` (a volume), each preferring the device and falling back to the host. `view/vis_png` is at `examples/00_blob_viz_tools/view/vis_png/` in orior. There is one engine, orior's (as M15), and `render/` is its own: no copy is left to compare. A WGSL raster arm is not built: `engine_view/gpu.js` draws meshes and is not an `orior_raster` arm. Once a WGSL arm exists it is checked byte for byte against the host and device arms, the way `render_test.py` holds the Python, C host and C device arms.

WANTS One renderer for the engine's state, taken from orior by a pinned dependency.

TRIED, AND WHAT IT GAVE orior's `bench_raster`: twenty sheet and twenty volume configurations, host against device byte for byte. Here, 20 of 20 `ok` on each of the four configurations, the device arm present and graded against the host on the three CUDA builds, the host graded alone on MinGW (**proved**, per configuration).

STATUS orior's, the one engine; built and run here

NEXT As M15.

M17. Memory operations

THE ALGEBRA IT HOLDS TO length, find, copy, move up (target above source, overlapping), compare (-1, 0, 1) and fill, each an exact byte operation; every arm answers exactly what a byte loop answers. Buffers are aligned and terminated by the caller (Doug's rule). An arm may therefore read an aligned block past the end and mask it.

DOES `scriptura`'s arms, one file an instruction set, chosen once at first use; below 32 bytes a call goes straight to portable. `portable`: 64-bit SWAR, overlapping head and tail words to 32 bytes, 32-byte blocks and an overlapping last block above. `avx2`: four blocks a branch (`vpminub` for length, OR for find, AND for compare), overlapping tails. `avx2-erms`: the same, with `rep movsb` and `rep stosb` for copy and fill from 16 KiB. `avx512-unrun` (masked loads and stores), `neon-unrun`, `sve-unrun` (predicated).

WANTS Every arm graded on its own hardware (AVX-512 and SVE have none here; NEON has the Pi 5). The small sizes at the C runtime's speed.

TRIED, AND WHAT IT GAVE The grade (`test/engine/runtime/scriptura/scriptura_arm_test.c`, every operation over 483 lengths from 0 to 4,160, the terminator, match or difference at four places): portable and avx2 agree with a byte loop on MinGW gcc 16, MSVC /W4, WSL gcc 13 and WSL clang 18, with 0 warnings (**proved**, per configuration). NEON and SVE build and link on aarch64 gcc 13 and clang 18 and emit `cmeq`, `shrn`; `whilelo`, `ld1b`, `brkb`, `cntp`; AVX-512 emits `zmm` masked byte compares and moves. Against the C runtime (`bench/bench_scriptura.c`, time-stamp ticks, best of 7, aligned buffers, an i7-5960X with ERMS), compare runs 0.31 to 0.77× memcmp on MinGW, and length and find 0.25 to 0.31× the MinGW runtime at 1 to 16 KiB and 0.93 to 1.25× glibc from 4 KiB. Copy and fill at 256 KiB and 1 MiB run 1.13 to 1.40× every runtime; string moves from 16 KiB bring them to 1.00×, and from 4 KiB they lose (1.28×): 16 KiB is the measured crossover on this part. Portable's short path is a leaf with no frame: overlapping words, the long path kept out of line (**measured**).

STATUS built; the x86 arms proved, the aarch64 arms built and emitting

NEXT The small sizes against glibc (16 to 64 bytes at 1.06× to 1.7×, one call through the arm table); the string crossover measured per part; the NEON arm run on the Pi.

M18. The period reading

THE ALGEBRA IT HOLDS TO Per axis of a lattice, the agreement count at every lag 1 to ⌊n/2⌋ over one set of start positions, and every lag shares one denominator. A candidate is a local peak: the agreement at p and at 2p each stands above both neighbor lags, and the height is the smaller rise. The floor is a drawn null per axis instead of chance: for each axis, every line along it is shuffled on its own: that axis loses its order while every other axis keeps its structure, and each shuffle reads its strongest height. P is the fundamental, the smallest candidate whose height stands strictly above every one of them, or above a set-wide top the program passes. Every comparison is an exact cross-multiply (A12).

DOES `period_read` and `period_draw` (src/cu/engine/analysis/period, module 19; its own base module, it returns P and never sets an order, and the program decides which order P sets): a device lattice of u16 lanes, rank 1 to 8. The request names `draws`, the number of shuffles; `content`, the lattice's content signum, from which the shuffles are seeded; `null_top`, one given top per axis for the set-wide band; and optionally `band` with `band_room`, at least rank · draws. It takes draws or a given top, never both: no draws without a top, or a top with draws of its own, is a request error. The caller passes the signum, for example the sample's lane root from obsignatio, because no module reaches another. Each draw shuffles each axis once (`period_line_shuffle_kernel`): every line along the axis is permuted alone by a keyed Fisher–Yates, keyed by the content, the draw, the axis, the line and the step; the four keys come from the signum's four words mixed with the counter draw · rank + axis. A draw's histogram must equal the original's; a mismatch is a logic error. Per axis it returns P (0 when none), the candidate (P itself, or the strongest peak when none clears), the agreement at the candidate and at twice it with the higher neighbor of each (`agreement_beside_candidate`, `agreement_beside_double`), the pairs per lag, the exact `margin` (the height over the pairs), and the band's `band_count` (the draws that reached a peak), `band_bottom` and `band_top` (both the given top when one is passed). It also returns N, Σc^2 as `collisions` (reported, not a floor), the band's heights sorted into `band` when it is given, and optionally every lag's count. `period_draw(request, draw, heights)` shuffles each axis once along its own lines, keyed by the content, the draw number and the axis, and returns each axis's strongest height (0 over Q where no peak is reached); the program keeps the top per axis across its series and passes the tops as `null_top`. The counts come from two kernels (a histogram and one agreement row per lag); the choice is made on the host. A volume of `2^32` voxels or more is a request error, because N^2 must fit 64 bits. The engine builds with it; no entry calls it yet.

WANTS The knee's orders read from it (the knee program: s = b = P, an odd P a request error, P = 0 order 0), with one band for the set: `period_draw` across its series, the top per axis passed as `null_top`. Flatten taking per-sample orders from it.

TRIED, AND WHAT IT GAVE The test (`test/engine/analysis/period/period_test_main.cu`, run by `test/engine/analysis/period/period_test.sh`): **proved**, 700 checks, 0 failed. Every lag's device count equals a host loop. P, the candidate and its height equal an independent host reference of the same rule (the smallest peak above the top; the strongest peak when none clears). Every band is sorted, and its ends are its bottom and top. A second read of the same content gives an identical reading and band. `period_draw` for draws 0 to 7

reproduces `period_read`'s band exactly, and the band's top given as `null_top` judges every axis as the band did. Nine refusals each give a request error from module 19: no lanes, rank 0, rank 9, an extent of 0, 2^32 voxels, agreement room short, no draws, band room short, a given top with draws of its own. Planted periods still read exactly and clear their bands under the per-axis null: 7 on one axis of 4,096, height 2,048/2,048 against a band top of 39/2,048; on 24 × 48 × 60, 5 in z (height 34,560/34,560, band top 109/34,560), none in y (no draw reached a peak), and 12 in x (height 34,560/34,560, band top 64/34,560). z is 24 deep, not 20, because the peak rule needs $2P + 1 \le \lfloor n/2 \rfloor$ and 20 slices allow $P \le 4$. **Measured** at 8 draws: a smooth ramp with noise (x + y + 2z plus uniform 0 to 63), 16 volumes of 24 × 64 × 64, a period on 3 of 48 axes (6.3%); on one such lattice, y's strongest peak, 15, has height 14/49,152 against a band top of 35/49,152 and is refused, and x reads 5, height 24/49,152 against a band top of 16/49,152. Uniform random u16, 64 volumes of 32 × 64 × 64: a period on 5 of 192 axes (2.6%), below the 1/(draws + 1) = 1/9 a permutation test allows (A12); the caller sets the false-period rate with draws (99 draws, 1%). **Measured** by `period_power` (M19), a period 5 planted along x of 32 × 64 × 64 under the camera law, 599 checks, 0 failed: at 8 draws x reads 5 on 32 of 32 at 64, 128 and 1,024 e, and on 31 with 1 multiple at 256 e; at 19 draws, on 32 of 32 from 64 e up. With the plant at 1,024 e the unplanted z and y read a period on 5 of 64 at 8 draws (against 64/9) and on 2 of 64 at 19 draws (against 64/20) (A12). A per-axis null draw is costly, and the band is set-wide (A12). Under the per-axis null (**measured**, 34 × 960 × 960, RTX 3070): `period_draw` 455.9 ms a call, and 450.1 ms a marginal draw in `period_read`; one band for the set holds the more (A12). Three rules do not hold and are not used: the strongest peak reads a multiple of P instead of P; the margin against the non-multiple lags reads smoothness as a period; and the chance floor $\Sigma c^2/N^2$ cannot tell structure from a rearrangement, since Σc^2 is unchanged by one.

STATUS built, proved against its reference; the local-peak rule, the fundamental above a per-axis drawn null or a set-wide top

NEXT The knee's set-wide band from `period_draw`; per-sample orders in flatten; the knee pulls it.

M19. The sims

THE ALGEBRA IT HOLDS TO A sim builds a lattice whose answer is known and grades a measurement against it, on the device, in exact integers. The camera law (`sim_camera.h`, the same in every sim): electrons S = the background + a ramp · x + an optional planted wave by phase + the brightness of every body whose ellipsoid holds the voxel, each body an integer center, velocity and semi-axes with a born and an ended frame and a parent. Shot noise is `sim_poisson_four_cumulants`: each expected electron adds 0, 1, 2 or 4 with chances 9, 8, 6 and 1 in 24. The count's first four cumulants equal Poisson's exactly, and it parts from Poisson at the fifth. `noise_terms` reads it at 16 electrons as κ3 15,925 and κ4 14,018 thousandths, against 16,000 and 14,000. The symmetric law `SIM_SHOT_SYMMETRIC` is Binomial(4S, ½) − S, with Poisson's mean and variance but symmetric, and it reads 74 and −9,282. Under this law, floor_match, floor_track, nbody_lattice, noise_floor, period_power and root_universal pass every check. knf_identity passes its 23 checks too. It ran with `TESSERA_OVERRIDE=1`,

since it declares 404,752,392 bytes over its kept peak of 370,311,168, and the lattice's entangled entropy reads 44.170 times the free null's mean. Read noise is Binomial($4r^2$, ½) − $2r^2$. The value is the offset + the fixed pattern at the voxel + the gain · the electrons + the read noise. Every draw is a keyed counter draw (splitmix64 of the key mixed with the counter), and nothing is a float.

DOES `engine/sims/`: eighteen sims, each an exact-integer program (on the device, except `ask_state`, `ka_psi` and `goodstein`, which run on the host, and `pi_tower`, whose turn runs on the host and whose BBP digits run on the engine's record machine) that `bash src/sims/run.sh <sim>` builds and runs, exiting 0 only when every check holds. `nbody_lattice`: moving and dividing bodies with their truth (the mass and the three coordinate sums a frame a body), and `--out <dir>` for `lattice.npy` (u16, T, Z, Y, X) and `truth.tsv`. `noise_floor`: the Kolmogorov mock (C11), the linear complexity of every bit plane, the photon transfer curve (C3, C7) with the motion removed by the truth and by the data alone, and the neighbor coherence of the frame differences (C5, C6). `period_power`: `period_read` (M18) on a planted period from no plant to a strong one, at 8 and 19 draws. `fixed_pattern` and `classify_reject_recover`: orior's ART-4-005 and ART-4-007 on the device, the phase dispersion ratio exact against a drawn null band. `root_universal`: the tower with lookup edges between its floors, on camera-law volumes (M20). `ask_state`: a qubit carried exactly as the answers of a complete ask, on the host in exact arithmetic (M21). `ka_psi`: the superposition theorem's inner function ψ, Sprecher's and Köppen's, computed exactly to depth 60 with the exponent never expanded, and the separation of its inner sums (A15; kolmogorov_arnold.md). `chaitin_omega`: Chaitin's Ω for Tromp's binary lambda calculus, where a closed term halts when it has a normal form, bracketed between two exact dyadics. Every closed term through L bits is run by normal-order reduction on the device, one term a thread (`omega_kernel`), or on host threads with `cpu`. The device takes the host's decisions in the host's order under budgets of at most 256 steps and tokens, and hands every term that reaches a smaller budget or outgrows its room back to the host, which runs it under the full budgets. The host's run of every term through 30 bits must give the device's fates and busy beavers, record holders included. `--ledger <path>` plans the run as jobs of `2^28` terms, each finished job's tally one flushed line, and a rerun runs only the jobs its ledger lacks. A normal form proves a halt, and Brent's watcher proves a loop when a term returns to a state it was in before. A term is proven to grow forever when a part of its spine, reduced only by head steps at or below its own position, comes back deeper on the spine (`omega_grows_forever`). A term still unsettled is proven to halt when it has a simple type (`omega_simply_typed`, Hindley's unification with an occurs check; a typed term normalizes, Tait 1967), and no loop or growth term may type. The rest stay open. The mass past L is counted with directed rounding instead of run. Given L alone, it runs on the engine's record machine (`omega_engine`), and every term the engine settled through 24 bits is run again by `omega_run`: a halt must give the same fate at the same step and the same normal form, a loop the same fate, and a term that grew, by the growth proof or parked as outgrown, is not run again (`chaitin_omega.cu`). At the default L of 28 lengths 25 to 28 are not cross-checked, and no default run has finished (item 12). `knf_identity`: the entropy history (M8) of `nbody_lattice`'s law in a 64^3 cube over 177 frames against spatial null permutations: the one-bit window rule, the 48 exact cube motions, the entangled entropy E against 19 free draws, the departure curves of two inverse nulls (inside

b^3 tiles, and whole tiles moved) at tiles 1 to 64 and over each body's box, the scene with no bodies, and alike-voxel controls (A13, two_crystals.md). `goodstein`: Goodstein's sequences stored as hereditary trees and never expanded, with b^L − 1 at a large L stored as one run of b − 1 over the exponents 0 to L − 1; with the base read as ω each tree is an ordinal below ε_0: the start 16 is ω^ω^ω and 65536 is ω↑↑4 (A13, two_crystals.md). `pi_tower` (pi turning at the boundary, the idea behind tower recursion): the rotation by π − 3 on the circle, stored as the integer turn nA mod 2^P with π bracketed by Machin's formula. Its closest returns are the floors, π's partial quotients. At 2^k cells, the step that fills every cell is read from the floors by the three-gap theorem, and each gap's question is a first hit answered by Euclid's descent. Whether the integer turn lands in the real turn's cell is itself a first hit, checked at every resolution. The resolutions come with the request: `run.sh pi_tower -- n` reads 2^n, `-- from to` reads 2^from to 2to, and no argument reads 2^1 to 2^100. P is 3n + 64 rounded up to a word and at least 384, and a width that cannot hold 3(P + 32) + 64 bits is refused with the limbs it needs. Its deepest turns run on the engine. The turn at depth n is α's bit n, and the BBP formula (Bailey, Borwein and Plouffe [7], Math. Comp. 66, 1997) reads it where it stands. Each term i ≤ d is a lane of the record machine: 16^(d − i) mod 8i + j by base-16 powering, each square taken mod 8i + j, then its fraction floored to W bits. A tail program takes the terms past d, and a pair program adds two lanes mod 2^W, and rounds of it reduce a sweep on the record machine. keymath sizes every register, the scheduler lays them with reuse, a sweep is one `cycle_record_run` over as many lanes as the device keeps resident, and tessera admits the job. The error, 4(N + T) + 1 units over N terms and T tail terms, decides the leading bits, and only those are printed. A resolution past 2^20, whole or as base^exponent, is stored as (2, n): P and the exact width a host turn would need print exact, and the engine sums the depth-n terms sweep by sweep, printing the terms done and the rate. Neither tower, T or T^{-1} (A13), is in it. `noise_root`: the root noise of an object's box. Each shared term is planted alone at three strengths, and every yank is priced through the tower and compression's coder (E16). `noise_terms`: each noise vector planted in the camera law and read back (noise_vector_integration_table.md). `floor_match`, `floor_track`, `pi_plane` and `omega_computer` are not described here yet. `SIM_EXACT_LIMBS` sets `ORIOR_EXACT_LIMBS` for every object a sim links. Every sim that uses the device is one tessera job (`sim_job.cu`, M14): before its first allocation it declares those bytes, under a signum over its name and arguments, and its admission and release are two of its checks. The files are in engine/README.md.

WANTS The engine run on `nbody_lattice`'s lattice and graded against its truth (not yet run). F_4 in bits (C1) from a sim's planted σ^2: not computed, because $\log_2$ of $2\pi e\sigma^2$ is irrational and the sims carry no floats.

TRIED, AND WHAT IT GAVE **nbody_lattice**, 9 checks, 0 failed (11 with `--out`): 24 frames of 20 × 96 × 96, 16 bodies, 10 founders and 3 divisions at frames 8, 12 and 16. **Proved:** the device truth equals the host's walk of each body's box; the signal equals the host's on all 4,423,680 lanes, 0 differ; no lane clipped; a noiseless camera renders the signal exactly; a second render is byte-identical. **Measured**, the camera law's two moments at gain 1, read variance 3, offset 100, fixed pattern 0 to 8: the residuals sum to 3,271 against a spread of the square root of 636,863,096 (between 25,236 and 25,237), and their squares sum to

635,890,703 against the law's 636,863,096, a ratio of 0.99847 (the check allows 1%). **noise_floor**, 11 checks, 0 failed. The Kolmogorov mock (C11): 4,096 streams of 512 bits, each from a linear generator of degree 8 to 64 and a drawn seed; Berlekamp–Massey on the device, one thread a stream, puts the complexity within the degree on 4,096 of 4,096 and regenerates each whole stream from its first L bits on 4,096 of 4,096 (**proved**). The description, 2L bits a stream, is 286,744 of 2,097,152 bits (13.67%). Keyed draws of the same count read 252 to 262 against n/2 = 256. The camera lattice's bit planes, 8,640 streams of 512 consecutive lanes a plane, random meaning within 16 of n/2 (**measured**): planes 0 to 8 random on 8,640 of 8,640 (mean L/n 0.5004), plane 9 on 850 (0.0638), plane 10 on 29 (0.0029), planes 11 to 15 on none. The photon transfer curve (C3, C7) on $d = I(t + 1) - I(t)$, 4,239,360 pairs, planted $g = 1$ and $r^2 = 3$ (**measured**): with the motion removed by the truth (S unchanged, 4,226,574 pairs), d^2 on S has slope 2.0046 against $2g^2 = 2$ (the check allows 2%) and intercept 5.643 against $2r^2 = 6$ (the check allows 6 either side). From the data alone, d^2 on $I(t) + I(t + 1)$ over every pair has slope 7.0460 against $g = 1$, and the neighbor coherence (C5), $(d - d')^2$ on the four lanes' sum over 4,195,200 pairs, has slope 4.5292 against 1: sharp-edged moving bodies bias the data-only gain, and the neighbor coherence does not remove the bias. That is the measured gap between removing the motion by the truth and removing it from the data. The neighbor correlation of d (C6), $\Sigma dd'/\Sigma d^2$, is $-71{,}727/1{,}161{,}503{,}631$ on truth-static pairs (the check: within 5σ of 0) and 403,193,327/2,288,244,270 (0.1762) over every pair. **period_power**, 599 checks, 0 failed under the fundamental and the per-axis null: from 64 e up x reads 5 on every volume at 19 draws, and the unplanted axes' false periods stay within 5σ of 1/(draws + 1) (M18, A12; the counts in ledger.md). **fixed_pattern** (ART-4-005 ported), 10 checks, 0 failed: frame 8 × 8, 48 frames, swing 20, pattern 40, 8 draws, key 0xF17A. The detector reads period 64, the frame, at a dispersion ratio of 179.565 against its own shuffle's 0.972. The batch and Welford routes (reduced fractions, on the device) agree on 64 of 64 phases, the broken route splits from each on 61, and the residual equals the moving scene on 3,072 of 3,072 values, a reduction of exactly 100% (**proved**). The shuffled stack reads period 11 at 2.161, and rejecting at the wrong period, 63, removes 0.2298%. Against the band top of 5.241 over 8 shuffles, the matched pattern (179.565) is removed at 100%, no pattern (1.477) is declined and returned untouched, and impulses (2.064) are declined. A static scene feature cannot be told from a fixed pattern and is removed with it: at depth 0, 5, 15 and 30 the reduction is 100.0000%, 99.9263%, 99.3370% and 97.3480%. It reads the raw integers, not the reference's byte view, and draws by key, not by Python's Mersenne Twister. The numbers are not the reference's; the claims are. **classify_reject_recover** (ART-4-007 ported), 7 checks, 0 failed. A fixed pattern over a varying subject is found (179.565 against 5.241), the subject exact on 3,072 of 3,072, a reduction of 100%, 0 of 64 pixels flagged. Incoherent noise of variance 1,600 over a varying subject is declined, but its best ratio, 3.523 at period 3, clears the band top of 3.294: it is declined only because the period is not the frame, a 1-in-9 event under the null. Impulses on a repeat: 0 flagged, exact on 64 of 64. Incoherent noise of variance 144 on a repeat: 55 of 64 flagged, and where the count and the median agree, exact on only 4 of 9 (**measured**). Agreement between the routes does not make a strict majority, and a strict majority does not make the value exact: the mode and the median can agree on a wrong value. **root_universal**, 1,268 checks, 0 failed (M20, A14). **ask_state**, 33 checks, 0 failed (M21, A15). **ka_psi**, 152 checks, 0 failed (A15): Sprecher's ψ descends, Köppen's is strictly

increasing and continuous in both readings, and Braun and Griebel's separation Lemma 3.4 is false at k = 1 and restored by a corrected bound with a narrower ramp (**proved**). **chaitin_omega**, 13 checks, 0 failed, at L = 30 on the host (2,048 steps and 2,048 tokens a term). Of 647,463 closed terms, 645,942 halt, 934 are proven to loop, 213 are proven to grow forever, 0 are proven to halt by a type, and 374 stay open (86 out of steps, 288 outgrew the space). 0.12254295… ≤ Ω ≤ 0.12599214…. The lower bound is the halted mass plus the normal forms past L. The upper bound is the halted mass, the open mass and the closed mass past L, and it equals their exact sum. At 200,000 steps and 16,384 tokens (73 s) the same 374 stay open (47 out of steps, 327 outgrew), and the bracket is the same. At L = 33 on the device (14 checks, 0 failed), of 3,910,730 closed terms 3,900,966 halt, 5,663 loop, 1,180 grow forever, 0 type and 2,921 stay open; 0.12312353… ≤ Ω ≤ 0.12599095…. The table equals the host's L = 33 run line for line. At L = 36 (14 checks, 0 failed), 0.12358892… ≤ Ω ≤ 0.12598988…; at 256 steps and tokens (13 checks, 0 failed, the strict busy beaver check dropped under 2,048) the bounds move by 2^ − 28.2 and 2^ − 25.5 against a gap of 2^ − 8.7. At L = 44 and 256 steps and tokens (13 checks, 0 failed), 3,392,860,908 terms, the first typed halt, and 0.12443857… ≤ Ω ≤ 0.12598796…. Every bracket holds 1/8, which puts Ω at 0.00… in binary: 2 bits **proved**, and the third is not settled. The checks include the closed-term counts against a parse of every code through 22 bits, no loop or growth term typing, the device against the host through 30 bits, and the busiest normal form at each length against BusyBeaverWiki's BB λ (OEIS A333479) at every length through the smaller of L and 33, up to 1812 at 33: at most the published value and equal where every term halted under any budgets, and equal everywhere at 2,048 steps and tokens or more. An open length holds only a lower bound: meeting the published maximum means the run reached the record holder. A known defect, and not fixed: the ledger keeps a line cut inside its last field once a rerun has ended it, and takes it over the whole line after it. At L = 31 that gave a wrong normal form record holder with every check passing (**measured**; ledger.md). The fates and the bracket cannot be hit. L = 44 at 256 steps and tokens on the idle device (RTX 3070): 28,080 ms, 120.8 million terms a second (**measured**). **As tessera jobs** (M14), each device sim runs in a scratch `TESSERA_STATE`, and the job's two checks are added to its count. Declared bytes, then the measured peak: period_power 262,144 and 145,915,904 (601 checks, 0 failed); nbody_lattice 26,542,080 and 175,276,032 (11, 0 failed); noise_floor 9,142,272 and 187,858,944 (13, 0 failed); root_universal 339,510 and 152,207,360 (1,270, 0 failed); fixed_pattern 141,056 and 145,915,904 (12, 0 failed); classify_reject_recover 118,016 and 145,915,904 (9, 0 failed). `ask_state` (33, 0 failed) and `ka_psi` (152, 0 failed) run on the host and submit no job. **The Ω engine run** at L = 16 (16 checks, 0 failed, declared 30,256 bytes, peak 286,425,088): 226 closed terms, all halt, 0.11664483… ≤ Ω ≤ 0.12600284…, 2 bits **proved**, BB λ equal to the published value at every length through 16, and every term the engine settled gives, under `omega_run`, the same fate at the same step and the same normal form. The host path at L = 16 (13 checks, 0 failed) gives the same table line for line. **knf_identity**, 23 checks, 0 failed (a tessera job, admitted with 404,752,392 bytes reserved, peak 370,311,168). **Proved:** the host's walk equals the engine's history at all 262,144 voxels; of 8,192 single flipped bits 1,500 leave it unchanged, and the window rule names all 8,192; the 48 exact motions carry it, with E and the cloud unchanged, on 48 of 48; the whole is its tiles plus its seams at all 7 tile sizes, and a rigid move keeps every tile's inside on 133 of 133 draws; alike voxels, 4 of 64 volumes

identified, within 5σ of 1/20. **Measured:** E is 44.626 times the free null's mean, above all 19 draws; the inside and between departure shares cross between tiles of 2 and 4 and sum to 0.590 at 4; each moving body's curve runs 0.112 to 0.196 at 2 up to 0.947 to 1.019 at 32; with no bodies E is −10.675 times the null's mean, not identified (the table in two_crystals.md). **goodstein**, 7 checks, 0 failed. **Proved:** the bump keeps the tree on 80,256 of 80,256 values below 20,000 at bases 2 to 6; 2↑↑k in hereditary base 2 is ω↑↑k for k = 1 to 4; the starts 1, 2 and 3 reach 0 in 1, 3 and 5 steps, each value the numeric sequence's; the starts 4, 16 and 65536 each run 1,000,000 steps to base 1,000,002 with the ordinal falling on every step, and every value that fits 64 bits equals the numeric sequence's (all 10^6 steps for 4). **Measured:** 65536's top run holds 7,625,597,484,984 terms as one block, and all three starts end on the same tail, $\omega^2 + \omega\cdot 8 + 572861$ (an observation). **pi_tower**, 8 checks, 0 failed. **Proved:** floor(π·2^384) is one integer at both ends of Machin's bracket (416 bits, 116 terms), and after the point π begins 0x243F6A8885A308D3; the bracket certifies 109 partial quotients, the first 21 the published [3; 7, 15, 1, 292, ..., 1, 84]; the turn's 66 record closest returns to 2^112 steps are the convergent denominators q_j in order; the integer turn lands in the real cell on every step searched at 2^1 to 2^100 cells; the fill step and last cell read from the floors equal a walk of every step at 2^1 to 2^24; at every resolution the last cell's first touch is the fill step. **Measured:** 2^64 cells fill at step 81,856,329,715,838,968,403 (8.186e19) and 2^100 at 1.593e30 (q_57 − 1). The fill takes 1 to 5 times the cells except inside a large floor: up to 71.9 inside 292 (2^8), 20.9 inside 84 (2^34) and 23.8 inside 99 (2^61). The last cell filled is often just behind the start, at 1 − α, 1 − 2α, 1 − 3α (an observation). Then 17 checks, 0 failed, for the arc and balance. **Proved:** the helix over our disk (radius $1/(2\pi)$, rising $1/\pi$ a turn), in $\mathbb{Q}(\pi)$ at four quarter turns, has $\kappa^2 = 4\pi^6/(\pi^2+1)^2$, $\tau = 2\pi^2/(\pi^2+1)$ and $\tau/\kappa = 1/\pi$, the tangent of its plane's tilt; its Darboux vector lies along our axis; the torque about the axis is zero and $L_z^2 = 1/(4(\pi^2+1))$. From the bracket to 12 places: κ = 5.705134342735, τ = 1.816000663299, tilt 17.656787151412°, L_z = 0.151657235526. In the square billiard from the corner at slope π, unfolded, the segment in cell (i, j) carries L = (−1)^(i+j)((j + ½) − π(i + ½)) about the center. Over 2^24 columns walked from the integer turn with every floor certain, no wall hit is a corner, no L is zero, and the record near-corners are exactly the q_j. **Measured:** 52,707,178 floor-wall hits to 16,777,216 side-wall hits; the lead changes hands 35,929,962 times in 69,484,394 segments and is never kept longer than 2; the nearest tie, |L| = 1.910e-8, is at $\pi \approx 5419351/1725033$, a convergent. Then 18 checks, 0 failed. At every resolution, H is the first step after the fill in cell 0 and C the first step after H in the fill's last cell, each one first hit, with the certainty check run to C. Both equal a walk at 2^1 to 2^24. **Measured:** H lands on a record return or a sum of them, and the wait H − T is either a few steps or about one floor's q. The fill is within 4 steps of the next record return at 64 of 100 resolutions. The second run C − H has the first run's period at 13 of 100, and is shorter at 86 by a floor combination (a record return at 80, c·q_j at 6) and longer at 1 (2^6). Then 23 checks, 0 failed, for the residue, the golden helix and any 2^n. **Proved**, the residue: on every floor with q_j ≤ 2^21 (j = 0 to 11), π's first q_j steps have the whole parts of the permutation $n \mapsto n\cdot p_j \bmod q_j$, and at step q_j the walk stands $\delta_j = q_j\pi - P_j$ off the whole, with $q_j|\delta_j| < 1$ certified. The identities are 3/1, 22/7, 333/106, 355/113, 103993/33102, ... 5419351/1725033, with the residues +1.415e-1, −8.851e-3, +8.821e-3, −3.014e-5, +1.912e-5, ... −3.820e-8. **Proved**, the golden helix, with every residue bracketed from the turn on

floors q_j ≤ 2^112 (j = 0 to 65): the residues flip sign every floor; q_j ≥ F_{j+1} and |δ_j| < 1/q_{j+1}; and the shrink |δ_j|/|δ_{j − 1}| lies between ½ and 1 exactly where a_{j+1} = 1, each side of 1/φ decided at both ends. **Measured:** of 65 shrinks, 45 fall below 1/φ and 20 above (every one above on a floor of 1, while 8 floors of 1 fall below), and 28 lie between ½ and 1. The growth a floor, q_65^{1/65}, is 3.210576 against φ = 1.618033. **Proved**, any 2^n: at 2^1000 (`SIM_EXACT_LIMBS=512`, 16,384 bits; P = 3,072, Machin at 3,104 bits and 864 terms, 888 floors certified), all 23 checks hold. The default width refuses 2^1000 by name, "the turn needs an exact width of 9376 bits: run with SIM_EXACT_LIMBS=512". **Measured** at 2^1000: the fill is step 1.288e302, 12.02 times the cells, on floor 600 (a = 106), and the second run is not the first's period. Then 41 checks, 0 failed, for the engine. **Proved** on the engine's record machine: π's first 64 bits after the point, 0x243F6A8885A308D3; the hex digits at Bailey's published positions (D. H. Bailey, "The BBP Algorithm for Pi", 2006, table 1 and its text), 26C65E52CB4593 at 10^6, 6C65E52CB459350050E4BB1 at 10^6 + 1, 17AF5863EFED8D at 10^7 and ECB840E21926EC at 10^8; and the engine's cell at 2^100, its low 64 bits, equals the exact turn's, 0x5a308d313198a2e0. **Measured** (RTX 3070, 70,656 lanes a sweep): 10^8 terms in 1,416 sweeps and 3.887 s. At 2^(2^30) cells (hex position 268,435,440), 268,435,441 terms in 3,800 sweeps and 10.638 s, 2.52e7 terms a second, 127 bits certified, the cell's low 64 bits 0x8293097a8232c37f. At 2^googol cells (n = 10^100, hex position 2.5e99), 46,844,928 of 2.5e99 terms in 663 sweeps and 64.0 s, 7.31e5 terms a second, about 1.08e86 years at that rate; the run was stopped, and no digit at that depth was printed. The job declared 3,413,540 bytes and peaked at 221,413,376: the declaration counts the sim's own buffers, not the record kernel's per-thread file. The runs are in ledger.md. **noise_root**, 72 checks, 0 failed (a tessera job; E16). **noise_terms**, 109 checks, 0 failed (noise_vector_integration_table.md). **omega_computer**, 5 checks, 0 failed (runs/omega_computer_run.txt).

STATUS built; every sim run recorded here passes every check, the device sims as tessera jobs, Ω runs on the engine, and π's BBP digits run on the engine

NEXT The engine on `nbody_lattice`'s lattice against its truth.

M20. The root universal

THE ALGEBRA IT HOLDS TO The tower is the additive skeleton of the Kolmogorov–Arnold form: sums, collapsed in order onto the floor. The lookup table is its one-variable edge. Laid between the floors, the edges make the collapse an outer sum with nonlinear edges. Inside the reversible stream an edge must be a bijection: it permutes the low bits of a coefficient and passes the high bits. Any other function rides widened, (x, y) → (x, y ⊕ f(x)), the Bennett embedding: a dimensional expansion, and the bound is one of width, not of kind (A14).

DOES `tower_lift` and `tower_lower` take edges. `TowerEdge` {floor, index_bits, forward} permutes the low index_bits of every approximation coefficient on floor k through `forward`, a permutation of [0, 2^index_bits). Floor k is the approximation just before level k's lifting; floor = floors is the collapsed floor. Lift lays E0, L0, E1, …, L(N − 1), EN, and lower undoes EN^{-1}, $L(N-1)^{-1}$, …, $L0^{-1}$, $E0^{-1}$: the edges on one floor unwind as a stack. index_bits runs 1 to 20. Every edge is checked before any device work (a permutation, the width in range, the floor

at most floors) and refused otherwise. With no edges the tower is unchanged. The crystal path in `engine.cu` passes no edges.

WANTS The edges in the crystal and seal path, with the `.kcr` recording the edge program (Doug's call). Which functions to lay, without fitting (item 3 of kolmogorov_arnold.md). A fitted edge's price.

TRIED, AND WHAT IT GAVE `tower_edge_test`, 25 checks, 0 failed (**proved**; 20 for the edges and 5 for the Bennett edges, A14): one edge, stacked edges on one floor, and edges on every floor including the collapsed one each rebuild the exact lanes; an edge changes the crystal; a non-permutation, a width of 0, a width of 21 and floor 50 are each refused; a clean edge after the refusals still round-trips; a function that is not a bijection rides as a Bennett edge, $(x, y) \to (x, y \oplus f(x))$, at the price of width: $|x|$ and $x \geq 100$ as 16-bit edges round-trip on 64 of 64 lattices and read out exactly, and $|x|$ laid bare is refused (A14). `root_universal` (M19's camera law), 1,268 checks, 0 failed. **Proved**, the root stays exact: 48 volumes, 24 of $4 \times 12 \times 48 \times 48$ and 24 of $3 \times 13 \times 37 \times 41$, 6 floors each, carry 171 random edges (1 to 6 a volume, widths 1 to 12 bits, and one 20-bit edge per extent) through lift, code, wipe of the stored coefficients, decode and lower: plain exact on 48 of 48, edged exact on 48 of 48, the crystal changed on 48 of 48. **Proved**, the fold keeps order: on floor 1, 8-bit edges a then b lay the same crystal as the single table b(a(i)) on 8 of 8; b then a lays a different crystal on 8 of 8; a on floor 1 and b on floor 2 lay a different crystal on 8 of 8: a floor between two edges blocks the fold. **Measured**, the price of an unfitted edge (one keyed random permutation a floor, coded bits a voxel, 8 volumes of $4 \times 12 \times 48 \times 48$): 6.802 with no edge; at widths 2, 4, 8 and 12, floor 0 (110,592 coefficients) 6.810, 6.908, 8.719 and 12.608, floor 3 (72) 6.802, 6.802, 6.804 and 6.817, the collapsed floor 6 (1) 6.802 at every width, every floor 6.808, 6.911, 8.742 and 12.714.

STATUS built and proved in `tower`; not in the crystal path

NEXT Doug's ruling on the `.kcr` recording the edge program; a fitted edge's price against the unfitted.

M21. The ask and the state

THE ALGEBRA IT HOLDS TO The terms. An **ask** (a measurement, formally a POVM) is finitely many outcomes with positive semidefinite operators E_i summing to the identity. A **state** ρ (the quantum functional) is a positive semidefinite matrix of trace 1; for a qubit $\rho = (I + r \cdot \sigma)/2$, $|r| \leq 1$. The **answer distribution** is $p_i = \mathrm{Tr}(\rho E_i)$: "the order falling out of an ask", not an order imposed. One basis's ask returns the **magnitude**, the squared size of each amplitude; the **phase**, the relative angles between the amplitudes, it cannot see, and other asks can. A **complete ask** (informationally complete) is one whose answers determine the state, one to one; it needs at least d^2 outcomes. The **valid set** is the distributions some state gives, a proper subset of all of them. The **crossing rule** is the fixed linear map from a complete ask's answers to any other ask's; it is not classical total probability, and the difference is interference (A15; the protocol is query_protocol_table.md and the plan engine_plan.md).

DOES `engine/sims/ask_state`, on the host, no GPU work: a qubit carried exactly as the answers of a complete ask. Rationals on the exact integer, reduced by a word gcd when both parts fit a word and carried exact, unreduced, when they do not. The SIC's numbers in $Q(\sqrt{3})$, $a + b\sqrt{3}$, multiplied with $(\sqrt{3})^2 = 3$, each sign decided exactly by comparing a^2 with $3b^2$. Two asks, both $E_i = (I + v_i \cdot \sigma)/4$: a rational tetrahedral ask, $v = (\pm 1, \pm 1, \pm 1)/2$ with an even number of minus signs, and the SIC, $v = (\pm 1, \pm 1, \pm 1)/\sqrt{3}$.

WANTS Two statements, not built (A15): the sparse form of ψ's terms, and the countable numbers represented exactly.

TRIED, AND WHAT IT GAVE `ask_state`, 33 checks, 0 failed. **Proved** for each ask: the operators sum to the identity and are positive, and the frame is $c \cdot I$, and the ask is complete (the rational ask: $|v|^2 = 3/4$, not rank one, frame I, $r = 4 \Sigma\, p_i\, v_i$; the SIC: $|v|^2 = 1$, rank one, frame $(4/3)I$, $r = 3 \Sigma\, p_i\, v_i$). 25 states, 21 pure (14 on the equator from rational t, $((1 - t^2)/(1 + t^2), 2t/(1 + t^2), 0)$, and rational points of the sphere) and 4 mixed, go state → answers → state exactly, 25 of 25; the pure lie on the boundary, $|r|^2 = 1$, 21 of 21, and the mixed strictly inside, 4 of 4. The crossings onto the +x, +y and +z asks equal $(1 + r_a)/2$, 75 of 75. The phase falls out of the ask: the 14 equator states all answer ½, ½ to the 0/1 ask, yet the complete ask tells apart all 91 of their pairs. The crossing weights onto +x are $3/2, 3/2, -1/2, -1/2$ (the rational ask) and $½ + ½\sqrt{3}$, $½ + ½\sqrt{3}$, $½ - ½\sqrt{3}$, $½ - ½\sqrt{3}$ (the SIC): a weight is negative: the crossing is not classical total probability. (1, 0, 0, 0) is refused, no state, and crosses onto +x at 3/2 and at $½ + ½\sqrt{3}$, values that are no probabilities. The SIC's answers cross into the rational ask's exactly, 25 of 25. **Measured:** of the 455 distributions with denominator 12, 55 are states for the rational ask and 87 for the SIC; of the others, 104 and 72 still cross to probabilities on the x, y and z asks, and the valid set is stricter than "the three axis crossings are probabilities" (both counts recounted independently in exact fractions).

STATUS built as a sim; proved exact on the host

NEXT Not set (Doug's).

M22. Exact qubit states

THE ALGEBRA IT HOLDS TO A state of n qubits is stored with every amplitude an exact number, never a float. The field is $Q(\sqrt{2})[i]$, the numbers $a + b\sqrt{2}$ with a and b Gaussian rationals; the entries of X, Y, Z, S, H, CNOT, CZ and the controlled T phase all lie in it. Its rational functions in ω, `Q(√2)[i](ω)`, hold a phase left as a symbol. Two forms: the dense statevector, `2^n` amplitudes, and the matrix-product state, a chain of site tensors whose bond after each two-site gate is cut to its exact rank by Gaussian elimination ($M = C \cdot F$). The boundary-lens expectation $\langle\psi|O|\psi\rangle$ contracts the chain from both edges inward (A17).

DOES `examples/qasm/`, error module 20, host only: the exact qubit states in C. `qasm_field_number.c` and `qasm_field_rational.c`: rationals and $Q(\sqrt{2})[i]$ on the exact integer (128 limbs by default); past that width a value refuses and never wraps. `qasm_dense.c`: the dense statevector with X, Y, Z, S, H, CNOT, CZ, the controlled phase and a general 2×2 or 4×4 matrix gate. `qasm_chain_apply.c` and `qasm_chain_matrix.c`: the matrix-product state, run

unchanged over either field, and the boundary-lens expectation. `qasm_symbolic_function.c` and `qasm_symbolic_polynomial.c`: `Q(√2)[i](ω)` in rat_make's canonical form, evaluated at a number. `qasm_lens_hash.c` and `qasm_lens_rank.c`: the reduced-round SHA-256 wedge-field invariants. `qasm_exact.c`: exact fractions, angles and fixed point. The witness root is the engine's seal (`obsignatio_seal`). The files are in engine/README.md.

WANTS qasm in the engine library build: `src/build_engine.sh` (MODULES and INGEST) and `src/cu/CMakeLists.txt` name no qasm source; it is a module with its own test and not part of the engine library. orior's fixed-point qasm (`qasm_bitstring.cu`, `build.sh` and `run.sh`) sits beside the exact host qasm, whose types are `QasmExact*`.

TRIED, AND WHAT IT GAVE `examples/qasm/test/run.sh`, on the host instead of as a tessera job: **39 checks, 0 failed**. Two are regression checks: the bond at the widest site index reads 0 instead of past the tensors; and w^(bits − 1) at w = 2 evaluates to 2^(bits − 1), which an extra power 2^bits must not refuse. Every value below is checked by `qasm_exact_test` (**proved** for these circuits). Dense: Bell and GHZ3 amplitudes "1/2*sqrt2 + 0 i", ⟨ψ|ψ⟩ exactly 1; Bell and controlled T, |11⟩ = "1/2 + 1/2 i"; five gates and then their exact inverse return |000⟩ to the bit. Chain: Bell, bond [2], 8 elements; GHZ-6, bonds [2,2,2,2,2], 40 elements; GHZ-4 and controlled T, bonds [2,2,2], 24 elements, ⟨1111|ψ⟩ = "1/2 + 1/2 i"; GHZ-100, widest bond 2, 792 field elements, norm exactly 1; the scrambler (10 qubits, depth 6), bonds [2,3,6,8,8,8,6,3,2], 552 elements, norm exactly 1; a five-qubit circuit and its inverse return |00000⟩ exactly; on 6 qubits the chain and the dense state agree on all 64 amplitudes for the GHZ chain, GHZ and controlled T, and the scrambler at depth 6. Symbolic, lens and refusals in A17. **Measured**, one run at BelowNormal priority: the dense section 350 µs, GHZ-100 15.9 ms, the scrambler 466 ms, the whole chain section 732 ms, the symbolic section 89 ms (its new check evaluates w^4095), the lens 8.6 ms. **The device code**, `examples/qasm/qasm_self_program.cu` and `qasm_self_run.cu`, `qasm_self_run` (`QasmSelfRequest`, `QasmSelfReading`). A circuit becomes record floors of integer steps, four parts an amplitude (the real rational, the real √2, the imaginary rational, the imaginary √2), each floor scaling D by the lcm G of the gate's entry denominators. It is imprinted and laid with register reuse, and one `cycle_record_run` launch runs every lane through the whole stack. Host and device records are compared word for word. The suites give qasm_test 86 passed, 0 failed; qasm exact test 39 checks, 0 failed; qasm self test 44 checks, 0 failed; the CLI on bell.qasm exit 3, bitstring 00, and on bernstein_vazirani.qasm exit 0, bitstring 101101001101; qasm run.sh exit 0. Every circuit's device records equal the host's word for word, and every host lane equals the dense exact port. Bell: 50 steps, 20 file limbs, 1 in limb, 3 out limbs. The 4-qubit scrambler (15 gates): 1856 steps, 98 file limbs, 4 in, 42 out. GHZ5 and controlled T: 804 steps, 134 file limbs, 8 in, 29 out. 2^200 in a 5-qubit lane is past the register file, and the lay refuses it before any run.

STATUS built; exact on the host, checked by `qasm_exact_test`; the device code proved by `qasm_self_test`, 44 checks, 0 failed; qasm not in the engine library build

NEXT orior is this repository instead of an external dependency: the sift is at `src/cu/engine/nbody/orior/`, and there is no submodule to track.

M23. The refinement loop

THE ALGEBRA IT HOLDS TO Two systems work at the machine's lowest layer. The generator compiles a program from its current understanding of the problem, and hands over the binary or its behavior. The critic takes that program as a raw, noisy object, strips the instructions, paths and structure that carry no signal, and hands back the signal. The generator recompiles to match and hands it back, and the cycle repeats. On this machine (a reading, not a ruling), the programs are record programs, the compiler's automata (M10, item 9 below). A verdict is a field, zero false and nonzero true. Two programs agree when their verdicts are equal on every lane, which the machine checks exactly, as it checks the device against the host.

DOES Nothing of the loop is built. What it would stand on is built: record programs imprinted, laid and run (M10), each step's width read at imprint, and every program's device run compared with its host run word for word.

WANTS The loop, with the generator and the critic both on the record machine, and each round kept only when its verdicts equal the last round's on every lane. The engine's own cost, with one best answer: the least cost at the same verdicts. The loop's progress is read as a departure from a null, as the entropy history's identity (M8, M19) and the period (M18) are read against their draws. The outcomes: the program sheds operations toward the fewest cycles the problem needs, near its Kolmogorov complexity; an error or exploit put into the code is flagged by the other system as noise, isolated and patched; and forms no one would write by hand may emerge.

TRIED, AND WHAT IT GAVE none

STATUS wanted, not built

NEXT The cost metric, Doug's choice: from what the imprint already reads (the step count, each step's width, the register file's limbs) or from counted device cycles. Then the critic's first form, and the null its curve is read against.

M24. The ask

THE ALGEBRA IT HOLDS TO Every ask is `[ADDRESS] -> (QUALIFIER) -> [COST] -> BIT` (E17, A18): unbound it returns its cost, bound by β it returns one bit. Candidates pass a gate of exact cases, and the survivors are ranked by cost read against its own spread; the two are never added together. A chain's links are read by asks over the rows of a Hadamard matrix, and contention by the square of how many links an ask covers. The protocol, step by step, is query_protocol_table.md.

DOES On the host: `query_ask` (`compiler/bootstrap/query_ask.{h,c}`) holds, reads equal or advancing, and bounds a run by the clock it finds (16 checks, 0 failed, Q1), and `query_interface_walk` asks addresses in a child the cell can lose (8 checks, 0 failed). The gate (`gate_descent.c`) keeps exactly what `chain_build` keeps on every relation the ladder holds (Q4). `ask_order` solves every link at 3 to 63 links in exact integers (1,231 checks, 0 failed,

Q5), and `ask_links_read` reads contention (Q7). One bit a check takes the fewest asks on every relation (Q15). On the part: `loop_back_if` is asked of sm_86 (Q16, item 14 below): 8 of 2927 forms come back on every count, every one a `BRA`, and none is cheaper above its spread. A writing of one instruction is searched for on the part for every precept and every ladder relation, every arrangement of the `.kdm` is written from them, run and read back (Q16), and the cases the descent places stand on the part as on the host (Q4).

WANTS The run channel made of these asks (`run_channel.h`, Open 1 of the query protocol table): a container that puts a chain's covered links and reads the part's clock around them, so that NVRTC, nvJitLink and the CUDA runtime leave the loop. Every position a `.krs` writes by hand asked the way `loop_back_if` is (Q16), and every program checked against NVIDIA's compiler (Q17).

TRIED, AND WHAT IT GAVE Slicing by adjacent cuts is refuted (Q6). One combined score and membership by agreement inside a floor are not so (Q12, Q13).

STATUS **proved** (the ask on the host and in the cell, the gate, the order and its solve, the 8 forms), **measured** (the contention read, one bit a check, the cost of the 8 forms, the writings and arrangements on the part), **theory** (the run channel)

NEXT The run channel (Open 1 of the query protocol table), then Q17's first step: the 16 lanes `sass_lane_needs` writes, run beside the cubins of their C route.

8.4 THE MACHINE'S ALGEBRA

Every line is exact. No line assumes a scale: orders, windows, widths and spacings are symbols the request fills.

8.4.1 *Ao. Exact numbers (M1): `decimal_double`*

A decimal d = (−1)^neg · digits · 10^ex is 2^{ex} · 5^{ex} · digits. Its binary value's leading W bits are found by multiplying or dividing by the widest power of 5 that fits a limb, chunk by chunk, with the fives stripped from a fraction first: the result is the exact integer ⌊|d| · 2^{−e2}⌋ with e2 chosen so the integer has W bits, and `terminates` says whether the rest is zero. When the value needs more than W bits, `needed_bits` says how many and `fits` = 0. Nothing is rounded.

8.4.2 *A1. The residual (M2): `residual_program`, `keymath`, `cycle`, `unit_sweep`*

Per axis a, with [1 1] the unit step:
L_n = [1 1]^n ∗ I, B_s ∗ B_b = B_{s+b}, R = 2^{|b|} L_s − L_{s+b}

- **The two terms are two rungs of one ladder.** B_b applied to B_s I is B_{s+b} I, and R is the difference of rungs s and s + b of one binomial ladder of the same image. Every residual for every (s, b) is a difference of two rungs, and one sweep that keeps its rungs gives all of them. Orders per request, and a sweep over candidate periods, cost one pass (theory).

- **The kernel sums to zero.** B_s sums to 2^{|s|}, and 2^{|b|} B_s and B_{s+b} both sum to 2^{|s|+|b|}. A constant added to I leaves R unchanged: the medium's mean cancels exactly.

- **The heat identity.** In centered steps H = [1 2 1] = [1 1]², with m = b/2 per axis:

  4^m δ − H^m = Σ_{k=0}^{m − 1} 4^{m − 1 − k} H^k (4δ − H), and 4δ − H = [− 1 2 − 1] = − Δ²

  so, per axis, R = Σ_{k=0}^{m − 1} 4^{m − 1 − k} · (− Δ² S_k), with S_k = H^k S and S = H^{s/2} I. H/4 is one explicit step of the discrete heat equation, and R is 4^m times what m heat steps remove from the smoothed field: the residual is exactly a weighted sum of negative discrete Laplacians down the ladder. In three axes the same telescoping runs axis by axis: 2^{G}δ − B_{b_z}⊗B_{b_y}⊗B_{b_x} = (2^{b_z}δ − B_{b_z})⊗2^{b_y+b_x}δ + B_{b_z}⊗(2^{b_y}δ − B_{b_y})⊗2^{b_x}δ + B_{b_z}⊗B_{b_y}⊗(2^{b_x}δ − B_{b_x}) (theory: exact algebra, not built as a pass).

- **Parity.** 2δ − [1 1] = [1 − 1], a first difference, centered half a voxel off. An odd background order therefore puts the subtracted term half a voxel from the kept one, and an odd smooth order moves both by the same half voxel. That is ruling (a): refuse odd b, hold odd s with its offset.

The positive set is P(x) = [R(x) > 0].

8.4.3 *A2. The component tree (M3):* `max_tree`

- c(x) is the dense rank of R(x) among the admitted voxels. Each tree node n has a level λ(n); ν(x) is x's own node, the node at level c(x) that holds x. The component at level r holding x is the ancestor a of ν(x) with λ(a) ≥ r > λ(parent(a)). Its voxels are exactly those x′ with ν(x′) in a's subtree.

- Number the nodes in DFS order; a's subtree is one range [in(a), out(a)).

- **Probes by LCA.** Two voxels share a component at level r exactly when r ≤ λ(LCA(ν(x), ν(x′))). Sort k probes by in(ν). The partition at every level at once is given by the k − 1 levels λ(LCA(p_i, p_{i+1})) of neighbors in that order: at level r, neighbors i, i + 1 are together exactly when r ≤ that level, and a probe whose own level is below r is absent. For u before v in DFS order, λ(LCA(u, v)) is the least level among the parents of the nodes in (u, v]: each of those nodes lies under the LCA, and the LCA's child toward v is one of them. It is not the parent of the lowest-level node in (u, v], since a node in u's own branch can sit lower than the LCA's child toward v. The slide's bisection over levels becomes k − 1 LCA queries (built: `max_tree_probe_levels`, with `max_tree_probe_partition` for one level from them; proved against a flood fill).

- **The overlap at every level from one count.** For a lag v, the own-node pair table J(v)[n, n′] = |{ x : ν_t(x) = n, P_{t+1}(x + v), ν_{t+1}(x + v) = n′ }|. Then for components A at level r of t and B at level r′ of t + 1:

  `C_{r,r′}[A, B](v)` = Σ_{n ∈ sub(A)} Σ_{n′ ∈ sub(B)} J(v)[n, n′]

a rectangle sum over the two DFS ranges, and one Fenwick sweep of J answers every asked pair of levels. The census (one partner, unpaired, mutual, forked) is the degrees of the support of C at the chosen levels. The tracker's overlap still recounts per sampled cut (built: `max_tree_pairs` and `max_tree_overlap_sums`, one Fenwick sweep instead of a summed-area table; proved against direct counts).

- **Built**, in `src/cu/engine/nbody/max_tree/`: `max_tree_nodes` (the nodes in DFS order, each with its level as a dense rank, its parent, its subtree's end and the root, and $\nu(x)$ for every voxel), `max_tree_node_at`, `max_tree_probe_levels`, `max_tree_probe_partition`, `max_tree_pairs` and `max_tree_overlap_sums` (`max_tree.h` :99, :105, :126, :130, :155 and :176). No caller outside the test.

- **Proved:** `max_tree_levels_test` (`test/engine/nbody/max_tree/`, host only, with run.sh beside it), a tessera job: "max_tree levels test: 110 volumes, 18071 checks, 0 failed". Each piece is checked against an independent 6-connected flood fill of {rank $\geq$ r}: the node structure; the component at every level in both frames; the probe partition at every level, with duplicate and absent probes; C at every pair of levels (r, r′) at random lags in $[-1, 1]^3$, against direct counts; and the refusals. The volumes are small and random, with values in $[-2, 5]$ and $[-40, 60]$, plus values across the 32-bit limb boundary.

- **Prior art, cited from knowledge.** The component tree of an image's upper level sets, the max-tree: P. Salembier, A. Oliveras and L. Garrido, "Antiextensive connected operators for image and sequence processing", IEEE Trans. Image Processing 7(4) (1998) 555–570; L. Najman and M. Couprie, "Building the component tree in quasi-linear time", IEEE Trans. Image Processing 15(11) (2006) 3531–3539. The LCA and its reduction to a range minimum over the DFS order: D. Harel and R. E. Tarjan, "Fast algorithms for finding nearest common ancestors", SIAM J. Computing 13(2) (1984) 338–355; M. A. Bender and M. Farach-Colton, "The LCA problem revisited", LATIN 2000, LNCS 1776, 88–94. The Fenwick sweep: P. M. Fenwick, "A new data structure for cumulative frequency tables", Software: Practice and Experience 24(3) (1994) 327–336.

8.4.4 *A3. The moments (M4)*

With own-node moments $\mu(n) = \Sigma_{\nu(x)=n} (1, x, x\,x^T)$, every node's moments are $\Sigma_{n' \in sub(n)} \mu(n')$: one prefix over the DFS order gives every component at every level. The centered second moment is $K = m\,M - S\,S^T$, carried with its mass, never divided.

8.4.5 *A4. The correlation operator (M5)*

For frames t, t′, a shift v, a label a of t and a label b of t′ (or ∅, a positive voxel under no label):

`C_{t,t′}[a, b](v)` $= \Sigma_x [L_t(x) = a] \cdot [P_{t'}(x + v)] \cdot [L_{t'}(x + v) = b]$

With a weight $\varphi(x) \in \{1, x, x\,x^T\}$ at v = 0, t′ = t and b = a, the same sum gives the moments. Its reductions:

- the argmax over v of the sum over all labels (`shift_agreement`);

- a local argmax per label from a start lag (`climb_machine`);
- a transpose: `C_{t′,t}[b, a](−v) = C_{t,t′}[a, b](v)`;
- box sums over nested widths around a lag, from one summed-area table over v (the core: each width is the box on top of the one below);
- the autocorrelation of a box, a tent, over one frame's own C (the contact form).

The labels may be any level's components: with A2, C at every pair of levels is a rectangle sum of J. The level is therefore one more axis of the same box, and all of the tree's frame-to-frame reading is C over (v, r, r′), counted once.

8.4.6 *A5. The marginal (M6):* `marginal`

For sources S and targets T with integer weights ω(s, t) and a null weight ω(s, ∅) per source, an arrangement α maps each source to a target or ∅, no two sources to one target:

Z = Σ_α Π_s ω(s, α(s)), Z_{s→t} the same over α(s) = t, P(s → t) = Z_{s→t} / Z

Z is the permanent of the |S| × (|T| + |S|) matrix [ω(s, t) | diag ω(s, ∅)]. **It factors over the connected components of the support graph** (s ~ t where ω(s, t) ≠ 0): the only coupling between sources is a shared target, and the sum over all arrangements is the product of the sums over components, and Z_{s→t} / Z over s's component is the global ratio exactly. A local set smaller than the component is exact only when it is the component. The cost lives in the component's size; a permanent is hard in general, and the measurement to report is the component's size (theory).

8.4.7 *A6. Matching (M7):* `heaviest_matching`

The one-to-one set of largest total weight on exact integer costs; no weight is scaled or compared as a fraction.

8.4.8 *A7. The entropy history (M8)*

For voxel x, bit j and frames 0 … F − 1, with f_j(x) the number of transitions where bit j flips: f_j(x) ≡ bit j of (I_0(x) ⊕ I_{F − 1}(x)) (mod 2). A voxel where this fails means the history was not taken whole; it is the pass's own proof. The windows partition the transitions; their width is a symbol (see the audit).

8.4.9 *A8. The files (M9)*

Integer lifting (predict, then update) takes each step's integer quotient by a power of two, and the inverse step subtracts the identical quotient, computed from the same integers. Each step is therefore a bijection on integers and nothing is lost: the tower rebuilds every voxel. The proof is not the algebra but the read-back: every sample is rebuilt from the file on the device and compared voxel for voxel, and again from a second read of the source pixel for pixel.

8.4.10 *A9. The golden ladder (M11)*

β(n) = the number of Fibonacci rungs at or below n after the first, over the rungs that fit 64 bits.

8.4.11 *A10. The seal (M13): `obsignatio`*

Each level ℓ keys its nodes: K_ℓ = BLAKE3-derive-key-context("obsignatio aeterna 2026-09-23 " ‖ ℓ), H_ℓ(m) = BLAKE3-keyed(K_ℓ, m). For a sample of extent (T, Z, Y, X):

ρ(t, z, y) = H_row(I[t, z, y, ·]), π(t, z) = H_plane(ρ(t, z, ·)), υ(t) = H_volume(π(t, ·)), Λ = H_lanes(υ(·)) κ(c) = H_chunk(LE64(ℓ_c) ‖ bits_c), Σ = H_stream(offsets ‖ B ‖ κ(·)) σ = H_side(H_side stored(deflated) ‖ H_side inflated(inflated)), μ = H_members(tables ‖ names) Ω = H_sample(head ‖ Λ ‖ Σ ‖ σ ‖ μ), Θ = H_set(Ω_1 ‖ … ‖ Ω_n)

- **Locality.** A change at (t, z, y, x) changes exactly ρ(t, z, y), π(t, z), υ(t), Λ, Ω and Θ. A change in chunk c changes exactly κ(c), Σ, Ω and Θ. Any other node agrees except with probability 2^ − 256.
- **Size is strength.** A k-bit check misses a random corruption with probability 2^ − k, whatever the function. The DAG's gain over CRC-64 is structure: nonlinearity, locality and composition instead of a smaller word.
- **The tree is BLAKE3's.** Pairing each level and carrying the odd node up is BLAKE3's left-heavy tree exactly (its left subtree is the largest power of two of chunks below the whole), and the level-parallel device fold equals the host's stack fold.
- **Where it stops.** The tower lifts across all four axes, and a decoded row can be checked only after a whole decode. The stored bytes check alone.
- **The shared shape.** The seal folds the lattice x, y, z, t, as the tower's floors lift it and the moments' prefix sums it (A3). The residual, the tree, C and the seal are each one lattice read once with a different combining operator. The seal's operator alone is not associative. Its tree shape is therefore fixed, and that fixed shape makes a mismatch locatable (observation).

8.4.12 *A11. The scheduler (M14): `tessera`*

The device holds C bytes; U(τ) is in use. Each admitted job j has a declaration d_j, a reservation r_j (from d_j, only growing) and a measured use u_j(τ) by pid:

O(τ) = U(τ) − Σ_j u_j(τ), H(τ) = C − O(τ) − Σ_j max(r_j, u_j(τ))

- **Admission:** k is admitted when d_k ≤ H(τ), and admission alone keeps H ≥ 0. The reservation rule, that what a job asks for is reserved and a lesser actual cost remembered for next time, is not the code's: the code reserves max(d_k, p_σ) instead (`tessera_ledger_wants`), which is not approved (M14).
- **Standing** (the rule is s_k plus d_k). A job's process already holds s_k bytes as it asks: its context and whatever it kept. The daemon reads them by pid, or from the process's own

report under WSL. The job wants w_k = max(s_k + d_k, p_σ). U(τ) already counts s_k. Admission, the head's shadow and the backfill weigh only n_k = w_k − s_k, the bytes it has yet to find. An admitted job is reserved w_k and counted as using s_k until its first sweep. The withholding rule is d_k alone against p_σ.

- **Growth:** u_j(τ) > r_j sets r_j = u_j(τ) and warns. Only growth and outside processes drive H negative, and both are measured: the tally equals C − H at every sweep.
- **Memory of a request:** p_σ = max over a run of u_j(τ), kept under the request's signum σ. It is a measurement at the sweep's resolution, not a byte-exact constant: identical requests have read peaks 4,194,304 bytes apart (M14). The engine is deterministic in its request and its result, not in the bytes a sweep samples. A new declaration d > p_σ is over budget: asked, withheld, overridable.
- **Liveness:** a ticket is (pid, start time), and a reused pid is a different process. A closed socket or a gone process releases r_j at once.
- **Backfill with the head kept:** the history keeps (p_σ, t_σ). The head's shadow S is the least time with H + Σ_{e_j ≤ S} max(r_j, u_j) ≥ n_h, and the spare is X = that sum − n_h. A job k behind the head goes now if n_k ≤ H and (now + t_σk ≤ S or n_k ≤ X). A signum with no history is never backfilled.
- **Time:** every deadline sits in one min-heap keyed by time to change; the root is the next event, handled in order.

8.4.13 *A12. The period reading (M18):* `period_read`

On an axis of extent n with the other axes holding M = N/n voxels, take L = ⌊n/2⌋ lags and the start positions whose coordinate on the axis is below n − L. Every lag ℓ ≤ L then compares the same Q = (n − L)·M pairs:

a(ℓ) = #{x : I(x) = I(x + ℓ·e)}, r(ℓ) = a(ℓ)/Q

- **The local peak.** A candidate p must stand above both its neighbor lags, and so must 2p:

  a(p) > max(a(p − 1), a(p + 1)) and a(2p) > max(a(2p − 1), a(2p + 1))

  Its height is the smaller rise, h(p) = min(a(p) − max(a(p − 1), a(p + 1)), a(2p) − max(a(2p − 1), a(2p + 1))), an integer count over the axis's Q pairs, and its margin is the rational h(p)/Q. The candidates run over 2 ≤ p with 2p + 1 ≤ L, which gives P ≤ ⌊(L − 1)/2⌋: 8 on an axis of 34 slices (L = 17), 5 on 24, 4 on 20.
- **The floor: a drawn null per axis.** The request names d draws. For axis a, draw k permutes each line along a on its own by π_k, and the shuffled lattice I∘π_k holds exactly the same values on every line, its histogram is the original's, and every other axis keeps its structure; only a's own order is destroyed. Each draw reads a's strongest height, h_k = max_p h_{I∘π_k}(p); the draws that reach a peak make a's band, sorted exactly. A read of rank r makes r·d shuffles, one per axis per draw.

- **The choice: the fundamental.** P is the smallest candidate whose height clears the band's top:

  $P = \min \{ p : h(p)/Q > \max_k h_k/Q \}$

  with each comparison a 128-bit cross-multiply. A period P also peaks at 2P and 3P with heights matching P's, and the strongest peak picked a multiple; the smallest peak above the top picks P. An empty band, where no draw reached a peak, or a given top of 0 lets any peak through, and the smallest peak is P. When none clears, P = 0 and the reported candidate is the strongest peak (the largest height, the smallest p on a tie). A lag that makes no peak at p and 2p is never a candidate. Nothing is rounded, and no constant is written in.

- **The fundamental leaves the rate alone.** Some peak clears the top exactly when the strongest peak does. An axis therefore reads a period under the fundamental exactly when it did under the strongest peak; the rule changes which period is read, not whether one is (theory).

- **The set-wide band.** The knee's ruling is 73,158 draws a series, enough to expect fewer than one false period over the set's 24,386 series × 3 axes = 73,158 axes. **Measured** (RTX 3070, on 34 × 960 × 960 = 31,334,400 voxels, a smooth ramp x + y + 2z plus uniform noise 0 to 63): the per-axis null makes one shuffle and one count per axis. `period_draw` takes 455.9 ms a call, the mean of 8 after a warm-up; a call is one count of the original, then a shuffle and a count on each of the 3 axes. `period_read` takes 1,934.2 ms at 4 draws and 9,136.5 ms at 20: a marginal draw costs 450.1 ms and the fixed cost is 133.7 ms (each as the run prints it from the unrounded times). From the measured draw instead of a measured set run: about 9.1 hours a series and about 223,000 GPU-hours for the 24,386 series. So one band serves the whole set. Its limits: one run on one lattice; host wall-clock with the syncs; and `period_draw` allocates its buffers on every call, and part of the 455.9 ms may be allocation (not separated). The timing program is a scratch program outside the repo's tests. `period_draw` shuffles each axis once along its own lines, keyed by the content, the draw number and the axis, and returns each axis's strongest height (0 over Q where no peak is reached). The program calls it across its series, keeps the top per axis, and passes those tops to `period_read` as `null_top` with no draws of its own; a given top with draws is a request error. P is the smallest peak above the given top, and a top of 0 lets any peak through. The tops compare exactly across shapes, because the cross-multiply carries each Q. Whether one band is fair across shapes is Doug's call.

- **The false-period rate.** With no structure along axis a, each line's order along a is one more arrangement of its values, whatever the other axes carry, and if the draws behave as uniform shuffles the d + 1 heights are exchangeable (theory). The lattice's own strongest height then stands strictly above all d others with probability at most 1/(d + 1), exactly that when no two tie. The caller sets the rate with d: 1/9 at 8 draws, 1% at 99. **Measured,** 5 of 192 random axes at 8 draws (2.6%, M18). With a period 5 planted along x at 1,024 e, the unplanted z and y read a period on 5 of 64 at 8 draws (against 64/9) and 2 of 64 at 19 draws (against 64/20) (`period_power`, M19): the per-axis null holds the bound when another axis carries a strong period.

- **Measured: the fundamental is read** (`period_power`, M19). A period 5 is planted along x of 32 × 64 × 64 under the camera law, at amplitudes 0 to 1,024 e, 32 volumes each. At 8 draws x reads 5, a multiple (10 or 15), or another lag: 10, 6, 0 at 16 e; 26, 3, 0 at 32 e; 32, 0, 0 at 64 and 128 e; 31, 1, 0 at 256 e; 32, 0, 0 at 1,024 e. At 19 draws it reads 5 on 32 of 32 from 64 e up.
- **The strongest peak does not hold and is not used: it reads a multiple.** The candidate is the largest height, the reading lands on a multiple of P, and the multiples' peaks are as high as P's, and the noise picks among them.
- **The global shuffle does not hold and is not used: it reads a period where another axis carries one.** One Feistel shuffle of the whole index per draw serves every axis, and an unplanted axis reads a period: the global shuffle destroys the planted axis's structure too, the null's agreement falls below the live lattice's at the unplanted axis's lags, and the exchangeability the false-period rate rests on fails there.
- **The margin rule does not hold and is not used.** For a candidate p with $2p \le L$, the lags that are not multiples of p number B, with $A = \Sigma\, a(\ell)$ over them (the total less the multiples), and the margin $m(p) = (a(p) + a(2p))/(2Q) - A/(B{\cdot}Q) = (B{\cdot}(a(p) + a(2p)) - 2A) / (2{\cdot}B{\cdot}Q)$ is the largest positive margin the candidate. A smooth lattice agrees more at every short lag than at the long ones, and the rule reads smoothness as a period: a smooth ramp with noise reads P = 2 or 3 on every axis.
- **Why chance could not do this.** The old floor was $m \ge \Sigma c^2/N^2$. Σc^2 is unchanged when the values are rearranged, and a floor built from it cannot tell structure from a rearrangement of the same values, and uniform random data passed 63 of 192 axes (33%). The drawn null keeps the histogram and Σc^2 exactly and changes only the arrangement, and a period is an arrangement. Σc^2 is still reported, as `collisions`.
- **The shuffle.** A keyed Fisher–Yates [12] on each line along the axis, one thread a line: for step s from n − 1 down to 1, the element at s swaps with the one at h mod (s + 1), h a 32-bit hash of the line index and s under four keys (`PERIOD_ROUNDS`). Each key is the content signum's words mixed with c · 4 + round, for the counter c = draw · rank + axis. The seed is the lattice's content, and the same content draws the same band. Superseded: a Feistel network on the whole index, 4 rounds, with cycle-walking to stay below N (the global shuffle).

8.4.14 *A13. The record machine (M10): `keymath`, `key_schedule`, `cycle`*

A record program is a list of steps s_1, …, s_n over registers of limbs, each step reading earlier registers and writing its own, run for every record in one sweep.

- **The steps are n** (the scheduler's steps, with the floors stacked).
  - The step count has no bound. `ENGINE_RECORD_STEPS_MAX` is gone: it sized a per-thread array holding one sign per step, and each sign now sits beside its register in the file.

  - The binding resource is the file: `ENGINE_RECORD_LIMBS_MOST` = 256 limbs of live registers.
  - A stack of floors runs as one program in one launch. Each floor is a round that reads the floor below and leaves its state, often wrapped, for the next.
  - The layout frees each register in a single pass, bucketed by its last reader. It frees the same registers at the same steps as the per-step scan it replaced.
  - The algebra of the stack, what it compresses and what it leaves, with the stacked-against-chained measurement (13 to 22 times, records equal): vertical_time_compression.md.
  - **Proved:** 700 floors of a round over four 32-bit words (xor, and, sum, xor with a constant, a 32-bit wrap), 4,204 steps, run in an 8-limb file. On 1,024 lanes the device equals the host word for word, and both equal the same rounds on the CPU's own 64-bit two's complement at every tapped floor (`test/engine/compiler/cycle/record_bitwise_test`).

- **Xor, and, and the two's complement wrap.**
  - `ENGINE_RECORD_XOR` and `_AND` work bitwise on each operand's two's complement, sign-extended without end, one bit wider than the wider operand. The imprint narrows that where it can show a register is never negative. An and with such a register is no wider than it, and an xor of two such is no wider than the wider. The result's sign is read from the operands' signs.
  - `ENGINE_RECORD_WRAP` takes left modulo `2^w`, read back signed, with $w \geq 4$ in `right`.
  - **Proved:**
    * Every output bit is rebuilt from the raw fields one bit at a time, and each sign still holds 64 bits past its written width. This holds on 4,096 lanes in the 64-limb file and 512 in the 256-limb file.
    * Hand-worked answers match.
    * The narrowed widths are exactly the rule's: 32, 20, 20, 32, 41, 41, 32 and 32 over unsigned 32- and 20-bit fields and a signed 40-bit field. All 4,096 lanes equal the oracle past every narrowed width.
    * A wrap below 4 bits is refused at imprint (`test/engine/compiler/cycle/record_bitwise_test`, 24 checks, 0 failed).

- **Register reuse** (opt-in). Register r is live from the step that writes it to its last reader. With reuse on, a liveness free-list returns r to the file once its last reader has run, and a chain needs only the registers live at once instead of one per step. **Proved** on a 21-step additive chain: the same output from 3 limbs as from 21. Off by default: a program that does not ask keeps one register per step: each register placed at the running total of the limbs before it, the same bump, and a program with no tables keeps the default layout, by construction.

- **The lookup table.** A table step reads the low b bits of one register's magnitude, $b \leq 32$, and returns `T[|x| mod 2^b]`, an `out_bits`-wide value, from `2^b` rows of $\lceil \text{out_bits}/32 \rceil$ limbs. The

file holds a magnitude and a sign: a negative x reads the row of |x|, and x and −x read the same row. A register that can be negative is made never negative before it indexes a table. It is a general one-variable function on the lane's alphabet: 2^16 entries for a u16 lane, built once by pushing every value through. **Proved:** filled by running the ordinary operations over the whole 16-bit alphabet, the table equals them on 4,096 lanes, on the device and the host (M10).

- **Composition keeps the order.** Reading g through f gives the table (g∘f)[x] = g[f[x]], one table for the chain: the floor is 2-D, but the operations need not be, for a chain of them is collapsed in order onto it. Composition regroups, (h∘g)∘f = h∘(g∘f), but does not reorder: g∘f ≠ f∘g. **Proved** with f(x) = 65535 − x and g(y) = |y − 30000|. Regrouping: g read through f equals the ops for |35535 − x|, and one table composed on the host equals the two. No reordering: f∘g differs on some lane.

- **The two towers** (a second tower over the first one's boundary, inverted, and the two collapse together).

  - Floor division is a record floor: ⌊v / 2^k⌋ = EXACT_QUOTIENT(v − AND(v, 2^k − 1), 2^k), rounding toward −∞ for every integer v, as `tower_floor_shift` does.
  - With it, the crystal's 5/3 lifting T (A14) and its inverse are sums, differences, constants and that division: stacks of record floors. An operation program F over the crystal and T^{-1} compose into one stack F ∘ T^{-1} by the regrouping law.
  - **Proved:** one 5/3 level over 8 signed 16-bit samples and its inverse, 129 steps in one program and a 12-limb file. On 4,096 lanes the device equals the host word for word, the forward floor equals tower.cu's formulas, and the inverse returns every sample exactly (`test/engine/compiler/cycle/record_bitwise_test`).
  - Derived in vertical_time_compression.md:

    * The file holds live registers only. A field counts against it only while its register is live; the atom stays in the record.
    * keymath folds no constant, drops no dead step and merges no duplicate step.
    * A record is addressable to 2^32 bits for each member; past that, device memory bounds it.
    * A lifting cone spans (2^{L+2} − 3)^D samples, yet one level over a 4-D cone of 625 samples runs in a 15-limb file.
    * keymath's widths grow every register 1 bit a level, the first level 2, where the low-pass coefficients' values grow 0.585 bits and the high-pass coefficients' 1 (the linear forms of A16; E4). The constant-divisor narrowing is **proved** (`test/engine/compiler/cycle/record_divide_test`, 19 checks, 0 failed).
    * T is a bijection; an operation G between the towers runs as $T^{-1}GT$, and the levels above G cancel past it.
    * The heap (the bits a floor's values occupy) mirrors exactly across the crystal. The ring (the imprint's widths) does not, until a wrap at the mirror makes it symmetric

to one bit a register. The crystal with the ring's framing is a code length bounding K(x) from above, and by counting no lens shortens most lanes.

– Open: the neighbor gather in D axes (the halo in the record, or one axis per sweep), $F \circ T^{-1}$'s error in D axes, which operations commute with T, the heap and the wrap at the mirror as committed tests (both measured in scratch runs; **proved** by `test/engine/compiler/cycle/record_boundary_test`), the oval (Doug's call), and the whole crystal as one stack (vertical_time_compression.md).

– **Where the two towers stand in the engine.**

  * In the engine, T and T^{-1} are `tower.cu`'s own kernels (`tower_forward_kernel` and `tower_inverse_kernel`, behind `tower_lift` and `tower_lower`). The crystal path (`engine.cu`) and `root_universal` call them.

  * The tower module also emits them as record floors. `tower_record_lift` and `tower_record_lower` pass a ruleset (E4) through a block's extent into a caller's program, one block of the lattice to a lane. Each position's register goes in and comes out in t, z, y, x order, and T leaves the crystal where `tower_lift` leaves its coefficients. With no ruleset named, the ruleset is the kernels' 5/3. The steps go in at the caller's count; with no room given, the call only counts them, which sizes the program. A register that is not earlier, a room too small, an empty extent, and a step with a zero weight, no taps or a sign other than ±1 are each refused, and the program is left as it was. A program composes $F \circ T^{-1}$ by laying its own steps after T^{-1}'s registers.

  * **Proved** (`test/engine/compiler/cycle/record_tower_test`, 209 checks, 0 failed, a job on tessera). There are seven blocks: $1 \times 1 \times 1 \times 64$, $1 \times 4 \times 4 \times 4$, $2 \times 3 \times 4 \times 4$, $1 \times 3 \times 5 \times 7$, $3 \times 1 \times 2 \times 9$, $2 \times 2 \times 2 \times 2$ and $1 \times 1 \times 1 \times 1$. Each carries 256 volumes: all 0, all 65,535, the two alternating, a ramp, and drawn. On every block:

    · with no ruleset named, and with the 5/3 named as its two steps, T equals `tower_lift`'s crystal at every coefficient on 256 of 256 volumes, and T^{-1} run on it returns the source on 256 of 256;

    · a host oracle's 5/3 equals `tower_lift` and returns the source on 256 of 256; against that oracle, the S transform's T and the four-tap predict's T each equal it on 256 of 256, and their T^{-1} returns the source on 256 of 256;

    · for all four rulesets, $T \circ T^{-1}$ as one program returns 256 of 256 crystals drawn anywhere in the fields' widths;

    · $F \circ T^{-1}$ as one program, F the block's sum and its energy along x (the sum of the squared differences of x neighbors), equals F on `tower_lower`'s samples on 256 of 256;

    · every program runs on the device and the host, and the records agree word for word.

  * **Measured**, the steps and the file (limbs of live registers, with reuse), the 5/3's first:

block	T steps, file	widest coefficient	T^{-1} file	$T \circ T^{-1}$ file	S transform T steps, file	four-tap T steps, file, T^{-1} file
1 × 1 × 1 × 64	697, 70	23 bits	71	71	317, 67	1,015, 73, 80
1 × 4 × 4 × 4	1,192, 71	23 bits	75	139	542, 68	1,735, 74, 80
2 × 3 × 4 × 4	2,264, 103	24 bits	119	215	1,070, 100	3,207, 106, 142
1 × 3 × 5 × 7	2,216, 112	25 bits	136	234	1,108, 109	3,009, 115, 193
3 × 1 × 2 × 9	1,046, 61	24 bits	61	111	510, 58	1,449, 64, 74
2 × 2 × 2 × 2	356, 23	21 bits	23	43	162, 20	519, 26, 26

  $T \circ T^{-1}$ on 1 × 3 × 5 × 7, 105 samples, holds 234 of the file's 256 limbs: a block is one lane with its samples as fields, and the file bounds the block.
  * The record programs the engine ran before (the cell program's, Ω's reduct in `chaitin_omega`, π's BBP terms in `pi_tower`) read their own fields and use neither tower.
  * **Built:** the towers as kernels, and as record floors from any ruleset. **Not built:** $F \circ T^{-1}$ in the crystal path (`engine.cu` calls the kernels); the gather across blocks, since a lane holds one block; edges as record floors; and the kernels taking a ruleset.
  * Posits (theory, neither built nor tested): the scheduler is bypassed and the whole tower beamformed using its own inverse energy potential; and a two wave-vector magnitude interference pattern, with reinforced alignment in the ruleset and the exact no rounding as the destructive interference.

- **The two crystals** (the finite towers and their limit, with an infinite delta between them).
  - A register is a 2-adic integer, and a wrap to w is the projection $\mathbb{Z}_2 \to \mathbb{Z}/2\text{^}w$. Sum, difference, product, xor and and commute with every projection. An exact quotient by an odd c is the product by c^{-1} in $\mathbb{Z}_2$.
  - **Proved** (`test/engine/compiler/cycle/record_coherence_test`, 14 checks, 0 failed): 393,216 of 393,216 lane-widths agree three ways over 16 random programs, 73,728 of 73,728 for the odd divisors, and a quotient and a comparison break the agreement. Mod 3^5, 3^10, 5^4 and 7^3, 262,144 of 262,144 lane-moduli agree three ways over 16 ring-only programs, the CRT join with the 8-bit window equals the exact run on all of them, and xor breaks mod 3.
  - **Proved** (`test/engine/compiler/cycle/record_boundary_test`, 48 checks, 0 failed): the crystal of 64 samples at 4 levels as a boundary. T's reach is 3ℓ at the level-ℓ low-pass coefficients and $3\ell - 2$ at its high-pass coefficients and T^{-1}'s is L + 2, each met on the device; arbitrary crystals return through T^{-1} then T; constants and 2^{3L}$\mathbb{Z}$^n moves pass through T, negation and doubling do not; the heap mirrors, the ring is ring_0 + 6(n − n/2^ℓ); the quotient toward zero keeps order and a sum within one; T and T^{-1} keep Haar measure, counted; det M = ±1 exactly, its sign +1 in the run. T against 8 null shuffles a lane identifies each structured class past the null's 1/(d + 1) bound and noise within it, and the crystal is a one-to-one ID: a draw changes it exactly when it moves a value, and every single flipped bit changes the image.
  - Derived in two_crystals.md: what does not factor through the projection, the lifting as a homeomorphism of $\mathbb{Z}_2$^n whose level-L low-pass coefficients read the samples

to w + 3L bits, the windows' union $\mathbb{Z}$ and the projections' limit $\mathbb{Z}_2$ with the solenoid between them, what passes to a limit, the top projection's limit $\mathbb{R}$, the odd crystals and the places of $\mathbb{Q}$, the count as coset counting, and the posits bounded (the anchors' complexity open; the fingerprint of T against null permutations, proved; the knf's identity by spatial null permutation, built; the pairwise agreement between sections, open).

  - **Proved** (`knf_identity`, 23 checks, 0 failed): the knf of the nbody lattice (64^3, 177 frames) against spatial null permutations. The host's walk equals the engine's knf at every voxel, and a single flipped bit leaves it unchanged exactly where the window rule says (1,500 of 8,192): the knf is many-to-one. The 48 exact cube motions carry the knf and keep its entangled entropy E and its cloud. The whole is its tiles plus its seams at every tile size. Alike-voxel controls are identified within 5σ of 1/20. **Measured:** E is 44.6 times the free null's mean, the inside and between departure curves cross between tiles of 2 and 4 and sum to 0.590 at 4, and each body's box has its own curve. Written in two_crystals.md.

- **The full Kolmogorov–Arnold form.** With linear steps only, the machine represented the theorem's skeleton with linear edges; the table gives it general one-variable edges, which is item 1 of kolmogorov_arnold.md and the LUT of noise_sieve_tower.md §2. Which functions to tabulate, without fitting, is still open (item 3 there).

8.4.15 *A14. The root universal (M20): `tower`*

The tower and the table are separate objects; laid together they are one. Let W_k be the approximation on floor k, L_k level k's lifting, and E_k the edges laid on floor k.

- **The edge is a bijection.** E(w) = (w with its low b bits x replaced by π(x)), π a permutation of [0, 2^b), the high bits passed through. So E is a bijection on the coefficient word and E^{-1} uses π^{-1} (proved by construction; the round trips below are the check).

- **The order.** Lift is $E_N \circ L(N-1) \circ \ldots \circ E_1 \circ L_0 \circ E_0$, and lower is its inverse, $E_0^{-1} \circ L_0^{-1} \circ \ldots \circ E_N^{-1}$. The edges on one floor are a stack, undone last laid first. **Proved:** every edge program in `tower_edge_test` and `root_universal` rebuilds the exact lanes, 171 random edges on 48 volumes through the whole crystal path.

- **The fold.** Two edges on one floor are one edge: π_b ∘ π_a, laid as one table, lays the same crystal. It keeps the order, since π_a ∘ π_b lays another, and a lifting level between two edges blocks the fold. **Proved** on 8 volumes each.

- **What it is** (theory). The tower is the Kolmogorov–Arnold form's additive skeleton, the table its one-variable edge, and with edges between the floors the collapse is an outer sum with nonlinear edges.

- **Its two bounds** (theory; kept). It is universal over the finite coefficient alphabet: every function of the indexed bits is one table, exactly. It is not the continuous-function theorem

of Kolmogorov and Arnold. And inside the reversible stream every edge is a permutation, and the second bound is one of width, not of kind (below).

- **The dimensional expansion** (reaching a boundary means a dimensional expansion realizes the transform). A function f that is not a bijection (|x|, a comparison) rides the stream through the Bennett embedding, (x, y) → (x, y ⊕ f(x)): a permutation of the pair for any f, since y comes back as the carrier ⊕ f(x) (proved by construction). The price is width: a permutation takes b_x bits, any f takes b_x + b_y, within the 20-bit limit. No engine change; the tables are built on the host. **Proved** by `tower_edge_test`, 25 checks, 0 failed. |x| of an 8-bit two's complement field and x ≥ 100, each a 16-bit Bennett edge (8 bits of x, 8 of carrier) on the collapsed floor of 1 × 8 × 8 × 8, alternate over 64 lattices: they round-trip on 64 of 64; f(x) reads out of the carrier (x kept, the carrier moved by exactly f(x), the high bits passed) on 64 of 64; the edge touches only its one coefficient on 64 of 64. The two edges on floors 1 and 2, where lifting mixes their outputs, round-trip. |x| laid bare on 16 bits, not embedded, is refused, since x and −x collide. The quantum forms of the same expansion are standard theorems (theory here): Naimark (an ask is a projective measurement in a larger space), Stinespring (a transform is a unitary in a larger space, then a discard) and purification (a mixed state is part of a pure state in dimension d^2); the ask is A15.
- **The price** (measured, M20). An unfitted edge is exact but not free. On floor 0, where every coefficient is, a random 12-bit permutation raises the coded bits from 6.802 to 12.608 a voxel; on the collapsed floor, one coefficient, the cost does not show at three decimals at any width. The reading, which is not measured: a random permutation of the low bits sends small magnitudes to large ones, and the magnitude coder pays for each: the cost scales with the coefficients the edge touches. Whether a fitted or data-named edge lowers the bits, not only raises them, is not measured.

8.4.16 *A15. The ask and the state (M21):* `ask_state`

The terms are M21's. The idea is standard: QBism (Fuchs and Schack) and the SIC-POVMs.

- **The ask returns the state.** For a qubit and $E_i = (I + v_i \cdot \sigma)/4$, the answers are $p_i = (1 + r \cdot v_i)/4$. With $\Sigma\, v_i = 0$ and $\Sigma\, v_i\, v_i^T = c \cdot I$, $\Sigma\, p_i\, v_i = (c/4)\, r$, which gives $r = (4/c)\, \Sigma\, p_i\, v_i$ and the state is read back from the answers alone (theory; **proved** on 25 states for each ask, c = 1 and c = 4/3). A complete ask needs at least d^2 outcomes, here 4 (theory, standard).
- **The phase falls out of the ask.** The equator states differ only in phase: each answers ½, ½ to the 0/1 ask, which sees magnitudes only. The complete ask tells all 91 pairs of the 14 apart (**proved**). So the phase is not lost; one basis's ask cannot see it, and a complete ask holds it.
- **The crossing rule.** The chance of + on axis a is $(1 + r_a)/2 = \Sigma\, p_i\, (½ + (2/c)\, v_{i,a})$, a fixed linear map from the complete ask's answers (theory; **proved**, 75 of 75 for each ask). Its weights onto +x are 3/2, 3/2, −1/2, −1/2 for the rational ask and ½ ± ½√3 for the SIC. Classical total probability never carries a negative weight, and the difference is interference.

- **The valid set.** A distribution is a state exactly when $|r|^2 \leq 1$. (1, 0, 0, 0) gives $|r|^2 = 12$ for the rational ask and 9 for the SIC, and it is refused, and it crosses onto +x at 3/2 and ½ + ½√3, no probabilities (**proved**). On the 455 distributions with denominator 12 the valid set is 55 (rational) and 87 (SIC), and 104 and 72 more pass the three axis crossings without being states (**measured**): the valid set is stricter than the axis crossings.

- **Exact numbers.** Every value is exact: rationals on the exact integer, and the SIC's √3 carried by its relation, $(a + b\sqrt{3})(c + d\sqrt{3}) = (ac + 3bd) + (ad + bc)\sqrt{3}$, with the sign of $a + b\sqrt{3}$ read from a^2 against $3b^2$, never equal unless both are 0, since √3 is irrational (proved by construction). The SIC's answers cross into the rational ask's exactly, 25 of 25 (**proved**).

- **The dimensional expansion** (theory, standard theorems). Naimark: an ask is a projective measurement in a larger space. Stinespring: a transform is a unitary in a larger space, then a discard. Purification: a mixed state is part of a pure state in dimension d^2. The engine's form is the Bennett embedding of A14, proved.

- **The statements** (the first built in `ka_psi`, the rest theory). The exp is symbolic after the second nth exp: ψ's terms $\gamma^{\wedge} - \beta(r)$, with $\beta(r) = (n^r - 1)/(n - 1)$, are carried as a digit and an exact integer exponent, a sparse form whose cost is polynomial in the scale instead of the expanded digits. `ka_psi` carries ψ this way to β(60) = 1,152,921,504,606,846,975 (built; 152 checks, 0 failed). ψ is the inner function of Sprecher's version of the superposition theorem, as J. Braun and M. Griebel state it ("On a constructive proof of Kolmogorov's superposition theorem", Constructive Approximation 30(3) (2009) 653–675, doi:10.1007/s00365-009-9054-2, the record checked at Crossref; the preprint read, pages 1 to 5). Their Theorem 2.1 takes integers $n \geq 2$, $m \geq 2n$, $\gamma \geq m + 2$ and $a = [\gamma(\gamma - 1)]^{-1}$, and writes f(x) = Σ_{q=0}^{m} Φ_q ∘ ξ(x_q), with ξ(x_q) = Σ_{p=1}^{n} α_p ψ(x_p + qa), α_1 = 1 and α_p = Σ_{r≥1} $\gamma^{\wedge} - (p-1)\beta(r)$. ψ is defined on the terminating base-γ rationals D_k = { Σ_{r≤k} i_r $\gamma^{\wedge} - r$ } (their 2.2), and in Köppen's form (their 2.7) level k adds its digit at the weight $\gamma^{\wedge} - \beta(k)$. So $\gamma^{\wedge} - \beta(r)$ is the γ-adic weight of level r. What ψ is, strictly increasing and continuous in Köppen's form and not so in Sprecher's, is proved exactly by the sim `ka_psi` (M19), to depth 60 with the exponent never expanded, in "Kolmogorov's inner function, exactly" of kolmogorov_arnold.md. Its "Separation" part finds the paper's Lemma 3.4 false at k = 1 and restores Lemmas 3.7 and 3.8 at every k with a corrected bound and a ramp narrowed by γ^2. The outer functions' contraction is not built. A two-sqrt product represented as two ints and their words of math operations is countable: the numbers named by finitely many integers and a finite word of operations (the algebraic, the computable, π as its program) are countable, and the exact storage holds each of them. And π compresses to 1 kb: π as a spigot program.

8.4.17 *A16. The Gaussian step (M10, M19):* `test/engine/compiler/cycle/record_gaussian_test`

BBP's base is a power of a Gaussian prime (the shape of the algorithm dictates the geometric ruleset to enforce, and where the geometry looks absurd the algorithm is adjusted until it is fluid). Each rule carries one label: **cited** (a published or standard theorem), **derived** (exact algebra worked

here and checked in floating point by a scratch script; theory, not built), **proved** (the run named) or **open** (not known).

- **The ring and its prime** (cited, standard). $\mathbb{Z}[i] = \{a + bi : a, b \in \mathbb{Z}\}$ is Euclidean under the norm $N(a + bi) = a^2 + b^2$. $2 = -i(1 + i)^2$, and so $1 + i$ is the only prime over 2, ramified, with $N(1 + i) = 2$. The completion there is $\mathbb{Z}_2[i]$, with $1 + i$ its uniformizer: $v(1 + i) = 1$ and $v(2) = 2$.

- **Eight steps are sixteen** (derived; proved below). $(1 + i)^2 = 2i$, which gives $(1 + i)^8 = (2i)^4 = 16$. The powers for $k = 1$ to 8 are $1 + i$, $2i$, $-2 + 2i$, -4, $-4 - 4i$, $-8i$, $8 - 8i$ and 16.

- **The step turns and scales** (derived). $1 + i = \sqrt{2} \cdot e^{i\pi/4}$: $z \mapsto z(1 + i)$ turns z by $\pi/4$ and scales it by $\sqrt{2}$, and N doubles exactly. Eight steps are a whole turn and a scale of $16 = 2^4$. So BBP's digit shift, $\times 16$, one hex digit, is eight Gaussian steps.

- **The record floor** (derived; proved below). On $z = a + bi$ the step is $(a, b) \mapsto (a - b, a + b)$: one DIFFERENCE and one SUM, the matrix $[[1, -1], [1, 1]]$, determinant 2.

  - Its image is the pairs of equal parity: the ideal $(1 + i)$, of index 2. Each floor therefore drops one bit, and after k floors the image is $(1 + i)^k\,\mathbb{Z}[i]$, of index 2^k. v rises by 1 a floor and N doubles: the bit leaves the archimedean side for the $(1 + i)$-adic one.
  - The inverse floor is $((c + d)/2, (d - c)/2)$: two EXACT_QUOTIENTs by 2, exact on the image. A pair of mixed parity is exactly a lane the machine refuses (derived; proved below).
  - SUM and DIFFERENCE commute with every wrap (E1, proved by `test/engine/compiler/cycle/record_coherence_test`), and the floor runs exactly in any window. In a window of w bits each floor keeps an index-2 part of the pairs, and $2w$ floors send every pair to 0, since $(1 + i)^{2w} = (2i)^w \equiv 0 \bmod 2^w$ (derived).

- **The product formula over $\mathbb{Q}(i)$** (the formula cited, standard; the values derived). Take $|x|_{\mathbb{C}} = x \cdot \bar{x}$ and $|x|_{1+i} = 2^{-v(x)}$; every other place gives 1 on these values. Then $|16|_{\mathbb{C}} \cdot |16|_{1+i} = 256 \cdot 2^{-8} = 1$. A floor multiplies $|\cdot|_{\mathbb{C}}$ by 2 and $|\cdot|_{1+i}$ by ½, and the product holds at 1 on every floor: the floor moves a bit from one place to the other and loses none.

- **π from the step** (the series cited; the rest derived).

  - BBP (Bailey, Borwein and Plouffe [7], Math. Comp. 66, 1997): $\pi = \Sigma_k\, 16^{-k}(4/(8k + 1) - 2/(8k + 4) - 1/(8k + 5) - 1/(8k + 6))$.
  - Expanding $1/(1 - x^8)$ term by term gives $\Sigma_k\, 16^{-k}/(8k + j) = 2^{j/2} \int_0^{1/\sqrt{2}} x^{j-1}/(1 - x^8)\,dx$. So $\pi = \int_0^{1/\sqrt{2}} (4\sqrt{2} - 8x^3 - 4\sqrt{2}x^4 - 8x^5)/(1 - x^8)\,dx$, the integral their proof runs through. The upper limit is $1/\sqrt{2} = |1/(1 + i)|$, and $(1/\sqrt{2})^8 = 1/16$.
  - **Adjusted: four of the eight roots of unity are poles instead of all eight.** $1 - x^8$ vanishes at the eighth roots of unity, $e^{ik\pi/4} = ((1 + i)/\sqrt{2})^k$. The ruleset as first stated put a pole at each. The numerator is $-8(1 + x^2)(x - 1/\sqrt{2})(x^2 + \sqrt{2}x + 1)$, which cancels $\pm i$ and $e^{\pm 3i\pi/4}$. What remains is $8(1/\sqrt{2} - x)/((1 - x^2)(x^2 - \sqrt{2}x + 1))$, with poles at ± 1 and $e^{\pm i\pi/4} = (1 \pm i)/\sqrt{2}$ only, and it vanishes at the upper limit.

  - With y = √2x, the form BBP's proof reaches: π = ∫_0^1 16(y − 1)/((y² − 2)(y² − 2y + 2)) dy. y² − 2y + 2 = (y − ρ)(y − ρ̄) with ρ = 1 + i. The poles are the Gaussian prime, its conjugate, and ±√2 = ±|ρ|. By partial fractions the integrand is 2/(y − √2) + 2/(y + √2) − 2ρ/(y − ρ) − 2ρ̄/(y − ρ̄). The residue at the prime is −2ρ, and the four residues sum to 0.
  - Over [0, 1] the poles at ±√2 give −2 ln 2 and the real part at ρ gives +2 ln 2, and they cancel. What remains is 4·arctan 1 = 4·(π/4), and π/4 is the angle [0, 1] subtends at ρ, arg(−i) − arg(−1 − i). So π is four times one Gaussian step's turn, and BBP's logarithms, the step's scale (ln N(ρ) = ln 2), cancel exactly: the series keeps the turn and drops the scale.

- **The width, made fluid** (the keymath rule; built, and proved at every floor).
  - The superseded widening takes a SUM or DIFFERENCE to max + 1, with floor k at w + k bits, 25 to 32 at w = 24, and is not used.
  - Floor k's values are (a·x_k − b·y_k, a·y_k + b·x_k) with (1 + i)^k = x_k + i·y_k, and |x_k| + |y_k| = 2^{⌈k/2⌉}. So they grow half a bit a floor. The script's corner figures (25, 26, 27, 27, 28, 28, 28 and 28) are two's complement widths, sign included (checked by hand at floors 2, 3, 7 and 8). The register holds the magnitude with its sign beside it, and there floor k needs 24 + ⌈k/2⌉.
  - The cause: the old rule bounded each register alone. An odd floor's values fill a turned square, |c| + |d| ≤ 2^{⌈k/2⌉}·2^{w − 1}, and a per-register box keeps its bounding square, which the next floor turns into twice the reach. Over 8n floors the imprint grew 8n bits and the values 4n.
  - The rule built (`keymath.cu`): every register is a linear form over atoms, integer coefficients plus a constant. A sum or difference adds its operands' forms, a product by a constant scales the other's, a constant is its value, and a wrap that passes its register through keeps that register's form. Every other register is an atom. The width is the fewer of the operation's own rule and the form's bound, |x| ≤ |c| + Σ |c_i|(2^{b_i} − 1). A coefficient past 2^62 makes the register an atom. On the Gaussian floor the coefficients are the parts of (1 + i)^k: the bound is 2^{⌈k/2⌉}·2^w and the width is w + ⌈k/2⌉.
  - Read through Mathai and Thiang (arXiv:1505.05250): T-duality turns the bulk-boundary map ∂ into a restriction ι*. Here the width, a boundary quantity, is read by restricting the bulk form to its atoms' widths, not carried register by register (an analogy, not a theorem about keymath).
  - The wrap at w + ⌈k/2⌉ + 1 bits a floor is not needed.

- **Proved** (`test/engine/compiler/cycle/record_gaussian_test`, 16 checks, 0 failed, a job on tessera). The program is two signed 24-bit fields and eight floors of one DIFFERENCE and one SUM, 18 steps with every floor an output. It ran on 4,096 lanes: every pair of the five edge values (0, −1, 1, −2^23 and 2^23 − 1), and keyed draws for the rest. Its checks:
  - the program imprints, lays and loads, and floor k is 24 + ⌈k/2⌉ bits wide: 25, 25, 26, 26, 27, 27, 28 and 28;

  - some lane fills every floor's width, |value| ≥ 2^{bits − 1}, and no width is loose;
  - the device equals the host word for word;
  - every floor equals z(1 + i)^k, taken on the CPU from the powers above;
  - floor 4 is −4z and floor 8 is 16z on every lane;
  - each floor's two registers share their parity;
  - floor 8's values lie in [−2^27, 2^27), 28 bits, and reach −2^27;
  - eight inverse floors take 16z back through z(1 + i)^{8−k} to z on every lane, in the host's records and the device's alike;
  - the chosen pairs w(1 + i)^k, for k from 0 to 9 with six drawn w of valuation 0, and then 0, have valuation k on the CPU;
  - through 1 to 8 inverse floors the host and the device agree on every run, j inverse floors refuse a pair exactly where (1 + i)^j does not divide it, and where they run each level is z/(1 + i)^k, the Gaussian division on the CPU. The run printed 272 runs and 216 refused. Derived: 61 pairs through 8 floor counts are 488 runs; a pair of valuation k runs the j ≤ min(k, 8) of them, 6·(0 + 1 + … + 8 + 8) + 8 = 272, and 488 − 272 = 216;
  - tessera admits the job and it releases.

- **The rule across the other record tests** (each a job on tessera, run one at a time): guide 12, table 16, divide 20, bitwise 25, coherence 15 and order 18 checks, 0 failed. The boundary ring law is ring_ℓ = ring_0 + n + 2n(1 − 2^{−ℓ}) in E4's widths, and boundary is 49 checks, 0 failed.

- **Where it stands in the engine.** The width rule is in `keymath.cu`, and every record program takes it. The Gaussian step itself is built only in the test. `pi_tower` runs 16^{d−i} mod m a nibble at a time (E13) and does not use the step.

- **The bridge** (open).

  - The digit at d is {16^d π} = {$4S_1 - 2S_4 - S_5 - S_6$}, where S_j = Σ_{i≤d} (16^{d−i} mod m_i)/m_i plus a tail, with m_i = 8i + j.
  - Each term's power reduces by Carmichael's λ (cited, standard): on m_i's odd part, the exponent is taken mod λ; on its power of 2, the term is 0 once 4(d − i) reaches that power (derived). Still, d + 1 terms remain.
  - The Gaussian form does not shorten them. 16 is a rational integer, and (1 + i)^{8(d−i)} mod m_i in ℤ[i]/(m_i) is 16^{d−i} mod m_i (derived).
  - What would close it is a sum over i taken at once: a modular form or Hecke eigenvalue whose coefficient is Σ_i r_i/m_i mod 1. The Hecke recursion, T_{p^{n+1}} = T_p·T_{p^n} − p^{k−1}·T_{p^{n−1}} on level-1 forms of weight k (cited, standard), makes a coefficient at 2^{4d} a power of a 2 × 2 matrix, about log d steps (derived).
  - No form whose coefficients give BBP's sum is known. No method is known to us that finds π's hex digit at d in fewer than about d steps (open).

8.4.18 *A17. Exact qubit states (M22): `qasm`*

The module is the exact qubit states in C at `examples/qasm/`, and each reading below is checked by the qasm tests, `examples/qasm/test/run.sh` (39 checks, 0 failed, on the host). Two of the checks are regression checks: the bond at the widest site index reads 0, and w^(bits − 1) at w = 2 evaluates to 2^(bits − 1).

- **The field.** Q(√2)[i] is the numbers a + b√2 with a and b Gaussian rationals. H's 1/√2 is √2/2, and the T phase e^{iπ/4} is (1 + i)·√2/2. A circuit over X, Y, Z, S, H, CNOT, CZ and the controlled T phase keeps every amplitude in the field. Each rational sits on the exact integer (`arithmetic/no_rounding`, 128 limbs by default), and a value past that width is refused, never wrapped (built).
- **The chain.** Each qubit is a site tensor. After a two-site gate the pair's matrix M is factored as M = C·F by Gaussian elimination, and the bond between them is M's exact rank; nothing is truncated. The same code runs over Q(√2)[i] and over `Q(√2)[i](ω)`. The readings are in M22 (**proved** for those circuits).
- **The symbolic phase.** `Q(√2)[i](ω)` holds rational functions in ω, each in rat_make's canonical form, and evaluates them at a number. Readings (**proved**):
  - Delta-Bell: bond [2]; ⟨00⟩ = "1/2sqrt2", ⟨11⟩ = "(1/2sqrt2)w", ⟨ψ|ψ⟩ = 1.
  - ⟨X0X1⟩ = "(1/2 + (1/2)w^2) / (w)", ⟨Z0Z1⟩ = 1, ⟨X0⟩ = 0.
  - Delta-GHZ3: bonds [2,2], and ⟨X0X1X2⟩ is the same function.
  - |+++⟩ with a phase on wire 0 keeps bonds [1,1].
  - At ω = e^{iπ/4}, ⟨X0X1⟩ equals the numeric controlled-T reading, "1/2sqrt2", and the norm is exactly 1.
- **The boundary lens.** ⟨ψ|O|ψ⟩ is contracted from both edges of the chain inward. `qasm_lens_hash.c` and `qasm_lens_rank.c` compute the reduced-round SHA-256 wedge-field invariants: rank, complement, degree-2 lift rank, curvature vanish counts, fibers and clock closure. At mid 13, p 13, q 8, 6 bits and seed 2024, rounds 16 to 30 each read raw rank 126, complement 130, lift extra 0, vanish 30 and 14 of 400, and fibers 64 and 64; the clock closed and the root was stable (**proved** at those arguments).
- **The witness.** The root is the engine's seal (`obsignatio_seal`). The checks are that a seal repeats, that one rational moved by 1/10^9 changes the dense state's seal, and that one flipped generator bit changes the lens's (**proved**).
- **Refusals.** Each is a request error from qasm: a gate past the state, a division by zero, a two-site gate on the last site, inverting the zero function, and an aperture past 20 bits (**proved**).
- **The device code** (M22) is built, on the engine's fully vectorized self run: record floors of integer steps, four parts an amplitude, one `cycle_record_run` launch over every lane, host and device records equal word for word. A tessera job: qasm_test 86 passed, 0 failed; qasm exact test 39 checks, 0 failed; qasm self test 44 checks, 0 failed; the CLI on bell.qasm exit 3, bitstring 00, and on bernstein_vazirani.qasm exit 0, bitstring 101101001101.

- **Not built into the engine library.** qasm in the engine library build: `src/build_engine.sh` (MODULES and INGEST) and `src/cu/CMakeLists.txt` name no qasm source. The QASM ingester is `qasm_read.c` and `qasm_lexer.c`.

8.4.19 *A18. The ask (M24): `query_ask`, `gate_descent`, `ask_order`, `interface_sass_probe`*

The algebra of each step and the run behind it are P1 to P8 of query_protocol_table.md. What is settled:

- **The ask holds on the host and in the cell** (**proved**, Q1). `query_ask` finds memory the test owns holding and leaves it as it was, reads a written word as that word and no other, reads a word nothing writes as not advancing, and reads a bound with no clock as past the bound. `query_interface_walk` keeps an ending as the answer of the address that caused it.
- **The gate is a conjunction** (**proved**, Q4). The descent's survivors equal every candidate asked every case, and both equal what `chain_build` keeps with its sweep: add 1,202 of 1,227, take 1,047 of 1,083, up 411 of 698, down 408 of 559, same 0 of 8, 0 lost.
- **The order and its solve** (**proved**, Q5). `4·Sᵀ − 2·J` times `S` is `(n + 1)` times the identity entry by entry, every ask covers `(n + 1)/2` links and any two share `(n + 1)/4`, and integer link costs solve back exactly at 1 to 8 passes. Against one ask a link the gain is $\sqrt{(n+1)}/2$ (**measured** 1.09, 2.07 and 8.16 at 3, 15 and 255 links).
- **Contention** (**measured**, Q7): found on 3.0% with none, 83.6% with up to 80 a pair and 100% with up to 200, over 15 links and 45 sweep asks.
- **One ask, many answers** (**measured**, Q15). One bit a check takes the fewest asks on every relation. Past a failing share of $(3 - \sqrt{5})/2$ asking one check at a time costs no more than any scheme that asks a set as one bit (cited from knowledge: P. Ungar, "The cutoff point for group testing", Comm. Pure Appl. Math. 13 (1960) 49–54).
- **A form asked of the part** (**proved**, Q16). On sm_86, 8 of the 2927 forms in `loop_back_if`'s place answer every count, every one a `BRA`, and `sass_loop_walk` agrees with all 8. The next above the least costs 0.0525 ns a turn more against a spread of 0.9260 (**measured**): no form is cheaper, and `sass.krs` keeps its `BRA`.

8.5 THE SCALE AUDIT: EVERY NUMBER IN THE MACHINE THAT FIXES A SCALE

A number in this table is a debt unless it is a word's width or a format's specification. Each is to come from the request or the data, or to be measured and reported, never tuned to a program.

number	where	what it fixes	why it is a debt
the even-order guard	`residual.cu`, `keymath.cu`, `unit_sweep.cu`	refuses every odd order	stricter than the algebra (A1); ruling (a) replaces it

number	where	what it fixes	why it is a debt
`ENGINE_RESIDUAL_LIMBS` 9	engine_config.h	the residual's width: 16 input bits plus the orders' growth	sized for one program's orders; the width should follow the request's orders (each step adds a bit). The planes path derives it (input bits + Σ orders + 1) and returns it (M2); the 16-bit path and the key still use 9
`UNIT_SWEEP_INPUT_BITS` 16, unsigned 8 and 16 bit input only	unit_sweep.cu, `engine_source_read`	the input's depth	a source deeper or signed needs a representation, not a refusal (the RSNA ruling: signed stored + 2^15 into the u16 lane, Doug to confirm). The unit sweep takes planes of any declared width (M2); its volume path and `engine_source_read` stay at 16
`ENGINE_AXES` 3	engine_config.h	three spatial axes	a word of the problem, but the residual, the tree and C are separable per axis
`ENGINE_HISTORY_WINDOW` 11, `ENGINE_HISTORY_WINDOWS_MAX` 16	engine_config.h	the entropy window and how many fit	a window is a scale; it belongs in the request
`ENGINE_COEFFICIENT_LIMIT` 2^30	engine_config.h, read by tower	the largest lifted coefficient	a word's width; an overflow is a request error, not a silent clip
`COMPRESSION_BLOCK` 64, 64 blocks a chunk, `COMPRESSION_K_BITS` 5, `COMPRESSION_ESCAPE` 24	compression.cu	the Rice coder's block and chunk	chosen; measure them against the data, or let the file carry them
`CRC_SEGMENT_PIXELS` 64	crc.h	the CRC's parallel segment	no longer read by the tower (the seal replaced its CRC); crc.h goes with stage B
free / 3 · 2	schedule.cu	the share of free device memory the schedule plans against	chosen (flagged in the RSNA work); ingest, prove and flatten consult no free memory at all; tessera arbitrates the device between processes (M14)
extents ≤ 2^20 and lanes ≤ 2^40	krep history read, `krep_lanes`, `entry_lanes`	the largest sample	chosen ceilings; the words hold more
`MARGINAL_ARRANGEMENTS_MOST` 2^16, `MARGINAL_TARGETS_MOST` 64, `MARGINAL_LIMBS` 16	marginal	the largest local set	OrganoidTracker's bound and not measured here; A5 says report the component's size instead
`SHIFT_AGREEMENT_LONGEST_AXIS` 2^23	shift_agreement	the NTT's longest padded axis	the prime's transform length; a word's width, but a larger view needs a second prime, not a refusal

number	where	what it fixes	why it is a debt
ENGINE_RECORD_LIMBS_MOST 256, CYCLE_RECORD_BLOCKS_MOST 4096, ENGINE_RECORD_MEMBERS_MAX 3	engine_config.h, cycle.h	the record machine's reach; 256 limbs of live registers is the binding resource (A13)	not measured
ENGINE_RECORD_WRAP_BITS_LEAST 4	engine_config.h	the narrowest two's complement wrap, a nibble (A13)	a floor the request sets instead of a resource; there is no step cap, since each sign sits beside its register
ENGINE_RECORD_TABLE_INDEX_BITS_MOST 32	engine_config.h	a table's index, the low bits of one register (A13)	a word's width: the index fits one limb
TOWER_EDGE_INDEX_BITS_MOST 20	tower.h	a tower edge's widest index, 2^20 entries of 4 bytes, 4 MiB a table (A14)	not measured
BODY_OVERLAP_CHUNK 256, CLIMB_MACHINE_BUNDLE 32, MAX_TREE_BUCKETS 256, UNIT_SWEEP_SHARED_BYTES_MOST 48 KiB	kernels	device grains	the device's, not the data's; measure per device
DEVICE_POOL_SLICE_ALIGN 256 bytes, DEVICE_POOL_PAGE_BYTES 2 MiB	device_pool.h	where a pool's slices sit, and the grain a pool is rounded up to (M9)	the device's allocation grain, not the data's: on this machine four allocations cost 14,680,064 bytes where one pool of the same buffers cost 10,485,760; measure per device
STACK_UNCACHED_BLOCK 64 MiB, NPY_FORTRAN_BLOCK 16 MiB	stack, npy	IO grains	size by experiment to the disk's throughput (build plan 22); there is no header-room cap: the npy header is bounded by its own length field and the file, and the nrrd header is read in doubling pieces until its blank line
flatten's slab, half the free device memory	flatten	frames per residual pass	measured 107.0 s against 9.0 s one frame at a time; a fraction of memory is not a scale of the problem
SCRIPTURA_DIRECT_BYTES 32	scriptura.c	below it a memory call skips the arm table for portable	the portable short path's reach (four words), not the data's; measured only on this part
SCRIPTURA_AVX2_STRING_BYTES 16 KiB	scriptura_arm_avx2.c	where copy and fill switch to `rep movsb`/`stosb`	the part's, not the data's: the measured crossover on a Haswell-E with ERMS (4 KiB lost 1.28×, 16 KiB and up at 1.00× the runtime); other parts move it, and each part is measured on its own

number	where	what it fixes	why it is a debt
`PERIOD_THREADS` 256, `PERIOD_GRID_ROWS_MOST` 65,535	period.cu	a block's width and the grid's row limit	the device's, not the data's: the columns are read from the device (multiprocessors × resident blocks), and the lags beyond the row limit are walked by stride
`PERIOD_ROUNDS` 4	period.cu	the keys of the line shuffle's hash in each draw of the null (A12; the Feistel shuffle's rounds before it)	a mixing depth, not a scale of the data
fewer than 2^32 voxels a reading	period.cu	the largest lattice a period is read on	a word's width: N^2 and Σc^2 must fit 64 bits; a larger lattice needs wider counts, not a refusal
`ORIOR_EXACT_LIMBS` 256 in the cell_tracking build (the default is 128)	`cell_tracking/build_engine.sh` and `build_driver.sh`, fitted from `ENGINE_RECORD_LIMBS_MOST` by a define, with no change to the source	8,192 bits, 1,024 digits	a word's width, derived and not chosen; `decimal_double` reports `needed_bits` past it

There is no npy zip directory cap (64 MiB).

8.6 WHAT THE ENGINE OFFERS PROGRAMS, AND WHAT IT ASKS

the engine offers	through
exact numbers from decimal strings, with `needed_bits`	M1
the residual at any orders and any input width the request names, its width returned	M2
the component tree at every level: cuts, probe partitions, the bipartite census over probe links	M3
moments per component	M4
one correlation C and its reductions	M5
the exact marginal and one-to-one matching on the caller's integer weights	M6, M7
the entropy history and its cloud	M8
proved files, and every source format read exactly	M9
a seal on every crystal and set, and the place of any mismatch	M13
one device shared between processes by memory, each job measured by pid	M14
the sift: a pattern found by anchors whose count the corpus's coherence sets	M15
a step's state drawn as a sheet or a volume, host and device alike	M16
the record machine for the program's own verdicts	M10
reversible lookup edges between the tower's floors, the root universal	M20
the root noise of a box, the shared term whose yank saves the crystal the most bits, and the box back from the yank exactly	E16
a stateful error on every failure	M12

the engine offers	through
length, find, copy, move, compare and fill at each instruction set's speed	M17

What the engine asks of a program: every scale (orders, windows, widths, spacings, lags, probes) in the request, and nothing about what the program's bodies are. Programs on it: cell tracking (cell_tracking_table.md) and RSNA knee MRI.

8.7 WHAT TO WORK TOWARD, IN ORDER

From the build plan (build_plan.md):

1. **The seal** (M13): stage B (history, bodies, flattened, key, schedule), then the universal root and its trust anchor. The RSNA converter is done (M9); the export follows.
2. **Errors in every module** (M12), and its two fail-closed defects.
3. **The scale audit**, row by row: the entropy window into the request, ruling (a) at the three guards, the residual's width from the orders, the marginal's limits reported instead of set.
4. **Orders from the per-axis coherence reading** (M2), now built as M18 (`period_read`, A12), and orders come from the data (order = period); the knee pulls it.
5. **The compression floor, by noise functionals** (M8, M9, M5) is its own theory: its plan and order of work are the compression workbook's, compression_table.md.
6. **The algebra as passes**: the ladder-keeping sweep (A1), the LCA probe partition and the DFS-range overlap (A2), moments at every level (A3), C as the one pass (A4), the marginal over the whole component (A5).
7. **The port into orior** (close the gaps before anything moves). These gaps stand:
   - (a) The layout is pinned to a commit hash before they re-check it.
   - (b) Our `arithmetic/no_rounding` arms, `render` and `nbody/orior` carry their full doc text and are byte-identical to their files (their code is identical). Our `exact_integer.{c,h}` is ahead of theirs: it has the open width, the multiplication ladder, the division and Newton's division (M1), and theirs multiplies by long multiplication alone and has no division. This tree is the source their engine is to be copied from (orior). The root `bench/` still differs from theirs in every file.
   - (c) `build_engine.sh` and `build_driver.sh` compile the exact integer from this tree's `arithmetic/no_rounding` by default, and `ORIOR_EXACT_ROOT` only overrides, and moving their `no_rounding` does not break our build.
   - (d) `engine/CMakeLists.txt`: every path is this tree's, the C-only warnings are kept off the CUDA sources, the CUDA arms join where a CUDA compiler is found, and `bench_lattice` is built only where the compiler has C99 `_Complex`. It builds and runs every bench on four configurations (M15).

- (e) Their C builds host-only (CMake, MinGW, a Pi 5, WSL), but obsignatio, tessera and most modules are `.cu` only. Each module needs a host-only path, or CUDA optional per module.
- (f) Their `src/engine/n_body` holds older copies of modules we have (and some we retired). One definition, and the old copies retired in the same change.
- (g) The sift and the exact integer are outside M12's stateful errors: wrap at the boundary or wire them.
- (h) The `representation` package, the Python model of games, particles, sound, text and proteins, sits in the engine at `src/engine/python/representation/` (game/, particle/, atom/, constants/, structure/, sound/, exact.py). Proofs under `evidence/proofs/` and examples under `examples/` (any_corpus, art, 0_experimental, chemistry, crystallography) import it. The engine is not clear of domain code while `representation/game/chess.py` sits under it.
- (i) `src/cu/CMakeLists.txt` sets `ORIOR_ROOT` to the parent of `src/engine`, which is `src/`, one level short of the repository's root. The four tests it guards lie under the root's `test/engine/`, and its `if(EXISTS ...)` blocks skip `test_steer`, `test_adversarial`, `test_arm_agreement` and `test_o2_spawn` without a word. The bench targets name `../bench/`, a path under `src/`, and the tree's `bench/` sits at the root. The fix is the root two levels up from `src/engine`. Measured with the fix: a configure, build and grade as a tessera job, 890.067 s, exit 0, "[+] all graders passed", with `test_steer`, `test_adversarial` and `test_arm_agreement` among the graders. `maint/engine/build_engine.sh` neither builds nor runs `test_o2_spawn`: its targets and its graders leave it out, and the fix only makes it a target. Built by name as a tessera job, it passes: "0 case(s) failed", exit 0, 2.286 s.
  - **The descent**, in `test/engine/nbody/orior/test_o2_spawn.c`. No $\log_2$ N floor applies to isolating one of N alignments: the case isolates its 2,029 in 9 and prints so. It is given the needle, reads every survivor against the needle's byte at each candidate offset (`steer_truthy_after`, `src/cu/engine/nbody/orior/orior_steer.c`), and places the offset leaving the fewest standing, a tie going to the earlier candidate. Its probes are not answers to yes or no questions about an unknown target. They form a certificate, a set of offsets at which every other alignment differs from the needle, and no general bound above one holds for its size. The comments make no bound claim. **Prior art, cited from knowledge.** A smallest certificate is a smallest set cover: each offset covers the alignments that differ from the needle there. Placing the offset that prunes most is the greedy set cover algorithm, which finds a cover within a factor $H(N - 1) \leq 1 + \ln(N - 1)$ of the smallest cover drawn from the same offsets (Johnson 1974; Lovász 1975; Chvátal 1979). The cost case counts the survivors entering each placed level: the pruning field is 3,925 over 9 levels, against 18,261 for a re-scan, and the barely pruning field 18,514 over 20 levels, against 40,580. 3,925 is 1.93 times the 2,029-alignment universe (Derived), matching the case's own note "near twice the universe". Choosing each level's probe also reads its survivors once for each

candidate offset, up to 20. The composition and the re-scan are counted per alignment on the same footing, and neither count is of bytes read.

- Their proposed order: new modules land as siblings under their `src/engine/c`; the nbody modules then replace their `n_body` copies; their own base/nbody/sims split comes last. Since gap (c) closed, that split no longer has to land with a change to our build.

8. **The two towers in the engine.** T and T^{-1} emitted as record floors by the engine itself, not only by the tests, and a program then runs $F \circ T^{-1}$ on the crystal as one stack, checked against `tower.cu`'s kernels lane for lane (A13; vertical_time_compression.md). **Built and proved**: `tower_record_lift` and `tower_record_lower` focus each floor from a ruleset (E4), and `test/engine/compiler/cycle/record_tower_test` has 209 checks, 0 failed (A13), 210 since it reads the stack limit (M10). Still open: $F \circ T^{-1}$ in the crystal path, the gather across blocks, edges as record floors, and the kernels taking a ruleset.

9. **The record machine as a compiler** (the kernel is a compiler). Its compiled programs are named automata: finite, and of fixed width, and an automaton can be part of a UTM program's toolbox. A UTM program then sits one level up, with storage that grows between runs and automata from the toolbox as its steps (theory). Once the automata are emitted, the stack return (M10) is dropped. **Built and measured** (`src/cu/engine/analysis/cycle/`, the record program compiled):

   - **The automaton.** At load, a program's steps become CUDA source. Each step's value is a local array of its own exact limbs, and each operation is a template over those limbs, the interpreter's own arithmetic with the widths fixed. NVRTC, loaded at run time (`nvrtc64_130_0.dll`, `libnvrtc.so.13`), compiles it to a cubin for the device's architecture. The cubin is loaded as a library and launched in the interpreter's place, and the build links nothing new.
   - **Kept.** A process finds a program again by its whole source. Across processes, a cache file holds the source and its cubin (`$CYCLE_CACHE`, else `%LOCALAPPDATA%\cycle` or `~/.cache/cycle`), and it is used only where its source is this source, byte for byte. A compile took 72 to 1,604 ms, and a cache read 0.4 to 1.4 ms.
   - **The interpreter stays**, as the oracle and the fallback. A program of more than 1,024 steps, or one NVRTC refuses, runs on it. Past 4,096 unrolled limb operations a program's loops are left rolled. The cap came from the runs: the bitwise test's 4,204-step stack kept NVRTC past 1.2 GB, and the order test's 12,293-step program past 13 GB, before each was stopped.
   - **Switches.** `CYCLE_RECORD_INTERPRET=1` keeps every program on the interpreter. `CYCLE_RECORD_CHECK=1` runs both on every launch and refuses the launch where their records or refusals differ (`ENGINE_ERROR_LOGIC`). `CYCLE_RECORD_REPORT=1` says on stderr where each program came from and how long it ran. `cycle_record_compiled` says which ran.
   - **Proved.** The ten record tests, each run compiled, then checked, then on the interpreter: 30 runs, 0 failed (speed 12, bitwise 25, boundary 49, coherence 15, table 16, divide 20,

order 18, tower 210, gaussian 11, guide 12 checks). The check compared both machines' records and refusals on 58 launches, and every one was the same.

- **Measured** (`test/engine/compiler/cycle/record_speed_test`, 4,194,304 lanes, the best of 5 runs timed on the host around `cycle_record_run`, RTX 3070):

program	interpreted	compiled	times faster
chain: the Gaussian step's 8 floors, 18 steps	3.818 ms	0.272 ms	14.0
powers: x to x^8 and their sum, 15 steps, products to 8 limbs	33.307 ms	1.920 ms	17.3
mix: 6 rounds of xor, product, sum and wrap, then the ladder, 32 steps	6.010 ms	0.500 ms	12.0

- **The divisions compiled.** Quotient, remainder, gcd and exact quotient are the interpreter's own Knuth D, Euclid on it, and multiply and mask. Each holds its working values at its operands' own limbs, where the interpreter holds them at the register file's. The five dividing tests, each run compiled, then checked, then on the interpreter: 15 runs, 0 failed (divide 20, coherence 15, bitwise 25, boundary 49, tower 210 checks). The check compared 101 launches, refusals included, and every one was the same. The kernels' own time, summed over the checked launches: the tower's 37, 11.518 ms compiled against 401.126 ms interpreted; the boundary's 14, 17.155 ms against 471.493 ms. The divide test's 4,096 lanes of 160-bit numerators took 0.926 ms compiled against 9.553 ms, and its 512 lanes of 2,048-bit numerators 27.634 ms against 99.652 ms and 16.607 ms against 22.055 ms. The cost is the compile. The tower's dividing programs of 761 steps took 66.9 s and 69.5 s, and its 1,012-step program 122.9 s for 328,744 bytes of cubin, paid once a program and then read from the cache.
- **Where a division runs.** Inlined into every step, the divisions made the compile the cost. Called, one shared body a width, they halved the compile and slowed the tower's kernels. Each way was built by nvcc for the record tests' 52 dividing programs into a cache folder of its own, and the tests ran checked against each folder, three ways back to back and twice. The interpreter's own time on every run was the control: about 400 ms for the tower, 490 ms for the boundary, 160 ms for the divide test and 213 ms for the speed test. Every launch was the same on both machines. The kernels' time summed over each test's checked launches, first run and second:

the divisions	nvcc, 52 programs	tower, 37 launches	boundary, 14	divide, 8	speed, 15 (no division)
inlined	1,541 s	10.8, 11.3 ms	17.0, 17.0 ms	45.0, 45.8 ms	12.9, 13.2 ms
called	864 s	61.7, 64.3 ms	124.8, 111.9 ms	36.7, 37.3 ms	12.8, 12.7 ms
called, operands copied first	not summed	103.1, 99.4 ms	106.6, 101.7 ms	39.5, 40.0 ms	12.7, 12.3 ms

2,897 of the 2,904 division sites in those programs divide by one limb, and a call saves every live register around it. Copying the operands out kept the steps' own arrays in registers and was slower still. nvcc's fast compile (`--Ofast-compile=mid`) built the 1,012-step program in 21.7 s, and its tower launches took 241.0 ms in one run (the interpreter's 409.7 ms).

- **Built: a divisor of one limb inlined.** It runs the interpreter's own one-limb path, a 64-bit division a numerator limb from the top, known at compile time (`cycle_divide_limb`, and `cycle_exact_quotient_limb`, which refuses a rest or a quotient past its limbs). Any wider divisor stays the one call (`cycle_divide_long`, `cycle_exact_quotient_inverse`, `cycle_gcd`). The ten record tests, each run compiled, then checked, then on the interpreter: 30 runs, 0 failed, and the check compared 128 launches, every one the same. Each test's NVRTC compiles and its checked launches summed, against the runs of the inlined and the called builds, with the interpreter's time on the same run:

build	tower: compiles, kernels (interpreter)	boundary	divide
inlined	706.9 s, 11.518 ms (401.126 ms)	71.1 s, 17.155 ms (471.493 ms)	20.6 s, 45.710 ms (161.301 ms)
called	346.7 s, 49.179 ms (311.450 ms)	35.2 s, 105.951 ms (513.240 ms)	23.9 s, 39.030 ms (162.370 ms)
one limb inlined	462.3 s, 14.782 ms (402.223 ms)	44.1 s, 17.719 ms (462.862 ms)	23.8 s, 40.696 ms (156.173 ms)

- **The compile grows with about the square of the steps.** The 1,012-step program cut to its first 128, 256, 512, 768 and 1,012 steps, each step's value kept live by one xor: cicc, NVRTC's front end and optimizer, took 2.2, 4.3, 18.1, 40.2 and 97.2 s, and 1.7, 3.2, 10.2, 20.5 and 52.4 s with its optimizer off. On the 381-step program ptxas took 0.6 s against cicc's 7.5 s. The cost follows the kernel's length, with or without the optimizer, and the loop below answers it where no flag does.
- **No step cap.** A program of any length compiles. The interpreter runs a program only where one of its steps is not supported by the compiler, where NVRTC or nvJitLink cannot be loaded, or under `CYCLE_RECORD_INTERPRET=1`.
- **The operator block** (every program's operators split into one file that all programs access, for a faster and smaller build). Every record operation is one function in one module: the interpreter's own arithmetic, with its widths taken as arguments. NVRTC compiles the module once for the device as relocatable code, and the cache keeps it (1.4 s, 164,864 bytes of cubin). A program's source is its steps alone, each one call into the block with its places and widths as constants. NVRTC compiles that as relocatable code, and nvJitLink links it against the block (loaded at run time, `nvJitLink_130_0.dll`, `libnvJitLink.so.13`). The register file and the signs lie in the program's local frame, and each operator reads its operands and writes its register there by address. The divisions and the ladder share one scratch, laid at the program's widest division.
- **The compile, with and without the block** (one program each):

build	program	compile	link	cubin
stage 1, the file in registers	tower, 761 steps	37.9 s	none	133,288 bytes
stage 1, the file in registers	tower, 1,012 steps	96.4 s	none	231,976 bytes
stage 1, loops rolled	tower, 2,094 steps	778.7 s	none	1,450,024 bytes

build	program	compile	link	cubin
the operator block	coherence, 164 steps	0.17 to 0.22 s	0.7 to 1.3 ms	91,360 to 96,480 bytes
the operator block	boundary, 968 steps	1.29 s	2.7 ms	366,560 bytes
the operator block	boundary, 1,992 steps	5.11 s	4.1 ms	731,744 bytes
the operator block	bitwise, 4,204 steps	9.67 s	5.4 ms	1,201,888 bytes
the operator block	order, 12,293 steps	217.5 s	27.1 ms	4,592,480 bytes
the operator block	order, 16,379 steps	209.5 s	24.9 ms	4,747,616 bytes

The bitwise stack of 4,204 steps and the order test's 12,293 steps both compile. Past about 4,000 steps the compile grows faster than the steps, and why is not yet measured.

- **Proved on the block.** Each test run compiled, then checked against the interpreter, then on the interpreter. Every checked launch had the same records and refusals on both machines. The kernels' time summed over each test's checked launches, beside the interpreter's on the same run:

test	checks	checked launches	compiled	interpreted
bitwise	25	6	8.566 ms	21.748 ms
boundary	49	16	94.854 ms	438.492 ms
coherence	15	37	14.459 ms	143.879 ms
divide	20	8	42.628 ms	169.002 ms
gaussian	11	1	0.264 ms	2.138 ms
guide	12	1	0.330 ms	2.357 ms
order	18	2	19.827 ms	87.672 ms

The cost is the call. The register file lies in memory and the operators read it by address. The boundary's kernels ran 4.6 times the interpreter's speed, against 26 times when the file was in registers (17.719 ms against 462.862 ms, the one-limb-inlined build above). The speed, table and tower tests, and the block built as LTO-IR and linked with link-time optimization (`CYCLE_RECORD_LTO=1`), are running.

10. **Resident programs.** Automata that run on the device on their own. Each reports its status to its own block and loops, talks to other programs through their blocks, and can be chained to another program or pointed at one. Built from these, a solution is small autonomous kernels composed on the GPU, with the host out of the loop.

    - **The block** (`EngineProgramBlock`, `engine_config.h`). Its core: the offset (the execaddr, where the program resumes), the step (the evacaddr, where it left), the span (the register map), and the time to live, the watchdog, the launch time, the run time and the exec time. Beside those it holds:
      - the program's signum, the run's generation, and the launch that owns it;
      - the scheduler's command (run, yield, stop) and the grant: registers a thread, threads, device bytes (the local frame times the threads) and shared bytes (`engine_config.h`);
      - the state, the exits, and the error's module and site;

  - the check-ins, and the progress: lanes and refusals;
  - the wiring: the blocks it reads and the blocks that read it, each link's words produced and consumed, the result, and the parent;
  - a CRC-64 that seals the block whenever no launch holds it.
- **Yield.** A launch runs its time to live and leaves, and its block holds where it stands. The next launch resumes from there, and no launch outlives Windows' 2 s watchdog.
- **The exits.** True, false, malformed (no lattice was built), and answered: an answer table on the device, keyed by the signum of the program and its inputs, holds each answer once. An exit short-circuits every program reading the block.
- **Two machines pointed at each other.** Running a machine that halts only on yes against one that halts only on no is dovetailing. It decides exactly the questions whose yes side and no side are both semi-decidable (Post [42], 1944). A network of such processes wired through channels is a Kahn process network.
- **What it must hold** (the adversarial pass):
  - Programs that wait on each other must be resident together. One cooperative launch guarantees that, and separate launches do not.
  - One fault ends every program in its CUDA context.
  - Programs in separate processes cannot share device blocks under WDDM.
  - A block's data is fenced before its state is written.
  - The answer table's key is the program and its inputs, the full signum, confirmed on a hit.
  - Admission reads the compiled registers.
  - Wiring is data in the block and is never compiled in.
  - A link needs a bound or a handshake.
- **Stages.**
  a) The block, the register file in registers, and yield and resume, proved on the eleven record tests.
  b) Resident programs as one cooperative launch, chained through their blocks.
  c) The exits and the answer table.
  d) Tessera admits with the compiled grant and reads the blocks from host memory.
- **Stage 1, built** (`src/cu/engine/analysis/cycle/`). The block, yield and resume, and the register file in registers: every place a constant in the source, and file_limbs registers hold the file. The thread blocks take lanes a round at a time from one counter and leave past the time to live (500 ms unless `CYCLE_RECORD_TTL` names one). The block's command can also send them out. The thread block that opens a launch always runs one round, and every launch moves the program on. The last one out writes the offset, the times and the state, and the host launches again while the state is yielded. On the tower test the programs hold 128 to 255 registers a thread with local frames of 0 to

752 bytes. Each runs in one launch and two check-ins. The 761-step program holds 183 registers, a 0-byte frame, and 0.442 ms for 256 lanes. Stage 1's compiles take 10.2 to 778.7 s a program, which the operator block (item 9) answers.

- **The block in device memory.** Check-ins stay on the device. The host lays the block, seals it and sends it before the first launch. It reads the block back once a launch has ended, checks the seal before the next launch, and seals and sends it again. A fault is left sealed on the host's copy.
- **Still owed for stage 1:** a forced yield, `CYCLE_RECORD_TTL=1` on the eleven record tests, matching the interpreter. The eleventh, the lane test, comes with `ENGINE_RECORD_LANE`.

11. **The emitter over any ruleset.** A record program's lane written directly as NVIDIA PTX (`src/cu/engine/rmc/`, being built), with PTX's own rules stored as data: its carry chains, its register classes and its calling convention, the construction set. Once this works it is split into functionals and support expanded to intel, amd, arm, riscv, vhdl and the rest, to prove it a universal compiler where it knows the hardware ruleset.

    - **(a) Functionals.** Each operation's emitter is a functional over a target ruleset L. PTX is the first L. **The bootstrap test** (future work, not built): the emitter written as a record program and compiled by the emitter to PTX, then that PTX run on the same record program. The test passes when the two emitted texts match byte for byte, the fixed point a three-stage compiler bootstrap compares (GCC's bootstrap, from knowledge). It needs two things first: the operator block emitted as PTX by the same construction set, and the emitter expressible as a record program.

        – **The bootstrap, built.** The device writes a lane's text with the text program, itself a record program run a lane a byte, and `codegen_test` includes the bootstrap: the text program's own lane written by the device with the text program. The production route does the same: with `CODEGEN_DEVICE=1` the device writes the assembly printer's own lane too, and while it does, the assembly printer's own program runs on the interpreter, the one program not compiled (`s_cycle_codegen_depth` in `src/cu/engine/analysis/cycle/cycle_compile_route.cu`). A text the device writes and matches once in a process is not written again. The production route has not run.

        – **The emitter on the device, a reading** (a reading; built below). The emitter can recompile itself to run on the device. Emission is parallel by step: lane l reads step l from the step table and writes its text, with the templates as table data from the `.krs`, and a scan of the text lengths places each step's text. The analysis across steps, first and last puts, lifetimes and the live estimate, is scans and minima over lanes. On NVIDIA the device can write PTX, but assembly to SASS is the driver's, on the host, and a kernel cannot branch into code it wrote (from knowledge): self-hosting there leaves the host assembling and loading, and the host becomes a loader only once a SASS `.krs` exists, derived by the probes of stages 4 and 5. MaxAs and CuAssembler recover SASS encodings by probing (cited from knowledge; the sources are unread). A CPU process can write code and run it after

remapping its pages executable, as a JIT does, where the platform allows it, and that makes stage 6's second target the likely first to host itself fully.

  * **Built** (everything on the device). The emitter's decisions, keymath's imprint and key_schedule's lay are each one source the host and the device compile (`codegen_core.h`, `keymath_core.h`, `key_schedule_core.h`). The device lays a program's step table, decides each step's forms a thread a step, cuts the lane into states in one thread, and writes the lane's text with the text program (`asm_printer.h`, `codegen_device.h`). `CODEGEN_DEVICE=1` has the record machine's lanes written by the device and checked against the host's text byte for byte. **Measured**, each device run a tessera job on Windows (RTX 3070): `codegen_test` 121 checks, 0 failed, the bootstrap included, the emitter's text in PTX 170,922, C 412,182 and VHDL 554,754 bytes; `CODEGEN_DEVICE=1 record_bitwise_test` 25 checks, 0 failed, 8 programs written by the device byte for byte the host's, the largest 4,204 steps and 2,051,022 bytes. Not exercised: `engine.cu`'s `engine_record_lay_device`.

– **Tessera emitted for the device, a want** (a want; its ledger built on the device, below). Tessera is to run on the device and be emitted as soon as the compiler can. Tessera is host C (`tessera_run.c` and the daemon's `tessera_*.c` files). Once the emitter can express it, tessera becomes a program the emitter compiles for the device, and its decisions run where the programs run. This ties item 10's stage 4, "Tessera admits with the compiled grant and reads the blocks from host memory", to the emitter on the device above. The part that can move is the ledger's arithmetic and the decisions over the blocks: each block's command (run, yield, stop) and its grant (registers a thread, threads, device bytes and shared bytes, the registers read from the compile), read and written by a resident program the way item 10's programs talk through their blocks. Three parts stay on the host, a reading. Admission watches operating-system processes, their tree, their signals and their peaks in processors and bytes, and the device sees none of it. A program enters the device through the host's loader (above). One fault ends every program in its CUDA context (item 10, what it must hold), and a scheduler in that context dies with the program it watches. Tessera is meant to be part of the compiler itself. **Tessera on every target** (built, below): the daemon, `tessera_run` and the seal build with no CUDA toolchain, the Pi first, and runs there become tessera jobs.

  * **Tessera on every target, built.** Where `__CUDACC__` is not defined `obsignatio.cu` compiles as C++. The functions the host and the device share are marked `OBSIGNATIO_SHARED`, and with no CUDA toolchain `obsignatio_many` and `obsignatio_bits` refuse every request as `ENGINE_ERROR_RESOURCE`. A POSIX part with no nvcc builds the seal as C++ and links the daemon and `tessera_run` with c++; the measure and job tests, which take device memory, are not built there, and say so. **Measured:** on a Raspberry Pi 5 (Linux aarch64, no CUDA toolchain, jobs given 2 of its 4 cores), `obsignatio_test` host only, host vectors 210 cases, level keys 18, seal 778, device refused 4, each 0 failed; tessera's tests exit 0, and `tessera_run` 15 checks, 0 failed.

On Windows, each a tessera job, `obsignatio_test`'s 15 tallies at 0 failed, measure 9 cases and job 25 checks at 0 failed, `tessera_run` 14 checks, 0 failed. On WSL with no nvcc on the non-interactive path, `obsignatio_test` host only at 0 failed, `tessera_run` 15 checks, 0 failed; on a loaded machine two peaks of 2.149 and 2.116 processors against the 2.100 bound give 15 checks, 2 failed, and on a quieter machine they read 1.979 and 1.915.

  * **Tessera's ledger on the device, built** (there should be two). The ledger's decisions are one source (`tessera_ledger_core.h`): the host daemon's ledger grows its rooms and runs the core, with no CUDA context and the Pi build unchanged; the device's tessera (`tessera_device.{h,cu}`) runs the same calls in one thread over rooms in device memory. **Measured**, each a tessera job on Windows: the daemon's `run.sh` exit 0 (`tessera_ledger_test` through the core 0 failed, job test 25/0, `tessera_run` 14/0); `tessera_device_test` 601 checks, 0 failed, 200 rounds of 400 calls, every round growing the device's rooms.

- **(b) Other targets**, each its own L: x86 (Intel, AMD), the AMD GPU instruction set, ARM (AArch64), RISC-V, and VHDL, where the program becomes a circuit.
- **(c) The universal compiler and its proof.** Given L, the program compiles to it. The proof is the same program on every target matching the host word for word, the way the record tests hold the compiled path to the host and the interpreter (item 9).
- **(d) Deriving L.** The engine helps a target derive its own ruleset, and the target's questions are then coherent.
- **(e) Unknown L.** Where ruleset L is unknown, more and more basic questions are asked, fully unbound. A small instance is being built: the emitter holds no constant for PTX's `.version` or `.target`. It compiles an empty kernel with NVRTC and reads back the header, and the part of L it does not know it gets by asking the toolchain.
  - Either a target's registers and their operators, its opcode or assembly, or a partial description of what the hardware is believed to be.
  - Given a simple set of math, the answers are checked and how the target works is known at once.
  - Illegal operations can be asked about: what crashes the process, since some systems stop a process that performs an operation their system holds absurd.
  - **Three ways into L, a reading.** The first statement names three ways in: registers with their operators, the opcode list or assembly, or a partial description of the hardware. PTX gives the emitter registers, operators and assembly. The question it still asks is the NVRTC header probe above. Status: not built.
  - **The membership query, a reading.** The second statement is a membership query: a small sum answered by the target's own hardware returns its rule. Take q = x / 0 on integers. x86 raises #DE, delivered on Linux as SIGFPE, and INT_MIN / −1 raises it too. AArch64's SDIV and UDIV return 0. RISC-V's M extension returns all ones for the quotient and x for the remainder. PTX's div leaves the value

unspecified. One question separates the three rulesets whose manuals define the answer. PTX's manual leaves it undefined, and the emitter treats undefined as refused. The value a given GPU returns can equal one of the other three, and an observed value cannot pick PTX out. Cited from knowledge, with the sources unread: the Intel SDM, the Arm ARM, the RISC-V unprivileged specification's division table and the PTX ISA. The record machine refuses a zero divisor (`cycle_record_kernel.cu` and `cycle.c`). The emitter's own test for a zero divisor before each `div.u32` and `div.u64` is not built. Measured (11(f) 3): 7 / 0 faults on x86 and answers 0 on AArch64, the signed division SDIV; RISC-V is not measured.

  - **Illegal operations, a reading.** The third quote is the negative half of L: the operations the target refuses by killing the asker. The record machine holds its own half already, since a refused lane is a question L does not answer. The target's half has to be asked from a process that can die. A crash is an answer, and the signal or exit status names the rule broken. A tessera job has the same structure. On CUDA an illegal address leaves the context unusable, and the unit of a probe has to be a process. SIGILL on an instruction the part lacks is how an unknown CPU reports which extension sets it has. Cited from knowledge, with the sources unread.
  - **More basic questions, a reading.** When a question kills the process, the next question has to be more basic and asked from a fresh process. The cell that watches the probes, and the limit on what it knows, are posit 12 (wants.md).

- **(f) The cell's machinery, in build order.** Step 1's stage 1 and step 3 are built, each with its status below. The cellular machinery for this universal compiler is called wilder than what von Neumann envisioned by decades of orders of magnitude, a comparison no figure in the tree measures.

  a) **The PTX emitter** (item 11). Stage 1 is built and proved. Every PTX launch is checked against the interpreter on the record tests, and the PTX is then compared with the C path. **Both paths kept, a reading.** The PTX path and the C source path both stay, and every program is measured on both. The relation between the two is the data, and rule (i) is set from it. Owed from this stage:

  - (i) a rule that routes each program by the measured relation of the two paths and keeps both paths in the engine. The first quantity measured to separate them is the local frame against the device's stack limit ((iii), the frame against the stack limit). After the PTX build and its hold, the rule reads the kernel's local frame and the stack limit (`cudaLimitStackSize`). A frame within the limit runs as PTX. Past the limit the rule builds the C source as well, and the smaller frame runs: the C source's wherever it fits and the PTX's does not. The other build's hold is given back (`cycle_program_release`). The PTX runs where the C source does not build, where its frame cannot be read, or where the two frames are equal. Routing a launch by its lanes is later work. Status: built (`cycle_record_route` and `cycle_program_release` in `cycle_compile_route.cu`) and proved. **Measured.** The eleven record tests ran, each a tessera job under `CYCLE_RECORD_CHECK=1` against a fresh cache, once as PTX and once

as C source. Both had 0 failed (guide 12, coherence 21, boundary 59, gaussian 16, bitwise 25, divide 20, order 18, table 16, tower 210, speed 12 and lane 17 checks), and in each all 687 checked launches gave the same records and refusals as the interpreter. Rule (i) sent 7 program loads to C, on boundary (1,992 steps) and tower (4,537 to 6,510 steps): their PTX frames were 1,120 to 1,576 bytes against the 1,024-byte limit, and their C frames 176 to 184 bytes. The routing is proved. Its time is not measured apart from the device: it saves the stack growth of 7.449 to 10.130 ms a run ((iii), the frame against the stack limit), paid at a run's first launch and outside the kernel sums;

– (ii) a thread count chosen per launch, for a launch of a few thousand lanes to reach every SM and not only a few blocks' worth. Status: built (`cycle_record_resident` in `cycle_record_launch.cu`) and proved with stage 1;

  * **The launch rule against the device's state** (Measured). The boundary test ran as C (`CYCLE_RECORD_NVRTC=1`, `CYCLE_RECORD_CHECK=1`, a fresh cache each run), alternating the launch rule with (ii)'s: old, new, old, new. The compiled launches' host-timed total over the 19 checked launches, in ms, was 123.635 and 172.857 under the old rule and 149.581 and 164.806 under the new, with 121.307, 123.950 and 129.188 in three runs of the new before them. Every run passed its 59 checks. The two rules' totals overlap, and at 1,048,576 lanes both print the same grid, 92 thread blocks of 256 threads at once. (ii) did not slow the C path. The 21:24 run that gave the C column below its 41.793 ran the interpreter in 375.689 ms. The eight C runs since took 553.082 to 844.936 ms on the interpreter, and the two of them on that code took 553.082 and 842.056. The device ran faster at 21:24 than in any run since, a reading, and what changed in its state is not known. The RTX 3070 also drives the display, and nvidia-smi showed no other compute process when checked. Times from different runs are read against each run's own interpreter total, a reading: the C column below and the C times in the two readings under (iii) come from the 21:24 run.

– (iii) the early store: each record word stored once its last put lays it, not every word at the lane's end. Built and proved with stage 1. **Measured.** Each record test ran as its own tessera job under `CYCLE_RECORD_CHECK=1` against a fresh cache, lanes as PTX, and all eleven passed: guide 12, coherence 21, boundary 59, gaussian 16, bitwise 25, divide 20, order 18, table 16, tower 210, speed 12 and lane 17 checks, 0 failed. All 687 checked launches gave the same records and refusals as the interpreter. The kernels' time in ms, summed over each test's checked launches:

test	PTX, the early store	PTX, the lane-end store	C source	PTX, both paths [its interpreter]	C, both paths [its interpreter]
tower	147.265	172.080	912.861	138.824 [2397.359]	286.545 [2151.482]
order	25.414	61.358	276.034	6.825 [146.721]	86.893 [250.027]
gaussian	138.086	193.010	315.176	99.448 [2663.893]	141.549 [2719.230]
speed	17.656	19.948	65.380	17.313 [238.495]	63.052 [252.659]

test	PTX, the early store	PTX, the lane-end store	C source	PTX, both paths [its interpreter]	C, both paths [its interpreter]
lane	2.868	4.001	31.517	2.671 [54.044]	9.916 [61.539]
coherence	10.970	13.629	30.300	9.888 [133.787]	14.070 [152.511]
bitwise	1.870	3.257	23.563	1.606 [21.927]	4.070 [22.768]
guide	0.264	0.208	0.484	0.216 [1.461]	0.652 [1.811]
boundary	24.636	50.019	41.793	55.927 [872.781]	126.399 [638.947]
divide	106.697	479.441	111.690	102.468 [156.013]	105.988 [167.289]
table	2.044	3.849	2.096	1.838 [2.207]	2.059 [2.103]

PTX with the early store is faster than the C source in every test, and faster than the lane-end store in every test but guide. The C column is run in the faster device state (ii)'s comparison found. The two both-paths columns are stage 1's proof run (Measured), both paths under the same code, each with its run's interpreter total in brackets. There PTX is faster than C in all eleven tests, by 1.03 to 12.7 times (Derived: divide 1.034, table 1.120, gaussian and coherence 1.423, tower 2.064, boundary 2.260, bitwise 2.534, guide 3.019, speed 3.642, lane 3.712, order 12.732). Divide and table, which otherwise spill and run slower than C (479.441 against 111.690, 3.849 against 2.096), do not with the early store. The device's state moves between runs on the same code: boundary's interpreter total is 593.667 ms in the early store's run and 872.781 in the both-paths run. The columns therefore compare the two paths and do not isolate rule (i) from the device. The divide test's largest program, 5 steps over 512 lanes, holds 241 registers, a 616-byte local frame and 96.901 ms with the early store, against 255 registers, a 1,256-byte frame and 467.075 ms with the lane-end store. The order test's 12,293-step program holds at most 56 live words a lane, 68 registers and a 136-byte local frame, and does not spill, where the lane-end store holds 255 registers and a 16,696-byte frame. Its assembly and link take 70.8 s, against 440.5 s for the lane-end store.

* **The live estimate** (built). The most 32-bit words a lane holds live at once, reckoned from the lifetimes the PTX text gives ptxas: each value's limbs and sign from its step to its last reader, each record word from its first put to its last, each atom word from its first reader to its last, each step's own temporaries at that step, and the words every lane holds throughout. It is reported with each program, and the live-word counts in this item are its output.
* **Live words against the two paths** (one row a launch, each launch's live-word estimate, registers, frame and lanes beside its PTX and C times). Of the 11 launches past 320 live words, PTX is slower on 4: bitwise at 348 (0.557 against 0.515 ms), divide at 385 (7.326 against 7.086), boundary at 369 (8.342 against 3.491) and tower at 370 (11.137 against 3.898). The other 7 run PTX 1.04 to 14.5 times as fast (Derived). Two tower launches alike in live words (370), registers (255), frame (1,144 bytes) and lanes (256) split: one ran 16.970 ms as PTX against 54.642 as C, the other 11.137 against 3.898. No budget of live words separates the two paths yet.

    * **The frame against the stack limit** (Measured: the early run's logs, one row a launch, its frame, registers and lanes beside its host-timed and device times and the gap between the two). On the 680 launches whose local frame is 960 bytes or less, the gap runs 0.127 to 1.630 ms. On the 7 whose frame is 1,120 to 1,576 bytes, it runs 7.449 to 10.130 ms. The device's stack limit, 1,024 bytes a thread (the tower test prints it before and after its blocks), falls between the two. A frame past the limit makes the runtime raise it at a run's first launch and reserve that frame for every resident thread, and `cycle_stack_return` gives the reservation back once the run is done, by design (`cycle_launch.cu`: a 7,600-byte frame holds 467,668,992 bytes past the 1 KiB default, and setting the limit back returns them in 5.0 ms). The gap's jump at the limit is that growth and return, paid once a run, a reading. Boundary's 1,992-step program shows both sides: as PTX it ran 0.893 ms on the device and 8.342 ms host-timed, and as C 2.920 and 3.491.

b) **The ruleset as data.** PTX's rules move out of the emitter's code into a table the emitter reads: carry chains, register classes, the calling convention and the header. Each operation's emitter becomes a functional over L. This is (a)'s split.

   – PTX output is one of the n rulesets the engine supports.

   – **Stage 2's plan, a reading** (a plan; 2a, 2b and 2c build it, below, except the canonical form and its signum identity). The emitter splits into a core that knows no target and a ruleset L it reads at run time. The core holds what every target shares: each record word's first and last put and the early store; the refusal sites and the words in flight; the value and atom-word lifetimes and the live-word estimate; the unrolling by width and the branch-free selects. L holds what a target owns: mnemonics, register classes and their declarations, labels, address syntax, the calling convention, the header, and the toolchain that builds it (nvJitLink for PTX, NVRTC for C). The C source path is a ruleset too, and stage 2 brings the two paths under one core. A ruleset maps each operation to a native form or to a construction from more basic ones: the carry is native in PTX (`add.cc`, `addc`), x86 (`adc`) and AArch64 (`adds`, `adcs`), and on RISC-V it is built from `sltu`, the compare stage 5 uses to the same end. The core moves to `src/cu/scaffolding/`, each L to a data file in `src/engine/compiler/codegen/rulesets/`, and the `cycle/` files keep the runtime: launch, share, check, block and cache. Rule (i) becomes routing among n rulesets, its first quantity the frame against the stack limit. The proof: the PTX ruleset writes PTX byte-identical to stage 1's, every program of the eleven record tests a cache hit, and the C ruleset passes the checked run.

   – **The ruleset file** (a plan, not built). The extension is `.krs`, for kolmogorov ruleset. A ruleset is plain ASCII, one entry a line, with the extension `.krs`. The first line is `krs 1`, the format and its version. The first word of each line after it names its kind: `header`, `toolchain`, `class`, `form`, or `construct`, whose block runs to a line `end`. `#` starts a comment, and a template is the rest of its line, verbatim. The engine writes each file's canonical form itself, and a

ruleset's identity is the obsignatio signum over the canonical bytes. The first two files are `ptx.krs` and `c.krs`. JSON, YAML, TOML and S-expressions are ruled out, and `.reg`, the Windows registry's extension.

– **Stage 2a, the ruleset as data** (built and proved). There are n rulesets, each a `.krs` file whose first line is `krs 1`, read once a process from `src/engine/compiler/codegen/rulesets/` or from the folder `CYCLE_RULESETS` names. `CYCLE_RULESETS` names a folder, as `CYCLE_CACHE` does. The six switches stay `CYCLE_RECORD_CHECK`, `CYCLE_RECORD_INTERPRET`, `CYCLE_RECORD_LTO`, `CYCLE_RECORD_NVRTC`, `CYCLE_RECORD_REPORT` and `CYCLE_RECORD_TTL`.

  * The entry kinds. `ruleset <name>` names the target the file describes. `toolchain <name>` names what turns the text into a program the target loads (`nvjitlink` in `ptx.krs`). `header <name>` names where the text's opening lines come from (`probe_nvrtc` in `ptx.krs`, the header asked of NVRTC, (e)). `bank <name> = <definition>` is a bank of registers, `{n}` the register's number in it. `fixed <name> = <definition>` is one register every lane holds throughout. `form <name> <parameter>... = <text>` is a template. `construct <name> <parameter>...` is a form built from more basic ones, its lines running to `end`, and the reader reads it (`ruleset_core_read.h` reads the construct kind).

  * The text rules. A form's text is the rest of its line after =, byte for byte, with `\t` a tab, `\n` a line's end and `\\` a backslash. `{p}` is parameter p's argument where p is one of the form's parameters, and any other brace stands for itself. A line that begins with # is a comment, a blank line is nothing, and a carriage return at a line's end is dropped. A form is kept cut at its parameters, and writing one appends its pieces with each argument between them.

  * `ptx.krs` holds 9 banks (file, sign, out, atom, temporary, wide, predicate, member, immediate), 12 fixed registers and 88 forms in 170 lines.

  * **Stage 2c, the C source as a ruleset** (built and proved with 2a). `c.krs` holds 83 forms in 161 lines, with toolchain `nvrtc` and header `prelude` (the operator block's prelude, `s_cycle_prelude`, `target.h`). One reader serves n rulesets through a schema each language reads against. A refused `c.krs` leaves a program the PTX does not hold on the interpreter, with the reason "its C ruleset, c.krs, was refused", and rule (i) keeps the PTX. `utils/test/src/cu/engine/rmc/ruleset_read_test.cu` reads every ruleset against its schema and holds the files each is read from against each other for absurd relations (`ruleset_core_relation.h`). The emitter writes through the ruleset, and the byte-for-byte match of a separate C printf is theory. 2a and 2c are proved together on the device (the proof, below).

  * **SASS** (built). `sass.krs` is NVIDIA SASS for sm_86, the machine code the device runs, read back by the cell's probes. It holds 9 banks, 12 fixed registers, 69 forms and 2 constructs in 301 lines.

  * **Yosys** (built). `yosys.krs` is Yosys's generic synthesis of a lane GHDL wrote as Verilog, a script Yosys reads a command a line (`yosys -s`). It holds 5 forms in 22 lines.
  * The emitter writes no PTX of its own. It decides what a step does, and the ruleset defines it for the target. The PTX left in `cycle_internal.h` is the interpreter's inline multiply-add (`mad.lo.cc.u32`, `addc.u32`), device code the emitter does not write.
  * Refusal. A ruleset that lacks a bank, register or form the emitter names, holds one it does not name, or gives a form other parameters is refused whole. A form given the wrong number of arguments marks the program broken. Either sends the program to the C source, and the reason is reported.
  * **The proof** (Measured). The eleven record tests ran as tessera jobs with `CYCLE_RECORD_CHECK=1`: once as PTX against a cache seeded from stage 1's final PTX run, and once as C source against one seeded from its final C run. A program whose text differs from stage 1's by a byte misses the cache and is written anew. None was, as PTX or as C: the PTX run read 165 programs from the cache and the C run 196, and no ruleset was refused. Both modes had 0 failed: guide 12, coherence 21, boundary 59, gaussian 16, bitwise 25, divide 20, order 18, table 16, tower 210, speed 12 and lane 17 checks. The PTX coherence test's first build stopped in the host compiler (`cl.exe` exit oxC0000142) before any stage 2 code ran. It was run again alone, from the same snapshot with a cache seeded from the first PTX run: 21 checks, 0 failed, its 38 programs read from the cache and none written anew. The PTX ruleset writes PTX byte-identical to stage 1's, and the C ruleset writes stage 1's C source.
  * The built reader departs from the plan above in three places: `class` is `bank` and `fixed`; a template is read with escapes where the plan reads it verbatim; and the canonical form with its signum identity is not built. `c.krs` carries out the plan's second file.

– **Stage 2b, the emitter in `src/cu/transpiler/vendor_bin_layouts/`** (built and proved). The emitter is `src/cu/transpiler/vendor_bin_layouts/`: `container_layout.{h,c}` holds a container's layout as rows in a file, the offsets, widths, tags and names a format uses, which the emitter carries none of; `container_write.{h,c}` is one emitter for every container, which reads a layout file and writes what that layout describes, carrying over untouched what it does not understand; and `nvidia/elf64_nvidia.tsv` is one such layout, beside the cubin writer that reads it. The rulesets and the code generation are `src/cu/engine/rmc/`, the rulesets at `codegen/rulesets/`. The emitter reads no device and loads no library. The `cycle/` runtime names the target through a `CycleTarget` built by `cycle_target` (the operator block's hash, the device's compute capability, NVRTC's version and the prelude), and compiles, links, caches and launches what the emitter writes. Every

build of the engine compiles the emitter with it: `build_engine.sh`, `build_driver.sh`, `examples/qasm/build.sh`, `sims/run.sh` and the twelve record and residual tests' scripts.

  * **The move.** The emitter is cut from the record machine into `src/cu/scaffolding/`: `static` is dropped from the five calls the record machine makes, the target passed in place of its own statics, with the ruleset folder and the include. Every file is ASCII.
  * **The proof** (Measured). The eleven record tests ran as tessera jobs with `CYCLE_RECORD_CHECK=1`: once as PTX against a cache seeded from stage 2's PTX runs, and once as C source against one seeded from its C run. No program was written anew, as PTX or as C: the PTX run read 203 programs from the cache, the 165 of stage 2's PTX run and the 38 of its coherence run again, and the C run read 196, and no ruleset was refused. Both had 0 failed: guide 12, coherence 21, boundary 59, gaussian 16, bitwise 25, divide 20, order 18, table 16, tower 210, speed 12 and lane 17 checks. The emitter moves without changing a byte of what it writes.
  * **The build check** (Measured). `examples/qasm/test/run.sh` checks qasm: `qasm_test`, its exact test 39 checks and its self test 44 checks, 0 failed. **In the driver's build.** `examples/cell_tracking/build_driver.sh`'s `FUNCTIONALS` (`build_driver.sh`) names `cu/includes/file_defs/krep` and `c/engine/rmc`, with no emit entry and no `base/` path.

– **The code generator and its language classes** (built). The code generator is a base class, `CodeGenerator` (`code_generator.h`, the register lane that names no language), and each language inherits it and names its ruleset: `PtxTarget` (`ptx_target.h`, `ptx.krs`), `CTarget` (`c_target.h`, `c.krs`) and `VhdlTarget` (`vhdl_target.h`, `vhdl.krs`). The `cycle/` module is cut into functional chunks, the split item 11's first statement asks for. **Measured**: the eleven record tests pass as PTX, 11 of 11 with 203 programs read from the cache, and as C source, 11 of 11 with 196, and no program is written anew.

– **The test folders.** The engine's test folders sit out of `src/engine` under `test/engine`, which mirrors the category tree: `test/engine/runtime/obsignatio`, `examples/qasm/test`, `test/engine/runtime/daemon` and `test/engine/nbody/max_tree` each run from there with 0 failed. The cell_tracking README states that `.kcr` is its own compression format.

– **The invariance theorem, a reading.** K_U(x) ≤ K_V(x) + c_{U,V}, with c_{U,V} the length of an interpreter for V written for U (Solomonoff 1964, Kolmogorov 1965, Chaitin 1969; Li and Vitányi, Theorem 2.1.1; cited from knowledge). The engine's device interpreter, `cycle_record_kernel` (`cycle_record_kernel.cu`), has that form for the record machine on the device. The emitter core with a ruleset L runs on the host and translates each record program into an L program whose length grows with the record program's, which bounds K_L(x) by m K_rec(x) + h (Derived), m the longest template a

step and h the header: a multiplicative bound. The `.krs` file's canonical length enters the additive constant when the core and the ruleset are carried as an L program. Carried as an L program that runs the code it writes, the core and the ruleset give K_L(x) ≤ K_rec(x) + |core as L| + |krs| + c, c the driver that runs the emitter and jumps to its output (Derived), and the bootstrap's fixed point (11(a)) shows the core running as an L program. On NVIDIA the jump needs the host's assembler and loader, and the device interpreter keeps the additive form with no host step. A ruleset derived by probes is a measured description of an unknown machine.

– **Across machines, a reading** (a plan, not built). Cross-hardware communication is then no problem from zero. Values: every ruleset compiles the same record program, and each target is proved by matching the host word for word. Two targets each proved on a program give the same bits for it, and equal obsignatio signums show it. Wire order: the record format fixes the byte order once, as little-endian 32-bit limbs, the way the seal already writes its lengths and offsets as LE64 (κ and Σ in the engine in math). Ordering: every `.krs` carries a memory-model section beside its arithmetic, defining acquire, release and a full fence at each scope. The core emits "publish the block" once, and each target defines it (from knowledge): x86 is ordered as TSO, where acquire and release need no fence and a full fence (`mfence` or a locked instruction) orders a store before a later load; AArch64 uses `dmb ish`, or `ldar` and `stlr`; RISC-V uses `fence` with its predecessor and successor sets; PTX uses scoped fences, `fence.acq_rel.gpu` and `fence.acq_rel.sys`. This carries item 10's rule, "A block's data is fenced before its state is written", across targets. Matching the host proves a program's values; the order between programs rests on each target's fences, and a run that matches does not prove it, a reading. Posit 19 replaces this as the direction (stage 4).

– **Prior art, cited from knowledge.** GCC's machine descriptions, each target's instruction patterns stored as data the compiler's core reads. LLVM's TableGen target descriptions and its legalization, which expands an operation a target lacks into ones it has (an add with carry out becomes an add and `sltu` on RISC-V). C. W. Fraser and D. R. Hanson, *A Retargetable C Compiler: Design and Implementation* (Addison-Wesley, 1995), lcc, whose back ends are tree grammars compiled by lburg. N. Ramsey and M. F. Fernández, "Specifying representations of machine instructions", ACM TOPLAS 19 (1997) 492–524, the New Jersey Machine-Code Toolkit. For the memory models: L. Lamport, "How to make a multiprocessor computer that correctly executes multiprocess programs", IEEE Transactions on Computers C-28 (1979) 690–691; P. Sewell, S. Sarkar, S. Owens, F. Zappa Nardelli and M. O. Myreen, "x86-TSO: a rigorous and usable programmer's model for x86 multiprocessors", Communications of the ACM 53 (2010) 89–97; J. Alglave, L. Maranget and M. Tautschnig, "Herding cats: modelling, simulation, testing, and data mining for weak memory", ACM TOPLAS 36 (2014), article 7; the RISC-V Instruction Set Manual, Volume I, its RVWMO memory model; D. Lustig, S. Sahasrabuddhe

and O. Giroux, "A formal analysis of the NVIDIA PTX memory consistency model", ASPLOS 2019.

c) **The cell, a probe runner** (posit 12). Each ribosome, a small probe program, runs in a child process the cell can lose. The cell records how each one ends (exit status, signal or fault, and output) and sees everything inside it. A tessera job has this shape already. Status: built (`src/cu/transpiler/lstar/interface/interface.{h,c}`, `interface_probe_run`, error module 23, `ENGINE_MODULE_INTERFACE`), not in the engine library build: its test builds it. `interface_probe_run` starts a probe, a small program that asks the target one question, in a child process, and records how it ended. On Windows the probe starts suspended in a job object that ends every process in it when the cell lets it go, with the fault dialogs off (`SetErrorMode`, inherited). On POSIX it runs in a process group of its own, and a pipe that closes on exec tells a probe that never started from one that exited. Its output and errors go to one file the cell reads back, its input is empty, and a probe past its limit is ended with every process it started. The answer is how it ended (exited, signaled, faulted, out of time, not started), its code, the rule a signal or a Windows NTSTATUS names (arithmetic, address, instruction, stack, trap, abort, other), its output and its wall time. An exit status with its top bit set on Windows is read as the fault that ended the process.

   – **Measured.** `test/engine/compiler/interface/interface_test.sh` builds the cell and `test/engine/compiler/interface/interface_probe.c`, one question a process, built with no optimization and volatile operands, and holds each ending to the host's rules: 17 checks, 0 failed on Windows (MSVC, x86-64) and on WSL (gcc, x86_64), each a tessera job. On Windows 7 / 0 faulted 0xC0000094, INT_MIN / −1 0xC0000095, a read of address 16 0xC0000005, `ud2` 0xC000001D, `__debugbreak` 0x80000003, unbounded recursion 0xC00000FD, and `abort()` 0xC0000409 (the C runtime's fail-fast). On WSL both divisions gave SIGFPE, the address and the recursion SIGSEGV, `ud2` SIGILL, `int3` SIGTRAP and `abort()` SIGABRT. A probe that hangs was ended at 0.522506 s on Windows and at 0.501591 s on WSL, by SIGKILL, against a 0.5 s limit. A program that is not there was not started: code 2 on Windows, errno 2 on WSL. On both, exit 0 and exit 3 exit as asked, the output and then the errors come back in one file (50 bytes on Windows, whose C runtime ends a line with two, 48 on Linux), a room of 8 keeps 7 bytes and counts the whole output, and after each death a fresh probe exits 0. The test holds each division to the part's own rule, chosen when it is built (`INTERFACE_TEST_DIVISION_FAULTS`, `INTERFACE_TEST_ZERO_QUOTIENT` and `interface_test_answered`, `test/engine/compiler/interface/interface_test.c`): x86 faults on both, AArch64 answers 0 and INT_MIN, and RISC-V answers −1 and INT_MIN. A part the test does not know stops its build. 17 checks, 0 failed on three hosts: Windows and WSL as before, and a Raspberry Pi 5 (Linux aarch64, Cortex-A76, gcc; CPU part 0xd0b, 4 cores and 8 GB, as read from its `/proc/cpuinfo` and `free`). The Pi ran the test as a plain process at nice 10, since tessera does not build there yet: its seal, `obsignatio.cu`,

and its link need nvcc. On the Pi 7 / 0 exited 0 printing 0, and INT_MIN / −1 exited 0 printing −2147483648; neither trapped. The rest matched WSL: SIGSEGV for the address and the recursion, SIGILL for `udf #0`, SIGTRAP for `brk #0`, SIGABRT for `abort()`, the hang ended at 0.500207 s, and the program that is not there not started, errno 2.

- **One signal for two rules, a reading.** POSIX gives the cell one signal for two rules twice over: SIGFPE for a zero divisor and for INT_MIN / −1, and SIGSEGV for an address out of range and for a stack past its limit. Windows names each apart (0xC0000094 and 0xC0000095, 0xC0000005 and 0xC000000FD). The cell reports a stack overflow on POSIX as an address fault. Telling the two apart there needs the faulting address, read on an alternate signal stack (`sigaltstack`, `si_addr`), and that is not built. x86 raises #DE for both divisions (cited from knowledge); Windows reports the two as 0xC0000094 and 0xC0000095, and Linux as SIGFPE for both. AArch64 answers both with no trap, 0 and INT_MIN, measured on the Pi. One question, 7 / 0, tells x86 from AArch64. RISC-V's −1 is in the test from knowledge and not measured.

d) **Membership queries and illegal operations, run.** A membership query for every instruction the ruleset uses, each answer checked against the host's exact integers. Probes of illegal operations: division by zero, INT_MIN / −1, an address out of range, and an instruction the part lacks. These are (e)'s readings, run.

- **Built and measured.** `ruleset_reader.h` gives a ruleset's forms to anything that writes the target's text. `test/engine/compiler/interface/interface_ptx_probe_main.cu` writes kernels from `ptx.krs` through them and runs one question a process under the cell: membership, where each of 55 forms answers 65,536 cases against the host's integers (25 questions), and the illegal operations: a load from address 16, a misaligned 32-bit load, trap, and `elect.sync`, which PTX gives sm_90 and later. `interface_ptx_test` holds each answer and asks again from a fresh process after every death. **Measured**, a tessera job, both rulesets (the engine's own and the flagless one, item 5): 10 checks, 0 failed each. Membership: 25 questions, 0 with a case that differs; `word_div` 62,093 defined cases and `wide_div` 65,345 agree, the rest (a zero divisor) undefined, the device answering ffffffff. Illegal address 700, misaligned 716, trap refused with a CUDA error, `elect.sync` refused by the toolchain for this device (sm_86), and a fresh probe after each answers 6 + 7 = 0x0000000d.
- **Not proved:** `cell_ptx` gives 2 failed in the matrix run, undiagnosed (item 13).
- **The wire, a want** (not built; wants.md posit 19). The probes of stages 4 and 5 turn on a transport, a bus, a network interface or a protocol's framing, each derived as a `.krs`. Listening is T^{-1} (E4), and a peer is recognized by the saving I(s : self) = K(s) − K(s | self), a reading. This replaces stage 2's byte order and fences as the direction. Boundary, proposed, Doug's to rule on: hardware we own, and wires we are entitled to hear.

  - **Identity and the forgery guard, wants** (not built; answers to posit 19's reading, wants.md). Identity is proved where a time series is encoded into it. Every cell carries an identity from its spawn: a secret drawn from the noise, with keys derived from it and its lineage. A hash chain over the cell's series, its head signed under the spawn key, proves the series' holder and order. It does not prove that the series is true, a reading. The forgery guard is not needed yet. The forgery guard, meaning a check that refuses bytes or a claim of identity made by anyone but the cell named, is a want. It is not scheduled.

e) **More basic constructions** ((e)). When a question kills its process, or its answer disagrees with the host, that instruction leaves L. Its operation is built from more basic ones and asked again: a quotient from subtraction and shifts, a product from sums, a carry from a compare. This is (e)'s "more and more basic, fully unbound".

  - **Built and measured.** A ruleset entry may be a construct, `construct <name> <parameters>` through `end`, each line a form of the same ruleset, with `{p}` a parameter and `{bank:n}` the n-th scratch register of a bank, fresh for each use. A construct is expanded wherever its name is written, recursively, and its scratch comes from the step's own temporaries. `test/engine/compiler/codegen/rulesets/flagless/{ptx,c}.krs` is `ptx.krs` with no condition code: the 13 carry and borrow forms rebuilt as constructs on a `%carry` register, RISC-V's shape, which has no carry flag. **Measured**, each a tessera job under `CYCLE_RECORD_CHECK=1`: the eleven record tests on the engine's own rulesets, the caches seeded from 2b's, as PTX 11 of 11 at 0 failed, read from the cache 203, written anew 0, and as C source 11 of 11, read from the cache 196, written anew 0; on the flagless ruleset against a fresh cache, 11 of 11 at 0 failed, 151 programs written as PTX and 5 as C, every launch checked against the interpreter.

f) **A second target**, AArch64 first, then RISC-V, each as its own L ((b)). It is proved by the same record programs matching the host word for word ((c)).

  - **The host oracle on a second part, built and measured** (stage 6's first step). `test/engine/compiler/cycle/record_host_test.c` imprints, lays and runs (`cycle_record_run_host`) programs over all 18 record operations on 4,096 lanes, and each program prints an FNV-1a digest of its inputs and of every output word, taken from each word's lowest byte. The part's byte order therefore does not enter. It also holds the reused register layout to the unreused one, and a zero divisor and an inexact exact quotient to a refused run. It builds with no CUDA toolchain where there is none. **Measured**, each a tessera job, the same lines on three parts, 9 checks, 0 failed on each: Windows (MSVC, x86-64), WSL (gcc, x86_64) and a Raspberry Pi 5 (gcc, AArch64, under the tessera built there with no CUDA toolchain), `ORIOR_EXACT_LIMBS` 128. Records: arithmetic fefaad30b21bf7be (the same with registers reused), division 21c04f0b581ef777, bitwise f1bc8fdc0316c355, tables 39b3224a57bbc124,

members 468a0e0e48772ff3. The Pi's build gave no warnings (gcc -Wall -Wextra). The device's record tests hold the device to the x86 host word for word. The Pi's host therefore agrees with the device too.

- **Plan** (not built). AArch64 runs natively on the Raspberry Pi 5, with `cycle_record_run_host` (`cycle.c`, portable C) as the host oracle there. RISC-V follows on an ESP32-C6 and an ESP32-P4, each a 32-bit RISC-V part (from knowledge). An ESP32-S3 is Xtensa.
- **The ESP32's JTAG, a reading** (cited from knowledge). The ESP32 has JTAG. Over JTAG the host holds the cell and the part is the probe. A fault halts the core, and the host reads its cause and address from it: `mcause`, `mepc` and `mtval` on RISC-V. The C6, the P4 and the S3 carry a USB-Serial-JTAG bridge on the chip, Espressif's OpenOCD drives it, ESP-IDF's panic handler halts when a debugger is attached, and `ebreak` enters debug mode when `dcsr.ebreakm` is set. Read from the halted core, a stack overflow can be told from an address fault.
- **The form limit** (built). An overflow in keymath's linear forms is caught: `keymath_sum_held` tests a sum before forming it. `record_host_test` carries a check, "form limit". **Measured**: 11 passed and 0 failed on Windows and on the Pi. The form limit records `8e0b1ff8af17a325`.
- **VHDL, a target on the Pi** (built). The register lane is a base class, `Code Generator` (`code_generator.h`), and `PtxTarget` (`ptx_target.h`) and `Vhdl Target` (`vhdl_target.h`) are languages over it. The rulesets gain the bank `immediate`, `vhdl.krs` is the VHDL ruleset (`vhdl_target.h`, GHDL analyzes and runs it), and `record_vhdl_test.cpp` holds its programs to the host. **Measured**: as PTX, 11 of 11 with 203 programs read from the cache and none written anew; with no flags set, 11 of 11; the cell's PTX test, 10 passed and 0 failed; the host test, 11 passed and 0 failed. On the Pi under GHDL 5.0.1, bitwise, tables, members and form limit equal the host. 5 programs are not checked, since they call the operator block.
  - **The lane in clock states** (built). State forms cut the lane into clock states, and `vhdl.krs` writes a clocked entity; the tree holds `vhdl_target.h`'s `scheduled()` and `program_schedule_model()` and `test/engine/compiler/codegen/vhdl_construction_set.cpp`. **Measured, on the Pi:** 12 passed and 0 failed, and every program equals the host both cut only where it must be (4 states) and with every form alone (141 to 289 states). `ghdl --synth` accepts it. For form limit with every form alone over 256 words, Yosys gives 167,697 generic cells and a longest path of 38 cells, and infers no RAM yet: the memory becomes flip-flops.

g) **Clustering.** The footprint of one cell, its bytes and its processes, and how many cells run at once, is worked as arithmetic first. The cells cluster, since each occupies very little memory.

h) **Vertical growth, then the bootstrap.** First a plan for the transform in which more than one program occupies a register vertically and never horizontally (posit 3, wants.md, theory), and the plan comes before any build. Then (a)'s bootstrap fixed point.

- **How a cell's gain survives, a question for Doug before the plan.** A gain escapes the noise floor and keeps enough energy to propagate, not to fall back down and dissipate through localized cancellation. Any tipped scale above the mean-field noise energy is irrevocable.
- **Two thresholds, a reading.** The first quote names two. Escaping the noise floor is nucleation: a gain has to pass a barrier before it can grow at all (Volmer and Weber 1926; Becker and Döring 1935). Propagating is criticality: a line whose members leave a mean of k copies a step dies out for certain at $k \leq 1$, unless every member leaves exactly one, and survives with a chance above 0 only at $k > 1$ (Harris 1963).
- **Irrevocable, a reading.** A path's reversal is e^(−ΔS/k_B) times as likely as the path, ΔS the entropy the path produces (Crooks 1999). A tipped scale is irrevocable by degree: the odds of undoing it fall exponentially in its margin over the noise and reach 0 at no finite margin, and the mean field's noise energy marks no step in them.
- **A new gain's survival, a reading.** A variant with a small advantage s, starting as one copy, survives its early rare phase with probability about 2s where offspring counts are Poisson (Haldane 1927). Started as k independent copies, it is lost with probability about $(1 - 2s)^k \approx e^{(-2sk)}$. At s = 0.01 one copy is lost about 98 times in 100, and about 230 copies bring the loss to 1 in 100 (Derived).
- **The question for the plan.** How many copies a new variant starts with, or a protected nursery that holds it until it is established. The plan waits on Doug's answer.
- **Prior art, cited from knowledge.** J. B. S. Haldane, "A mathematical theory of natural and artificial selection, Part V: Selection and mutation", Proc. Camb. Phil. Soc. 23 (1927) 838–844. G. E. Crooks, "Entropy production fluctuation theorem and the nonequilibrium work relation for free energy differences", Phys. Rev. E 60 (1999) 2721–2726. T. E. Harris, The Theory of Branching Processes (Springer, 1963). M. Volmer and A. Weber, "Keimbildung in übersättigten Gebilden", Z. phys. Chem. 119 (1926) 277–301. R. Becker and W. Döring, "Kinetische Behandlung der Keimbildung in übersättigten Dämpfen", Ann. Phys. 24 (1935) 719–752.

– **The question the cells are selected for** (a want, not built). Entropy is not simulated but adopted as a core principle, the cells are selected for one fixed thing, and a wider question can be asked that makes them evolve to answer it.

  * **A moving target, a reading.** The cells are selected now for one fixed thing. The want is a target that moves or stays open, a question no cell answers

once and for all. Ω is the candidate: each of its bits is a finite fact, and no single program computes them all (Chaitin 1975). The engine's own Ω run is item 12.

* **Prior art, cited from knowledge.** Novelty search drops the objective and selects for behavior not seen before (J. Lehman and K. O. Stanley, "Abandoning objectives: evolution through the search for novelty alone", Evolutionary Computation 19 (2011) 189–223). POET evolves the environments beside the agents that solve them (R. Wang, J. Lehman, J. Clune and K. O. Stanley, "Paired Open-Ended Trailblazer (POET): endlessly generating increasingly complex and diverse learning environments and their solutions", arXiv:1901.01753, 2019). Tierra and Avida are the counter-reading, systems built for open-ended evolution whose novelty leveled off in the runs reported (T. S. Ray, "An approach to the synthesis of life", Artificial Life II, 1992, 371–408; C. Ofria and C. O. Wilke, "Avida: a software platform for research in computational evolutionary biology", Artificial Life 10 (2004) 191–229), and open-ended evolution is still counted an open problem (T. Taylor et al., "Open-ended evolution: perspectives from the OEE workshop in York", Artificial Life 22 (2016) 408–423). G. J. Chaitin, "A theory of program size formally identical to information theory", J. ACM 22 (1975) 329–340.

- **The name.** The library is orior.
  - **The name, a reading.** The name is Doug's, and it is Latin (Telos, τέλος, "end" or "purpose", is Greek). The conclusion that this leads inevitably to AGI is Doug's expectation, and no figure in the tree bears on it.

12. **Ω's device phase has no budget** (`chaitin_omega.cu`). Recorded only. A budget would change the sim's behavior, and that is Doug's call.

    - **The exits.** The engine's loop runs while a job waits or a term is live, and no count of rounds, steps or tokens ends it: "There is no step budget and no token budget". A term leaves the device by one of four exits: a normal form halts it, Brent's watcher proves a loop, `omega_grows_forever` proves growth, or the device parks it as outgrown.
    - **Parking is memory pressure.** While the device cannot hold a round's 19 rooms, the stepping term with the largest reduct is marked grown and the reducts are laid again without it, one term at a time. A term that keeps growing with no growth proof, and still fits, keeps the run going.
    - **The 32-bit index.** A round whose pool passes `2^32 − 1` tokens fails a check and stops the run (`OMEGA_ENGINE_INDEX_MOST`). It is a hard stop, and it settles no term.
    - **The cross check.** The terms the engine settled are run again by `omega_run` through 24 bits (`OMEGA_ENGINE_CROSS_MOST`). At the default L of 28 lengths 25 to 28 are not cross-checked.
    - **Measured, the record of runs** (every printed run found on this machine):
      - through 20 bits: finished, "5 rounds, 8010 records swept, 0 parked as outgrown, 118 ms" (16 checks, 0 failed);

  - 20 jobs: round 4096 with 5 live, 18 of 20 jobs done, 194,378,537 records swept, the largest term 57,330 tokens, 25,283 ms, with 5 live from round 16 on and no final line;
  - the default L of 28 with the cross check raised to 28 (a tessera job): 88 live from round 64 on, 18 of 24 jobs done from round 8 on, and the largest term 33,787 tokens at round 64, 1,081,339 at 128 and 138,412,027 at 256. Round 256 took 1,427,203 ms and swept 6,038,215,855 records. It is stopped there with 1,398 CPU s spent and no check reached, and the cross check returns to 24.
- No run at the default L of 28 has finished.

13. **The engine in categories.** The engine files sit in category folders, 300 to 500 lines each, with no leftover tokens or mangled names and no British definition in the file names.
    - **The categories.** `src/engine` is in category folders, and the test tree mirrors it under `test/engine`:
      - arithmetic: `no_rounding`, `decimal_double`, `double_fields`;
      - analysis: `compression`, `entropy_history`, `golden_bands`, `residual`, `residual_survey`, `shift_agreement`, `unit_sweep`, `tower`, `period` and `noise_detector`;
      - codecs: `blosc`, `deflate`, `inflate`, `lz4`, `snappy`, `zstd`, `crc` and `zip`;
      - compiler: `cycle`, `codegen`, `emit`, `keymath`, `key_schedule`, `cell`, `bootstrap`, `cubin` and `gnascor`;
      - formats: `cfg_json`, `hdf5`, `nifti`, `npy`, `nrrd`, `stack`, `tiff`, `krep`, `dicom` and `zarr`;
      - quantum: `qasm`;
      - runtime: `daemon` (its `service` included), `device_pool`, `schedule`, `scriptura`, `obsignatio` and `radix_keys`;
      - `nbody`, `prg_sch`, `render` and `sims` are categories of their own.
    - **Splits.** No C, C++ or CUDA file under `src/engine` or `test/engine` is over 500 lines. The pieces of a file share an `*_internal.h`. The cycle module is sixteen files in `compiler/cycle/` (the compile's cache, route and toolchain, the launch, the prelude, the record kernel and launch, the sweep, the host's `cycle.c`, and their headers).
    - **Names.** The emitter is the code generator (`codegen/`: `Target`, `CodeGenerator`, `Machine Instr`, `Opcode`, `RegisterClass`, `Schedule`, `AsmPrinter`). The text program is the assembly printer (`asm_printer`), and `EMIT_DEVICE` is `CODEGEN_DEVICE`. `ENGINE_RECORD_LIMBS_MOST` is `ENGINE_RECORD_LIMBS_MAX`, most is max, tally is results, reading is measurement, good is ok, `X_REFUSED` is `X_ERROR`, and `X_TOOK` is `X_STATUS_CHECK`. Kept: tessera, scriptura, obsignatio, signum, crystal, steer, climb, lane, ruleset, the `.krs` keywords, and tessera's admit, withheld and hold_until.
    - **Definition.** American throughout, file names and outputs included: `noise_detector_neighbors.cu`, `maint/chain/fetch_labeled.py` and `blocks_labeled.json` carry American definition, and the engine writes `noise_neighbors.tsv`, `pi_rings_center.png` and `shuffled_rings_center.png`, and the flicker table's columns

`neighbor_pairs` and `neighbor_squares`. The signed `maint/signing/manifest.tsv` still names the two chain files by their British definitions under `tools/chain/`, and re-signing it is Doug's.

- **Format.** clang-format with the repository's `.clang-format` over all 604 engine files.
- **Not proved.** The whole test matrix has not run. Three failures stand undiagnosed: the qasm build, `cell_ptx` 2 failed, and a `record_bitwise_c` hang. The bootstrap route (item 11 (a)) has not run.

14. **The ruleset asked of the part.** The compiler derives a target's ruleset by putting questions to the target and reading what comes back. sm_86 is the first subject instead of the target. nvdisasm and ptxas are the oracle: no vendor source is read and no shipped ELF is taken apart to learn a rule.

    - **Three searches, each wider than the last.** A listing gives the forms some compiler happened to write. Widening turns each of a form's 128 bits over and keeps every encoding the disassembler names whole. The sweep cuts the opcode out of a carrier the part ran and puts all 4096 values back, from each of 8 carriers, which no compiler's output bounds.
    - **A form is its operation and the kind of each operand together.** `SHF.L.U32` with a register in the shift operand and `SHF.L.U32` with a number there carry one name and are two forms, and the assembler puts a number in a different place for each. The search had kept a decode only where it named an operation other than the one it came from, and had widened only the 118 forms a listing gave, never the 433 it found.
    - **The kind of an operand is a three bit field.** On sm_86 the low word reads 0x2 for a register in the second and the third operand, 0xa and 0x8 for a constant and a number in the second, 0x6 and 0x4 for a constant and a number in the third. A register to a number is two bits, through the constant that lies between. One round reaches the constant and the round after reaches the number. `SHF.L.S32`, `SHF.L.S64`, `SHF.L.U64` and `SHF.L.W.U32` had their number forms only because the sweep's carriers happened to hold those mode bits, and the sweep holds the high word fixed.
    - **Measured** (RTX 3070, sm_86, CUDA 13.3): 118 forms listed widen to 2436 over 2 rounds, the sweep adds 373 on top, and the part's machine file holds 2927 forms. 0 without fields, 0 differing when read back, 0 past what it holds. Of about 3200 generated lines across 16 programs, 88 find no form.
    - **The walk is capped at 2 rounds.** Run to its own end it does not close: 3128 forms from 118 in 19 minutes, flat at 177 a minute, no plateau. The component reachable a bit at a time is most of what the part decodes. It would pass the 16384 a machine holds and end in `refused`, reporting nothing, and a form reached far out carries the operand bits of a chain of forms it has nothing to do with, which is `PLOP3.LUT` at depth.
    - **The 88 that are left are not a search gap.** All 34 forms the part has with a number in the first source operand are single source: BREV, FLO, IABS, I2I, LDC, B2R, BAR. No operation taking three sources has one there. `SEL Rd, Ra, Rb, P` takes a register

at Ra and no number. These are sass.krs's `wide_select` and `word_select` handed a number for `chosen`, and the fix is the ruleset's or the generator's.

- **Registers.** Ro through R254 hold and R255 is RZ. What holds is gated by the count the ELF declares: R238 and R254 error under a frame declaring fewer. `register_file_holds()` is 238.
- **Three comparisons no compiler wrote.** ptxas reads zero and below off the negations of NE and GE. `ISETP.EQ.U32.AND`, `ISETP.EQ.U32.AND.EX` and `ISETP.LT.AND` were in no listing and nothing had run them. Each was reached one bit out, and each answered right on the part.
- **The rotate the part refuses.** The funnel idiom every compiler writes was asked first on `SHF.R.U32.HI` and `SHF.L.U32` with one register handed to both halves, and the part answered wrong: `.U32` has no funnel. `SHF.R.U64` and `SHF.L.U64.HI` answer right, and both came out of widening instead of out of any compiler.
- **The resident is not asked of the ruleset.** ptxas turns ptx.krs's 110 PTX instructions into 248 SASS that runs. A program is put together by writing into that cubin's `cycle_lane`, leaving `cycle_program` as ptxas wrote it, and 247 of the 248 survive. `program_unit_written()` answers 0 for SASS and sass.krs is never asked for a resident. Its 6 refusals are fail closed and correct: a constant bank past 0, `PLOP3.LUT` operand 6 with no field, `BAR.SYNC.DEFER_BLOCKING` operand 0 with no field, and a call to a symbol its form was not learned with.
- **The listing is not the instruction.** `LDG`'s bits 32-39 change nothing nvdisasm prints. Two instructions that print the same can differ there. 968 of 976 match byte for byte and all 976 read back right.
- **A loop asked of the part** (Q16, A18). `loop_back_if` is derived by asking: every form in the machine file is put in its place with its operands filled by their kinds, at the end of a body that counts N down with `word_add` and sets the flag with `test_word_nonzero`. On sm_86, 2533 of the 2927 forms assemble, 1715 fall through and 8 come back on every N, every one a `BRA`. None is cheaper than the next above the spread of its own runs, and `sass.krs` keeps its `BRA`. `interface_sass_probe` given `loop` and a machine file puts this ask alone against the cubins of an earlier run (`SASS_PATTERN` in `interface_sass_test.sh`).
- **Read back with no disassembler.** `sass_encoding_read` reads an encoding back into its instruction through the machine file's forms, and `sass_loop_walk` checks one emitted instruction as a loop's way back, on or off, and never changes it (`compiler/cubin/sass_assemble.h`). The walk holds that the guard is the flag alone, that one label or number operand added to the address after the instruction lands on the label, and that the first operand writes nothing the loop keeps. A caller can stop it at any step, and with it off what was emitted is what runs, as code written to run in constant time needs. It gives the expected step on ten known cases out of ten, and agrees with all 8 forms above.
- **A branch's distance starts at bit 34**, in four-byte steps. Bits 32 and 33 are the operation's own, as `BRA`, `BRA.U` and `BRA.DIV` show the same distance with 0, 1 and 2 there. Of

the 2927 forms, 2383 read back as they were listed, and the rest are known limits: two names the disassembler prints for one encoding (`IMAD.MOV`), addresses with a uniform register, absolute 64-bit `CALL.ABS` and `JMP` targets, and 380 forms whose operands the assembler cannot place.

- **Not proved.** The suite has not run since the machine file was replaced. The run that wrote it passed every check except the comparison against the committed file it replaces.

9

VERTICAL TIME COMPRESSION: FLOORS STACKED INTO ONE SWEEP

Purpose: the algebra of the record machine's stacked floors: what a stack is, what it compresses, what it leaves as it is, what bounds its file and its record, how its values and widths fill the floors of the two towers, and what is proved, measured and derived about it.

Scope: the record machine (M10 and A13 in engine_table.md): `keymath` imprints a program, `key_schedule` lays it out, `cycle` sweeps it. The shape is Doug's: a floor, the operations in order above it, the next floor, and so on up, with the top conjunction performing the whole transform.

Labels.

- **Proved:** a named test checked it on the machine.
- **Measured:** timed on the machine.
- **Derived:** argued here from the engine's source or by elementary algebra, or computed by the replica. Not run on the machine.
- **Open:** not settled.
- **The replica** is a script that copies keymath's width rules and key_schedule's reuse (first fit, coalesced, a register freed as the step after its last reader begins) and evaluates each program exactly. It reproduces the proved lifting program below: 129 steps and a 12-limb file. Its widths include the constant-divisor narrowing (below: widths across levels).

9.1 THE OBJECTS

- **Lane.** A lane ℓ reads its atom a_ℓ, the fields of its records.
- **Floor.** A floor S_k is the set of registers live at a cut between two rounds. It is the state one round leaves and the next reads.
- **Round.** A round f_k is a straight-line list of steps. It reads S_{k − 1} and leaves S_k. Its steps fill the register file while they run. At the floor, register reuse has freed everything except S_k.
- **Stack.** The stack is F = f_n ∘ … ∘ f_1, imprinted once as one key. The imprint derives every register's width across the whole composition before any lane runs.
- **Sweep.** A sweep applies F to every lane in one launch: R_ℓ = F(a_ℓ).

9.2 THE ALGEBRA

1. **Composition regroups and keeps its order.**
   - $(h \circ g) \circ f = h \circ (g \circ f)$, but $g \circ f \neq f \circ g$ in general.
   - A stack can be cut into programs at any floor and the programs chained, and every result stays the same.
   - **Proved** for tables (M10): a table read through a table equals the ops, and the reversed order differs on some lane.
   - **Proved** for floors (below): 700 floors run as one program equal the same floors run as 700 chained programs, record for record, at four lane counts.
2. **The file holds live registers only.**
   - A register is live from the step that writes it to its last reader. With reuse on, the file needs the most limbs live at any one step, plus any gap first fit leaves between registers of different limb counts.
   - The atom is not in the file. A field step copies its field from the record, in device memory, into a register. That register counts against the file only until its last reader. A program that reads many fields, each read late and used at once, runs in a small file: one lifting level over a 4-D cone reads 625 fields in 15 limbs (derived, below).
   - The file's size follows the order of the steps. At a floor it holds that floor's registers. Inside a round it holds the registers of the floor below still to be read, and the round's own registers not yet read for the last time.
   - The step count has no bound of its own. Each sign sits beside its register. The file (`ENGINE_RECORD_LIMBS_MOST`, 256 limbs) is the only bound on the program.
   - The record's own length is a separate bound (below).
   - **Measured:** 4,204 steps in an 8-limb file (the stack test), and in a 12-limb file with its words carried as 33-bit fields (the chained measure).
3. **Widths are derived, never declared.**
   - Each step's width follows from its operands' widths.
   - A register the imprint can show is never negative carries that fact forward:
     - a field read unsigned, a constant, an absolute value, a gcd, a table entry;
     - a sum, product, quotient or xor of two such registers;
     - an and with one such register.
   - With that fact, an and is no wider than its never-negative operand, and an xor of two never-negative registers is no wider than the wider one. A stack of rounds on fixed-width words then keeps its words at their width through every floor, instead of growing a bit per round.
   - Every width is checked against the values: the bitwise oracle rebuilds each output bit from the raw fields and requires the sign to hold 64 bits past the written width.

4. **Time: what the stack compresses.**
    - **Chained**, each of the n floors pays a launch, a synchronization, and a round trip of its state through device memory, on top of its steps.
    - **Stacked**, the lane reads its atom once, runs every step in registers, and writes its record once. The time between floors is gone.
    - The lanes run side by side on the device. Until the device is full, a stack over L lanes costs one tower's depth for the whole batch.
    - **Measured** (below): stacked is 13 to 22 times faster than chained.
5. **Time: what the stack leaves as it is.**
    - Each lane still evaluates every step in order. Once the device is full, a sweep's time grows linearly with the number of steps.
    - The only ways to shorten that are fewer steps:
        - a table (a sub-function of up to 32 input bits becomes one lookup, and composition keeps its order);
        - narrower registers (fewer limbs a step);
        - folded, dropped and shared steps (below: steps and their reduction).
6. **Exactness.**
    - Every step is exact integer arithmetic.
    - The host evaluates the same key with the exact integer library.
    - A device that equals the host word for word proves the port, for a stack as for any program.

9.3 PROVED AND MEASURED

- **The stack test** (`test/engine/record_bitwise_test`). 700 floors of one round each, over four 32-bit words: xor, and, sum, xor with the floor's constant, and a 32-bit wrap.
    - 4,204 steps, one program, register reuse on, an 8-limb file.
    - On 1,024 lanes the device equals the host word for word. Both equal the same rounds run on the CPU's own 64-bit two's complement at every tapped floor.
- **Stacked against chained.** The same 700 floors, run as one program of 4,204 steps (a 12-limb file) and as 700 programs of 13 steps. Each chained sweep reads the previous sweep's records from device memory. RTX 3070, the second sweep of each timed:

lanes	stacked, one sweep	chained, 700 sweeps	ratio	records
1,024	4,274 µs	55,899 µs	13.1×	equal
16,384	4,122 µs	71,419 µs	17.3×	equal
262,144	26,048 µs	572,534 µs	22.0×	equal

lanes	stacked, one sweep	chained, 700 sweeps	ratio	records
1,048,576	98,783 µs	2,088,411 µs	21.1×	equal

  - At 1,024 and 16,384 lanes the stacked sweep takes the same time: the device is not yet full, and the batch costs one tower's depth.
  - From 262,144 lanes the time grows with the lanes, and the ratio settles near 21×. That ratio is the cost of the time between floors, which the stack removes.
  - At 1,048,576 lanes the stacked sweep evaluates 4.4 × 10^9 steps in 98.8 ms, about 4.5 × 10^10 steps a second.
  - The measure's source is stored outside the repository and has not been rerun from it.

9.4 WHAT THE STACK IS NOT

- It does not evaluate fewer steps than the program has. The depth stays; the compression is of the time between floors and across the lanes.
- It does not change a result: a stack and its chained programs give the same records.

9.5 STEPS AND THEIR REDUCTION

- **A lane runs every step.** Derived from `cycle.cu`: one thread runs one lane's steps one after another. A lane's time grows with its step count, not with the program's dependency depth: two steps with no path between them still run in turn. The device runs lanes side by side, never the steps of one lane.
- **keymath takes the program as given.** Derived from `keymath.cu`: the key holds exactly the steps asked for, in their order (`key->steps = request->count`).
  - A step whose operands are all constants is not folded into one constant.
  - A step no output reads is not dropped.
  - Two equal steps are not merged.
  - Every reduction of the step count is the program writer's.
- **The reductions.** Each keeps every record the same.
  - Fold: a step over constants only becomes one constant.
  - Drop: a step no output depends on is removed.
  - Share: a value computed twice is computed once and read twice. The shared register lives from its first reader to its last, and the file pays for that span.
  - Table: a chain of one-register steps becomes one table step (below: tables).
- **Sharing trades steps for file.** Derived (replica). The proved lifting program emits its four distinct constants (1, 2, 3 and 4) 33 times: two for each of its 16 floor divisions, and one 2 read by both the low-pass coefficients and the evens.

  - Each constant emitted once and shared: 100 steps in place of 129, and a file of 15 limbs in place of 12.
  - The forward still equals tower.cu's formulas, and $T^{-1} \circ T$ is still the identity, on 500 random lines.

9.6 THE RECORD'S LENGTH

Derived from `key_schedule.cu` and `cycle.cu`:

- A member's record length, `in_limbs`, is an unsigned int with no cap of its own. Laying the key checks only that each field lies inside it: field_offset + bits $\leq$ 32 · in_limbs.
- A field's offset is a 32-bit bit offset, and the kernel computes the bit it reads in 32-bit arithmetic.
  - A lane can address the first 2^32 bits of each record: 2^27 limbs, 512 MiB.
  - The lay check runs in 64-bit arithmetic. It does not refuse a field that ends past bit 2^32 of a record that long, and the kernel's bit arithmetic would wrap for that field.
- A lane reads at most 3 members (`ENGINE_RECORD_MEMBERS_MAX`), one record from each, chosen by the index. Each member is its own device array. Nothing in the kernel stops the three from being the same array.
- The records sit in device memory: bodies × in_limbs × 4 bytes for each member. A record's address is computed in 64-bit arithmetic. Past the 2^32-bit offset, the device's memory is the practical bound on a record's length.
- A lane's output record holds each output at its width plus one sign bit. The lay checks their sum stays below 2^31 bits.
- The two bounds are separate. The record bounds what a lane can read, and the file bounds what it can hold at once.

9.7 THE TWO TOWERS

Doug's: a second tower stacked over the first one's boundary, inverted, and the two collapse together. One is the crystal, the tower of the data's lifted floors. The other is the tower of operations, the stack above.

- **The crystal's tower T** is the 5/3 integer lifting of `engine/base/tower/tower.cu` (A14 in engine_table.md). One level over a line of samples x:
  - the high-pass coefficient d_j = x_{2j+1} − ⌊(x_{2j} + x_{2j+2}) / 2⌋;
  - the low-pass coefficient s_i = x_{2i} + ⌊(d_{i − 1} + d_i + 2) / 4⌋;
  - an edge repeats its neighbor.

- **Its inverse T^{-1}** runs the same two lines backward and in the other order: x_{2i} = s_i − ⌊(d_{i − 1} + d_i + 2) / 4⌋, then x_{2j+1} = d_j + ⌊(x_{2j} + x_{2j+2}) / 2⌋.
- **Floor division is a record floor.** For every integer v and k ≥ 0:

  ⌊v / 2^k⌋ = EXACT_QUOTIENT(v − AND(v, 2^k − 1), 2^k)

  - The and reads v's two's complement without end. With the never-negative mask 2^k − 1 it gives v's residue modulo 2^k, in [0, 2^k), for a negative v as for a positive one.
  - v less its residue is a multiple of 2^k. The exact quotient divides it with nothing left over, and the quotient rounds toward −∞, as `tower_floor_shift` does.
  - The residue is no wider than the mask, k bits, by the never-negative width rule (3).
- **T and T^{-1} are stacks of record floors.** Every step of a lifting level is a sum, a difference, a constant or that floor division. A level is a floor of the record machine, L levels are L floors, and the inverse is L more.
- **The operation tower over the crystal.** A program F that reads the crystal's lifted floors, and the inverse T^{-1} that brings them back, compose into one stack $F \circ T^{-1}$ by the regrouping law (1). No step between them leaves the lane, and $T^{-1} \circ T$ is the identity on the machine exactly.
- **Proved** (`test/engine/record_bitwise_test`): one 5/3 level over 8 signed 16-bit samples and its inverse, 129 steps as one program, register reuse on, a 12-limb file, 4,096 lanes of edge-shaped samples.
  - The device equals the host word for word.
  - The forward floor's 4 low-pass coefficients and 4 high-pass coefficients equal tower.cu's formulas, computed on the CPU, on every lane.
  - The inverse floor returns all 8 samples exactly on every lane.

9.8 THE LIFTING AS RECORD FLOORS

- **The cone.** Derived.
  - One level: a high-pass coefficient reads 3 samples (x_{2j} to x_{2j+2}), a low-pass coefficient reads 5 (x_{2i − 2} to x_{2i+2}).
  - L levels along a line: a level-L low-pass coefficient reads 5 consecutive level-(L − 1) low-pass coefficients, spaced 2^{L − 1} samples apart. Its span w_L = w_{L − 1} + 4 · 2^{L − 1}, with w_1 = 5, gives w_L = 2^{L+2} − 3 samples. It reaches 2^{L+1} − 2 samples past its center on each side.
  - Separable in D axes, one level along each axis in turn: the span is the product, (2^{L+2} − 3)^D. One level in 4-D reads 5^4 = 625 samples.
  - The replica agrees for L = 1 to 6 in 1-D and for D = 1 to 4 at one level. Its forward equals tower.cu's formulas, computed on the CPU, on every line it ran.

- **The file of one cone.** Derived (replica). Only the steps one interior low-pass coefficient reads, in depth-first order, with each field read just before its first use:

levels L (1-D)	samples read	steps	file	the low-pass coefficient's derived width
1	5	28	6 limbs	20 bits
2	13	119	11 limbs	24 bits
3	29	338	17 limbs	28 bits
4	61	813	23 limbs	32 bits
5	125	1,800	29 limbs	36 bits
6	253	3,811	40 limbs	40 bits

axes D (one level)	samples read	steps	file	the all-low-pass coefficient's derived width
1	5	28	6 limbs	20 bits
2	25	163	9 limbs	24 bits
3	125	838	12 limbs	28 bits
4	625	4,213	15 limbs	32 bits

  - The file stays far below the samples read. The count of samples read is not a bound on the file.
  - Depth-first is one order among many, and the order with the smallest file is not known here. Finding the smallest register file for a program without recomputation is NP-complete in general (Sethi, "Complete register allocation problems", SIAM J. Comput. 4, 1975).

- **Widths across levels.** Derived.

  - **The constant-divisor narrowing**. keymath gives a quotient or exact quotient whose divisor is a constant c the dividend's width less ⌊log2 c⌋ bits, never under 1. The engine's quotient rounds toward zero and the exact quotient does not round: |q| ≤ |v|/c < 2^{w − ⌊log2 c⌋} for a w-bit dividend. The floor toward −∞ belongs to the lifting's composite of an and, a difference and an exact quotient, never to one operation. The exact quotient now works at its numerator's width and checks q · c against the whole numerator. **Proved** (`test/engine/record_divide_test`, 19 checks, 0 failed): 40-bit factors times 3, 12 and 2^32 + 7 divide back exactly, a 64-bit value over 2^32 + 7 equals the library's quotient, and the widths are exactly the rule's.
  - With it, keymath gives a high-pass coefficient of 16-bit samples 18 bits and a low-pass coefficient 20. Each level adds 4 bits to a low-pass coefficient: 16, 20, 24, 28, and 40 at level 6, two limbs. Before it, 7 bits a level: 23, 30, and 58 at level 6. The proved program's file is 12 limbs under either rule.
  - The values grow far less. The low-pass coefficient's linear part is (−1, 2, 6, 2, −1)/8 over x_{2i−2} to x_{2i+2}, with Σ|c| = 1.5. Its floors add less than 3/4.
  - Over samples |x| ≤ B: |s| ≤ 1.5B + 1 and |d| ≤ 2B. The edge cases (a line of 2 or 3, either end) repeat a neighbor and keep Σ|c| ≤ 1.5 for a low-pass coefficient and 2 for a high-pass coefficient.

  - The low-pass coefficient's bound grows log2 1.5 ≈ 0.585 bits a level and the high-pass coefficient's 1 bit, against keymath's 4.
  - A wrap to w = bit_length(bound) + 1 bits after each level holds every value in range exactly: [− 2^{w − 1}, 2^{w − 1}) contains [− bound, bound].
  - A wrap is silent: the machine does not check that its value is in range. A wrong wrap gives a wrong record on the device and the same wrong record on the host, and device = host still holds. Only an oracle outside the machine, such as tower.cu's formulas, catches it.

- **A whole line, down and back.** Derived (replica). 64 samples, all 6 levels, then all 6 inverse levels, in one program, in the order they are emitted:

widths	steps	widest register	file	$T^{-1} \circ T$ = id on 200 lines
derived by keymath	1,966	58 bits	130 limbs	yes
wrapped at the bound after each level	2,092	39 bits	122 limbs	yes

  - The 126 extra steps are the wraps, one for each coefficient.

9.9 THE NEIGHBOR GATHER

- **The limit.** In one sweep a lane reads at most 3 records. A coefficient's cone must lie in those records.

- **One axis: tiles, nothing duplicated.** Derived. Lay the line as records of t samples each. A lane reads its left, own and right tiles as its three members through the index. Its L-level coefficients reach 2^{L+1} − 2 samples to each side, which lie in the three tiles when 2^{L+1} − 2 ≤ t.

- **D axes.** Derived. A tile's one-level cone touches 3^D − 1 neighbor tiles: 8, 26 and 80 in 2, 3 and 4 axes. Three members reach the neighbors along one axis only. Two ways remain.

  - **The halo in the record.** Each record carries its tile and the halo its cone reads. One level on a tile of (2m)^D samples reads (2m + 3)^D, duplicating ((2m + 3)/(2m))^D. A lane then lifts its tile alone and recomputes what its neighbors also compute.
  - **One axis per sweep.** Each sweep lifts along one axis, with the three members as the left, own and right tiles along it. A D-axis crystal of L levels takes D · L sweeps, with no duplication and no recomputation. Each boundary between sweeps pays the chained cost.

- **What a tile costs.** Derived (replica). A lane per tile, one level, every coefficient its tile owns, in depth-first order:

D	tile	samples read	steps a sample	the whole grid, steps a sample	file
1	2	5	14.0	8.6	6 limbs
1	16	19	9.2	8.5	8 limbs
2	4	25	46.5	16.2	13 limbs
2	256	361	18.9	16.1	179 limbs
3	8	125	127.8	23.8	40 limbs
3	512	1,331	38.9	23.7	659 limbs
4	16	625	330.9	31.4	167 limbs
4	256	2,401	115.5	31.3	1,153 limbs

  - One level of the whole grid costs 7.5 steps a sample for each axis plus one field step: 8.5, 16, 23.5 and 31 in 1 to 4 axes. The replica's grids add one constant for each line.
  - A tile recomputes its halo. In 4-D a 16-sample tile runs 331 steps a sample against the grid's 31.
  - In this order a tile's file passes 256 limbs at 512 samples in 3-D and 256 samples in 4-D.
- **What a split costs.** Derived from the stacked-against-chained measure, which has not been rerun.
  - At 1,048,576 lanes the stacked sweep spends about 0.0224 ns a lane for each step.
  - The chained run's 700 × 13 steps account for about 214 ms of its 2,088 ms. The rest, about 2.68 ms a floor, is about 2.55 ns a lane, or about 114 step-times a lane at each boundary.
  - That floor carried 4 words a lane. The measure does not separate launch, synchronization and memory, and a boundary carrying more words may cost more.
  - On these numbers, one axis per sweep with t samples a lane pays about 114/t step-times a sample at each boundary. The halo of a 16-sample 4-D tile costs about 300 extra steps a sample.
- **Open:** which way costs less for the crystal, until the chained cost is measured apart; and a tile's file under a better order.

9.10 THE TWO BOUNDARIES

Doug's: the bottom boundary of the inverted tower and the bottom boundary of the upright tower are the same boundary, right next to one another, never touching.

- **The floors.**
  - T stands up from its floor 0, the samples. T^{-1} hangs down and ends on its floor 0, the rebuilt samples.
  - An operation program F stands up from its own floor 0, the atom. In $F \circ T^{-1}$, T^{-1}'s bottom is F's bottom.
  - A floor is a cut between steps. Its registers are the ones live at that cut.
- **Never touching, literally.**

  - T's floor 0 is the atom: fields in the record, in device memory, which field steps read.
  - T^{-1}'s floor 0 is registers in the file, written by steps and put into the output record.
  - **Proved** (`test/engine/record_bitwise_test`): on 4,096 lanes the rebuilt samples equal the input fields, value for value. They are never the same register. One side lives in the record read, the other in the file and the record written.

- **T is a bijection.** Derived.
  - Each lifting step adds to one half of the line a function of the other half only. The high-pass coefficients subtract a floored prediction made from the evens, then the low-pass coefficients add a floored update made from the high-pass coefficients.
  - A step $(a, b) \mapsto (a, b - P(a))$ is undone by $(a, c) \mapsto (a, c + P(a))$ for any function P, floors included: the same value comes back off.
  - Every level and every axis is a composition of such steps (Sweldens, "The lifting scheme", 1996; Calderbank, Daubechies, Sweldens and Yeo, "Wavelet transforms that map integers to integers", 1998).
  - T is a bijection of the integer vectors of each shape. $T^{-1} \circ T = \mathrm{id}$ and $T \circ T^{-1} = \mathrm{id}$ on every input, for every level count and axis order, when three things hold: the inverse repeats the forward's edge rule, it runs the steps in reverse order, and the machine refuses nothing (no register past $32 \cdot 256$ bits, every wrap in range).

- **The pair cancels.** Derived. $F \circ T^{-1} \circ T = F \circ (T^{-1} \circ T) = F$ by the regrouping law. F run on the crystal's floors through T^{-1} gives the same records as F run on the samples.

- **The pair costs steps and yields nothing.** Derived.
  - keymath does not see the identity: $T^{-1} \circ T$ emitted as steps runs every one of them.
  - In the proved program T is 69 steps (8 field steps and 61) and T^{-1} is 60. F on the samples needs only the 8 field steps.
  - When the record holds the samples, F on the samples saves every step of T and T^{-1}.
  - When the record holds only the crystal, $F \circ T^{-1}$ is the way down to the samples, and T^{-1}'s steps are its price.

- **Operations between the towers.** Derived. An operation G on the lifted floors, between T and T^{-1}, gives the program $T^{-1} \circ G \circ T$ on the samples: G conjugated by T.
  - Chains cancel inside: $(T^{-1}G_2T) \circ (T^{-1}G_1T) = T^{-1}(G_2G_1)T$. A chain of operations on the crystal pays one T and one T^{-1}, not a pair for each operation.
  - $T^{-1}GT = \mathrm{id}$ exactly when $G = \mathrm{id}$. The full cancellation survives only an identity in the middle.
  - $T^{-1}GT$ is a bijection exactly when G is one, and its inverse is $T^{-1}G^{-1}T$.
  - The levels above G cancel past it. Write $T = T_L \circ \ldots \circ T_1$, level 1 first. Level j reads and writes only the all-low-pass corner that level $j - 1$ left. A G that touches only coefficients outside the corner of level ℓ (the high-pass coefficients of level ℓ or below)

acts on other coordinates than every T_j above ℓ, and commutes with them. Then $T^{-1}GT = T_1^{-1} \circ \ldots \circ T_\ell^{-1} \circ G \circ T_\ell \circ \ldots \circ T_1$: a G on level ℓ's high-pass coefficients costs ℓ levels each way instead of L.

- **An edge is a record floor.** Derived from `tower_edge_kernel` (A14).
  - An edge permutes the low k bits of a coefficient through a table π ($k \leq 20$) and passes its high bits.
  - For every integer v: E(v) = (v − AND(v, 2^k − 1)) + TABLE_π(AND(v, 2^k − 1)).
  - The and is v's residue, never negative and k bits wide, and the table step indexes by it. v less its residue has k low zero bits, and the sum puts π's value in them. For a 32-bit word it agrees with the kernel's (word & ~mask) | π(word & mask).
  - E is a bijection of the integers. $T^{-1}ET$ is a bijection of the samples with inverse $T^{-1}E^{-1}T$. An edge on level ℓ's high-pass coefficients costs ℓ levels each way.
  - An edge moves a coefficient by less than 2^k. A wrap placed after an edge needs its bound raised by 2^k, or it is silently wrong.
- **Open:** which G on the lifted floors make $T^{-1}GT$ shorter than T, G and T^{-1} emitted apart. A G that commutes with T gives $T^{-1}GT = G$, run on the samples with no lifting at all. Which edges commute with T is open.

9.11 THE HEAP AND THE RING

Doug's framing (paraphrased): domain and range are complete and defined across the whole function. Information heaps, and stretched out it looks fuzzed, but it always stays within the domain's bound, "ever ringed". Abstractly it is a fuzzy oval brush stroke, and the more oval the shape, the more complex the information.

- **The heap** of a floor, on one lane: Σ over the floor's registers of the value's magnitude bits, plus 1 for the sign where the value is nonzero.
- **The ring** of a floor: Σ over the floor's registers of the imprint's widths. It belongs to the program and is the same on every lane.
- **Measured in a scratch run** (`ovoid_measure.cu`, not a committed test). One program: T over 64 signed 16-bit samples at 6 levels, then T^{-1}, with the widths before the constant-divisor narrowing.
  - 4,096 lanes in four classes by lane mod 4: a clean ramp, the ramp with noise of ±8, the ramp with noise of ±1,024, and raw noise over the whole 16-bit range.
  - The round trip is exact on 4,096 of 4,096 lanes at 3 and 6 levels, and the device equals the host word for word.
  - The mean heap:

floor	ramp	±8	±1,024	noise	ring
T0	857	851	856	960	1,024
T1	435	542	754	957	1,344
T2	229	389	702	955	1,504
T3	131	314	673	956	1,584
T4	87	279	658	956	1,624
T5	71	266	651	956	1,644
T6, the crystal	70	265	650	956	1,654
T^{-1} 5	71	266	651	956	1,663
T^{-1} 4	87	279	658	956	1,703
T^{-1} 3	131	314	673	956	1,827
T^{-1} 2	229	389	702	955	2,163
T^{-1} 1	435	542	754	957	3,011
T^{-1} 0	857	851	856	960	5,059

  - The heap pinches at the crystal by 12× for the ramp, 3.2× at ±8 and 1.3× at ±1,024, and not at all for noise.
  - The live bits peak at the last cut (5,079), not at the crystal (1,656).
  - 128 samples at 7 levels are refused: 93-bit registers overflow the 256-limb file.

- **The ring is the domain's bound.** Derived. The imprint's width W_r bounds register r's magnitude for every input in the fields' domain: |v_r| < 2^{W_r}. The magnitude part of the heap is at most the ring on every floor of every lane. The sign bits can add one bit a nonzero register: heap $\leq$ ring + the floor's nonzero registers. A 16-bit field holding $-32{,}768$ fills its 16 bits and takes a sign bit besides.

- **The crystal is the mirror's plane.** Derived. T^{-1} undoes the levels in reverse: after T_L^{-1} to T_{k+1}^{-1}, the line holds $T_k \circ \ldots \circ T_1$ of the samples. The inverse floor k holds the forward floor k's values, register for register, and the heap is an exact mirror across the crystal on every lane. The replica checks this on 300 lines.

- **The ring is not a mirror.** Derived. A width follows the path of steps that made the register, never its value, and nothing narrows it back. keymath cannot see $T^{-1} \circ T = \mathrm{id}$. The inverse side's widths grow from the crystal's: at T^{-1} 0, registers of 82 bits before the narrowing, and 58 after it, hold 16-bit values.

- **Two fixes.**
  - The constant-divisor narrowing (built and proved, above).
  - The wrap at the mirror (derived, built). The inverse floor k holds the forward floor k's values, each within its forward register's width b. WRAP(v, b + 1) on each rebuilt register passes its value through unchanged, and WRAP(v, 16) on the rebuilt samples is exact on 16-bit input. The bound comes from the mirror and holds whatever the values are. It needs $T^{-1} \circ T = \mathrm{id}$: with an operation between the towers the mirror breaks, and a wrap placed by it is silently wrong.
    * **Note.** Built. `boundary_mirror` (`test/engine/record_boundary_test.cu`) emits WRAP(v, b + 1) on every rebuilt register, b its forward twin's width, after the forward tower is imprinted alone for those widths. The rebuilt samples take the same wrap. Their twins are the 24-bit signed fields (`BOUNDARY_TEST_FIELD_BITS`,

:43), and the wrap there is WRAP(v, 25). The checks are at :1155, :1293 and :1296. WRAP(v, 16) at the samples' own width on 16-bit input is not exercised.

- **The ring under the three rules.** Derived (replica). The first column reproduces the scratch run's ring exactly.

floor	before the narrowing	narrowed	narrowed, wrapped at the mirror
T0	1,024	1,024	1,024
T1	1,344	1,216	1,216
T2	1,504	1,312	1,312
T3	1,584	1,360	1,360
T4	1,624	1,384	1,384
T5	1,644	1,396	1,396
T6, the crystal	1,654	1,402	1,402
T^{-1} 5	1,663	1,408	1,398
T^{-1} 4	1,703	1,434	1,388
T^{-1} 3	1,827	1,514	1,368
T^{-1} 2	2,163	1,730	1,328
T^{-1} 1	3,011	2,274	1,248
T^{-1} 0	5,059	3,586	1,024
steps; widest register; file	1,966; 82 bits; 171 limbs	1,966; 58 bits; 130 limbs	2,092; 43 bits; 84 limbs
128 samples, 7 levels	93 bits; 330 limbs, refused	65 bits; 261 limbs, refused	47 bits; 164 limbs

  - Wrapped at the mirror, each inverse floor's ring is the forward floor's plus one bit for each wrapped register. The wrap's signed range needs one bit over the forward register's magnitude width. Floor 0 is exact at 1,024.
  - The narrowing alone leaves 128 samples at 7 levels over the file. The wrap at the mirror brings it to 164 limbs.
  - **Measured in a scratch run** (on the device, 4,096 lanes; the round trip is exact and the device equals the host in every run):
    * Narrowed: the ring equals the replica's column floor for floor, in a 130-limb file. The run has 1,955 steps; the replica's 11 more are its own constant 2s. The live bits peak at the last cut, 3,606.
    * Wrapped at the mirror: T^{-1} 5 to T^{-1} 1 equal the replica's column, in an 84-limb file of 2,081 steps. The run wrapped the samples to 17 bits, not 16, and its T^{-1} 0 reads 1,088. The live bits peak at 1,546, at cut 1,023, against 1,010 at the crystal.
    * 128 samples at 7 levels: refused under the narrowing alone, its widest register 65 bits. Wrapped at the mirror, it loads in 164 limbs and 4,193 steps, with the ring 2,048 at the samples, 2,810 at the crystal and 2,176 at the rebuilt samples, and the round trip is exact on every lane.
- **The crystal as a code length.** Derived.
  - The heap alone is not a code length: nothing in it says where one value ends and the next begins.

  - The ring supplies the framing. For each register r, write its magnitude's bit count m_r in $\lceil \log_2(W_r + 1) \rceil$ bits, then, where $m_r > 0$, the $m_r - 1$ bits under the leading one and the sign. The decoder knows every W_r from the program.
  - The code's length is $\Sigma_r \lceil \log_2(W_r + 1) \rceil + \Sigma_r m_r$: the heap plus the framing, less one bit for each nonzero register. Any prefix code on the counts m_r serves in place of the fixed field.
  - T is a bijection. Decoding the crystal and running T^{-1} gives the lane back: $K(x \mid \text{the program}) \leq$ that length + c, for a constant c, the length of a fixed decoder.
  - The code states each coefficient alone, by its length and its bits, the same way for every register of a floor of the tower. It is a memoryless coder of the tower's coefficients, the class of rung F2 in compression_table.md. Its length is at least F2's multinomial terms for the same coefficients, and F2 is the tighter rung. As a two-part code it is F3′ with an empty model (kolmogorov_arnold.md, item 5).
  - The bound runs one way. K can be far below the crystal's code: a lane printed by a short program whose samples show no smoothness gets a noise-sized crystal.

- **Which shape is the oval.** Stated and not resolved; it is Doug's call.
  - The heap across the floors is an hourglass for simple data, pinched at the crystal. Its waist fills in as the complexity rises, and noise shows none.
  - The space between the heap and the ring is an oval, largest for simple data.

9.12 THE LENS

Doug's posit, the crystal as a lens between the physical data and its information content, is in wants.md. What the engine measures and derives about it follows.

- **What the measure supports.** Measured in the scratch run above: the crystal concentrates a lane's structure. The ramp's 857 bits go to 70, 12×, and the pinch ranks the four classes by their complexity. A cleaner crystal, a sparser heap, is a sharper focus, and the pinch is a per-lane reading of how much structure the lens sees.
- **What bounds it.** Derived.
  - T is a bijection: the crystal holds the lane exactly and adds or loses nothing. No prefix code of the crystal is shorter than $K(x \mid \text{the program}) - c$. The lens reaches toward K and never below it.
  - By counting, fewer than 2^{n-j} codewords are shorter than $n - j$ bits. Of the 2^n lanes of n bits, fewer than a 2^{-j} fraction can be coded j or more bits short, by any lossless coder, the crystal included. No lossless map shortens most inputs.
  - Noise gets no focus: measured, 960 bits to 956.
  - The lens's gain on a lane is at most $n - K(x \mid \text{the program}) + c$, the lane's own compressibility. The 5/3 lens sees only the smoothness its lifting predicts, and its gain is usually far less.

- **The tetration.** Derived. A gain of tetrated size needs a lane whose description is tetrated-small against its length. Such lanes exist, printed by tiny programs, but by counting they are a vanishing fraction, and the 5/3 lens need not find them. Any such gain comes from the input's description, never from the lens.
- **Open.**
  - Whether a ladder of lenses (more levels, other liftings, tables at the crystal) comes closer to K on a named class of lanes.
  - What "tetrated resources" measures, and against what bound.

9.13 F ∘ T⁻¹ FOR A LINEAR F

Derived.

- T^{-1} is not linear, because its floors are not. $F \circ T^{-1}$ is not a matrix even when F is one, and it cannot be folded into one matrix exactly.
- Its distance from the linear inverse W^{-1} (the same two lines without floors), for the same coefficients c, along one line:
  - An even sample's floor, $\lfloor (d_{i-1} + d_i + 2)/4 \rfloor$ against $(d_{i-1} + d_i)/4$, moves the even by $-1/2$, $-1/4$, 0 or $1/4$.
  - An odd sample's floor, $\lfloor (a + b)/2 \rfloor$ against $(a + b)/2$, moves the odd by 0 or $-1/2$.
  - The high-pass coefficients are read exact from c. An inverse level that receives its low-pass coefficients with error at most ε returns evens with error at most $\varepsilon + 1/2$ and odds with error at most $\varepsilon + 1$.
  - Over L levels, $\|T^{-1}c - W^{-1}c\|_\infty \le L$.
  - For a linear F with $\|F\|_\infty$ its largest row sum of |entries|: $|F(T^{-1}c) - F(W^{-1}c)| \le L \cdot \|F\|_\infty$ along one line.
- In D axes the high-pass coefficients of one axis carry the error of the axes inverted before them. The bound there is open.
- As a stack, $F \circ T^{-1}$ saves the time between F and T^{-1} (item 4), not steps: its step count is F's plus T^{-1}'s.

9.14 THE LANE INDEX AND THE LATCH

- **The lane index.** Derived from `cycle.cu`: the kernel holds each lane's number as a 64-bit loop variable, and no step reads it. The step `ENGINE_RECORD_LANE` writes it into a register.
- **The latch.** Derived. The first lane that meets a condition is the minimum over lanes of ℓ where the condition holds, with ∞ where it does not. The minimum is associative, commutative and idempotent. Any grouping returns the same lane as a serial scan from lane 0: a

tree of depth ⌈log2 lanes⌉, or a device-wide atomic minimum. `cycle_record_latch` reads it on the device by a tree over each warp and one atomic minimum, and `cycle_record_latch_host` reads it on the host (`cycle.h`).

9.15 TABLES

Derived.

- A chain of steps that each read one register, starting from a register of b ≤ 32 bits, is one function of that register. One table step of 2^b rows replaces the chain, and composition keeps its order (M10, proved).
- The table reads the low b bits of the register's magnitude: the file holds a magnitude and a sign.
  - A register that can be negative indexes by |v|, and v and −v read the same row.
  - A signed register goes into a table after it is made never negative, for example by adding 2^{b − 1} to a b-bit signed field.
- Two never-negative registers u and v, of b_u and b_v bits with b_u + b_v ≤ 32, index one table as u · 2^{b_v} + v. A constant, a product, a sum and the table step compute any function of the pair: 4 steps, or 3 when the constant is shared.
- A table of b index bits and out_bits bits holds 2^b · ⌈out_bits/32⌉ · 4 bytes: 4 GiB at b = 30 with one-limb rows, 16 GiB at b = 32. On an 8 GB device one table of one-limb rows fits up to b = 30.

9.16 OPEN

- **Depth by tables.** Which rounds' sub-functions fit a 32-bit index, and how much depth that removes.
- **The chained cost apart.** Launch, synchronization and the state's round trip through memory, each measured on its own.
- **The neighbor gather.** For D axes: the halo in the record, or one axis per sweep. Which costs less waits on the chained cost apart. A tile's file under a better order than depth-first is also open.
- **The smallest file.** An order of the lifting's steps with a smaller file than depth-first, or a proof that none exists.
- **$F \circ T^{-1}$ in D axes.** The bound on its distance from the linear $F \circ W^{-1}$, and any particular F that runs through T^{-1} in fewer steps than the two apart.
- **Operations that commute with T.** Which operations on the lifted floors, edges included, commute with T, and run on the samples with no lifting.

- **The heap and the wrap at the mirror as tests.** The scratch runs committed as tests, and the heap tabled under the narrowed widths. **Proved** since: `test/engine/record_boundary_test` (41 checks, 0 failed) runs T then T^{-1} over 64 samples at 4 levels with the mirror wraps, every floor an output. The heap mirrors on every lane, the ring is ring_0 + 6(n − n/2^ℓ) at floor ℓ and one bit wider for each wrapped low-pass coefficient at its mirror, and the pinch orders the four classes; the ring derived in two_crystals.md.
- **The oval.** Which shape it is, the heap's hourglass or the gap between heap and ring, is Doug's call.
- **The lens.** Whether a ladder of lenses comes closer to K on a named class of lanes, and what "tetrated resources" measures.
- **The whole crystal as one stack.** The proof above is one level along one line. All levels along all four axes as record floors, against tower.cu's own crystal, is not built.

10

THE TWO CRYSTALS: THE RECORD MACHINE OVER THE 2-ADIC INTEGERS

Purpose: the 2-adic structure of the record machine: the wrap as a projection, the operations that commute with every projection and the test that proves it, the exact quotient by an odd divisor as a 2-adic product, the lifting on the 2-adic integers and the bits it reads, the two limits of the finite windows and the solenoid between them, what passes to a limit and what does not, the crystal as a boundary measured on itself, the top projection and its limit $\mathbb{R}$, the odd crystals, the places of $\mathbb{Q}$, the count each crystal keeps, and Doug's posits, each bounded.

Scope: the record machine (M10 and A13 in engine_table.md) and the 5/3 lifting T as record floors (vertical_time_compression.md). The shape is Doug's: two crystals, the finite towers and their limit, with an infinite delta between them.

Labels.

- **Proved:** a named test checked it on the machine.
- **Measured:** a number a named run reported, where no check holds it to a value.
- **Derived:** argued here from the engine's source or by elementary algebra. Not run on the machine.
- **Open:** not settled.
- **Posit:** Doug's, stated in the field's terms, with what the math bounds of it.

10.1 THE OBJECTS

Derived.

- **The 2-adic integers $\mathbb{Z}_2$.** An element is a sequence of bits x_0, x_1, … without end, read as Σ x_k 2^k. Sums and products carry upward, as in binary, and never stop. Equivalently, $\mathbb{Z}_2$ = lim← $\mathbb{Z}/2^w$: an element is a sequence of residues r_w ∈ $\mathbb{Z}/2^w$ with r_{w+1} ≡ r_w mod 2^w (Koblitz, ch. 1 [27]; Gouvêa, ch. 3 [16]).
- **A register is a 2-adic integer.** The file holds a magnitude and a sign. XOR and AND read a register as its two's complement sign-extended without end (`engine_config.h`): zeros past the top bit for a value that is not negative, ones for a negative one. That bit sequence is the value in $\mathbb{Z}_2$. For example -1 = …1111, since 1 + …1111 carries to 0. The inclusion $\mathbb{Z} \subset \mathbb{Z}_2$ is a ring embedding.

- **The wrap is a projection.** WRAP(v, w) takes v modulo 2^w and reads it back signed, in [−2^{w−1}, 2^{w−1}); the machine takes $w \geq 4$. It is π_w: $\mathbb{Z}_2 \to \mathbb{Z}/2^w$, each residue class written by its signed representative.
  - The projections cohere: π_w ∘ π_{w+1} = π_w, and WRAP(WRAP(v, w + 1), w) = WRAP(v, w).
- **A window.** W_w = [−2^{w−1}, 2^{w−1}) ∩ ℤ, the values a wrap to w returns. π_w restricted to W_w is a bijection onto ℤ/2^w, and a wrap to w leaves every value of W_w as it is.
- **An odd c is a unit of $\mathbb{Z}_2$.** For example $1/3 = \ldots10101011: 3 \cdot 1011_2 = 33 \equiv 1 \bmod 16$, and the digits 10 repeat above the lowest two. A wrap to 8 bits gives −85, and $3 \cdot (-85) = -255 \equiv 1 \bmod 256$.
  - $\mathbb{Z}_2 \cap \mathbb{Q}$ is the rationals with odd denominators. Their expansions are exactly the eventually periodic ones (Gouvêa [16]).

10.2 COHERENCE: FIVE OPERATIONS COMMUTE WITH EVERY PROJECTION

Derived.

- π_w is a ring homomorphism. (a + b) mod 2^w depends only on a mod 2^w and b mod 2^w, and likewise a − b and a · b: a carry goes up, never down.
- Bit k of x xor y, and of x and y, reads bit k of x and bit k of y only. π_w keeps bits 0 to w − 1 and commutes with both.
- **The coherence theorem.** Let F be a straight-line program over SUM, DIFFERENCE, PRODUCT, XOR and AND. Write F_w for the same program with every input and every step wrapped to w. By induction on the steps, π_w ∘ F = F_w ∘ π_w: on every lane, WRAP(F(x), w) = F_w(WRAP(x, w)).
- The exact F's registers widen with the program. F_w's do not: each wrapped step reads operands in W_w, and its result is no wider than 2w bits before its wrap.

10.3 PROVED: THE COHERENCE TEST

Proved (`test/engine/record_coherence_test`, 9 checks, 0 failed; 14 checks, 0 failed, with the odd crystals below).

- **The programs.** 16 random programs of 10 operations. Each operation is drawn from the five and reads two values from among the 4 signed 24-bit fields and the operations before it. A program has at most 2 products, and its last operation reads the one before it. The widest exact register is 73 bits.
- **The lanes.** 4,096 lanes for each program. Each field is 0, −1, the most negative or the most positive 24-bit value, 1, or random (random 3 times in 8).

- **Three reckonings** at w = 5, 8, 13, 16, 31 and 32, all in one record program for each drawn program:
  - (a) the exact run, then a wrap to w;
  - (b) the same program with every input and every step wrapped to w;
  - (c) the CPU's own 64-bit two's complement over the same inputs, reduced to w and read signed.
- **Result.** 393,216 of 393,216 lane-widths agree three ways (16 programs × 4,096 lanes × 6 widths). The device's records equal the host's word for word.
  - (c) is itself a projection of the same 2-adic run: the CPU's word is F_64, and π_w ∘ π_64 = π_w.
- **Broken, as the theorem expects.** One signed 24-bit field at 8 bits: a quotient by 3 and a comparison with 0, each run on the field and on its wrap, then wrapped. The quotient disagrees on 2,212 of 4,096 lanes and the comparison on 1,790. The device equals the host.

10.4 WHAT DOES NOT FACTOR THROUGH THE PROJECTION

Derived. An operation G factors through π_w when WRAP(G(x), w) is a function of WRAP(x, w). For each failure below, two inputs share their w low bits and the outputs differ in theirs (w ≥ 4 throughout):

- **ABSOLUTE.** −1 and 2^w − 1: their absolute values 1 and 2^w − 1 wrap to 1 and −1.
- **COMPARE.** −1 is below 0 and 2^w − 1 above it.
- **QUOTIENT.** Broken on the machine (above).
- **REMAINDER** by 3. 1 and 1 + 2^w leave 1 and (1 + 2^w) mod 3, which is 0 or 2: 2^w is not a multiple of 3.
- **GCD** with 3. gcd(3, 3) = 3 and gcd(3 + 2^w, 3) = 1.
- **LADDER.** It reads the ratio of two magnitudes and refuses a right operand that is not positive. 2^{w−1} is positive and wraps to −2^{w−1}.
- **TABLE.** It indexes by |x| mod 2^b. −1 and 2^w − 1 read rows 1 and (2^w − 1) mod 2^b, two different rows for b ≥ 2.
  - A table indexed by the two's complement residue x mod 2^b would factor through π_w for every w ≥ b: it reads only bits the projection keeps.
- **The floor shift** ⌊v / 2^k⌋, k ≥ 1. Its bits 0 to w − 1 are v's bits k to w + k − 1: it factors through π_{w+k}, not π_w. v and v + 2^w give floors 2^{w−k} apart.
- **EXACT_QUOTIENT by an even c** holds the floor shift inside it: for c = 2^k and v a multiple of it, v and v + 2^w give quotients 2^{w−k} apart.

10.5 THE EXACT QUOTIENT BY AN ODD DIVISOR

- **The 2-adic map.** Derived. Division by an odd c in $\mathbb{Z}_2$ is the product by the unit c^{-1}, and it commutes with every projection: $\pi_w(c^{-1}\, x) = \pi_w(c^{-1}) \cdot \pi_w(x)$.
- **The machine's operation is not that map on wrapped inputs.** Derived. EXACT_QUOTIENT refuses a lane whose numerator c does not divide as an integer, and a wrapped multiple of c is often not a multiple of c: $3 \cdot 50 = 150$ wraps at 8 bits to -106. The wrapped program F_w replaces the exact quotient with a PRODUCT by the constant $c^{-1} \bmod 2^w$ and a WRAP.
- **The identity.** Derived: for c odd and $c \mid v$,

  $\mathrm{WRAP}(v / c, w) = \mathrm{WRAP}(\mathrm{WRAP}(v, w) \cdot (c^{-1} \bmod 2^w), w)$

  Both sides are $\pi_w(v \cdot c^{-1})$. At 8 bits with $v = 150$ and $c = 3$: $c^{-1} \bmod 256 = 171$, and $(-106) \cdot 171 = -18{,}126 \equiv 50 \bmod 256$.
- **Proved** (`test/engine/record_coherence_test`, its third part). c = 3, 7 and 12345, and v = PRODUCT(u, c) for a signed 24-bit field u, on the same inputs.
  - At the six widths, WRAP(EXACT_QUOTIENT(v, c), w) and WRAP(PRODUCT(WRAP(v, w), CONSTANT($c^{-1} \bmod 2^w$)), w) both equal WRAP(u, w).
  - 73,728 of 73,728 lane-widths agree (3 divisors × 4,096 lanes × 6 widths). The device equals the host.
  - The host's c^{-1} is Newton's iteration in a 32-bit word, four rounds from x = c.
- **The kernel computes c^{-1}.** Derived from `cycle.cu` (`cycle_record_exact_quotient`).
  - It shifts the numerator and the divisor past the divisor's low zero bits.
  - It grows the odd part's inverse modulo $2^{32 \cdot \mathrm{work}}$ by Newton's $x \leftarrow x(2 - cx)$ from one word, where work is the larger of the numerator's limbs and the quotient's.
  - Newton converges 2-adically. An odd $c = 2m + 1$ has $c^2 = 4m(m + 1) + 1 \equiv 1 \bmod 8$: it is its own inverse to 3 bits. If $cx \equiv 1 \bmod 2^b$, then $c \cdot x(2 - cx) = 1 - (1 - cx)^2 \equiv 1 \bmod 2^{2b}$. In one word, four rounds give 6, 12, 24 and 48 bits, past the word's 32. The stored limbs then double each round up to work.
  - The quotient is the low limbs of numerator · inverse, and the product back against the whole numerator proves it.
- **The 2-adic reading.** Derived. The inverse register holds $\pi_{32 \cdot \mathrm{work}}(c^{-1})$, a window of an element of $\mathbb{Z}_2$ outside $\mathbb{Z}$ for every odd c other than ±1. Without the product back, the low limbs of numerator · inverse are $\pi_{32 \cdot \mathrm{work}}(v \cdot c^{-1})$ for any v, a window of a 2-adic integer. The check against the whole numerator makes the result the integer v / c, or refuses the lane.

10.6 THE LIFTING ON $\mathbb{Z}_2$

Derived.

- **The 2-adic floor shift.** For x ∈ $\mathbb{Z}_2$ and k ≥ 0, write x = r + 2^k y with 0 ≤ r < 2^k, x's residue. Then ⌊x / 2^k⌋ = y, the digits moved down k places. On $\mathbb{Z}$ it is the floor toward $-\infty$, the record floor of vertical_time_compression.md. On $\mathbb{Z}_2$ it is continuous: its bits 0 to w − 1 read x's bits k to w + k − 1.

- **T is a homeomorphism of $\mathbb{Z}_2$^n.** Every lifting step is a shear (a, b) ↦ (a, b ± P(a)), with P built from sums and floor shifts. It is defined on $\mathbb{Z}_2$ word for word and undone by the opposite shear. T is a bijection of $\mathbb{Z}_2$^n that extends T on $\mathbb{Z}$^n. T and T^{-1} are continuous, since each output's w bits read finitely many input bits.

- **T is not a wrapped program.** Two inputs equal in their w low bits can give high-pass coefficients that differ in bit w − 1: adding 2^w to x_{2j} moves ⌊(x_{2j} + x_{2j+2}) / 2⌋ by 2^{w − 1}. No F_w has π_w ∘ T = F_w ∘ π_w.

- **The bits one level reads.** Along one line, an interior coefficient (its neighbors inside the line), w ≥ 1. An output's w bits read each input to the count of bits given:

output	x_{2j+1}	x_{2j}, x_{2j+2}
the high-pass coefficient d_j	w	w + 1

output	x_{2i}	x_{2i − 1}, x_{2i+1}	x_{2i − 2}, x_{2i+2}
the low-pass coefficient s_i	w + 2	w + 2	w + 3

  - Upper bound. ⌊u / 2^k⌋ mod 2^w reads u mod 2^{w+k}. The high-pass coefficient reads the evens' sum to w + 1 bits. The low-pass coefficient reads d_{i − 1} + d_i to w + 2 bits, and a high-pass coefficient read to w + 2 bits reads its evens to w + 3.
  - Each count is exact. Adding 2^{m − 1} to an input read to m bits moves the output by 2^{w − 1} modulo 2^w:
    * the high-pass coefficient: x_{2j+1} + 2^{w − 1} moves d_j by 2^{w − 1}; x_{2j+2} + 2^w moves it by -2^{w-1};
    * the low-pass coefficient: x_{2i+2} + 2^{w+2} moves d_i by -2^{w+1} and s_i by -2^{w-1}; x_{2i+1} + 2^{w+1} moves d_i by 2^{w+1} and s_i by 2^{w − 1}; x_{2i} + 2^{w+1} moves both high-pass coefficients by -2^w and s_i by $2^{w+1} - 2^{w-1} \equiv -2^{w-1}$.
  - An edge repeats its neighbor and reads no more bits than the interior.

- **L levels.** A level-L low-pass coefficient's w bits read the samples to w + 3L bits, and a level-L high-pass coefficient's to w + 3L − 2.
  - Upper bound: a level reads the low-pass coefficients below it to 3 bits more, by the table.

  - Exact on the outermost sample of an interior cone. Of the level-($\ell - 1$) low-pass coefficients the cone reads, that sample reaches only the outermost, as its outer even. Adding 2^{m+2} to an outer even moves the low-pass coefficient above by exactly -2^{m-1} for $m \geq 1$. Adding 2^{w+3L−1} to the sample moves the outermost level-1 low-pass coefficient of the cone by 2^{w+3L−4} in magnitude, the level-2 low-pass coefficient by 2^{w+3L−7}, and the level-L low-pass coefficient by 2^{w−1}.
  - As a 2-adic modulus: $|T(x) - T(y)|_2 \leq 2^{3L} |x - y|_2$ along one line.
  - **Proved** on the machine, the bound and its exactness, per band: 3ℓ at the level-ℓ low-pass coefficients and $3\ell - 2$ at the level-ℓ high-pass coefficients ("The boundary", below).

- **The inverse reads fewer.** One inverse level: an even x_{2i} = s_i − ⌊(d_{i−1} + d_i + 2) / 4⌋ reads the low-pass coefficient to w bits and the high-pass coefficients to w + 2; an odd x_{2j+1} = d_j + ⌊(x_{2j} + x_{2j+2}) / 2⌋ reads the low-pass coefficients to w + 1 and the high-pass coefficients to w + 3. By induction down the levels, the samples' w bits read the level-ℓ high-pass coefficients to at most $w + \ell + 2$ bits and the level-L low-pass coefficients to w + L. T^{-1} reads the crystal to at most w + L + 2 bits: $|T^{-1}(c) - T^{-1}(c')|_2 \leq 2^{L+2} |c - c'|_2$. **Proved** exact on the machine at L = 4 ("The boundary", below).

- **What the counts mean.** A wrapped T at w needs its samples to w + 3L bits. The extra bits come from the floors: a SUM, DIFFERENCE or PRODUCT reads no bit above w. Precision flows down from the top bits, 3 a level along the low-pass coefficients forward and 1 a level back.

10.7 THE TWO CRYSTALS

Doug's: two crystals, the finite towers and their limit, with an infinite delta between them.

- **The inverse crystal, $\mathbb{Z}_2$.** Derived. $\mathbb{Z}_2 = \lim\leftarrow \mathbb{Z}/2^w$ under the projections: compact, and each element a coherent sequence of windows. Every register the machine writes lies in it, and every WRAP is one of its projections.

- **The direct crystal, as sets.** Derived. The windows W_w ⊂ W_{w+1} have union $\mathbb{Z}$, and every value the machine holds lies in some window. The union is of sets only.
  - A window is not closed under +: $2^{w-2} + 2^{w-2} = 2^{w-1}$ leaves W_w. Nor under ×.
  - No ring map $\mathbb{Z}/2^w \to \mathbb{Z}/2^{w+1}$ sends 1 to 1: $2^w \cdot 1 = 0$ would have to go to $2^w \neq 0$.

- **The direct crystal, as groups.** Derived. Under the injections $x \mapsto 2x$, $\mathbb{Z}/2^w \to \mathbb{Z}/2^{w+1}$, the direct limit is the Prüfer group $\mathbb{Z}[1/2]/\mathbb{Z}$ instead of $\mathbb{Z}$; $\mathbb{Z}/2^w$ sits in it as the multiples of 2^{-w}.
  - The Prüfer group is the Pontryagin dual of $\mathbb{Z}_2$ (Hewitt and Ross [18]). The two systems are dual map for map: the dual of a projection $\mathbb{Z}/2^{w+1} \to \mathbb{Z}/2^w$ is an injection $x \mapsto 2x$.

- **They agree on every finite projection.** Derived. π_w(ℤ) = π_w($\mathbb{Z}_2$) = ℤ/2^w for every w. Every x ∈ $\mathbb{Z}_2$ shares its w low bits with an integer, the signed representative of π_w(x) in W_w: ℤ is dense in $\mathbb{Z}_2$. No finite floor tells the two crystals apart.
- **The delta is infinite.** Derived.
  - ℤ is countable. $\mathbb{Z}_2$ is in bijection with the bit sequences {0,1}^ℕ and uncountable, by Cantor's diagonal. $\mathbb{Z}_2 \setminus \mathbb{Z}$ is uncountable.
  - Its part in ℚ, the rationals with odd denominators outside ℤ, is countable: the eventually periodic expansions. The machine reaches their windows through the exact quotient's inverse (1/3, above).
  - The computable elements of $\mathbb{Z}_2$ are countable, and a program can produce any window of one. All but countably many elements are not computable: each has a window at every w, as every element does, yet no program produces its windows for every w.
- **The solenoid.** Derived. The envelope joining the two crystals is the dyadic solenoid (Vietoris 1927 [57]; van Dantzig 1930 [55]):

  $\Sigma_2 = \lim\leftarrow(S^1, z \mapsto z^2) = (\mathbb{R} \times \mathbb{Z}_2) / \mathbb{Z}$, with ℤ embedded diagonally, n ↦ (n, n)

  - **The transversal.** Projecting to the first circle, (t, x) ↦ t mod 1, the fiber over a point is a copy of $\mathbb{Z}_2$, a Cantor set.
  - **The leaves.** The image of each line ℝ × {x} is a leaf: a line immersed injectively and dense in Σ_2. The leaf through (0, x) meets the fiber over 0 in the coset x + ℤ, and the leaf through 0 meets it in ℤ itself. The leaves are in bijection with $\mathbb{Z}_2$ / ℤ, uncountably many.
  - ℤ lies along one leaf as its integer points and inside the transversal as a dense subset.
  - **Dual.** Σ_2 is the Pontryagin dual of ℤ[1/2]. The exact sequence 0 → ℤ → ℤ[1/2] → ℤ[1/2]/ℤ → 0 dualizes to 0 → $\mathbb{Z}_2$ → Σ_2 → S^1 → 0: the solenoid is the circle extended by $\mathbb{Z}_2$, and its fiber is the dual of the direct crystal's Prüfer group.

10.8 GOING UP: WHAT PASSES TO THE LIMIT

Derived.

- **Unions of chains.** For a chain of structures A_0 ⊂ A_1 ⊂ … under embeddings, the union satisfies every ∀∃ sentence that holds in every A_i (Chang 1959 [10]; Łoś and Suszko 1957 [37]; Hodges [19], where the Fraïssé limit is built as such a union).
  - A ∀∃ sentence says: for every x there is a y with a quantifier-free relation. A witness y found in A_i stays a witness in the union, since the embeddings keep quantifier-free relations.
  - On the machine: the groups ℤ/2^w under x ↦ 2x satisfy "every x has a −x", and their union, the Prüfer group, does too.
  - The converse fails. The Prüfer group is divisible by 2, and no ℤ/2^w is: 1 has no half.

  - The windows W_w are not a chain of structures under + and ×. The theorem says nothing about them as rings.

- **Unions of bijections.** Let f_i: A_i → B_i be bijections with A_i ⊂ A_{i+1}, B_i ⊂ B_{i+1}, and f_{i+1} extending f_i. Then ∪ f_i: ∪ A_i → ∪ B_i is a bijection: any two points lie in one A_i, and any image point in one B_i.

  - Exactness passes. T restricted to W_w^n is a bijection onto its image, the restrictions extend one another, and their union is T on $\mathbb{Z}$^n, onto $\mathbb{Z}$^n. $T^{-1} \circ T = \mathrm{id}$ at each stage gives it on the union.
  - On $\mathbb{Z}_2$^n exactness comes from the shear argument directly (above), not from a union.

- **What does not pass.** Finiteness and termination.

  - ω is a transitive set: every member is finite, and ω is not.
  - Every W_w is finite, and $\mathbb{Z}$ is not.
  - Every F_w terminates in a fixed number of steps on w-bit words. T on $\mathbb{Z}_2$^n is not a finite computation: the machine computes $\pi_w \circ T$ from π_{w+3L}, one finite window at a time, never the whole.

10.9 THE BOUNDARY: THE CRYSTAL MEASURED ON ITSELF

Proved (`test/engine/record_boundary_test`, 41 checks, 0 failed; 48 checks, 0 failed, with the identity by null permutation in "Doug's posits"). A 5/3 tower T of L = 4 levels over n = 64 signed 24-bit samples runs as record floors. Each floor shift is three record steps: an AND with 2^k − 1, a DIFFERENCE, then EXACT_QUOTIENT by 2^k. The crystal is in Mallat order: the level-4 low-pass coefficients first, then the high-pass coefficients of levels 4 down to 1. Every program runs on the device and the host, and the records agree word for word.

- **T's matrix.** The test builds 2^12·M, column i the image of 2^12·e_i under T, and the same for T^{-1}. On multiples of 2^12 every floor is exact and the +2 offset drops out: M is T's linear part, with entries in $\mathbb{Z}[1/2]$.

- **The reach, per band.** A row's reach is 12 less the least 2-adic valuation of its entries. T's matrix reaches 12 bits from the level-4 low-pass coefficients, and 10, 7, 4 and 1 from the high-pass coefficients of levels 4, 3, 2 and 1: 3L at the low-pass coefficients and $3\ell - 2$ at the level-ℓ high-pass coefficients, the counts derived above. T^{-1}'s matrix reaches 6 = L + 2.

- **Flips.** 8,192 pairs of lanes for each map. A pair is a random input and the same input with one bit b flipped, b drawn from 0 to 22, below the sign bit: the flip moves the value by exactly ±2^b.

  - No flip moves an output below bit $b - 12$ for T, or below bit $b - 6$ for T^{-1}: 8,192 of 8,192 pairs each. The furthest reach met on the device is 12 for T and 6 for T^{-1}. Both bounds are exact.

  - A flip at b ≥ 12 moves the output by exactly ±2^b times the matrix's column: 3,918 of 3,918 such pairs for T and 3,837 of 3,837 for T^{-1} (the counts of such pairs are measured; the check holds every one).

- **Written onto the boundary and read back.** Arbitrary 24-bit crystals run through T^{-1} then T return exactly: 16,384 of 16,384 lanes. Every crystal is the crystal of some samples: the boundary is a whole coordinate chart of ℤ^n, and no crystal lies outside T's image.

- **What passes through T.** 2,048 pairs of each kind, the base samples below 2^21 in magnitude.

  - A constant c below 2^19 in magnitude, added to every sample, moves the 4 level-4 low-pass coefficients by c each and no other coefficient: 2,048 of 2,048.
  - A lattice move 2^12·z, z ∈ [−128, 127]^64, moves the crystal by exactly M·2^12·z: 2,048 of 2,048.
  - Negation and doubling: the check holds that neither passes on every pair. **Measured:** each passes on 0 of 2,048.
  - Derived. The floor commutes with adding an integer, ⌊(u + 2c) / 2⌋ = ⌊u / 2⌋ + c: a constant leaves every high-pass coefficient unchanged and moves every low-pass coefficient by itself, level by level. On 2^{3L}ℤ^n every floor is exact: T(x + 2^{3L}z) = T(x) + M·2^{3L}z. Negation fails on the floor, ⌊−u / 2⌋ ≠ −⌊u / 2⌋ for odd u, and doubling on the parity, 2⌊(a + b) / 2⌋ ≠ a + b for a + b odd.

- **Floors, heap and ring.** T then T^{-1} in one program, every floor's registers an output. Each rebuilt register is wrapped at the mirror to its forward twin's width + 1, the samples included, to 25 bits. A value's heap is its magnitude's bits and one sign bit when it is not zero. A floor's ring is the sum of the imprint's widths of its registers. Four classes of 1,024 lanes: a ramp (start below 2^19 in magnitude, slope from −2,048 to 2,047), the ramp ±8, the ramp ±1,024, and noise over the 24-bit field.

  - The last floor returns every sample exactly, on every lane.
  - The heap mirrors: the heap at floor 2L − k equals the heap at floor k, on every lane.
  - Every floor's heap lies inside its ring and one sign bit for each nonzero value, on every lane.
  - The ring at forward floor ℓ ≥ 1 is ring_0 + n + 2(n − n/2^ℓ), and at the mirror floor 2L − ℓ it is ring_ℓ + n/2^ℓ, one bit for each wrapped low-pass coefficient. The ring is widest at the crystal. Floors 0 to 8: 1,536, 1,664, 1,696, 1,712, 1,720, 1,720, 1,712, 1,696 and 1,600 (keymath's linear forms, `build/20260925_013426_record_boundary_test`). Under the widths before, the law was ring_0 + 6(n − n/2^ℓ), with floors 1,536, 1,728, 1,824, 1,872, 1,896, 1,880, 1,840, 1,760 and 1,600.
  - The heap's pinch, a class's heap at floor 0 over its heap at the crystal, falls from the ramp to ±8 to ±1,024 to noise (the order is checked). **Measured:** 9.61, 3.82, 1.76 and 1.00.

- **The ring, derived.** With keymath's linear forms (A16 of engine_table.md), every register level ℓ makes, low-pass and high-pass, is w + ℓ + 1 bits.
  - The first level turns the n samples of width w into n/2 high-pass coefficients and n/2 low-pass coefficients of width w + 2: the ring grows by 2n.
  - Each level after turns the n/2^{ℓ − 1} low-pass coefficients of width w + ℓ into n/2^ℓ high-pass coefficients and n/2^ℓ low-pass coefficients of width w + ℓ + 1: the ring grows by n/2^{ℓ − 1}.
  - Summed, ring_ℓ = ring_0 + n + 2n(1 − 2^{−ℓ}) for ℓ ≥ 1.
  - Under the widths before, the constant-divisor narrowing included (vertical_time_compression.md), a level turned the n/2^{ℓ − 1} low-pass coefficients of width W into n/2^ℓ high-pass coefficients of width W + 2 and n/2^ℓ low-pass coefficients of width W + 4, a growth of 6n/2^ℓ, and ring_ℓ = ring_0 + 6n(1 − 2^{−ℓ}).
  - The mirror floor adds one bit for each of its n/2^ℓ rebuilt low-pass coefficients, the wrap's + 1.
  - Under the widths before, the formula reproduced the replica's column in vertical_time_compression.md (16-bit samples, n = 64, L = 6): 1,024, 1,216, 1,312, 1,360, 1,384, 1,396 and 1,402 going up, 1,398, 1,388, 1,368, 1,328 and 1,248 coming down. The replica's T^{-1} 0 read 1,024, with the samples wrapped to 16 bits. The scratch run there wrapped them to 17 and read 1,088 = 1,024 + 64, as the formula gives.
  - The growth is n + 2n(1 − 2^{−L}) < 3n for every L: under 3 bits a sample, however many levels. Under the widths before it was under 6.
- **The ring's shape.** Derived: up the arc ring_0 + n + 2n(1 − 2^{−ℓ}), each level after the first adding half the level before it, widest at the crystal, and back down the same arc plus n/2^ℓ. Which shape is the oval is Doug's call. His words were a "fuzzy oval brush stroke" and "the more oval the shape, the more complex the information base".
- **Haar counted.** A tower of 4 samples and one level, T and T^{-1}, each reaching 3 bits. For w = 1 and 2 every input quantum at level w + 3 is run, all 2^{4(w+3)} residue vectors, from the representatives 0 ≤ x < 2^{w+3} and from −2^{w+2} ≤ x < 2^{w+2}. An output's low w bits name its quantum at level w. In 8 of 8 runs every output quantum receives exactly 2^12 = 4,096 input quanta (derived in "Counting quanta", below).
- **Volume.** det(2^12·M) is checked against ±2^768 modulo primes below 2^31, until their product passes Hadamard's bound plus 2 bits, about 779 bits: agreement modulo all of them is equality. det M = det M^{-1} = ±1 exactly, and the product of the two matrices is 2^24·I exactly. **Measured:** the sign is +1; the check accepts either.
  - **The sign, derived.** Each lifting step is a shear, determinant 1. The Mallat order is a chain of unshuffles, one a level: a band of 2k values into its k evens, then its k odds. The odd at 2i + 1 precedes the k − 1 − i evens after it: k(k − 1)/2 inversions, sign (−1)^{k(k−1)/2}. For k = 32, 16, 8 and 4 the exponents are 496, 120, 28 and 6, all even: det M = +1.

10.10 THE TOP PROJECTION AND ITS LIMIT ℝ

Proved (`test/engine/record_boundary_test`). The top projection τ_k(x) = x / 2^k toward zero, the machine's QUOTIENT by a power of two, keeps the top of the window where π_w keeps the bottom. (k, c) = (3, 3), (7, 5) and (5, 12,345), 65,536 lanes each, 196,608 in all, x and y random signed 24-bit fields.

- Nested quotients commute, every quotient toward zero: τ_k(x) / c = τ_k(x / c) on 196,608 of 196,608.
- A COMPARE through τ_k is never reversed: on every lane τ_k(x) against τ_k(y) is a tie or the order of x against y. **Measured:** 4 ties.
- A SUM through τ_k is off by at most 1: τ_k(x + y) − τ_k(x) − τ_k(y) is −1, 0 or 1 on every lane, and the check holds that a carry occurs. **Measured:** carried on 92,890.
- An 8-bit WRAP reverses comparisons τ_k keeps. **Measured:** 97,966 lanes.

Derived.

- **The two projections are dual.** π_w is a ring map and scrambles order. τ_k keeps order, weakly, and adds only up to a carry: a truncation drops a fraction of its argument's sign and of magnitude below 1, and the three dropped fractions net to −1, 0 or 1.
- **τ_k's fibers are not all one size.** $\tau_k^{-1}(0) = (-2\text{^}k, 2\text{^}k) \cap \mathbb{Z}$ holds 2^{k+1} − 1 integers, and every other fiber 2^k. π_w's fibers are the cosets of 2^wℤ, translates of one another: the Haar count ("Counting quanta") has no analog for truncation at 0.
- **The τ tower.** τ_j ∘ τ_k = τ_{j+k}: nested quotients toward zero, the check above with c a power of two.
  - Its inverse limit lim←(ℤ, τ_1) is the sequences (a_k) with a_k = τ_1(a_{k+1}). It maps onto ℝ by (a_k) ↦ lim a_k / 2^k: a step moves a_k / 2^k by at most 2^{−(k+1)}, and t ∈ ℝ has the preimage a_k = trunc(2^k t), truncated toward zero, since trunc(trunc(y) / 2) = trunc(y / 2).
  - The map is one-to-one except at the nonzero dyadic rationals, where it is two-to-one: 1 is the limit of a_k = 2^k and of a_k = 2^k − 1, the expansions 1.000… and 0.111….
  - π_w's tower adds a bit above at each stage, and its limit is $\mathbb{Z}_2$. τ_k's tower adds a bit below, and its limit is ℝ. The solenoid $\Sigma_2 = (\mathbb{R} \times \mathbb{Z}_2)/\mathbb{Z}$ holds both, ℝ along its leaves and $\mathbb{Z}_2$ in its fiber.
  - This is the derived form of Doug's inverted boundary ("Doug's posits", below).

10.11 THE ODD CRYSTALS

Proved (`test/engine/record_coherence_test`, its last part, 14 checks in all, 0 failed). The moduli m = 243, 59,049, 625 and 343 (3^5, 3^10, 5^4 and 7^3). 16 ring-only programs of SUM, DIFFERENCE and PRODUCT, 4,096 lanes each.

- Three reckonings agree on 262,144 of 262,144 lane-moduli: REMAINDER by m of the exact run; the run with every step reduced by REMAINDER by m; the host's arithmetic mod m. REMAINDER carries the numerator's sign, and each is read mod m.
- The CRT join y_2 + 2^8·(((y_p − y_2)·2^{−8}) rem m), with y_2 the run mod 2^8 and y_p the run mod m, equals the exact run mod 2^8·m on 262,144 of 262,144.
- XOR breaks the agreement mod 3. **Measured:** on 827 of 4,096 lanes.

Derived.

- REMAINDER by p^v, read mod p^v, is the projection $\mathbb{Z}_p \to \mathbb{Z}/p^v$ on the integers: the odd crystal's π. SUM, DIFFERENCE and PRODUCT commute with it, by the coherence theorem with p^v in place of 2^w.
- XOR and AND read base-2 digits and do not factor through mod 3. For XOR: 3 xor 1 = 2, while 3 ≡ 0 and 0 xor 1 = 1. For AND: 3 and 1 = 1, while 0 and 1 = 0. The test breaks XOR.
- **The crystals are orthogonal.** For m odd, $\mathbb{Z}/2^8 \cdot m \cong \mathbb{Z}/2^8 \times \mathbb{Z}/m$ (the Chinese remainder theorem), and the join above is the inverse of the isomorphism. Over every modulus, $\hat{\mathbb{Z}} = \varprojlim \mathbb{Z}/N = \prod_p \mathbb{Z}_p$: $\mathbb{Z}_2$ is one factor, and the odd crystals are the others.

10.12 THE PLACES OF $\mathbb{Q}$

Derived.

- **Ostrowski** (1916 [39]): every nontrivial absolute value on $\mathbb{Q}$ is equivalent to the real one or to a p-adic one. The completions of $\mathbb{Q}$ are $\mathbb{R}$ and the $\mathbb{Q}_p$, one for each prime. The machine's two limits are two of them: $\mathbb{R}$ by the τ tower and $\mathbb{Z}_2 \subset \mathbb{Q}_2$ by the π tower.
- **T at each place.**
  - On $\mathbb{Z}^n$, a bijection (proved, the written boundary).
  - On $\mathbb{Z}_2^n$, a homeomorphism that keeps Haar measure (derived; proved counted).
  - On $\mathbb{R}^n$, reached by the τ tower: T(x) − Mx is bounded on $\mathbb{Z}^n$, since each floor drops less than 1 and passes through a fixed number of fixed linear steps. For $x \in \mathbb{R}^n$, $2^{-k} \cdot T(\text{trunc}(2^k x)) \to Mx$ as $k \to \infty$: the real limit of T is its linear part M, and det M = 1 keeps Lebesgue measure.
  - On $\mathbb{Z}_p^n$ for odd p, T with floors does not extend. Parity is not p-adically continuous: x and x + p^N are p-adically close, $|p^N|_p = p^{-N}$, and have opposite parities. For x even, $\lfloor (x + p^N)/2 \rfloor - \lfloor x/2 \rfloor = (p^N - 1)/2 \equiv -1/2 \bmod p$, a p-adic unit however large N is.
  - M does extend. Its entries lie in $\mathbb{Z}[1/2] \subset \mathbb{Z}_p$ and det M = 1: $M \in SL_n(\mathbb{Z}_p)$, an automorphism of every odd crystal $\mathbb{Z}_p^n$ that keeps its Haar measure. $|\det M|_v = 1$ at every place v.

- **The adeles.** $\mathbb{A} = \mathbb{R} \times \Pi' \mathbb{Q}_p$, the restricted product, with almost every component in $\mathbb{Z}_p$. $\mathbb{Q}$ sits in $\mathbb{A}$ diagonally and discretely. $\mathbb{A} = \mathbb{Q} + (\mathbb{R} \times \hat{\mathbb{Z}})$ and $\mathbb{Q} \cap (\mathbb{R} \times \hat{\mathbb{Z}}) = \mathbb{Z}$, giving $\mathbb{A}/\mathbb{Q} \cong (\mathbb{R} \times \hat{\mathbb{Z}})/\mathbb{Z}$, compact: the full solenoid, $\lim\leftarrow(S^1, z \mapsto z^N)$ over every N (Tate 1950 [54]). The dyadic solenoid $\Sigma_2 = (\mathbb{R} \times \mathbb{Z}_2)/\mathbb{Z}$ is its quotient by the odd factors $\Pi_{p \text{ odd}} \mathbb{Z}_p$.

- **Zeta at the places.** Derived. Zeta has one factor at each place: the Euler product at the finite places, and $\pi^{-s/2}\Gamma(s/2)$ at $\mathbb{R}$, whose phase on the critical line is $\theta(t)$. The machine's $Z(t) = e^{i\theta(t)}\zeta(1/2 + it)$ is the main sum over the finite places turned by the real place's phase. What the record machine reads of the zeros there is in the analytic number theory workbook, "The zeros in the engine's field".

- **The product formula.** For $x \in \mathbb{Q}^\times$, $|x|_\infty \cdot \Pi_p |x|_p = 1$: with $x = \pm\Pi p^{v_p}$, $|x|_\infty = \Pi p^{v_p}$ and $|x|_p = p^{-v_p}$. For 12: $12 \cdot 1/4 \cdot 1/3 = 1$. Read as a conservation law, a rational large at some places is small at others in exact balance. The name is a reading; the theorem is the formula.

10.13 COUNTING QUANTA

Derived.

- **Haar measure as coset counting.** A quantum at level w in $\mathbb{Z}_2^n$ is a coset of $(2^w\mathbb{Z}_2)^n$, the points with one value of π_w. The Haar measure μ gives it 2^{-wn}.

- **A shear keeps μ.** A shear $(a, b) \mapsto (a, b + P(a))$, P continuous, is a translation of the b coordinates for each fixed a, and a translation keeps Haar measure. By Fubini the shear keeps μ on the product. T and T^{-1} are compositions of shears and keep μ, for every n and L.

- **The count.** An output's w bits read the inputs to $w + 3L$ bits: the preimage of a level-w output quantum is a union of level-$(w + 3L)$ input quanta. It has measure 2^{-wn} and each of them $2^{-(w+3L)n}$: it holds exactly 2^{3Ln} of them. For T^{-1}, reaching $L + 2$, it holds $2^{(L+2)n}$.

- **Proved** at $n = 4$ and $L = 1$, both reaches 3, $w = 1$ and 2, from two windows of representatives: $2^{12} = 4{,}096$ on every output quantum, 8 of 8 runs ("The boundary", above).

- The window does not matter: the quanta are cosets, and a map that reads $w + 3L$ bits sees the coset only. The test runs two windows to show it.

- On $\mathbb{Z}/2^w$ alone no such count holds for T: T is not a wrapped program. The count runs between two levels, $w + 3L$ to w.

10.14 THE LIMIT STAGE

Derived.

- **Limits with and without a modulus.** Chaitin's Ω (1975) [8] is limit computable: the halting programs, enumerated, give an increasing computable sequence of rationals converging to it, and its digits are Δ^0_2 by Shoenfield's limit lemma (1959) [47]. It has no computable modulus of convergence: one would compute Ω's digits, and Ω is not computable.
- T on $\mathbb{Z}_2$ has the computable modulus w ↦ w + 3L. The machine produces w bits of T(x) from w + 3L bits of x in a fixed number of steps, at every w.
- Quotient and compare have no 2-adic limit: neither extends continuously to $\mathbb{Z}_2$ ("What does not factor through the projection").
- **Past ω.** A machine that runs through ω steps and goes on from a limit configuration is an infinite time Turing machine (Hamkins and Lewis 2000 [17]): at a limit stage each cell takes the lim sup of its values. Koepke's ordinal Turing machines (2005) [28] run over ordinal time on a tape of ordinal length. The device runs finite stages only: every program is a finite record with a fixed step count, and every lane's run halts. A machine that loops a stack of floors until a halt register is set, with orders in $\mathbb{Z}$ and ±ω, is "The ordered machine".

10.15 THE ORDERED MACHINE

Doug's definition of the higher-order and negative-order hypercomputer (points 1 to 4) and his statement on the same machine are in wants.md. The orders in $\mathbb{Z}$ and ±ω below are this section's derived form of his "all the way to infinity and back".

- **The loop rule.** Every record program halts, since each has a fixed step count. An Ω for the record machine needs a machine R* that applies a stack of floors again and again until a halt register is set.
- **Order** (the reading Doug confirmed): how many times the loop runs a floor. A negative order counts runs of the floor's inverse.

The formalization.

- State: a crystal, a vector of 2-adic integers, stored one window at a time.
- Step: one floor stack F, made reversible. A shear (a, b) ↦ (a, b + g(a)) is a bijection for any g. A floor that merges states rides the Bennett embedding (x, y) ↦ (x, y ⊕ f(x)) (A14 in engine_table.md).
- Order k ∈ $\mathbb{Z}$: the state is F^k(x), with F^{−k} = (F⁻¹)^k. The counter n ↦ n + 1 is a shear too. The orders load themselves as the naturals load, and running the inverse extends them to $\mathbb{Z}$: Doug's point 4.
- Order ±ω: at the limit each bit takes the lim sup of its history, the limit rule of Hamkins and Lewis (2000). +ω runs F, and −ω runs F⁻¹.

Derived.

- A shear's inverse is (a, b) ↦ (a, b − g(a)): the a it reads comes through unchanged, and g(a) is taken back. The counter (x, n) ↦ (x, n + 1) is the shear with g constant 1, and its inverse steps n ↦ n − 1. (F⁻¹)^k undoes F^k one step at a time.
- **Negative orders need reversibility.** Doubling on the circle $S^1 = \mathbb{R}/\mathbb{Z}$ is 2-to-1: x and x + 1/2 have one image, and it has no inverse. On the solenoid Σ_2 ("The two crystals") it is an automorphism: Σ_2 is the dual of $\mathbb{Z}[1/2]$, ×2 is an automorphism of $\mathbb{Z}[1/2]$ with inverse ×1/2, and its dual is an automorphism of Σ_2. A point of $\Sigma_2 = \lim\leftarrow(\mathbb{R}/\mathbb{Z}, t \mapsto 2t)$ is a sequence (t_0, t_1, ...) with 2t_{i+1} = t_i, a backward orbit: Σ_2 is the natural extension of the doubling map, its two-sided orbit. On an irreversible floor only positive orders exist. Every Turing machine has a reversible simulation (Bennett 1973 [5]), and the Bennett embedding is the engine's form of it (A14, proved by `tower_edge_test`).

Theorems (cited).

- **Hamkins and Lewis (2000).** An infinite time Turing machine decides the halting problem for ordinary Turing machines. A flag cell is set on halt, and at stage ω it reads 1 exactly when the machine halted. Doug's point 2 holds in this sense: with the completed tower, the forever loop is answered.
- **Hamkins and Lewis (2000).** The halting problem for infinite time Turing machines is not decidable by an infinite time Turing machine: the diagonal moves up a level. Read in Doug's terms, the question about the tower as a whole, asked from inside it, is a question that does not construct.
- **Every halt is seen at a finite stage.** The halting set is computably enumerable. Running every program side by side, as `chaitin_omega` does, sees each halt. Only "never" needs the ω stage.
- **Chaitin (1987) [9].** A formal system, a checker, of description length K determines at most K + c bits of an Ω, with c a constant of the universal machine.

Derived, mine. The three were checked against how `record_order_test` is built.

- **(i) On a finite window the ω stage takes finitely many steps.** A bijection of a finite set is a permutation, and each state's orbit is a pure cycle, of length at most the set's size (2^16 for a 16-bit window). A bit is 1 infinitely often exactly when some state on the cycle has it. Its lim sup is the OR over the cycle, read after one full cycle. The ITTM's reach past ordinary machines needs an infinite state, here $\mathbb{Z}_2$, which the machine holds one window at a time.
- **(ii) The limit rule is not a step of F** and has no inverse: many histories share one lim sup. −ω is reached by F⁻¹'s steps from the same start, not by inverting +ω. On a finite window both runs trace one cycle, in opposite directions, and +ω = −ω.
- **(iii) The halt flag at ω** is the lim sup of H's indicator along the orbit. A flag that stays set, f ← f ∨ h(x), is not reversible. It is read at the limit and is not run as a floor. In the test the lim sup and the flag are gathered by an OR into registers beside the floor, and F stays a pure table permutation.

Proved (`test/record_order_test`, 17 checks, 0 failed, run 20260924_181659).

- **(a) The orders both ways.**
  - The floor G is four shears over 32-bit words: a ← wrap32(a + b·d); b ← wrap32(b xor wrap32(a + n)), the Bennett form; d ← wrap32(d + (a and b)); n ← wrap32(n + 1). G^{-1} takes the same shears off in reverse order.
  - One program runs orders 0 → 256 → −256 → 0: 1,024 floors and 12,293 steps, register reuse on, a 10-limb file. Every floor's four words are an output.
  - The device records equal the host's word for word, on 512 lanes of edge-shaped a, b and d, with n = 0.
  - On 512 of 512 lanes every visit to order k holds the CPU's G^k(x), whichever direction it came from. The counter reads k at every floor, negative orders included, and the run returns to x exactly.
  - The step of the induction, on the CPU: G carries each order to the next, from −256 to 256, on 512 of 512. The CPU's G^{-1} inverts G, and the orders form one orbit indexed by ℤ.
- **(b) The ±ω stage on a finite window.**
  - π is a permutation of the 2^16 states, checked to be one, and π^{-1} undoes it, on all 65,536. Half the states lie on cycles of at most 2,048, half on longer cycles. The halt set H is 48 random draws, read by a 1-bit table.
  - A run is 2,048 floors: 16,379 steps, a 6-limb file. The lim sup of each bit and the halt flag are gathered by an OR in registers beside the floor ((ii) and (iii)). The run is made once with π and once with π^{-1}.
  - The device equals the host word for word, both directions, on 1,024 random starts. Every window's OR, flag and last state equal the CPU's walk, both directions, 1,024 of 1,024.
  - Every orbit returns to its own start, 1,024 of 1,024: a bijection has no transient.
  - On the 528 decided lanes (the cycle fits the run), +ω is the OR over the cycle and the flag is set exactly when the cycle meets H, 528 of 528. −ω equals +ω, 528 of 528.
  - The decided lanes hold both answers. Past the run, every set flag is a halt on the lane's own cycle.

Measured (b).

- Decided: 329 halt and 199 run forever.
- Open (the cycle is longer than the run), 496: 391 have a halt seen, and 105 are clear but halt past the run. None is clear and never halts: with 48 halt states, every long cycle met H.

The two questions (Doug's point 3) are in wants.md. Their seed in the engine, "malformed questions fail to construct", is tested: keymath does not imprint a record whose step reads a later one (`record_divide_test`).

The field. Doug's posit, that the crystals feel the tensor field of the subject at the field speed, is in wants.md.

- What the machine shows that bears on it:
  - E, the knf's entangled entropy, is summed over every neighbor pair of the whole volume at once ("The knf's identity by spatial null permutation").
  - The seal's root changes with any one voxel. A voxel's change changes its chunk leaf and every node above it, up to a hash collision (obsignatio_seal.md).
  - A sweep runs every lane side by side.
  - The 48 motions of the cube carry the knf whole: knf(gX) = g·knf(X) on 48 of 48 (proved).
- "Field speed" is not defined here, and nothing is claimed of it past these.

Higher-order interference. The words spoken while `record_order_test` ran are in wants.md.

- "Higher-order interference" has a standard meaning, Sorkin's hierarchy (Sorkin 1994 [50]). Quantum theory has second-order interference and none of third order.
- The ask_state crossing rule's negative weights are second-order interference (A15 in engine_table.md).

10.16 GOODSTEIN: ω-TOWERS STORED AS FINITE OBJECTS

`goodstein` (engine/sims, a host sim in exact integers, 7 checks, 0 failed, run 20260924_191006). Doug asked for ω^ω^ω, then for the tetration of ω.

- **The objects.**
  - n in hereditary base b: n written in base b, every exponent written in base b again, down to the digits.
  - A step writes every b as b + 1 (the bump) and subtracts 1. Goodstein's sequence G(n) runs the steps from base 2.
  - A value is stored as its hereditary tree and never expanded. b^L − 1 at a large L is stored as one run of coefficient b − 1 over the exponents 0 … L − 1, each exponent written in the base the run was made at.
  - With b read as ω, the tree is an ordinal in Cantor normal form, below ε_0 = sup{ω, ω^ω, ω^ω^ω, …}. 2↑↑k reads as ω↑↑k: 4 is ω^ω, 16 is ω^ω^ω, 65536 is ω↑↑4.

Derived (the standard proof, Goodstein 1944 [15]).

- The bump leaves the tree unchanged, and with it the ordinal.

- Subtracting 1 lowers the ordinal strictly. A lowest term c·b^0 loses 1. A lowest term c·b^E with E > 0 becomes (c − 1)·b^E plus b^E − 1, whose terms all stand below b^E.
- ε_0 is well-ordered, and a strictly falling sequence of ordinals is finite: every Goodstein sequence reaches 0.

Theorems (cited).

- **Goodstein (1944).** Every Goodstein sequence reaches 0.
- **Kirby and Paris (1982) [25].** The statement is true and not provable in Peano arithmetic.
- **Gentzen (1936) [14].** Induction up to ε_0 proves Peano arithmetic consistent. ε_0 is the height the proof above needs.

Proved (the checks).

- **The bump keeps the tree**: 80,256 of 80,256 values below 20,000, at bases 2 to 6.
- **2↑↑k in hereditary base 2 is ω↑↑k** for k = 1 to 4, and the towers climb.
- **The starts 1, 2 and 3** reach 0 in 1, 3 and 5 steps (G(3) = 3, 3, 3, 2, 1, 0). Every value equals the numeric sequence's.
- **The starts 4, 16 and 65536** (ω^ω, ω^ω^ω and ω↑↑4) each run 1,000,000 steps, to base 1,000,002. The ordinal falls on every step. Every value that fits 64 bits equals the numeric sequence's: all 10^6 steps for start 4, 1 step for 16, none for 65536.

Measured.

- At 65536 the top run holds 7,625,597,484,984 terms as one block. Derived: the first step writes 3^(3^27) − 1 as one run over the exponents 0 … 3^27 − 1, and the exponents 0, 1 and 2 have split off into the tail since. 3^27 − 3 = 7,625,597,484,984.
- An observation and not a check: after 10^6 steps the three starts end in one tail, ω² + ω·8 + 572861. Start 16's first step writes 26 = 2·9 + 2·3 + 2 as its lowest terms, and that is start 4's first step (4 → 26). From there the tail evolves as G(4). The same happens one level up for 65536.

What it proves. The start 16 is carried as ω^ω^ω exactly, as a finite object. It falls a million times, and no value is ever formed. The sim checks the descent Goodstein's theorem rests on, step by step. It does not prove the theorem.

10.17 π TURNING AT THE BOUNDARY: THE FLOORS OF A ROTATION

`pi_tower` (engine/sims, 8 checks, 0 failed, run 20260924_192322).

Posit.

- At a boundary, the turn of π does not follow the boundary's angle. It turns more sharply, and passes through the boundary region for a stretch without touching the boundary. It therefore covers the boundary region gradually, over a very long run, before it has touched all of it and begins to pass over it again. The run never ends, and the boundary region it covers, though infinite, is countable.
- Watching π turn at the boundary gave the idea of tower recursion.
  - Recorded as the history of the idea. The floors below are the floors of the rotation, and the 5/3 tower's levels are a different object.
- **The reading.** The boundary is the circle of length 1. The turn is $x \mapsto x + \pi$, which on the circle is the rotation by $\alpha = \pi - 3$.

Derived.

- **It never closes.** π is irrational (Lambert 1761 [32]): nα is not an integer for $n \geq 1$, and no two points of the orbit coincide.
- **It is dense, and equidistributed** (Weyl 1916 [60]).
- **The orbit is countable.** It is indexed by ℕ, or by ℤ with the negative orders ("The ordered machine"). The circle is not countable. A countable set has measure 0: the orbit never covers 100% of the circle. Cut into N cells, the circle is filled at a finite step. Doug's "infinite … is countable" holds for the orbit.
- **The floors.** α = [0; 7, 15, 1, 292, …], with convergent denominators q_j, q_{j+1} = a_{j+1}·q_j + q_{j − 1}. The best approximations are the convergents (Khinchin [24]): the turn's record close returns are the steps q_j.
- **The drift.** Between q_j and q_{j+1} the intermediate returns are q_{j − 1} + c·q_j for c = 1 to a_{j+1}, and each shifts by ‖q_jα‖ from the one before. A large partial quotient is a long drift: a_{j+1} small shifts before the next close return. Doug's "travel for a really long … time".
- **The descent.** A first hit, the least n with nα in a window, recurses on (m mod a, a) with the window reflected, and unwinds x = ⌜(l + m·y)/a⌝: one Euclid step a level. Its levels are the floors.

Theory (cited, not checked by the sim). The first return map of the rotation to the arc under a close return is again a rotation, by the Gauss-map angle {1/α}, rescaled: one Euclid step, one floor, one partial quotient (Rauzy 1979 [43]; Khinchin [24]).

The method (exact, integers only).

- π is bracketed by Machin's formula [22] at 416 bits, 116 terms. floor(π·2^384) is one integer at both ends.
- The turn is stored as y_n = nA mod 2^384, with A = floor(α·2^384). A first hit is the descent above.

- The fill test at count n uses the three-gap theorem (Sós 1958 [51]; van Ravenstein 1988 [56]). Point i is followed by i + u, i − v or i + u − v, where u and v are the lowest and highest record steps below n. A gap from p holds a whole empty cell of size s exactly when (p mod s) + g ≥ 2s. p mod s is the turn by A mod s on s, and each gap kind is one first-hit query over an index range.
- The least covering count is found by doubling, then halving.
- The integer turn against the real one: α·2^384 lies in (A, A + 1), and step n's real place lies in (nA, nA + n). It changes cell only where (nA mod s) > s − n: one first-hit query per resolution.

Proved (the checks).

- **π bracketed.** floor(π·2^384) is one integer at both ends of the bracket. After the point, π begins 0x243F6A8885A308D3, the published value.
- **The floors.** The bracket certifies 109 partial quotients, [3; 7, 15, 1, 292, 1, 1, 1, 2, 1, 3, 1, 14, 2, 1, 1, 2, 2, 2, 2, 1, 84, 2, 1, 1, 15, 3, 13, 1, 4, 2, 6, 6, 99, …]. The first 21 after the 3 match the published expansion.
- **The closest returns are the floors.** 66 record returns up to 2^112 steps, each the convergent denominator q_j in order (1, 7, 106, 113, 33102, 33215, …, q_65 ≈ 8.47·10^32).
- **The integer turn is the real turn.** At every resolution from 2^1 to 2^100 cells, no step up to the one searched lands in a cell other than the real one.
- **The fill from the floors.** The fill step and the last cell, read from the floors, equal a walk of every step, at 2^1 to 2^24 cells.
- **The last cell.** At 2^1 to 2^100 cells, the last cell's first touch is the fill step.
- **The width.** No exact operation outgrew its width.

Measured (the fill step over the cell count).

- A large floor is a long drift.
  - Inside 292, at 2^7 through 2^14 cells: 29.1, 71.9, 50.7, 29.0, 15.4, 7.9, 4.0 and 2.0.
  - Inside 84, at 2^33 through 2^38: 16.2, 20.9, 13.6, 7.7, 4.0 and 2.1.
  - Inside 99, at 2^60 through 2^65: 20.6, 23.8, 15.2, 8.4, 4.4 and 2.3.
  - Elsewhere the ratio is 0.9 to 5.
- The fill steps:
  - 2^24 cells at step 25,510,581 = q_12 − 1;
  - 2^32 at 6,701,487,258 = q_20 − 1;
  - 2^64 at 81,856,329,715,838,968,403 (8.186·10^19);

  - 2^100 at 1,593,334,768,903,120,834,487,234,062,301 (1.593·10^30) = q_57 − 1.

- The longest stall in the walk: 949 steps with no new cell, at 2^22 cells.
- An observation and not a check: the last cell filled is very often just behind the start.
  - It sits at 1 − α = 0.858407346410 (2^24, 2^32, 2^38, 2^41, 2^100, …), at 1 − 2α = 0.716814692820, at 1 − 3α = 0.575222039230, at 1 − 4α = 0.433629385640, or at 1 − 6α = 0.150444078461.
  - The step is then a few short of a floor's q: q_j − 1 lands at −α.
  - Why: the place near −jα is reached only through (q − j)α, with q a close return. The place the turn came from is the last it fills.
  - It touches the negative orders of "The ordered machine". Nothing is claimed of it until counted (Open).

10.17.1 *The arc*

Posit.

1. If we were on one disk, and π on a separate disk balanced by its torsion, π's planes would be offset from ours by an angle.
2. This is the arc π follows, and it belongs in `pi_tower`. Following it misses forever, through a countable infinity of turns, until it has covered all of the boundary. It then continues without repeating, covering the boundary again from slightly different angles, with slightly different values and under slightly different field conditions, without end.

- **The construction.** Roll the boundary into the cylinder whose cross-section is our disk. The turn is the helix γ(s) = (a cos s, a sin s, b s), with a = 1/(2π) (circumference 1) and b = 1/(2π²) (rising 1/π a turn). It pierces our disk at the marks {nπ}.

Proved (`pi_tower`, 13 checks, 0 failed, run 20260924_193716).

- **In $\mathbb{Q}(\pi)$**, with π a free variable because it is transcendental (Lindemann 1882 [35]), from the derivatives at the four quarter turns, where every trig derivative is 0 or ±1:
  - the curvature κ = 2π³/(π² + 1), checked as $\kappa^2 = |\gamma' \times \gamma''|^2/|\gamma'|^6$, with no square root formed;
  - the torsion τ = 2π²/(π² + 1), from $\det(\gamma', \gamma'', \gamma''')/|\gamma' \times \gamma''|^2$;
  - τ/κ = 1/π = tan φ, checked as $\tau^2/\kappa^2 = 1/\pi^2$ and equal to the tangent's squared rise over run: φ is the tilt of the tangent against our disk (Lancret 1806 [33]);
  - the Darboux vector τT + κB, checked scaled by $|\gamma'|^3$, along our disk's axis.
- **From the bracket**, where both ends agree to 12 places: κ = 5.705134342735, τ = 1.816000663299, φ = 17.656787151412°, and 72.343212848587° against the axis. κ and τ rise with π and arctan(1/π) falls: each end brackets the value. arctan(1/π) is taken on the bracket with a bounded fixed-point error.

Derived.

- The principal normal points at the axis and lies in our disk's plane. The osculating plane's dihedral angle to our disk is then the tangent's elevation, $\varphi = \arctan(1/\pi) \approx 17.657°$: π's planes offset from ours by an angle (1).
- The Darboux vector along the axis: the frame turns about our disk's axis at a constant rate.

Measured, the re-etch: the step by which every cell has a second touch, by a walk, at 2^1 to 2^24 cells.

- At 2^24 the fill is at 25,510,581, and the re-etch by 51,021,159.
- Inside the 292 floor the re-etch comes almost at once: at 2^8, 18,416 and then 18,747. The long drift had already etched most cells several times.

Continuing. Posit.

1. This is because most of the angular momentum is applied at the moment of deflection.
2. If this holds, π is a hyperobject of infinite information that can be represented in a kilobyte.
3. π is not only infinite. The field's starting conditions do not matter: they perturb it differently every time, because it is irrational and does not repeat.
4. On the perturbations, consider a wave n whose magnitude varies without bound and whose direction is constant, meeting a period x = 1.
5. Not all of the angular momentum is applied at deflection. π is itself an angle, and it applies momentum to itself as it unfolds: the momentum is present but balanced.
6. The balance breaks at the boundary, where those forces lose equilibrium.
7. One must win.

What the model shows that bears on them (bounds, checked). Derived unless marked. The checks for 7 to 9 are proved and measured after (9).

- **(2) Etched, and etched again.**
  - At every finite resolution the turn etches every cell in finitely many steps (proved), then etches them again and again, never repeating a point.
  - The orbit's closure is the whole circle: every open arc, however small, is etched. In that sense "covered all of the boundary" holds at countable infinity. The points themselves have measure 0 and never cover the continuum.
  - The crossing angle is the same at every crossing: 17.657° on the helix, and 72.343° or 17.657° in the square billiard. What varies is the place within the cell. "Field conditions" have no counterpart in the model.

- **(3) Deflection.**
  - In the square billiard there is no force between walls, and L about the center, (x − ½)p_y − (y − ½)p_x, is constant there. A hit on a vertical wall at height y changes it by 2(y − ½)p_x: 0 at mid-wall, largest at the corners. All of the change, not most, is at deflection.
  - On the helix there is no deflection. The force is centripetal, L about the axis is constant, and κ is constant: the turning is spread evenly.
- **(4) Infinite length, finite information.** A program prints any window of π: K(the first n bits of π) ≤ K(n) + c = O(log n) (Kolmogorov 1965 [31]). A Machin program fits in a kilobyte, and "infinite information" does not hold. An infinite expansion is not infinite complexity ("The anchors" under Doug's posits). A Martin-Löf random real is the opposite: incompressible. "Hyperobject" has no definition here.
- **(5, 6) Two perturbations.**
  - A shifted start slides the whole etch rigidly: an isometry, Lyapunov exponent 0, ε stays ε, and the three gaps are the same. The rotation is uniquely ergodic (Weyl [60]; Walters [58]): every start has the same long-run statistics. "The field starting conditions do not matter" holds, and "perturb it differently every time" is the shift.
  - A changed magnitude, with the direction constant, is Doug's wave (6). The rotations by α and by α + ε separate by nε after n encounters with the period 1: linearly, without end, and after about 1/ε encounters the two etches are unrelated. It is not chaos: the growth is linear instead of exponential.
  - Every irrational magnitude etches its own never-repeating pattern. A rational one closes.
- **(7) Present but balanced.**
  - On the helix the acceleration $\gamma'' = -(a\cos s, a\sin s, 0)$ points at the axis, and its torque about the axis is 0.
  - $L_z = a\cdot\cos\varphi = 1/(2\sqrt{(\pi^2 + 1)}) \approx 0.15166$ per unit mass at unit speed, constant. $L_z^2 = 1/(4(\pi^2 + 1))$ lies in $\mathbb{Q}(\pi)$.
  - The helix turns π full turns per unit of axial height.
  - Mine: about a fixed point on the axis, L = r × v is not constant. The torque r × F = (k·a·b·s·sin s, −k·a·b·s·cos s, 0), with F = −k(a cos s, a sin s, 0), is horizontal, and it grows with the height above the point. The momentum is present and turns, and only its axial part is kept. "Present but balanced" is the helix's description.
- **(8) The unbalancing at the boundary.**
  - In the square billiard L is constant between walls. A wall hit is an impulse whose torque about the center is 2(y − ½)p_x.
  - The hit heights are the folded marks, and no two kicks are equal.

  - $|L| \le |p|\cdot\sqrt{2}/2$ always. The running sum of the kicks is L now less L at the start, which stays within $|p|\cdot\sqrt{2}$, and the mean kick goes to 0.
  - On the helix nothing unbalances: there is no wall.

- **(9) One must win.** Doug, clarifying: one force must win, because division by zero is impossible and the boundary is real in the information space. A tie is where the rule has no value, and π never ties. There are three instances, each ruled out by π's irrationality (Lambert [32]).
  - **(i) The corner.** At a corner the normal has no value, and the reflection $v - 2(v\cdot n)n$ has none: the geometric division by zero. After its start the unfolded line $(t, \pi t)$ never meets a lattice point (k, m), since $\pi k = m$ would make π rational. Every wall hit strikes exactly one wall.
  - **(ii) The cell edge.** $\{n\pi\}$ is never a dyadic $j/2^k$, since $n\pi - j/2^k$ an integer would make π rational. Every mark's cell is decided. `pi_tower` proves the exact integer turn decides the same cell as the real π at every resolution from 2^1 to 2^{100} ("The integer turn is the real turn", above).
  - **(iii) The lead.** L is never exactly 0. That would need the unfolded line to pass through an image $(k + \tfrac{1}{2}, m + \tfrac{1}{2})$ of the center, $\pi = (2m + 1)/(2k + 1)$, which is rational. At every instant one side strictly leads. The billiard orbit is minimal in each of its four directions (irrational slope). It passes arbitrarily close to the center on both sides with one direction, and L changes sign infinitely often. There is always a winner, and never a final one.
  - **The near-ties are the floors.** The corner miss at step q is $\|q\pi\|$. The record near-misses are the convergent denominators q_j (proved, "The closest returns are the floors"). Measured, from the floor table `pi_tower` prints: $8.9\cdot10^{-3}$ at 7, $3.0\cdot10^{-5}$ at 113, and $1.6\cdot10^{-34}$ at q_65. The records are proved, and "never exactly 0" is the irrationality itself.

Proved, posits 7 to 9 (`pi_tower`, 17 checks, 0 failed, run 20260924_194636).

- **(7) The helix.** In $\mathbb{Q}(\pi)$ at the four quarter turns, the torque about our axis, $x\cdot y'' - y\cdot x''$, is 0, and $L_z^2 = (x\cdot y' - y\cdot x')^2/|\gamma'|^2 = 1/(4(\pi^2 + 1))$. From the bracket, L_z = 0.151657235526 at unit speed, the same at both ends to 12 places (the floor of the root of the floor).

- **(8, 9) The billiard**, from the corner at slope π, unfolded.
  - Segment cell (i, j) is column i, with the rows $j = \lfloor \pi i \rfloor \ldots \lfloor \pi(i + 1) \rfloor$.
  - L about the center, with $p = (1, \pi)$, is $(-1)^{(i+j)}\cdot((j + \tfrac{1}{2}) - \pi(i + \tfrac{1}{2}))$. Derived: $x - \tfrac{1}{2} = \sigma_x(\{X\} - \tfrac{1}{2})$ and $y - \tfrac{1}{2} = \sigma_y(\{Y\} - \tfrac{1}{2})$, with $p = (\sigma_x, \sigma_y\cdot\pi)$, and along the segment $\pi\{X\} - \{Y\} = \lfloor Y \rfloor - \pi\lfloor X \rfloor$.
  - Its sign is the sign of $d - \alpha(2i + 1)$, with $d = (2j + 1) - 3(2i + 1)$ a whole number, flipped by the parity of i + j. d stands above $\alpha(2i + 1)$ exactly when $d > \lfloor \alpha(2i + 1) \rfloor$.
  - The floors $\lfloor \alpha n \rfloor$ are read from the integer turn's carries. Each is certain while the top word is short of all ones, since $n < 2^{320}$.

  - Over 2^24 columns, with every floor certain:
    * 0 corners: {α(i + 1)} is never 0;
    * no L is 0: {α(2i + 1)} is never 0;
    * the record near-corners at the side walls are exactly 1, 7, 106, 113, 33102, 33215, 66317, 99532, 265381, 364913, 1360120 and 1725033, the q_j ≤ 2^24.

Measured, the billiard over 2^24 columns.

- 69,484,394 segments: 16,777,216 side-wall hits and 52,707,178 floor-wall hits. The second is ⌊π·2^24⌋: the unfolded line crosses ⌊πN⌋ horizontals.
- The lead in L changed hands 35,929,962 times and was never kept longer than 2 segments.
  - Derived: the parity factor flips at every wall, and the raw sign flips only where the line passes a cell center's level, about once a column.
  - The steep path zig-zags floor to ceiling, and each bounce reverses its circulation about the center.
- One side led on 34,742,196 of the 69,484,394 segments, the other on 34,742,198: a difference of 2.
- The nearest tie is |L| = 1.910·10^−8, in cell (862516, 2709675), where π stands nearest (2j + 1)/(2i + 1) = 5419351/1725033. That is a convergent of π (q_11 = 1725033), odd over odd. The closest the lead comes to a draw is at a floor.

Doug's framing for 9 holds as built: every wall hit strikes one wall, and every segment has a leader. A corner is a tie for every rational slope m/k, at the lattice point (k, m). A zero L is a tie for the rationals odd over odd. The walk's near-ties are π's convergents.

Cell 0 again, and the second run. Continuing. Posit.

1. After π covers the boundary, how long until it covers cell 0?
2. After it covers cell 0, how long until that run covers the cell that closed the boundary in the first run, and are the two periods the same?
3. The fill always lands just before a floor.
4. Any 2^n can be reached directly in the tower, as one term.
5. If qα is nearly a whole number, its identity can be taken, and its null permutation makes it whole. It lacks its residue.

- **The objects.** T is the fill step and L its last cell. H is the first n > T with {nα} < 1/N, the next touch of cell 0. C is the first n > H in L. Each is one first hit from the step before it.

Proved (`pi_tower`, 18 checks, 0 failed).

- H and C equal a walk of every step, at 2^1 to 2^24 cells.

- The certainty check, that the integer turn is the real one, runs to C at every resolution.

Proved since (`pi_tower`, 23 checks, 0 failed). The new checks are the residue (check 11, under (14) below), the golden helix (check 12, three checks, in "The golden helix" after this) and the reach of the records at any 2^n (under (13) below). With no argument the sim runs 2^1 to 2^100, and every earlier result there is unchanged: the fill at 2^100 is 1593334768903120834487234062301, the same period holds at 13 of 100, and the billiard is unchanged.

Theory (cited).

- **Slater (1950 [48], 1967 [49]).** The return times to an interval under an irrational rotation take at most three values, and the largest is the sum of the other two. Derived: T sits inside one gap between consecutive visits to [0, 1/N), and H − T is at most the largest of the three.
- **Kac (1947) [23].** For an ergodic measure-preserving map, the mean return time to a set of measure μ is 1/μ. The mean return to cell 0 is N steps.

Measured (100 resolutions). The classification was made by a scratch script over the run's integer output instead of by the sim.

- **(10) H** lands on a record return or a sum of them: q_12 at 2^24, q_20 at 2^32, q_34 at 2^64.
  - The wait H − T is bimodal. It is 1 to a few hundred steps when the fill lands just before a return from above, or about one floor's q when it does not (33,104 at 2^15, 2.5·10^18 at 2^64).
- **(12) The fill before a floor.** The fill falls short of the next record return (a q_j, or an intermediate q_{j − 1} + c·q_j with c = 1 to a_{j+1}) by j steps.
  - j = 1 at 20 resolutions, 2 at 23, 3 at 10 and 4 at 11: within 4 at 64, and within 9 at 77.
  - The next return is a floor at 44 and an intermediate at 56.
  - The large j lie inside the big floors: 105 and 109 inside 292, 22,867 inside 84 (2^33), and 177 inside 99.
  - This is the fact of the last cell at −jα above: (q − j)α sits just behind −jα. Doug's "always" is "mostly": within 4 steps at 64 of 100.
- **(11) The second run**, C − H, against the first run, T.
  - The same at 13: 2^15, 2^32, 2^35, 2^48, 2^54, 2^59, 2^62, 2^73, 2^74, 2^79, 2^85, 2^89 and 2^92.
  - Shorter at 86, and longer at 1 (2^6, by 7).
  - Every shortfall is a floor combination: a record return at 80 (a full q_j at 52), and c·q_j at the other 6: 20·q_20 at 2^33, 6·q_24 at 2^42, 2·q_26 at 2^47, 28·q_32 at 2^60, 4·q_42 at 2^80 and 2·q_44 at 2^83.
  - Runs of resolutions share one shortfall: q_20 at 2^34 and at 2^36 to 2^39, and q_32 at 2^61 and at 2^63 to 2^66.

  - A reading, derived and not proved: the second run starts at {Hα}, a sliver inside cell 0, the offset one return gives. It reaches L one floor-return sooner.
  - Whether the two periods are the same: the same at 13 of 100, and otherwise apart by a floor combination.

Derived.

- **(13) One term.** T, H and C at any resolution are each one first-hit query: a single descent through the floors, with no walk of the steps. Certainty holds as far as the bracket reaches, and the bracket widens with n.
  - **Proved**. `pi_tower [n]` or `pi_tower [from] [to]` runs any 2^n. The bracket is P = 3n + 64 bits, rounded up to a multiple of 64, and at least 384. The records reach to + 12 bits (at least 112), and a check fails if any count searched passes that reach.
  - The turn needs an exact width of 3(P + 32) + 64 bits. When the build is narrower, the sim stops and names the width to run with. At 2^1000 on 4096 bits it prints "the turn needs an exact width of 9376 bits: run with SIM_EXACT_LIMBS=512". `run.sh` passes SIM_EXACT_LIMBS to every object. The walks and the billiard use ⌊α·2^384⌋ = ⌊A_P / 2^(P − 384)⌋ at every n.
  - **Measured** at 2^1000 (SIM_EXACT_LIMBS=512, 16,384 bits):
    * P = 3072, with Machin at 3104 bits and 864 terms; 888 floors certified.
    * The fill is step 1.288·10^302, 12.0231 times the cells. The last cell is at 0.575222039230, at floor 600 (a_601 = 106).
    * Cell 0 is etched again about 1.77·10^301 steps after the fill. The second run is not the first's period (0 of 1).
- **(14) The residue** (a bound, checked).
  - **Proved** (check 11), verbatim: "on every floor with q_j to 2^21, pi's first q_j steps have the whole parts of the permutation n -> n p_j mod q_j, and at step q_j it stands its residue off the whole". This covers floors j = 0 to 11. For n < q_j the 384-bit walk's whole parts equal ⌊n·p_j/q_j⌋. At step q_j the walk stands at whole p_j with place δ_j when δ_j > 0, and at whole p_j − 1 with place 1 − |δ_j| when δ_j < 0. Certainty: q(δ_M + q) < M for δ > 0, and q·|δ_M| < M with |δ_M| ≥ q for δ < 0.
  - For 1 ≤ n < q_j, n·p_j/q_j is never whole, and ⌊n·p_j/q_j⌋ = ⌈n·p_j/q_j⌉ − 1. The sign enters only through the side of the mark (the right-closed cell below for δ_j < 0) and through step q_j.
  - **Measured**, the printed identities P/q (π's own numerators) and residues qπ − P: 3/1 +1.415e-1; 22/7 −8.851e-3; 333/106 +8.821e-3; 355/113 −3.014e-5; 103993/33102 +1.912e-5; 104348/33215 −1.101e-5; 208341/66317 +8.114e-6; 312689/99532 −2.900e-6; 833719/265381 +2.312e-6; 1146408/364913 −5.877e-7; 4272943/1360120 +5.495e-7; 5419351/1725033 −3.820e-8.

- Write q_jα = p_j + δ_j, with p_j the nearest whole and δ_j the residue, signed (−1)^j. The identity is p_j/q_j. In π's own terms p/q are the convergents 22/7, 355/113, 103993/33102, …, and δ = qπ − P is the printed ‖qπ‖ with its sign.
- Replacing α by p_j/q_j makes the turn the cyclic shift k ↦ k + p_j mod q_j on q_j cells, one cycle since p_j and q_j are coprime. The n-th point sits at (n·p_j mod q_j)/q_j. It closes after q_j steps, with q_j·(p_j/q_j) = p_j whole: the null version, every point on a multiple of 1/q_j.
- For 0 ≤ n < q_j, nα = n·p_j/q_j + n·δ_j/q_j. Each point sits off its lattice point by n·δ_j/q_j (n·δ_j in cell widths), on the side of δ_j's sign, and within one cell since q_j·|δ_j| < 1.
- The points {nα}, 0 ≤ n < q_j, fall one to each of q_j cells: [k/q_j, (k + 1)/q_j) when δ_j > 0, and (k/q_j, (k + 1)/q_j] when δ_j < 0. With half-open cells [k/q_j, (k + 1)/q_j) and δ_j < 0 the count fails. At q_1 = 7 (δ_1 = 7α − 1 < 0), n = 0 at 0 and n = 1 at 0.1416 both fall in [0, 1/7), and [6/7, 1) holds none.
- The residues run Euclid's algorithm: δ_{j+1} = δ_{j − 1} + a_{j+1}·δ_j, with a_{j+1} = ⌊|δ_{j − 1}|/|δ_j|⌋. Each floor's residue is the next floor's step, the Gauss-map recursion. It holds by construction here, since the floors come from Euclid on (M, A).
- "Null permutation" read as the rational turn p_j/q_j is a reading of posit 14.

The golden helix. Continuing. Posit.

1. Their period is a contraction of the golden spiral, and π rides its inverse in the negative space.
2. π does ride it, and rides it exactly, because it is a helix.

Derived. Write ξ_k = [a_k; a_{k+1}, …] for the complete quotient at floor k, and r_j = |δ_j|/|δ_{j − 1}| for the shrink into floor j, in the sim's indexing (the table's "j (a_{j+1})" column below).

- **Every tower grows at least at the golden rate.** q_j = a_j·q_{j − 1} + q_{j − 2} ≥ q_{j − 1} + q_{j − 2}, with q_0 = 1 and q_1 = a_1 ≥ 1. Then q_j ≥ F_{j+1}, the Fibonacci numbers, for every irrational. The residues follow: |δ_j| < 1/q_{j+1} ≤ 1/F_{j+2}. Every tower's residues shrink at least at the golden ratio's rate, which is 15's "contraction" in the precise sense. The golden ratio [1; 1, 1, …] meets the inequality with equality on every floor.
- **The sign flips on every floor.** δ_j has sign (−1)^j: from above on even floors and from below on odd ones, a half turn each floor. This is 15's "negative space".
- **The shrink.** r_j = 1/ξ_{j+1}, and by Euclid r_j lies in (1/(a_{j+1} + 1), 1/a_{j+1}). It stands between 1/2 and 1 exactly on the floors of 1, and below 1/2 (below 1/φ) on every floor of 2 or more. For a_{j+1} = 1, r_j = 1/(1 + r_{j+1}), and r_j > 1/φ exactly when r_{j+1} < 1/φ. The golden ratio's shrink is 1/φ on every floor.
- **The helix** (a reading of Doug's word). Put floor j at (jπ, ln|δ_j|) on the cylinder $S^1 \times \mathbb{R}$, with the angle taken mod 2π. The points lie on a discrete helix, the log spiral (θ, ln r) lifted to

the cylinder. The turning is exact: a half turn on every floor, for every irrational. The pitch is the drop ln ξ_{j+1}. It is constant only for the golden ratio, at ln φ ≈ 0.481, and it varies for π. Doug's "exactly" holds for the turning and does not hold for the pitch.

Theory (cited). **Hurwitz (1891) [21].** Every irrational x has infinitely many p/q with |x − p/q| < 1/(√5·q²). √5 is the largest constant that works for the golden ratio, the worst approximable number.

Proved (check 12, three checks, `pi_tower`). Every residue is bracketed from the full turn as (qA − pM, qA − pM + q), on floors j = 0 to 65 (q_j ≤ 2^112). Verbatim:

- "the residues flip sign on every floor, a half turn: from above on even floors, from below on odd"
- "every floor is at least the golden one, q_j >= F_(j+1), and every residue is under the next floor's 1 / q_(j+1)"
- "the shrink stands between 1/2 and 1 exactly where the next floor is 1, each side of 1/phi decided at both ends". The side of 1/φ is decided exactly, as (2x + y)² < 5y² at both corners of the bracket.

Measured (65 shrinks, j = 1 to 65).

- 45 are below 1/φ and 20 above. 28 stand between 1/2 and 1: the floors of 1.
- Every shrink above 1/φ is on a floor of 1. Floors of 1 can still fall below it: j = 5 at 0.5758, 13 at 0.5858, 22 at 0.5157, 39 at 0.5378, 51 at 0.5190, 55 at 0.5620, 58 at 0.5275 and 61 at 0.5742.
- The first shrinks, as j (a_{j+1}) r: 1 (15) 0.0625; 2 (1) 0.9965; 3 (292) 0.0034; 4 (1) 0.6345; 5 (1) 0.5758; 6 (1) 0.7366; 7 (2) 0.3574; 8 (1) 0.7973; 9 (3) 0.2541; 10 (1) 0.9350; 11 (14) 0.0695; 20 (84) 0.0118; 32 (99) 0.0100.
- π follows the golden pitch on its runs of 1s (floors 4 to 6: 0.6345, 0.5758, 0.7366) and leaves it on every floor of 2 or more. It leaves farthest at 292, 99, 84, 15, 14, 13 and 12: the shrinks 0.0034, 0.0100, 0.0118, 0.0625 (j = 1) and 0.0652 (j = 24), 0.0695, 0.0723 and 0.0791.
- Growth per floor: q_65^(1/65) = 3.210576 (an integer root, 6 places), against φ = 1.618033 (from ⌊√(5·10^12)⌋). The mean pitch over the 65 certified floors is ln q_65 / 65 ≈ 1.166, against ln φ ≈ 0.481. The sum of the pitches, ln(|δ_0|/|δ_65|), differs from ln q_65 only by end terms.

Theory (cited). **Lévy (1936) [34].** For almost every real, q_j^(1/j) tends to e^(π²/(12 ln 2)) ≈ 3.2758, a mean pitch of π²/(12 ln 2) ≈ 1.187. Whether π obeys it is Open. The 1.166 above is π's measured mean over 65 floors instead of a limit.

10.18 SUBTRACTIVE COALESCENCE

Doug's name for T. Derived.

- The 5/3 lifting's steps are predict and update, the lifting scheme of Sweldens (1996) [52]. An odd sample less its prediction from the evens is a high-pass coefficient; an even plus a correction from the high-pass coefficients is a low-pass coefficient. Each step is undone by the opposite step.
- T is not a projection. It is a bijection (proved both ways), and T^{-1} returns every sample.
- The pinch is the predictable part moved instead of information lost. A ramp's high-pass coefficients are near 0 and its content sits in the few level-L low-pass coefficients. The heap at the crystal is 1/9.61 of the samples' heap for the ramp and 1/1.00 for noise (measured, above). The heap shrinks while the ring grows by n + 2n(1 − 2^{−L}) (6n(1 − 2^{−L}) under the widths), and the count is kept exactly (det M = 1, Haar counted).

10.19 PHYSICAL WALLS

Cited; both pages read, and only what they state is given.

- **Borsten and Kim [6]**, "Limits to Computational Acceleration Imposed by Quantum Field Theory and Quantum Gravity", arXiv:2604.00182, 31 March 2026. Their abstract: schemes that use curved spacetimes and exotic fields, for instance time dilation, to accelerate computation are "consistently thwarted by physical effects from quantum gravity (including swampland conjectures) and quantum field theory in curved space". An observer and a computer able to withstand energy scales up to order E accelerate computation by at most O(1)E e-folds per unit time, $(\ln \alpha)/\tau \lesssim E$. The Bekenstein bound is the memory analog: a computer of length scale D at energies up to order E with N memory states has $(\ln N)/D \lesssim E$.
  - The reading, derived from their bounds: a device of bounded energy and size runs finitely many stages in finite time and holds finitely many states. The limits $\mathbb{Z}_2$ and $\mathbb{R}$ are reached one finite window at a time, and the walls bound how many.
- **Aaronson [1]**, "On black holes, holography, the Quantum Extended Church-Turing Thesis, fully homomorphic encryption, and brain uploading", Shtetl-Optimized, 27 July 2022. AdS/CFT predicts that the boundary state $|\psi\rangle$ "encodes everything there is to know about the AdS bulk, including whatever is inside the black hole", and that "the information about what's inside the black hole will be pseudorandomly scrambled". He cites Bouland, Fefferman and Vazirani [11] (arXiv:1910.14646).
  - The contrast, derived: T's boundary is information-complete too (every crystal is the crystal of some samples, proved) and is not scrambled. Each coefficient reads a cone of reach 3L, and T^{-1} reads the crystal back in a number of steps linear in n.

10.20 THE KNF'S IDENTITY BY SPATIAL NULL PERMUTATION

`knf_identity` (engine/sims, 23 checks, 0 failed). The nbody lattice's law in a 64^3 cube over 177 frames: 176 transitions, 16 full windows of 11. The engine's `entropy_history_project` makes the knf (A7).

- **The objects.**
  - A section is one voxel's 16 windows: its bit-weighted flip density in each window, the kernel's own, less its mean over the windows.
  - E, the knf's entangled entropy: over every voxel and each of its z, y and x neighbors on the torus, the dot product of their two sections, summed.
  - A spatial null draw permutes the sections over the voxels. It keeps every section and the cloud, and E reads arrangement only.
  - A volume is identified when E stands above every one of d = 19 draws. The departure at a null is 19·E less the sum of the 19 draws' E.
  - Tile size b = 1, 2, 4, ..., 64. The inside null shuffles the sections within each b^3 tile and moves nothing across a seam. The between null moves whole tiles rigidly to shuffled places. At b = 64 the inside null is the free null, and at b = 1 the between null is.
  - A share is a departure over the free null's departure.

Proved (the checks).

- **One bit.** The host's walk equals the engine's knf at all 262,144 voxels. Of 8,192 single flipped bits, 1,500 leave the knf unchanged, and the window rule names the outcome of all 8,192 exactly.
  - The window rule, derived. A flip of bit j at frame f changes the transitions $f - 1 \to f$ and $f \to f + 1$. When bit j differs between frames $f - 1$ and $f + 1$, exactly one of the two transitions flips, before and after, and the count holds if both transitions lie in one window. On a window's edge, or at the first or last frame, the flip moves a count.
  - The knf is therefore many-to-one: no 1:1 ID in the way the crystal is. Its identity is its rank against the null.
- **The exact motions.** The 48 motions of the cube (6 axis orders × 8 reflections), each with a keyed torus translation. The data is moved over xyz and projected again. knf(gX) = g·knf(X), E unchanged and the cloud unchanged, on 48 of 48.
  - Derived: an orthogonal matrix with integer entries is a signed permutation matrix. These 48 are the only rotations and reflections that carry $\mathbb{Z}^3$ onto itself, and any other angle rounds on the lattice.
- **The tiles.** At all 7 tile sizes the whole is its tiles plus its seams exactly. A rigid move of whole tiles keeps every tile's inside exactly: 133 of 133 draws. The inside departure at b = 1 and the between departure at b = 64 are 0 exactly.

  - Derived: since a between draw keeps every tile's inside, its departure is the seams' departure alone.

- **The control.** 64 volumes of 32^3 with every voxel drawn alike (no bodies, no ramp, no fixed pattern), 19 free draws each. 4 are identified, where spatial exchangeability bounds the rate by 1/20 (3.2 expected); the check holds it within 5σ.

Measured, the lattice.

- E is 44.626 times the free null's mean and stands above all 19 draws.
- The departure curve, as a share of the free departure, * where E stands above every draw:

b	inside	between	sum
1	0.000	1.002*	1.002
2	0.126*	0.492*	0.618
4	0.349*	0.241*	0.590
8	0.647*	0.106*	0.754
16	0.896*	0.047*	0.944
32	0.974*	0.008*	0.982
64	1.000*	0.000	1.000

  - The two cross between b = 2 and 4. The bodies' semi-axes are 2 to 7.

- **The bodies.** Each body's box (its z, y and x bounds over its life) gets its own inside curve, as a share of its own free departure.
  - Every body but one reads 0.112 to 0.196 at b = 2, 0.254 to 0.502 at 4, 0.594 to 0.895 at 8, 0.900 to 0.989 at 16 and 0.947 to 1.019 at 32.
  - Body 3 does not move and never ends. It flips nothing, its share of the whole is 0.002, and its curve is noise (1.192 at b = 4).
  - The shares of the whole sum past 1, since the boxes overlap. The largest: body 7, 0.426 (the largest box); body 11, 0.175; body 10, 0.172; body 0, 0.164.
- **The same scene without bodies** (the same camera and ramp). E is -10.675 times the null's mean and is not identified. The free departure is near 0, and the shares are noise. One between mark, at b = 16, of 14, which is chance level.

10.21 DOUG'S POSITS

- **"Transitivity: T closed is a member of T open."** Derived bound. Read T closed as a finite stage (a window, a wrapped program F_w) and T open as the limit ($\mathbb{Z}$ or $\mathbb{Z}_2$).
  - Bare membership passes nothing: ω is transitive, every member finite, itself infinite.
  - Membership tied to structure, each stage embedded in the next and in the limit, passes the $\forall\exists$ sentences and exactness, and does not pass finiteness or termination.

- **"Two crystals preserve infinity; the delta between them is infinite."** Supported (derived above). The two crystals agree at every finite width, and the delta $\mathbb{Z}_2 \setminus \mathbb{Z}$ is uncountable. Every window of every element of $\mathbb{Z}_2$ is a value the machine can hold.
- **The anchors are of infinite complexity, and they bend the information field toward themselves.** Open. What the math states:
  - An infinite expansion is not infinite complexity. 1/3 = ...10101011 never ends, and a program a few bits long prints any window of it: K(π_w(1/3)) ≤ K(w) + c = O(log w).
  - For a computable x ∈ $\mathbb{Z}_2$, K(π_w(x)) ≤ K(x) + O(log w): run x's program to w digits, given w.
  - Haar-almost every element of $\mathbb{Z}_2$ is Martin-Löf random (Martin-Löf 1966 [38]). The Haar measure on $\mathbb{Z}_2$ is the fair coin on its digits, and the random sequences have measure 1. A random element is not computable.
  - The machine reaches finite windows only. Every register is π_w of something, and any w-bit window has K ≤ w + O(log w). Every constant in a program is a finite description, and the machine's reach is the windows of computable elements.
  - "Infinitely complex" can name only a limit object the machine never holds whole. Which object the anchors are, and whether they are random elements of $\mathbb{Z}_2$, is open. "Bending the information field" has no definition here to derive from.
- **The inverted boundary** (the posit is in wants.md, and A13 in engine_table.md). Its derived form is the τ tower: the finite windows extended a bit above at each stage, π_w's tower, have the limit $\mathbb{Z}_2$, and extended a bit below, τ_k's tower, the limit $\mathbb{R}$ ("The top projection and its limit $\mathbb{R}$").
- **The quanta** (Doug's posit, in wants.md). Derived bound: every output quantum at level w has exactly 2^{3Ln} input quanta at level w + 3L, at every w, and the count passes to $\mathbb{Z}_2$ as Haar measure ("Counting quanta"). The machine holds finite windows only, and the physical walls bound how many.
- **The recursion stack** (Doug's posit, in wants.md). Derived bound: every stack the device runs is finite. T's limit sits at the first limit stage ω, with a computable modulus. Stages past ω are the machines of Hamkins and Lewis and of Koepke, not built ("The limit stage"). Orders in $\mathbb{Z}$ and ±ω on a finite window: "The ordered machine". Ordinals below ε_0 held as finite trees and walked down a million steps: "Goodstein: ω-towers held as finite objects".
- **"Subtractive coalescence."** Posit, Doug's name for T. Derived bound: predict and update, a bijection, the pinch the predictable part moved ("Subtractive coalescence").
- **The identity of T by null permutation.** T's identity is taken by ranking T against T applied to null permutations of its input, and where T passes, that identity is an exact universal root ID for the structure. Posit. The identity is an ID, a fingerprint of the data's structure. It is not the identity map, nor the identity edge of A14.

  - The procedure. Run T on the samples x and on d null draws σ_1 x, ..., σ_d x, each σ_i a uniform random shuffle of the samples: A12's drawn null in engine_table.md. A lane is identified when its crystal's heap stands strictly below every draw's.
  - Derived. A shuffle keeps every value, the histogram and the count, and changes the arrangement only. x and its draws share one multiset of values, and any gap between T(x)'s heap and the draws' heaps reads arrangement alone. T keeps the count exactly (det M = 1, Haar counted): the gap is not T gaining or losing volume.
  - Derived. Under the null that x's arrangement is itself a uniform shuffle, x and the d draws are exchangeable, and the chance that x's heap stands strictly below all d draws is at most 1/(d + 1) (Hope 1968 [20]). This is A12's false-period rate, carried over.
  - **Proved** (`test/engine/record_boundary_test`, 48 checks, 0 failed). d = 8 Fisher–Yates shuffles a lane from the test's seeded generator, 256 lanes a class, the ID the crystal's total heap, the four classes of "The boundary". A shuffle keeps the samples' heap exactly, on every draw. Noise is identified no more often than 256/9 plus 5 standard deviations of the binomial count, and each structured class is identified past that bound.
  - **Measured:** lanes identified, with the lane's crystal heap over the draws' mean in brackets: ramp 256 of 256 (0.13), ramp ±8 256 of 256 (0.33), ramp ±1,024 254 of 256 (0.73), noise 21 of 256 (1.00).
  - The claim is the rate instead of every lane. The null bounds how often noise is identified; it promises nothing for any one structured lane. Two shallow ramps under ±1,024 were not identified: their noise swamps the slope, and their shuffles have little arrangement to destroy. A first form of the check asserted every structured lane and failed on those two.
  - **One to one** ("unique", not bit to bit; "change one bit and the permutation fails"). **Proved** in the same test.
    * 8,192 of 8,192 null draws change the crystal exactly when they move a value: T(σx) = T(x) exactly when σx = x. **Measured:** 0 draws moved none.
    * Every single flipped bit changes the image: 8,192 of 8,192 pairs through T and 8,192 of 8,192 through T^{-1}.
    * Derived: both follow from T being a bijection. The crystal is a one-to-one ID of its samples. The heap fingerprint is many-to-one: it reads the arrangement's structure instead of the samples.

- **The knf is unique: it is the broken edge of the crystal. Two sections of a knf agree only by chance, or when the knf belongs to more than one subset.** Posit, and the next one to test. The .knf is the entropy history (M8, A7 in engine_table.md); the name appears nowhere in the source.
  - The measurement, as Doug set it: the null permutation together with the identity by null permutation measures the knf, and it is the next posit to test. The knf is ranked against its spatial null permutation (`knf_identity`, below), and its identity is taken by the identity:null permutation, the procedure that identifies the crystal ("The identity

of T by null permutation", below). "Unique" is then read as that rank and that identity, as for the heap fingerprint.

  - What is already measured: 1,500 of 8,192 single flipped bits leave the knf unchanged (`knf_identity`). The knf is not unique bit for bit, and the test above asks whether it is unique by its rank against the two nulls.
  - What the source holds. Derived from A7 and `entropy_history`. For each voxel x, bit j and window of transitions, the history keeps f_j(x), the number of transitions where bit j flips. It reads the raw 16-bit volume, before any lifting, and its parity check f_j(x) ≡ bit j of I_0(x) ⊕ I_{F − 1}(x) proves it was taken whole.
  - Derived: the history is many-to-one instead of a bijection. A count keeps how many transitions flipped a bit and loses which: a bit that flips at transitions 1 and 2 and one that flips at 3 and 4 give one count in one window. "Unique" can hold for it only as a statistical ID, like the heap fingerprint above, not as the crystal's one-to-one ID.
  - Derived: agreement between two sections has a chance rate. A drawn null draws the rate (A12's form): a section agreeing past all d draws has chance at most 1/(d + 1) under the null. Agreement past that rate reads as shared membership, Doug's "more than one subset". The logic is the period reading's, where agreement past the null at lag p reads the lattice as belonging to its own shifted copy.
  - The null is spatial: permute the data over its spatial coordinates x, y and z, and take the identity of the whole null permutation. Built as `knf_identity` ("The knf's identity by spatial null permutation", above).
  - Open: "the broken edge of the crystal". The history reads raw bits, not the crystal or the part the prediction leaves. Whether the history computed on the crystal's high-pass coefficients is the edge Doug means is not settled.
  - Open: the agreement test between sections, comparing departure curves body against body. The curves are printed; the pairwise test is not built.

- **The departure curve**: every piece of information, however large, has its own departure curve, and that curve is the integral over all of its constituents. Posit. His posits on comparing curves, and on the mutation as a vector magnitude difference, are in wants.md with the pairwise test.

  - Derived: E is a sum over edges, and a departure is linear in E and in the draws' sums. The departure of a whole is the sum of its edges' departures, exactly: the integral of its constituents, with the edge as the constituent. The whole equals its tiles plus its seams at every tile size (proved).
  - Built: each body's curve over its box, the six bounds xmax, xmin, ymax, ymin, zmax and zmin.

- **The two nulls**: they should be close to one to one, each the inverse of the other. They may cross, and where they do, the crossing is mutual in volume and in overall magnitude. Posit.

  - Measured against it: the two are inverse, with the endpoints exact (inside 0 at b = 1, between 0 at b = 64), and they cross between b = 2 and 4. They are not 1:1: inside plus between dips to 0.590 at b = 4.
  - The gap 1 − (inside + between) is 0.41 at b = 4. A reading, and not proved: an inside-shuffled pair in a small tile still shares a body, and neither null removes that co-membership. The gap is then the share both nulls keep, shared membership at scale b, the mutual part of the posit.

- **The bulk and the boundary** (his posit is in wants.md).
  - Derived: T is a bijection from the samples (the bulk) to the crystal (the boundary). Each coefficient reads a cone of reach 3L, and each sample is rebuilt from a cone of reach L + 2 (proved along one line, "The boundary").
  - The holographic codes are isometries with redundancy: a bulk operator can be rebuilt on more than one boundary region (Almheiri, Dong and Harlow 2015 [2]; Pastawski, Yoshida, Harlow and Preskill 2015 [40]).
  - T has no redundancy ("Redundancy" in Open): each sample has one region. T is a bulk-to-boundary map with no error correction, the contrast drawn under "Physical walls".

10.22 OPEN

- **The anchors** (above).
- **The counts in D axes.** The reaches 3ℓ and $3\ell - 2$ for T and L + 2 for T^{-1} are proved exact along one line ("The boundary"). In D axes they are open.
- **The precision count as a test.** Proved since: `test/engine/record_boundary_test` flips input bits and meets the reach 3L on the device ("The boundary").
- **A table by the residue.** A table indexed by x mod 2^b in two's complement factors through π_w for w ≥ b. **Proved** since: `test/engine/record_coherence_test`, 21 checks, 0 failed. At b = 8, the wraps to 8, 13, 16, 31 and 32 bits read x's row on 20480 of 20480 lane-widths (Derived: 20480 = 5 × 4096), and the wrap to 5 bits, below b, breaks it on 2247 of 4096 lanes. A table indexed by the magnitude breaks at every width from 8 bits, on at least 778 of 4096 lanes. All 53248 reads are the row the host's two's-complement word names.
- **Operations that commute with T** (vertical_time_compression.md), now on $\mathbb{Z}_2$^n as on $\mathbb{Z}$^n. `test/engine/record_boundary_test` proves that constants and lattice moves in 2^{3L}$\mathbb{Z}$^n pass through T, and that negation and doubling do not. The general question is open.
- **The fingerprint's counts. Proved** since: `test/engine/record_boundary_test` (Doug's posits, above). The fingerprint per band, not only the total heap, is open.

- **The knf's agreement as a test** (Doug's posits, above). The knf's identity by spatial null is built and run (`knf_identity`). The pairwise test, one body's departure curve against another's, is not built.
- **The knf's uniqueness by the two nulls** (Doug's posits, above). The next posit to test: the knf ranked against its spatial null permutation, and its identity taken by the identity:null permutation. Not yet run.
- **The two nulls' gap.** Whether 1 − (inside + between) measures shared membership at scale b (a reading) is not proved.
- **`record_order_test`** ("The ordered machine"). **Proved** since: 17 checks, 0 failed.
- **Ω for R*.** The loop machine's Ω is not built.
- **The last cell behind the start** ("π turning at the boundary"). How many of the 100 resolutions end at −jα is not counted.
- **The arc's momentum as checks** ("The arc", posits 7 to 9). **Proved** since: `pi_tower`, 17 checks, 0 failed.
- **The residue as a check** ("The arc", posit 14). For each floor with q_j ≤ 364,913, the integer turn's carries equal the permutation's wraps at every step n < q_j (q·c_n + k_{n+1} − k_n = p), giving the in-cell offset exactly as n·δ. **Proved** since: `pi_tower`, 23 checks, 0 failed (check 11, floors to 2^21).
- **Any 2^n as a request argument** ("The arc", posit 13). **Proved** since: `pi_tower`, 23 checks, 0 failed.
- **Whether π obeys Lévy's constant** ("The golden helix"). π's mean pitch over 65 certified floors is 1.166, against Lévy's 1.187 for almost every real.
- **Redundancy.** T is n-to-n and every crystal is legal (the written boundary, proved). A corrupted crystal is another crystal, and T^{-1} returns other samples with no sign of it: T holds no error correction. **Built and proved** since: `test/engine/record_boundary_test`, 59 checks, 0 failed. Each coefficient c in [−2^21, 2^21) is carried as y = c + 2^21 in [0, 2^22), modulo 2053, 2063, 2069 and 2081, and two of the moduli cover the range (Derived: 2053·2063 = 4,235,339 ≥ 2^22 = 4,194,304). The decoder in the machine is Garner's mixed radix over all four, and a value past the range is the syndrome. It then reconstructs from each three, and the correction is the mean of the reconstructions inside the range. Over 16384 coefficients on 256 crystals: clean, 0 false alarms; one residue corrupted in 8115, every one corrected, and T^{-1} returns the samples on 256 of 256 crystals; two corrupted in 8244, every one detected, and the one-error correction misses all 8244. Without the code, the two information residues read 4098 coefficients as another's, and T^{-1} runs on all 256 crystals and returns other samples on 256. With two redundant moduli the code's distance is 3: it corrects one residue or flags two, and it does not guarantee both on one coefficient. How many of the 8244 double errors returned a wrong value in the range is not printed. The code is the classical analog, on this boundary, of the error-correcting codes of holography, a reading.

– Prior art, cited from knowledge. The mixed-radix conversion is Garner's (1959) [13]. A residue system with redundant moduli as a code that checks itself is Watson and Hastings (1966) [59], and its correcting power in the number of redundant moduli is Barsi and Maestrini (1973) [4]. Szabo and Tanaka (1967) [53] is the standard text.

11

KOLMOGOROV AND ARNOLD, COMPUTED EXACTLY

Purpose: Set the Kolmogorov–Arnold representation theorem beside the engine, part by part. The engine already has the theorem's shape in several places, arrived at by accident: sums of functions of one variable, fed through functions of one variable, stacked. This file says where that shape is, where it stops, and what finishing it would take. The next build can then finish it by design.

Scope: The theorem, the networks built on it (KANs), and the engine parts that take its form: the tower (M9), the residual and the component tree (M2, M3), the key and the cycle (M10), the crystal (M9, compression_table.md), the root universal (M20), and the theorem's inner function computed exactly (the `ka_psi` sim). Statuses follow README.md.

11.1 THE THEOREM, AND WHAT KANS DID TO IT

The theorem (Kolmogorov 1957 [30], Arnold 1957 [3]). Every continuous f on [0, 1]^n is a sum and composition of continuous functions of one variable:

f(x_1, …, x_n) = Σ_{q=1}^{2n+1} Φ_q(Σ_{p=1}^{n} φ_{q,p}(x_p))

The inner functions φ_{q,p} can be chosen once for every f; only the outer Φ_q depend on f. The statement is exact: an equality instead of an approximation. Its catch is that the inner functions it guarantees can be very rough (nowhere differentiable). For that reason it was long thought useless for computing (Girosi and Poggio 1989).

KANs (Liu et al., 2024 [36]) take the shape and give up the exactness. A layer is a matrix of one-variable functions on the edges and plain sums at the nodes, x_{l+1,q} = Σ_p φ_{l,q,p}(x_{l,p}), stacked to any depth. Each φ is w · (SiLU(x) + Σ_i c_i B_i(x)), a smooth B-spline on a grid, and every coefficient is fitted in floating point by gradient descent. A KAN therefore approximates f. Its splines are truncated to a grid, its SiLU is a float, and every sum and product rounds.

The engine takes the other road. It keeps the theorem's equality and gives up the fitting. Every function on an edge is an exact integer map, every node sum is exact in two's complement, and nothing rounds anywhere. That makes it not a KAN: it is the theorem's own form, computed exactly, on the finite integer domain the data actually lives on. Where the engine is "imperfect", it is incomplete instead of approximate.

11.2 ON A FINITE DOMAIN THE THEOREM IS EXACT AND ONE TERM SUFFICES

The engine's data never leaves a finite set of integers: a voxel is a u16 lane, a coordinate is below its extent. On a finite domain the representation needs no continuity argument and no 2n + 1 terms. With every x_p in [0, B):

f(x_1, …, x_n) = Φ(Σ_{p=1}^{n} x_p · B^{p − 1})

The inner functions φ_p(x_p) = x_p · B^{p − 1} are shifts. Their sum is the mixed-radix number of the point, which is a bijection onto [0, B^n). An outer Φ therefore exists and equals f read at that number. It is arithmetic instead of approximation: exact for every f (**proved** by construction; the mixed radix is a bijection).

This is also exactly what the engine's multi-limb keys do: they pack several quantities into one exact integer as radix digits. The component tree's key carries the face's name in its lowest limb below the residual's limbs (`max_tree_key_limb`), and that makes every key distinct.

It also says where the difficulty went. The one-term form moves the whole of f into Φ, a table of B^n entries. The theorem's real content, on any domain, is that Φ and the φ can be *small*. KANs look for that smallness by fitting smooth functions. The engine looks for it by reading structure out of the data exactly: separability per axis, a period, a ladder of rungs. Both are the same search for a short description, which is Kolmogorov's other result (below).

11.3 THE ENGINE AS THE THEOREM, PART BY PART

the theorem's element	the engine	exact?	status
an inner function φ on each input, then a node sum	**the tower's lifting steps.** Predict: h = x_odd − ⌊(x_left + x_right) / 2⌋ (tower.cu). Update: l = x_even + ⌊(h_before + h_after + 2) / 4⌋ (tower.cu). Each step adds to one value the outer function Φ(s) = ⌊s / 2^k⌋ of a node sum s of its neighbors. Each step therefore has the form x + Φ(Σ φ_p(x_p))	yes. The quotient is an exact integer function, and the inverse subtracts the identical quotient. Each step is therefore a bijection on the integers (A8)	**proved**: 25 of 25 samples rebuilt voxel for voxel
layers stacked: Φ_{L − 1} ∘ … ∘ Φ_0	the tower's floors: 8 floors over t, z, y and x reach one coefficient for a 100 × 64 × 256 × 256 sample	yes	**proved** (same run)
a layer's edges as one-variable functions of each coordinate	**the residual.** L_n = [1 1]^n ∗ I is separable per axis, one one-variable convolution per axis composed, and R = 2^{|b|} L_s − L_{s+b} is a sum of two such terms (A1)	yes; the kernel sums to zero exactly	**proved**: 0 of 10,485,760,000 lanes differ from the step-by-step residual
the outer Φ after the sum	**the residual's readers.** P(x) = [R(x) > 0] and the dense rank c(x) of R(x) are one-variable monotone functions of the sum R (A2)	yes	built

the theorem's element	the engine	exact?	status
KAN's B-spline basis B_i	**the binomial ladder.** [1 1]^n is the uniform discrete B-spline of order n: the n-fold convolution of the unit box, and that convolution defines a uniform B-spline (Schoenberg 1946 [45]), taken on the integer grid with integer coefficients	yes: binomial coefficients, no float	an exact identity (standard); built as the ladder
KAN's grid extension: refine the spline grid without refitting	**one ladder's rungs.** B_s ∗ B_b = B_{s+b}: a finer or coarser B-spline is another rung of the same ladder, and a sweep that keeps its rungs gives every (s, b) in one pass (A1)	yes	theory: the ladder-keeping sweep is not built
a sum node in the network	**the cycle's fold.** Every term of the imprinted key is summed into each output lane in two's complement, one launch	yes	**proved** (the key against the passes, above)
a composed network is itself a function to compose again	**keys of keys.** A composed key is a step in another program (noise_sieve_tower.md §9)	yes	**proved** for linear keys
a general nonlinear φ on an edge	**a pointwise step as a table on the lane's alphabet.** Every value a lane can hold is pushed through the step once, and the table is the function (noise_sieve_tower.md §2). On a finite alphabet any one-variable function is exactly a table. No spline is needed and nothing is fitted	yes, by construction	**proved** as the record machine's table step on x^2 and $\lvert x - 30000\rvert + 100$ (M10, A13 of engine_table.md; `record_table_test`, 4,096 lanes, device and host, 15 checks, 0 failed); on the tower's edges, as a bijection (M20, A14; `tower_edge_test`, 25 checks, 0 failed)
KAN's SiLU base term, which takes whatever the spline does not	**the noise, determined and not absorbed**: the medium subtracted exactly, the residue stored whole, and the floor measured per voxel and bit (the next section)	yes	**proved** (the residue lossless) and **measured** (the floor)
coefficients c_i fitted by gradient descent	nothing is fitted; a scale comes from the data (order = period, M2) or the request	nothing to round	absent, by the engine's rule (no chosen numbers)

11.4 SAME SKELETON, A DIFFERENT NOISE TERM

The skeleton is shared: one-variable functions on the edges, sums at the nodes, layers stacked. The two part at the term that holds what the functions do not explain.

In a KAN each edge is w · (b(x) + spline(x)), with b = SiLU. The base term is one fixed function, the same on every edge of every network and for every dataset, chosen before any data is seen. Whatever the spline does not fit goes into it, or into the loss, as a residual error in floating point. Nothing says what that remainder *is*: it is minimized and then ignored.

In the engine the remainder is determined, three ways and exactly:

- **Subtracted.** The residual takes the medium out whole: R = 2^{|b|} L_s − L_{s+b}, whose kernel sums to exactly zero: a constant background leaves R unchanged and nothing of it survives as error (A1; **proved**, 0 of 10,485,760,000 lanes differ).
- **Stored.** Every floor's high-pass coefficients, the residue, are kept whole in the crystal: never bounded, modeled or discarded, and rebuilt voxel for voxel (**proved**, 25 of 25).
- **Measured.** The entropy history measures the floor per voxel and per bit, as exact counts. On 44b6_0113de3b, bits 0 to 3 flip 499 to 500 times per thousand transitions in every window, at maximum entropy. Bits 5 to 10 fall from about 430 to about 20 where bodies hold them. The fixed pattern is the anchor bits, which never flip (**measured**; the counts **proved** on 2,000 voxels × 9 windows). The null draws read the field's own noise at a body's lag in frames far off in time. A body is real only where it stands above that reading (built).

So where a KAN has one generic base term and an unexplained remainder, the engine names the remainder at every voxel and bit and keeps every bit of it. That is also what makes the compression floor measurable (compression_table.md, F4): the noise is a measured quantity with a place instead of a loss value.

claim	status
the engine and a KAN share the skeleton: edge functions, node sums, stacked layers	the engine's instances are **proved** (the tower, the residual, the cycle); the skeleton is the theorem's
a KAN's base term is fixed and generic, and its remainder is minimized, never determined	the published construction (SiLU plus spline, fitted by loss)
the engine determines the remainder: subtracted exactly, stored whole, and its floor measured per voxel and bit	**proved** (subtraction, residue) and **measured** (the floor)

11.5 WHERE IT IS INCOMPLETE, AND WHAT FINISHES IT

1. **Nonlinear edges** (built: the record machine's table step, M10 and A13 of engine_table.md). The engine's other edges are linear: convolution taps, shifts, the lifting's neighbor sums. The theorem needs general one-variable functions. On the lane's alphabet these are exact tables: 2^16 entries a function for a u16 lane, built once by pushing every value through, and composable in program order. This is the LUT of noise_sieve_tower.md §2. **Proved** by `record_table_test`, 15 checks and 0 failed: x^2 and $|x - 30000| + 100$ as tables equal the ops on 4,096 lanes, device and host, and a table read through a table equals its composition and differs from it reversed. With it the partial form becomes the full one; which functions to tabulate is item 3. The tables also stand between the tower's floors (built: M20 and A14, the root universal). "The tower is its own object, the lut is its own object, if we integrate them both we get a root universal." The tower is the additive skeleton, sums collapsed in order onto the floor, and an edge between two floors makes the collapse an outer sum with non-linear edges (theory). **Proved:** 171 random edges on 48 camera-law volumes rebuild every lane through the whole crystal path, and two edges on one floor fold into one table in their

order and no other. Two bounds hold it (theory). It is universal over the finite coefficient alphabet, every function of the indexed bits one table, exactly and not approximately; it is not the continuous-function theorem. And inside the reversible stream an edge must be a permutation. A table that is not a bijection (|x|, a comparison) therefore rides widened: the Bennett embedding (x, y) → (x, y ⊕ f(x)) is a permutation of the pair for any f (Bennett 1973 [5]), and the bound is one of width, b_x + b_y bits within 20, not of kind ("a dimensional expansion"; **proved** by `tower_edge_test`, 25 of 25, A14).

2. **Outer functions beyond ⌊·/2^k⌋ and a threshold** (theory). The lifting's quotient and the residual's sign and rank are the outer functions in use. A general Φ is the same table machinery as item 1, applied after a sum.

3. **Choosing the functions without fitting** (open, Doug's call). A KAN finds its φ by descent, and the engine's rule forbids a chosen or tuned number. Two exact routes are open. The data can name the function the way it already names an order (a period read off the data). Or a fit can be solved exactly in rationals: least squares on integers has an exact rational solution through its normal equations, and nothing rounds. Whether any fitted function belongs in the engine at all is Doug's ruling.

4. **Many terms, or one** (theory). On the finite domain one term suffices (above), but its Φ is huge. Keeping the functions small is the point. The target is the fewest terms and the smallest tables that hold f exactly. That size is measurable, which is item 5.

5. **The two Kolmogorovs meet in the crystal** (theory). The representation theorem is Kolmogorov 1957. The crystal's name, `.kcr`, is Kolmogorov 1965 [31] (Kolmogorov 1968 in English): the complexity K(x), the length of the shortest program that prints x. They meet here. A set stored as an exact representation (the tables and sums of items 1 to 4) plus its exact residue is a two-part code: the model, then what the model does not predict. Its length bounds K(x) from above. An exact Kolmogorov–Arnold form of a set is therefore a compressor, and every table it saves is bytes off the crystal. That adds a rung to the ladder in compression_table.md: the two-part code, the representation's size plus the residue's F2.

claim	status
the tower's lifting steps have the theorem's form x + Φ(Σ φ_p(x_p)), exactly and invertibly	**proved** (25 of 25 rebuilt)
the binomial ladder is the uniform discrete B-spline basis, in integers	an exact identity; built as the ladder
on a finite integer domain the representation holds exactly with one term and radix inner functions	**proved** by construction
the engine is a KAN	**not so**: the same skeleton, but a KAN fits float splines, rounds throughout and absorbs its remainder in a fixed base term; the engine holds the theorem's form exactly, fits nothing, and determines its remainder

claim	status
nonlinear edges as exact tables on the lane's alphabet complete the form	**proved** for the table step (`record_table_test`, 15 of 15; M10, A13); the functions to tabulate are open (item 3)
the tower with permutation edges between its floors, the root universal, rebuilds every lane and folds edges in order	**proved** (`tower_edge_test`, 25 of 25; `root_universal`, 1,268 of 1,268; M20, A14)
a function that is not a bijection rides the reversible stream through the Bennett embedding, at the price of width	**proved** (|x| and x ≥ 100 as 16-bit edges, 64 of 64; |x| laid bare refused; A14)
the root universal is universal over the finite coefficient alphabet, exactly	theory; it is not the continuous-function theorem
an unfitted edge's price, in coded bits a voxel	**measured**: 6.802 with none, 12.608 with a random 12-bit permutation on floor 0, no change at three decimals on the collapsed floor (M20); a fitted edge's price is not measured
an exact representation plus its residue is a two-part code that bounds K(x)	theory; a rung for the compression table

11.6 KOLMOGOROV'S INNER FUNCTION, EXACTLY

The continuous theorem, in Sprecher's version, is built on one inner function ψ. The constructive proof rests on two of its properties, that it is strictly increasing and that it is continuous. Sprecher's own ψ is not increasing. This section holds ψ exactly, in the engine's arithmetic, and proves both properties. It then tests the proof's next step, the separation of the inner sums (Braun and Griebel's section 3), finds it false at the first level, and repairs it. It is graded by the sim `ka_psi` (`engine/sims/ka_psi/ka_psi.cu`, `bash src/sims/run.sh ka_psi`), run as `build/20260923_221011_sim_ka_psi`: 152 checks, 0 failed, about 3 s here. Items 1 to 3 print the same lines as the earlier build `build/20260923_215902_sim_ka_psi` (114 checks, 0 failed), and the separation adds the rest. The sim is orior's.

The setting (Braun and Griebel 2009, Theorem 2.1; A15 of engine_table.md). Take integers n ≥ 2, m ≥ 2n, γ ≥ m + 2 and a = [γ(γ − 1)]^-1. Then f(x) = Σ_{q=0}^{m} Φ_q ∘ ξ(x_q), with ξ(x_q) = Σ_{p=1}^{n} α_p ψ(x_p + qa), α_1 = 1, α_p = Σ_{r≥1} γ^−(p−1)β(r) and β(r) = (n^r − 1)/(n − 1). ψ is first defined on the terminating base-γ rationals D_k = { Σ_{r≤k} i_r γ^−r } and extended to [0, 1] as a limit. Here n = 2 and n = 3 with γ = 10, which meets γ ≥ 2n + 2.

The arithmetic (proved by construction). Every value on a grid is an exact integer over 2^L · γ^β(L). At depth, a value is a sparse sum Σ c_j γ^−e_j: each c_j an exact rational, and each e_j = β(L) stored as an integer and never expanded ("the exp is symbolic"). At level 60 for n = 2, β = 2^60 − 1 = 1,152,921,504,606,846,975, a number no expansion of digits could reach. A sign is decided exactly: the head is folded in until it outweighs a proven bound on the tail, and when the exponent gap is wide, bit lengths decide it. So every question of the form "is it past the limit" gets a true or false answer instead of a rounded one.

1. **Sprecher's ψ fails, exactly.** Sprecher's ψ (Braun and Griebel's 2.4) at n = 2 and γ = 10 gives ψ(0.58999) = 2207/4000 = 0.55175 > ψ(0.59) = 11/20 = 0.55, the paper's (2.5) reproduced

as exact rationals (**proved**). It is not increasing, and it descends between 10 of the 99,999 neighboring pairs of D_5, the first between 0.08999 and 0.09000 (**measured**). The paper's values need m_r = ⟨i_r⟩(1 + Σ_{s<r} [i_s] ⋯ [i_{r − 1}]), the empty product counted as 1, as in Sprecher's own form; the sum as typeset in (2.4), with [i_1] = 0 and no leading 1, gives ψ(0.59) = 0.501. The sim takes the reading that reproduces (2.5), and an independent recount in exact fractions agrees on both values and on the 10 descents.

2. **Köppen's ψ, in both of its readings.** Köppen's correction is recursive on D_k. A level-L point whose last digit is γ − 1 is carried: it takes a midpoint of the level-(L − 1) values ψ and ψ* on either side. The paper gives that midpoint two ways. (2.9)/(2.10), the form its proofs use, is ψ_L = ½ψ_{L − 1} + ½ψ*_{L − 1} + (γ − 2)/(2γ^β(L)), symmetric in the cell. (2.7) as printed puts i_k = γ − 1 inside the half, (γ − 1)/(2γ^β(L)). Of the two, (2.9) averages ψ_L(d − γ^ − L) with ψ_{L − 1}(d + γ^ − L). For n = 2 and n = 3 at γ = 10, on every point of D_1 to D_5 (10 to 100,000 points), in both readings (**proved**, exhaustively):

   - (a) the pair recursion, the scale step on (ψ, ψ*), equals the level-by-level recursion;
   - (b) ψ is strictly increasing on every neighboring pair;
   - (c) the least gap is exactly γ^ − β(L);
   - (d) the widest gap equals G_L of the recursion G_1 = 1/γ, G_L = ½G_{L − 1} − (s/2)γ^ − β(L), with s = γ − 2 for (2.9) and s = γ − 3 for (2.7). An independent recount in exact fractions agrees on (b), (c) and (d) for every level, both n and both readings.

3. **Every scale, by chaining the one step.** Checked symbolically to L = 60 for n = 2 and L = 38 for n = 3 (β = 675,425,858,836,496,044), in both readings (**proved**):

   - the step keeps the order: γ^ − β(L − 1) > (γ − 1)γ^ − β(L), on all 59 and 37 levels;
   - the least gap stays γ^ − β(L): ½γ^ − β(L − 1) > ((γ + 1)/2)γ^ − β(L), because γ^(n^(L − 1)) > γ + 1;
   - the widest gap stays above the least and at most 2^ − (L − 1)/γ, and within Lemma 2.3's bound (½)^(L − 2)[1/(2γ) + (γ − 2)γ^n/(γ^n − 2)];
   - the symbolic G_L equals the gap measured on D_2 to D_5, 4 of 4;
   - at a keyed depth-60 point (depth 38 for n = 3), ψ is a sparse sum out to exponent β(L), of 55 and 57 terms in the two readings for n = 2 and 36 for n = 3, and its gap ψ* − ψ lies between the least and the widest.

Why it holds, at every level (proved by induction; the sim checks each step's inequality exactly). Inside a level-(L − 1) cell of width D, write u = γ^ − β(L). The level-L points are ψ + i·u for i = 0 to γ − 2, then the carried midpoint M, then ψ*. Under (2.9) the two gaps beside M are each D/2 − (γ − 2)u/2. Under (2.7) they are D/2 − (γ − 3)u/2 and D/2 − (γ − 1)u/2.

- **The least gap.** g_L = min(u_L, ½(g_{L − 1} − (γ − 1)u_L)) in the worst case, and g_1 = u_1. Since u_{L − 1}/u_L = γ^(n^(L − 1)) ≥ γ^n > γ + 1, induction gives g_L = u_L > 0: ψ is strictly increasing on every D_L.

- **The widest gap.** G_L = ½G_{L − 1} − (s/2)u_L ≤ 2^ − (L − 1)/γ. Two points closer than γ^ − L lie in one level-L cell or two neighboring ones. Between them ψ moves by at most 2G_L, and that goes to 0: ψ is continuous.
- **The extension.** Between any x < y in [0, 1] lie two grid points d < d′ of some level, with ψ(d) < ψ(d′). The limit ψ on [0, 1] is therefore strictly increasing as well as continuous.

What it settles. Both readings give a continuous, strictly increasing ψ; the (2.7)/(2.9) discrepancy changes only the widest gap. Sprecher's proof needed this step and his ψ does not give it. Köppen's ψ does, and Braun and Griebel prove it by bounds (Lemmas 2.3 and 2.4). Here it is computed exactly to depth 60, with every inequality decided true or false.

11.6.1 *Separation*

The theorem needs more than ψ. The inner sum ξ(d) = Σ_p α_p ψ(d_p) must hold distinct grid points apart, for the outer functions to be built piece by piece on its image. Braun and Griebel prove this in three steps (the preprint, pages 12 to 15, read here).

- **Lemma 3.4.** For distinct d, d′ ∈ D_k^n, the difference μ_k = Σ_p α_p [ψ(d_p) − ψ(d′_p)] has |μ_k| ≥ γ^ − nβ(k).
- **Lemma 3.7.** The images T_k(d) = [ξ(d), ξ(d) + (γ − 2)b_k] are disjoint, where b_k = ε_{k,2} Σ_p α_p and ε_{k,p} = Σ_{r>k} γ^ − (p − 1)β(r) is the tail of α_p.
- **Lemma 3.8.** The supports U_k(d) = (ξ(d) − γ^ − β(k+1), ξ(d) + (γ − 2)b_k + γ^ − β(k+1)) are disjoint. These are the supports of the ramps ω, whose ramp width is γ^ − β(k+1). Distinct points' supports are disjoint when their ξ differ by at least (γ − 2)b_k + 2γ^ − β(k+1).

The unit below is u′ = γ^ − β(k+1). Because β(k + 1) = nβ(k) + 1, Lemma 3.4's margin γ^ − nβ(k) is γu′ = 10 units.

The method (the sim; recounted here). ξ is computed exactly on every point of D_k^n, with ψ in the (2.9) form:

- for n = 2 at k = 1, 2 and 3 (100, 10^4 and 10^6 points);
- for n = 3 at k = 1 and 2 (10^3 and 10^6 points).

The values are sorted, and the least neighboring gap is found. The sim cuts α_p at the terms ≥ γ^ − (β(k+1)+3) and bounds the dropped tail by twice its first term. Each gap it prints is therefore a lower bound. An independent recount in exact fractions cut α_p at two depths further apart. It finds the same tightest pair at every k, and the exact gaps below.

1. **Lemma 3.4 is false at k = 1 (proved).** The pairs are tabled below; one unit is 10^{-3} for n = 2 and 10^{-4} for n = 3.
    - n = 2, the pair (0.1, 0) and (0, 0.9): μ_1 = 1/10 − (9/10)α_2, with α_2 = 0.1010001…, which is 9.0999099999991… units, below the margin of 10.

- n = 3, the pair (0.1, 0, 0) and (0, 0.9, 0.9): $\mu_1 = 1/10 - (9/10)(\alpha_2 + \alpha_3) = 9.0999099991\ldots$ units, again below 10.
- At k = 1 the supports of Lemma 3.8 need 10.809 units (n = 2) and 10.881 (n = 3), more than the least gap, and they overlap.
- The images of Lemma 3.7 have width $(\gamma - 2)b_1 = 8.809$ and 8.881 units, less than the least gap, and they are disjoint, k = 1 included.
- From k = 2 on, the lemma holds with room at every level tested:

n	k	least gap, units	tightest pair	α cut at r ≤ k (3.11)
2	1	9.0999099999991…	(0.1, 0), (0, 0.9)	10, on the margin
2	2	49.50499999505…	(0.04, 0.50), (0.09, 0.05)	50
2	3	500,049.5000005…	(0.039, 0.500), (0.094, 0.005)	500,050
3	1	9.0999099991…	(0.1, 0, 0), (0, 0.9, 0.9)	10, on the margin
3	2	$5{,}000 + 5 \times 10^{-15}$	(0.05, 0, 0.04), (0, 0, 0.09)	5,000

2. **Where the proof breaks** (**proved** on the grids above; the argument read in the preprint).
   - The proof first bounds (3.11), μ with each α_p cut to its first k terms, $\alpha_p - \varepsilon_{k,p}$. That bound holds: its least gap on every grid tested is the last column, at least the margin, and exactly on it at k = 1.
   - It then says the tails $\Sigma_p \varepsilon_{k,p}A_{k,p}$, with $A_{k,p} = \psi(d_p) - \psi(d'_p)$, are "too small to annihilate" $\gamma^{\wedge} - n\beta(k)$. That is true, and it does not keep $|\mu_k|$ at or above $\gamma^{\wedge} - n\beta(k)$: at k = 1 the tails pull it from 10 units to 9.0999099….
   - So the lemma's bound fails at k = 1. At every k the proof's last step does not prove it.
3. **The repair.** Replace the lemma's bound and narrow the ramp.
   - **The corrected bound** (**proved**, given (3.11)). Since $\psi \in [0, 1]$, each $|A_{k,p}| \le 1$, and $|\mu_k| \ge \gamma^{\wedge} - n\beta(k) - \Sigma_{p \ge 2} \varepsilon_{k,p}$, about $(\gamma - 1)u'$.
   - **The images stay apart.** The bound beats the image width $(\gamma - 2)\varepsilon_{k,2}\Sigma_p \alpha_p$. The reason: for n = 2 and 3 the exponents $(p - 1)\beta(r)$ are distinct positive integers. Then $\Sigma_p \alpha_p < \gamma/(\gamma - 1)$, and $(\gamma - 2)\gamma/(\gamma - 1) < \gamma - 1$ for every γ.
   - **The supports stay apart with a narrower ramp.** Use $\rho = \gamma^{\wedge} - (\beta(k+1)+2)$, γ^2 narrower than the paper's $\gamma^{\wedge} - \beta(k+1)$. The supports are then disjoint whenever $\gamma^{\wedge} - n\beta(k) - \Sigma_{p \ge 2} \varepsilon_{k,p} - (\gamma - 2)\varepsilon_{k,2}\Sigma_p \alpha_p - 2\rho > 0$. Each sum is taken as an upper bound: the series is cut after two terms, plus twice the third, which is valid because the exponents grow by at least 1 a term.
   - **At every scale** (**proved** at each level, exponents never expanded). The inequality holds at every k from 1 to 57 for n = 2 and 1 to 35 for n = 3. With the paper's ramp the same corrected bound covers no level, 0 of 57 and 0 of 35; the actual grids from k = 2 on are still disjoint with it (item 1 of this part).

- **On the grids.** The narrow ramp separates the supports on every grid tested, k = 1 included (**proved**, exhaustively).
- So Lemma 3.4 as stated is false at k = 1, and its proof has a gap at every k. The corrected bound, together with a ramp narrowed by γ^2, restores Lemmas 3.7 and 3.8 at every k.

4. **The shifts** (Theorem 3.3's covering).
   - The (m + 1) shifted copies of the grid use x + qa, with $a = [\gamma(\gamma - 1)]^{-1}$. Then qa = 0.0qqq… in base γ and $qa \equiv q\gamma^{-k}/(\gamma - 1)$ modulo γ^{-k} (**proved**). The gaps between the cubes of each copy are therefore disjoint within each coordinate.
   - With m = 2n, a coordinate sits in at most one gap. So at least m − n + 1 of the shifts put the point inside a cube: 3 of 5 for n = 2, 4 of 7 for n = 3 (the sim's check).
   - With m + 1 = γ, two shifts share a gap. The theorem asks $\gamma \geq m + 2$ for this reason.

Not yet done (do not read it as done). Two things remain:

- the contraction of the outer functions (Theorem 3.3), with the narrowed ramp in place of the paper's;
- ψ at the shifted points x + qa, which are off the grid.

Until both are built, the engine holds the theorem's inner function and its separation on the grid exactly, short of the continuous theorem. A14's "not the continuous theorem" stands until Doug rules.

The literature (each record checked at Crossref or arXiv).

- D. A. Sprecher, "A numerical implementation of Kolmogorov's superpositions", Neural Networks 9(5) (1996) 765–772, doi:10.1016/0893-6080(95)00081-X; and "… II", Neural Networks 10(3) (1997) 447–457, doi:10.1016/S0893-6080(96)00073-1. The ψ that fails (item 1).
- M. Köppen, "On the training of a Kolmogorov network", ICANN 2002, LNCS 2415, 474–479, doi:10.1007/3-540-46084-5_77. The corrected ψ (items 2 and 3).
- J. Braun and M. Griebel, "On a constructive proof of Kolmogorov's superposition theorem", Constructive Approximation 30(3) (2009) 653–675, doi:10.1007/s00365-009-9054-2. The proof of Köppen's ψ by bounds.
- J. Actor and M. G. Knepley, "An algorithm for computing Lipschitz inner functions in Kolmogorov's superposition theorem", arXiv:1712.08286 (2017). A reading, not checked here: they computed in mpmath floats with no precision analysis, and their candidate ψ^{p,q} = α_p(x + qε) separates at every finite level but loses separation in the limit.
- F. Girosi and T. Poggio, "Representation properties of networks: Kolmogorov's theorem is irrelevant". The objection that the inner functions are too rough to compute. Neural Computation 1(4) (1989) 465–469, doi:10.1162/neco.1989.1.4.465.
- J. Schmidt-Hieber, "The Kolmogorov–Arnold representation theorem revisited", Neural Networks 137 (2021) 119–126, doi:10.1016/j.neunet.2021.01.020 (arXiv:2007.15884).

- A. C. Antoulas, I. V. Gosea and C. Poussot-Vassal, "Variable decoupling and the Kolmogorov superposition theorem for rational functions", arXiv:2605.07246 (2026).

claim	status
Sprecher's ψ is not increasing: ψ(0.58999) = 2207/4000 > ψ(0.59) = 11/20	**proved**, exactly (`ka_psi`; recounted independently)
Köppen's ψ, in both readings, is strictly increasing with least gap γ^ − β(L) and continuous with widest gap G_L ≤ 2^ − (L − 1)/γ	**proved**: exhaustively on D_1 to D_5, symbolically to L = 60 (n = 2) and 38 (n = 3), and by induction for every L
Braun and Griebel's Lemma 3.4 holds: |μ_k| ≥ γ^ − nβ(k) on D_k^n	**false** at k = 1 (9.0999099… units against 10, n = 2 and 3); holds at every grid tested from k = 2; its proof has a gap at every k
the corrected bound |μ_k| ≥ γ^ − nβ(k) − Σ_{p ≥ 2} ε_{k,p}, with the ramp narrowed by γ², keeps the images and supports of Lemmas 3.7 and 3.8 disjoint	**proved** at every k from 1 to 57 (n = 2) and 1 to 35 (n = 3), given (3.11), and on every grid tested
the outer functions' contraction (Theorem 3.3), and ψ at the off-grid shifted points	not built
the engine holds the continuous theorem	**not so** yet; Doug's ruling awaited

12

THE SCHEDULER: TESSERA

Purpose: The theory of the engine's device-job scheduler: how jobs from separate processes share one device by memory, what is measured and how, what the accounting guarantees, and what is still open.

Scope: `tessera`, a daemon per host and device, with its client calls. The rows are M14 and A11 in engine_table.md. Statuses follow README.md.

12.1 WHAT WAS RULED (23 SEPTEMBER)

- "just a normal job scheduling function, super common for stuff like this".
- Jobs are packed by memory: several run at once while they fit.
- The byte budget is a request field. The engine chooses no share, and schedule.cu's free/3·2 goes.
- One arbiter per host and device: "daemon or systemd on linux that self tears down/knocks up, single win pid, single docker hook ... this shit is getting deployed onto SaN".
- Clients reach it over a local socket.
- It measures each process's device memory by pid, "with periodic dead sweeps ... otherwise itll leak over time and lock down memory that is never used". Stale tickets "get dumped to lostandfound" (now `hst/lnf.log`).
- "caller declares, we measure, they overbudget it could be for a reason we ask they can override, they underbudget we grow and report via warn so accounting isnt perturbed".
- Over budget is judged at admission, against the last measured peak of the job's signum. An over-budget job is not thrown away: "it goes into holding before stalling and dumping to lostandfound" (now `hst/lnf.log`).
- Times (the holding time, the sweep and the idle time) are per-job fields.

12.2 THE ACCOUNTING

Let the device hold C bytes. At time τ, the driver reports U(τ) bytes in use on the device, and each admitted job j has:

- d_j, its declared bytes;
- r_j, its reservation: r_j starts at d_j and only grows;
- u_j(τ), the bytes its process holds now, measured by pid.

The bytes occupied by processes outside tessera are
$O(\tau) = U(\tau) - \Sigma_j\, u_j(\tau)$
A job reserves room it may not have allocated yet. It therefore blocks $\max(r_j, u_j(\tau))$. The headroom is
$H(\tau) = C - O(\tau) - \Sigma_j \max(r_j, u_j(\tau))$

Admission. A waiting job k is admitted when $d_k \leq H(\tau)$. Then $r_k = d_k$, and the headroom falls by d_k. "We reserve what they ask for and then if it cost less we remember that for next time".

Growth. If $u_j(\tau) > r_j$, then r_j becomes $u_j(\tau)$ and the client is warned: declared d_j, now r_j. Growth never stops a job already running, and a job can push H below zero. While $H < 0$, nothing new is admitted.

Invariant. Admission keeps $H \geq 0$. Only growth or an outside process can drive H negative, and both are measured. The sum the scheduler believes it has granted plus what it measured outside equals $C - H(\tau)$ at every sweep: the accounting is never perturbed, only corrected, and every correction is reported.

Release. When job j ends, r_j and u_j leave the sums. Its measured peak
p_j = max over its sweeps of $u_j(\tau)$
is kept under its signum σ_j, the BLAKE3 root of its request. The engine is deterministic in its request and its result. The same request needs about the same bytes again. The kept peak is a measurement at the sweep's resolution, though, not a byte-exact constant: identical requests have read peaks 4,194,304 bytes apart (below).

Over budget. At submission, a job whose signum has a kept peak p and whose declaration exceeds it ($d_k > p$) is asked. It waits in holding with its precalc intact. With the override set it is admitted on d_k. If no answer comes within its holding time, it is moved to lost and found. A signum never seen is admitted on its declaration and measured.

Dead sweep. A client that dies closes its socket, and the daemon sees the close at once. The periodic sweep is the backstop. Each ticket carries the pid and the process's start time. A pid reused by a new process is then not mistaken for the old one. A ticket whose process is gone releases its reservation and moves to lost and found with its last measure.

12.3 MEASURING BY PID

- **Linux:** NVML's per-process list gives each compute process's used bytes. The daemon learns a client's pid from the socket (`SO_PEERCRED`), translated into the daemon's pid namespace. A client inside a container is therefore measured under its host pid, and a client cannot misstate its pid.
- **Windows:** under the WDDM driver model, NVML always reports per-process memory as not available, because Windows manages it. There the daemon reads the counter Task Manager uses, `\GPU Process Memory(pid_<pid>_luid_*_phys_*)\Dedicated Usage`, through

PDH. It learns the pid from the pipe (`GetNamedPipeClientProcessId`). Confirmed on this machine: the counter lists each process's dedicated bytes by pid and adapter.

- **Both:** the device's total and in-use bytes come from NVML's device memory query, which works under WDDM too. The daemon never creates a CUDA context of its own and holds no device memory.

12.4 THE PARTS

- **The ledger** (`tessera_ledger.{c,h}`) is pure: the accounting, backfill and the deadline heap, with no socket, clock or device. Every rule above is a function of its inputs, and the suite proves it without a GPU.
  - Proved: 143,808 cases, 0 failed. The headroom identity holds on 2,000 random states. The deadlines fire in time order. Budget, hold, override and lost pass their 18 checks. Idle teardown passes its 4. Backfill never moved the head's shadow later, over 121,586 admissions in 20,000 random scenarios, every one of which ran to completion.
  - Found by the suite: a run released before its first measure had recorded a peak of 0, and every later run of that signum was withheld as over budget. A peak now enters the history only if the run was measured at least once.
- **The frame** (`tessera.h`) is 128 bytes, little-endian, the same on every platform: magic, version, kind, the override, the device's UUID (16 bytes) and LUID, the request's signum, its declared bytes, its three times, and the bytes, measure and identity of a reply.
  - Asks: submit, override, release, precalc saved.
  - Tells: admitted, asked, grew, lost, released, refused.
- **The client** blocks in its submit until the daemon admits the job, asks about it, or loses it. A grew frame can arrive while the job runs; the client drains those at release and reports the largest in its ticket, for the caller to warn.
- **The daemon** is one program per device, named by the device's UUID: the pipe `\\.\pipe\tessera-<uuid>` on Windows, the socket `tessera-<uuid>.sock` in the runtime directory on Linux. Each client has its own thread, all behind one lock; one timer thread waits on the deadline heap's root.
  - On each sweep it reads the device's memory and the job's pid, and warns if the job grew.
  - After every change it runs admission and tells each admitted client.
  - It keeps each process's identity: a handle on Windows, a pidfd on Linux. A reused pid can never pass for the old process.
- **Its files** live in a state directory per device. The lock is `tessera.lock`. The rest sit under `hst/`: the history `hst/head.log` (signum, peak, duration), sealed with the host BLAKE3; `hst/tail.log`, the copy each save writes before it renames it over `hst/head.log`; and `hst/`

`lnf.log`, one append-only file of sealed blocks, each a lost job's ticket or the note a client sends when it keeps that job's precalc.

12.5 ONE DAEMON PER HOST

- **Windows:** the daemon creates its pipe with `FILE_FLAG_FIRST_PIPE_INSTANCE`, and a second daemon fails at birth. A client that finds no pipe starts the daemon detached and retries.
- **Linux:** `bind()` on the socket path refuses a second daemon. Under systemd the socket unit holds the listening socket and starts the daemon on the first connection (socket activation: the socket arrives as fd 3). The daemon can then exit when idle and wake again. Without systemd, a client starts it as on Windows.
- **Docker:** the host's socket is bind-mounted into the container. The pid namespace translation above keeps the measure exact.
- **SAN:** device memory is local to a host. Each host runs its own daemon, and nothing is shared across the SAN but the data.

12.6 ORDER, TIME AND LOSS (23 SEPTEMBER)

- **Order among waiting jobs: backfill with the head kept.** A job behind the head may go first only if it cannot delay the head's admission. The head never starves, and the room the head cannot use yet is not wasted.

  "Cannot delay" is a statement about time, and it needs durations. The engine is deterministic in its request and its result. A signum's run time therefore repeats about as its peak does (each a measurement, not a constant), and the history keeps both: (p_σ, t_σ), the peak bytes and the wall time of the last run. With the head h not fitting (d_h > H):

  - The head's shadow time S is the earliest moment at which the running jobs' releases, taken in order of their expected ends e_j = start_j + t_σj, free enough room: the least S with H + Σ_{e_j ≤ S} max(r_j, u_j) ≥ d_h.
  - The spare at S is X = H + Σ_{e_j ≤ S} max(r_j, u_j) − d_h: room the head will not need even when it starts.
  - A waiting job k behind the head is admitted now if d_k ≤ H and either it ends before the shadow (now + t_σk ≤ S) or it fits in the spare (d_k ≤ X, and then X falls by d_k).
  - A job whose signum has no history has no duration: it can never be proved not to delay the head, and it is not backfilled. It waits its turn, and its first run records (p_σ, t_σ).
  - If a job overruns its expected end, the head's shadow moves later. That is measured and reported like a growth, never hidden.

- **Time: the soonest change is the root.** "what I want is the shortest time to change of anything to be the root because that is most likely event, in order". Every deadline goes

into one min-heap keyed by the time until it changes something: each live job's next sweep, each withheld job's holding expiry, the idle teardown. The daemon waits on its socket for at most the root's remaining time, handles the root, pushes that item's next deadline and takes the new root. Events are handled strictly in time order, and nothing is polled on a fixed tick. The shortest sweep among live jobs falls out of the heap without being chosen.

- **Lost and found keeps the work.** When a withheld job stalls past its grace time, the daemon appends its ticket (identity, signum, pid, declared, peak, measured, times, reason) to `hst/lnf.log` as a sealed block and tells the client, with the log's path. The client keeps its precalc and says so; the daemon then appends a sealed note for that identity. The job can be resumed and not recomputed.

The seal. The seal is `obsignatio_seal`, BLAKE3 keyed at the file level (`OBSIGNATIO_LEVEL_FILE`), and it runs on the host. The daemon still makes no CUDA call and holds no device context.

- **The history file** is its records, 48 bytes each (the 32-byte signum, then the peak and the duration, 8 bytes each), followed by a 32-byte seal over them. The save builds the whole file, seals it, writes it whole as `hst/tail.log` and renames it over `hst/head.log`. Each failure on the way (the allocation, the seal, the open, the write, the close, the rename) is printed on stderr. The call sites still don't branch on the save's result, but a failed save is never silent.
- **The load** refuses a history shorter than the seal, one whose length less the seal is not a whole number of records (a short tail, or a file with no seal), and one whose seal does not hold. It names the file and the daemon exits. A missing history is a fresh start.
- **A ticket** is one block of `hst/lnf.log`: its text, starting `identity <16 hex digits>`, then a last line `seal <64 hex digits>` sealing every byte of the block above it. A note, such as "precalc saved", is appended as its own sealed block that starts `identity <16 hex digits>` and then the note text. It goes in only when the log holds a ticket of that identity whose seal holds. A job whose ticket's seal is broken takes no note, and the precalc saved is not answered.

That was the gap before this date: the history was raw records and the ticket plain text. A damaged history of whole records read without complaint, a short tail was dropped silently, and a failed save went unreported.

The daemon opens its ledger and loads its history before it makes any endpoint: the pipe on Windows, the lock and the socket on Linux. A refused history ends the daemon before any client can reach it. A daemon that loses the race to be the only one has only read the history, which is safe, because a save swaps the whole file in at once. The unsealed 192-byte history from before the seal was moved aside as `history.unsealed` in the state directory instead of deleted.

12.7 THE DRIVER SUBMITS (24 SEPTEMBER)

A program's driver runs every job through tessera, one job a part. Its signum, its declaration, its parts and its runs are the program's, in the cell workbook (on_the_engine.md). Its times are 2 s holding, 20 ms sweep and 5 s idle.

- **The signum** is the BLAKE3 hash of the part's name, a NUL, and the whole effective request; a changed setting is a new signum.
- **The declaration** is the largest sample's lattice in 16-bit lanes. For `--ingest` it comes from the source's description (`engine_source_lanes`, which reads no voxel); for every other part, from the `.kcr` head.
- **The times** are 2 s holding, 20 ms sweep and 5 s idle.
- **The daemon** is `tessera_daemon` beside the driver, which the driver starts when none answers.
- **A part fails** if its job is not taken, or is withheld and lost; `--override` admits a withheld job on its declaration.

Measured on 44b6_0113de3b, in a scratch state directory:

run	declared and granted	peak measured
`--ingest`	838,860,800	5,091,037,184
`--run kcr-prove`	838,860,800	3,958,566,912
`--run kcr-prove` again	838,860,800	3,962,761,216

The history then contained two records and the seal (128 bytes), and the daemon ended once idle. A second run on the real state gave the same declarations and ingest peak, with the two prove peaks in the other order.

Two findings, both measured:

- **The kept peak is not a constant.** Identical requests measured peaks 4,194,304 bytes (2^22) apart. The peak is the largest of samples taken every 20 ms: a measurement at that resolution. The rule above is qualified to say so. Whether the sampling moves it has not been tested.
- **A job reserves its declaration instead of its kept peak.** The second prove declared less than its kept peak and was admitted on its declaration. Until its sweeps grew it, its reservation stood 3,119,706,112 bytes below what it went on to use. That is Doug's rule: reserve what the job asks for, grow and warn when it takes more.
  - The same day, the working tree's code was changed, uncommitted and not yet ruled on, to reserve the larger of the declaration and the kept peak (`tessera_ledger_wants`). With it, two proves each reserved 3,962,761,216 bytes.

12.8 THE SIMS SUBMIT (24 SEPTEMBER)

Every sim that uses the device is one job (`src/sims/cu/sim_job.cu`).

- **The signum** is the BLAKE3 hash of the sim's name and each argument, each ended by a NUL.

- **The declaration** is the bytes of the sim's first allocation, declared before it makes it.
- **The daemon** is `$TESSERA_DAEMON`, or `tessera_daemon` beside the sim, and `run.sh` builds it there.
- **The times** are the driver's, and `TESSERA_OVERRIDE=1` overrides.
- **The checks:** the job's admission and its release are two of the sim's checks. A sim that is not admitted fails.

Measured in a scratch state directory:

sim	declared	peak measured	checks
`period_power`	262,144	145,915,904	601, 0 failed
`nbody_lattice`	26,542,080	175,276,032	11, 0 failed
`noise_floor`	9,142,272	187,858,944	13, 0 failed
`root_universal`	339,510	152,207,360	1,270, 0 failed
`fixed_pattern`	141,056	145,915,904	12, 0 failed
`classify_reject_recover`	118,016	145,915,904	9, 0 failed
`chaitin_omega 16`, on the engine	30,256	286,425,088	16, 0 failed

- **The history** was then seven records and the seal (368 bytes).
- **The host-only sims:** `ask_state` and `ka_psi` run on the host and submit no job.
- **Every peak is above its declaration.** The smallest peaks sit near 146 MB whatever was declared. The peak is the process's whole dedicated memory, its CUDA context included, and the context is likely most of that; not measured apart.
- **Two peaks differ from the earlier runs by 2,097,152 bytes (`2^21`):** `nbody_lattice` and `fixed_pattern`. That is the same finding as the driver's.
- **The engine DLL:** no program in the repo loads it, and no other caller is left to submit.

12.9 LINUX (24 SEPTEMBER, NOT RERUN HERE)

- **The build:** Linux built under WSL 2 (gcc 13.3.0, CUDA 13.3), and its suite passes with 0 warnings. The fixes it needed: `PATH_MAX` under strict C11, the noinline helpers under gcc, `_GNU_SOURCE`, the timer thread's missing return, and the Windows-only strings.
- **The measure refuses WSL:** WSL runs the device through the Windows driver, and its NVML read a process as 0 bytes before and after it allocated 256 MiB. So no pid can be measured there. The measure refuses to open on a paravirtual device (`tessera_measure_paravirtual`), and the daemon refuses to run. The measure test and the job test both check exactly that refusal.
- **Socket activation:** the systemd user units `tessera@.socket` and `tessera@.service` are in `engine/daemon/service`. Under WSL, a client's connection started the daemon, and the measure refusal then ended it. systemd restarted it until its own start limit stopped it.

- **Docker:** a static client with no CUDA (`engine/daemon/test/tessera_socket_probe.c`) ran in a container under Docker Engine 29.1.3 inside the WSL distro. The host's socket was bind-mounted and named by `TESSERA_RUNTIME`. The client's submit reached the host's socket-activated daemon, which the WSL refusal then ended, and the submit was refused.
- **Not run** (this machine has no native NVIDIA Linux driver):
  - a native Linux host, where NVML measures a pid;
  - a daemon kept running on the activated socket;
  - a job admitted through the docker mount.

13

THE SEAL: OBSIGNATIO

Purpose: The theory of the engine's integrity layer: the dimensional Merkle DAG that seals every crystal, set and file, what it proves, what it costs, and where it stops.

Scope: `engine/base/obsignatio` (the BLAKE3 port and the level folds), the `.kcr` seal section, and the verify in `engine.cu`. The rows are M13 and A10 in engine_table.md. Statuses follow README.md.

13.1 WHY A DAG AND NOT A CRC

A CRC-64 is linear over GF(2): crc(a ⊕ b) = crc(a) ⊕ crc(b) ⊕ crc(0). So anyone can craft an edit that leaves it unchanged, it says nothing about where a fault sits, and two CRCs do not compose into a commitment on both. It still misses a random corruption only 2^{-64} of the time, and it catches every burst of 64 bits or fewer; its failure is structure instead of size.

A k-bit check of any kind misses a random corruption with probability 2^{-k}, whatever the function. A check smaller than 64 bits is therefore never stronger than CRC-64 against random flips, and no finite check is infinite. The seal takes k = 256 at every node:

- a random corruption passes one node with probability 2^{-256};
- a deliberate collision costs about 2^{128} work.

The DAG adds structure. It is nonlinear, and no edit can be steered past it. A mismatch walks down to its place. Every subtree seals on its own, in parallel, and the parents fold them exactly.

13.2 THE NODE FUNCTION

Every node is BLAKE3 in keyed mode, under a key derived once for its level (derive-key per level, dated context strings):

K_ℓ = BLAKE3-derive-key-context("obsignatio aeterna 2026-09-23 " ‖ ℓ), $H_\ell(m)$ = BLAKE3-keyed(K_ℓ, m)

The levels ℓ are row, plane, volume, lanes, chunk, stream, side stored, side inflated, side, members, sample, set, file and universal. Two nodes of different levels can only agree through a collision between two keyed BLAKE3 instances: domain separation is BLAKE3's own. A later change of format takes a new date, and old and new roots never meet.

BLAKE3 is itself a Merkle tree over 1 KiB chunks: its left subtree always holds the largest power of two of chunks that is smaller than the whole. On the device, pairing the nodes of each level and carrying the odd one up gives exactly that tree:

- n = 3 folds $(c_0c_1)c_2$;
- n = 5 folds $((c_0c_1)(c_2c_3))c_4$;
- n = 7 folds $((c_0c_1)(c_2c_3))((c_4c_5)c_6)$.

So the level-parallel device fold and the host's stack fold are one function, proved equal on every length from 0 to 102,400 bytes.

13.3 THE CRYSTAL'S DAG

For a sample I of extent (T, Z, Y, X), lanes little-endian u16 in key order:

ρ(t, z, y) = H_row(I[t, z, y, 0 … X − 1]) π(t, z) = H_plane(ρ(t, z, 0) ‖ … ‖ ρ(t, z, Y − 1)) υ(t) = H_volume(π(t, 0) ‖ … ‖ π(t, Z − 1)) Λ = H_lanes(υ(0) ‖ … ‖ υ(T − 1))

The stored stream: chunk c holds the ℓ_c bits from offset o_c to o_{c+1} (the last chunk ends at the stream's bit count B). Each chunk is repacked to start at bit 0, least significant bit first, as the coder writes them:

κ(c) = H_chunk(LE64(ℓ_c) ‖ bits_c), Σ = H_stream(LE64(o_0 … o_{C − 1}) ‖ LE64(B) ‖ κ(0) ‖ … ‖ κ(C − 1))

The source's other bytes (the DICOM headers) are stored deflated. The side root seals both forms (both):

σ_s = H_side stored(deflated bytes), σ_i = H_side inflated(inflated bytes), σ = H_side(σ_s ‖ σ_i)

The member tables and names: μ = H_members(the six tables as LE64 words ‖ the names).

The sample's root seals every section (every section):

Ω = H_sample(LE64(extent, C, B, lane offset, members, side bytes, deflated bytes, name bytes, lane nodes) ‖ Λ ‖ Σ ‖ σ ‖ μ)

The set's root seals every sample's in the order named: Θ = H_set(Ω_1 ‖ … ‖ Ω_n).

Every node is stored in the `.kcr` ("every floor of the tower"): the six roots, the TZY + TZ + T + 1 lane nodes, and the C chunk leaves. They sit right after the head. Both placements, after the head and at the end, were measured on the knee: the same 25,218,496 bytes, and a prove median of 76 ms each over 15 alternating runs. The seal is made from the source before the stream is written. Writing it last gains nothing. Written first, it arrives in one forward read with the head.

13.4 THE WITNESS

"we ingest seal and then witness pixel:pixel 1:1 once. then we have our hash and we know our input is verified and witnessed." Ingest seals the source as read, writes the crystal, reads it back and proves every node. Then it witnesses the crystal against the source twice (keep both):

- the decoded voxels against the source lanes still on the device;
- a second, independent read of the source against the rebuilt volume, pixel for pixel.

The second read is the only step that catches a flip in the first read, which would otherwise be sealed as the truth. After the witness the root stands for the input: every later check is hash against hash, and no source is needed again.

13.5 WHAT IT PROVES

- **Locality.** A change at lane (t, z, y, x) changes exactly ρ(t, z, y), π(t, z), υ(t), Λ, Ω and Θ, and no other node, except with probability 2^ − 256 per node. A change inside chunk c's bits changes κ(c), Σ, Ω and Θ, and no other node. A mismatch therefore walks down to its row or chunk. **Proved:**
  - 9,234 single-lane flips of a 2 × 3 × 3 × 513 volume each change exactly their four path nodes;
  - 57,216 single-bit flips of a stream (every bit, padding included) each change exactly the leaf that holds them, or nothing when they sit outside every message.
- **End to end.** The lane nodes are taken from the source's lanes at ingest and compared against the lanes decoded from the file. A decoder fault shows as a row as well as a flip on disk.
- **Order of the check.** The stored bytes are checked first (κ, Σ, σ_s, μ). A flip on disk is named before anything decodes corrupted data. Then the side bytes inflate (σ_i, σ), then the file decodes (ρ … Λ), then Ω.
- **Tampering, on a real crystal.** Single bits were flipped in copies of an RSNA knee crystal (1 × 24 × 640 × 640, 2,400 chunks, 15,386 lane nodes). Every case refused the crystal and named the place:

flipped	named as
the extent word	the head's shape disagrees with the seal's node counts
the stored sample root	one root
row node 5,120	row t 0, z 8, y 0
plane node 0	one node above the rows
chunk leaf 1,200	chunk 1,200
a limb in the stream	chunk 1,234 and the stream root, before any decode
the deflated side bytes	the side's stored leaf
a member name	the members root

The untouched copy proved clean.

13.6 WHAT IT COSTS

- **Nodes.** 32 bytes each: TZY + TZ + T + 1 lane nodes, C chunk leaves and 6 roots. The row level dominates: 32 bytes per row of 2X bytes, which is 16/X of the raw bytes. At X = 960 that is 1.7% of raw.

- **Measured on the knee** (re-ingested sealed):
  - 1 × 34 × 960 × 960: 25,218,496 bytes against 23,927,904 under CRC-64. That is +1,290,592 bytes: 32,676 lane nodes and 7,650 chunk leaves, less the CRC words. 5.4% of the crystal, 2.1% of raw.
  - The signed series, 1 × 24 × 640 × 640: 11,319,416 bytes against 10,750,104.

13.7 WHERE IT STOPS

- **The tower spans all four axes**, and no single plane decodes on its own. A check without a full decode is possible only on the stored bytes: the chunk leaves, the stream root, the side's stored leaf and the members root. A decoded row can be checked only after a whole decode, though a mismatch still names its row.
- **An unkeyed root seals against accident instead of against an author.** Whoever can rewrite a file can recompute its roots all the way up. Against a deliberate tamperer the universal root needs a trust anchor where they cannot write:
  - a signature over it, with the public key compiled in;
  - or a root published out of band.

  Doug's plan fuses the check into the engine's one swept cycle and injects the engine's own hash at compile time. The anchor is still open.

13.8 THE SHARED SHAPE

The lane DAG folds x, then y, then z, then t: the same lattice, in the same order, that the tower's floors lift and that the moments' DFS prefix sums (A3). The residual (A1) is a fold of one binomial step along each axis. The component tree (A2) is a fold of the level order. C (A4) is a fold of one count under a shift. The seal is a fold of one keyed hash. Each is one lattice read once, with a different combining operator: a sum, a convolution, a level merge, an integer lift or a hash. Only the operator carries the meaning.

Of these operators, only the seal's is not associative: $H(H(a \parallel b) \parallel c) \neq H(a \parallel H(b \parallel c))$. Its tree shape is therefore fixed and written down, where a sum could be regrouped freely. That fixed shape makes a mismatch locatable (observation).

13.9 WHAT THE ENGINE SHOWS ABOUT THE WIRE AND THE WITNESS

Doug's nine posits on the wire and the witness are in wants.md, numbered as he gave them. What the engine shows that bears on them follows (a to g, checked). Derived unless marked.

- **(a) The seal detects and locates a change; it does not restore the value.** Locality is proved ("What it proves"): a mismatch walks down to its row or chunk.
  - Derived: the seal can confirm a guessed repair. With t unknown flips in a segment of m bits, a search tries C(m, t) candidates. The right one proves, and a wrong one passes

with probability 2^ − 256. One flip in a row of 960 lanes (15,360 bits) is 15,360 hashes. That is a search, and the seal holds no error-correcting code.

  - "All crystals fail" (point 1), read against Locality: a change fails its own path of nodes up to the set root Θ. Every other sample's root Ω still proves.

- **(b) Every crystal is legal, and T carries no redundancy** ("Redundancy" in two_crystals.md). Rebuilding needs the source or the whole crystal, and it fails closed: every tampered case in the table above was stopped and named.

- **(c) A CRC with the seal as a locator.**

  - Erasure: a CRC of degree r with g(0) = 1 recovers any burst of at most r bits whose positions are known. The burst e at offset i leaves the syndrome x^i·e(x) mod g. g(0) = 1 makes x a unit mod g, and no nonzero e of degree below r is a multiple of g: the syndrome names e. At r = 64, a burst of up to 64 bits.
  - The seal locates to a row or a chunk instead of to bits. Inside the segment the burst's place is unknown, and that is burst correction: a linear code that corrects every burst of length b needs r ≥ 2b check bits (Reiger 1960 [44]; Peterson and Weldon 1972 [41]). A CRC-64 corrects bursts of at most 32 bits, and only with a generator chosen for it.
  - "We have the crc" (point 2) holds outside the `.kcr`: the `.bapx` body table carries a CRC-64, and the entropy history carries `payload_crc` and `cloud_crc`. The `.kcr`'s CRC words were replaced by the seal ("What it costs"). Adding one back is a format change, Doug's call.

- **(d) The set root is a joint function of every crystal's root.** It changes on any change, except with probability 2^ − 256 per node. This is classical binding, and it involves no quantum entanglement (points 3 and 4). Θ over a set is built. The universal root is ruled and still unbuilt (below).

- **(e) The floor as amplitude and the knf as phase** (points 6 and 7).

  - An exact pair is an exact complex amplitude when the pair is (re, im), as Gaussian rationals. A (magnitude, phase) pair is exact only with the phase an index k of a root of unity, ζ_N^k in ℤ[ζ_N].
  - The knf as phase is untested, and it is carried in wants.md.
  - An exact classical register holds a qubit's state vector exactly. `ask_state` (M21) carries a qubit exactly now. n qubits cost 2^n amplitudes, and exact classical registers give no Bell violation between separated parts. "Classic qubit" fits the reading of an exact simulation with no rounding.

- **(f) Witnessing the wire** (point 8). A hash reads the value it seals. An unknown quantum state cannot be copied (Wootters and Zurek 1982 [61]), and reading one disturbs it. A Merkle-witnessed wire is classical by construction.

- **(g) "Knows all noise from non noise"** (point 9). Exact equality catches any departure from a sealed state. The second, independent read of "The witness" guards the value before

sealing. Past that, the seal cannot say whether the sealed value was the true signal: shot noise in the source is sealed as the truth.

- **The words of point 2.**
  - The elevator clock: "The clock is the elevator", noise_sieve_tower.md §9 and §17. Every floor of the tower sits at a known offset.
  - The floor identity: the identity line, one null draw a floor (§18).
  - The locale: rules derived once at the top reach every locale (§1).
  - "Rebuildable", derived: from the whole crystal, yes. T^{-1} is exact, and the seal proves the result. From a part, only the samples whose cone (reach L + 2 along a line) lies inside it.
  - "The spine" has no definition in the engine, and it is carried in wants.md with Doug's question of point 5.

13.10 THE UNIVERSAL ROOT (RULED, NOT BUILT)

"ingest->seal kcr shards until we accumulate the grains to a lattice and the same goes for the output and any scheduling, we coalesce a universal hash for the state of the program". The plan:

- each `.kcr` is a shard sealed at ingest, and the shard roots fold into the set's lattice root Θ;
- outputs, schedules and keys seal the same way;
- H_universal folds every file the engine touches, inputs read and outputs written, plus a hash of the engine itself ("our specific engine signature gets hashed and stops the program if its different"; "we do compile time injection for the engine hash").

The engine is deterministic. Rerunning the same input on the same engine must give the same universal root: the seal doubles as a reproducibility proof, and a user who runs their own data seals it themselves.

14

THE BUILD PLAN: THE ENGINE

Purpose: The order the engine's wants in engine_table.md are built in, and every ruling on them, dated. Each build takes the next item in order, even when another looks nearer. Built in order, one item at a time; an item is done when its program runs on the device and its row below says so.

Two tables, two concerns: keep the cell table and the engine table separate; each is optimized against the other only through its interface section, never merged. **The engine is optimized for no scale:** no size, spacing, order, window or width is ever written into engine/. Each comes from the request or the data. The engine table's scale audit lists what is still in; this has been put in and ripped out repeatedly, and it is dangerous.

Scope: the engine's items of the build plan kept and item 38. The item numbers are the plan's own, and a number missing here is another workbook's: the cell program's items are in the cell workbook's build_plan.md, and the crystal's growth in the compression workbook's build_plan.md.

Statuses: `proved`, `measured`, `built`, `theory`, `refuted`, `not so`. "Not built" means nothing of it exists yet.

14.1 DONE BEFORE THIS PLAN

item	what it is	status
record programs	`keymath_record_imprint` proves every register's width, `key_schedule_record_lay` lays the register file out, `cycle_record_run` sweeps one launch	built

14.2 PHASE A: THE VOCABULARY, ONCE, USED BY EVERYTHING BELOW

Doug's rule for everything: truthy and falsy vector magnitudes, by which the machine steers itself iteratively. Every verdict is a field, zero false and nonzero true; AND is PRODUCT, OR is SUM, NOT is $1 - |\text{COMPARE}(t, o)|$, and a value is gated by PRODUCT(value, t). No thresholds and no host decisions: a pass writes its verdicts as records, and the next pass reads them.

item	what it is	status
A1	tuple lanes: a lane reads fields from 2 or 3 bodies through an index array (a pair, or a parent and two children)	built: the print swept through a reversed index lands 0 of 81,807 prints away from their body, 550 us; direct CRC-64 still `590f3a2c386647e3`; host port 0 of 81,807 differ
A2	absolute value and compare (−1, 0 or 1)	built: over K_zy of 81,807 bodies (22,364 below zero, 34,872 at zero, 24,571 above), 258 us; 0 break |K| against K, 0 disagree with the print's nonzero band, 0 differ from the host
A3	records are vector magnitudes: the imprint reports each output's field (offset, bits), and any sweep's records are the next program's input, read with FIELD_SIGNED	built: the print reports seven fields at 0:8 8:8 … 48:8; the pair program, its host proof and A2's check all read the print through the reported fields and hold (self-weight 1,275 on all 81,807, 0 differ from the host)
A4	each member of a lane reads its own array (the bodies, the last sweep's records, the tracker's per-body values such as the climbed lag). A pass reads its own verdicts, and the machine steers itself by sweeping again	built: a pass reads each body and its own print side by side with no index, recomputes the mass band, and writes 1 − |COMPARE| against the print's band: truthy on 81,807 of 81,807, 157 us, 0 differ from the host; two members naming the same array share one device copy

14.3 PHASE C: THE RESIDUAL AS UNIT SWEEPS

item	row	what it is	status
C5	3	the binomial kept as [1 1]^n unit sweeps with B_ narrow shared, proved lane for lane against the current key	built and proved (`unit_sweep` module; `--unit-sweep`, `--unit-sweep-prove`; `EngineResidualRequest.unit_sweep`). Centred [1 2 1] steps keep the key's mirror boundary exactly; one block a line, all of an axis's steps in shared memory, only the live limbs worked. 0 lanes differ on 500 frames over 5 samples. 14.0 ms a frame against the key's 29.0 (2×); edges identical. The default is still the key (Waiting on Doug 15)

14.4 PHASE D: PROVE AND MEASURE

D1

WHAT IT IS the host port of the record machine (`cycle.c`, orior's exact integer); every program's prints equal the device's bit for bit

STATUS built and run (`engine_record_host`): the print 0 of 81,807 differ; A1, A2, A4, the pair, B2's check and prediction and B4's triples all 0 differ from the device

D2

WHAT IT IS why `flatten` is slow: 138 to 170 s on 44b6_0113de3b, against 8.5 s for the same 81,807 bodies on the tracker's per-frame path

STATUS cause found (run on a scratch copy of the set, and `flattened.kcr` did not touch the data): the slab. `flatten` sizes its residual slab to half the free device memory (10 frames here, about 1.5 GB of residual limbs beside all 100 frames of lanes). With it, every stage runs 10 to 20× slower: codes 17.5 s, contract 44.0 s, label 9.2 s, the residual 25.7 s, 107.0 s in all. With a one-frame slab: codes 1.6 s, contract 2.1 s, label 0.6 s, the residual 3.0 s, 9.0 s in all. `flattened.kcr` is byte-identical either way. The grade (the exact-key contract on each sample's frame 0) is 3.4 s at the big slab and 70 ms at one frame, now timed as its own stage. The code still sizes the slab as before (Waiting on Doug 16)

14.5 MEASURED ALONG THE WAY: WHAT A REGISTER TIER COSTS

`-Xptxas -v`, sm_86. The record kernel's whole frame is its per-lane arrays (`file[WIDE]`, `sign[256]`, the ladder's `reached[WIDE + 2]`), all in local memory, 0 spills. Local memory is VM the driver reserves for every resident thread: the RTX 3070 holds 46 × 1,536 = 70,656.

kernel	frame a thread	VM reserved
record <64>, after A1	816 B	57.7 MB
record <256>, after A1	2,352 B	166 MB
linear <16> (HEAD)	336 B	23.7 MB
linear <64> (HEAD)	1,104 B	78 MB
linear <256> (HEAD)	4,176 B	295 MB

- The tiers 16/64/256 are not measured. `ENGINE_RECORD_LIMBS_MOST 256u` copies that tier; `CYCLE_RECORD_BLOCKS_MOST 4096ull` is invented. Neither is measured yet.
- The ladder's scratch `reached[WIDE + 2]` costs as much as the register file itself.
- `key_schedule_record_lay` gives every step its own register for the whole run and never reuses a dead one. The file grows with every step.
- The first sweep of a run pays about 15 ms that later sweeps do not (15,221 us, then 550 us).

14.6 WAITING ON DOUG

1. Answered: the unit sweeps become the default residual (tracker and `flatten`); the key stays for the proof. Done: `ENGINE_RESIDUAL_BY_UNIT_SWEEP` is 0, the zeroed default. `--residual-key` selects the key, and `--unit-sweep-prove` runs both and compares them. On 44b6_0113de3b by default: the unit sweeps take 1.41 s over 100 frames and the key 0; edges 38/12, unchanged.
2. Answered: `flatten` runs one frame at a time, as the tracker does. Done: `flatten_set` calls `engine_residual` a frame at a time (its own key, slab, atoms and device lanes are gone), and it takes the unit sweeps too. On 6bba_48816121 (scratch set): 7.7 s, the residual 1.5

s of it; components proved on 100 of 100 frames, 0 bits lost, 0 faces differ on the graded frame; `flattened.kcr` byte-identical (`cmp`) to the one the old slab wrote. 19b. Answered: the one microseconds clock is a static inline in engine_config.h; the base modules' copies and engine.cu's go. Done: `engine_clock_microseconds` lives in engine_config.h; max_tree, box_history, climb_machine and engine.cu no longer keep their own. 20b. Done: `climb_machine_extents(machine, const ClimbMachineExtentsRequest *)`; the review edits, the successor helper and the cast comments rebuilt clean and every proof held (C1 0 of 35,755; the box's three proofs 0; C2 0 of 11,780,208 and 0 of 9,021,600; the unit sweep 100 frames, 0 lanes; edges 38/12).

3. Answered: `engine_record_sweep` and its unchecked `*sweep_microseconds` stay as they are.

4. Answered: a cast across width or sign carries a one-line `//` comment saying what it widens, narrows or re-signs and why that is safe there. It is the only exception to the no-comments rule. Redundant casts are still removed.

5. Answered: fold both. The microseconds clocks become `engine_clock_microseconds`, and the key's successor builder becomes one helper in score_sample.cu.

6. Answered: reshape `climb_machine_extents` to take one const request struct.

7. Ruled: stateful graceful errors are the only acceptable error. Kinds: request, resource (a failed CUDA call or allocation) and logic (unforgivable; the result is withheld). Each request carries `EngineError *error`, and the engine keeps its own record too (first write wins). The record holds kind, module, site (line), status, execaddr, evacaddr, imagebase and up to 16 return-address frames. Builds emit a .pdb (`-Z7`, `-DEBUG`), and execaddr − imagebase steps straight to the source line. Wired so far, each built and checked on 44b6 (38/0/12, proved on 100 frames, 0 lanes differ):

- engine_frame_bodies; engine_residual (a BOTH_PROVED lane disagreement is a logic error);
- max_tree (every entry, slide and overlap included), flatten, decimal_double;
- unit_sweep, cycle (module 5), keymath (6), key_schedule (7), residual_program (8), grow (9);
- krep (10): its file layer (input, history and bodies read/write) takes an error. _IO errors carry errno; bad contents (header, bounds, CRC, trailing bytes) are logic errors.
- The engine calls that go through krep: iapx head/load/prove, ingest, entropy set/cloud/history read, and bodies read/write. The two set requests carry `error`, and the others take a trailing `EngineError *`.
- engine_entropy_set's keep path and schedule's per-sample skip each use a local error, because those reads may miss. The failure path was checked with a missing sample: module 10, site 178, status 2.

- compression (11) and tower (12): requests carry error; TOOK on every CUDA call. A malformed offset table or chunk count read from a file is a logic error. A coefficient overflow on lift is a request error; a rebuilt value outside 16 bits on lower is a logic error. entry_iapx_encode/decode take the error. Checked: 44b6 passed, and kcr-prove proves 1 of 1.
- For RSNA knee, the answers:
  - zip becomes its own module, engine/base/representation/zip/, with the ZIP64 walk moved out of npy.c;
  - dicom becomes engine/base/representation/dicom/;
  - one sample is one SeriesUID, the single */<SeriesUID>/ prefix in the zip;
  - "do not bound or truncate the data": DICOM header bytes go inside the .kcr as a new section. The .kcr framing is still open with Doug.
- entropy_history (13): windows(extent, error); the project request carries error. Broken parity is a logic error. Checked on 44b6.
- Doug's rulings on .kcr v2:
  - sections in order of how often they're read (the file is read FIFO): head and geometry, then pixel offsets and stream, then deflated DICOM header bytes (which engine_iapx_load hands back), then the member list;
  - CRC-64 per part, with no zip CRC-32 rejoin.
- Doug accepted the counter-proposal put to him:
  - zarr is untouched; dicom is a source kind reading through the zip module;
  - EngineArrayRead gains an optional EngineSideBytes *side;
  - the zip archive is kept open across samples, keyed by path and indexed by series prefix;
  - equal slice keys break ties exactly by InstanceNumber, then SOP UID; nothing is refused;
  - no bounding: nothing is refused for rescale, signed pixels or transfer syntax (signed pixels become stored + 2^15, pending Doug on the mechanism);
  - flatten takes smooth/background orders per sample, and one set holds every geometry.
- NO ROUNDING (Doug, emphatically). The old zebrafish orders {2,34,34}/{6,96,96} came from chosen scales and n = 4(σ/v)² rounded to even: a debt, never to be reused.
- Ruled:
  - Parity (a): refuse an odd background order per axis (request error). An odd smooth order is kept, with an exact half-voxel offset recorded as a residual field. The residual is 2^G·B_s x − B_b B_s x, and the centers differ by b/2. The guards to change are residual.cu, keymath.cu and unit_sweep.cu (narrow_bits at :259).
  - Count rule: order = period (n = P, both in voxel steps).

  - The per-axis coherence reading on volumes is new engine math, still to build (bench_recover_period is the 1-D model).

- Converter work order:

  1. the zip module (ZIP64 walk moves out of npy.c; module 14);
  2. dicom;
  3. EngineSideBytes on EngineArrayRead;
  4. .kcr v2;
  5. one series end to end, then ping RSNA (rsna-knee-abnormality-7d).

- Done: the zip module (14), dicom (16) and the engine wiring. Detection is PK + the member folder's first file carrying DICM. A file source maps sample to member, and zip_held_release runs at the end of ingest. Test series 1.2.826…903947 (34×960×960 u16) was ingested and proved from the file, CRC-64 45aea19b28ddd8b9, 38.1% of raw. An independent Python read agreed on the shape, the one prefix and 34 distinct keys. The key runs opposite to InstanceNumber. The set dir needed the 8.3 short path because set + two UIDs exceeds MAX_PATH 260; the engine's opens need a long-path form (to do).

- The RSNA census: all 819,635 members are Explicit VR LE, 16-bit allocated, one sample per pixel, native. 201,276 slices are signed.

- Ruled:

  - signed pixels: sign-extend from HighBit to 16 bits, then add 2^15 into the u16 lane. Read BitsStored and HighBit; the .kcr records the offset. A slice whose raw words are not exactly the sign-extended form keeps its raw pixel bytes in the side bytes.
  - Deflate encoder: a full LZ77 + dynamic Huffman module, proved by inflate round-trip.

- Doug named the information crystal `.kcr` (Kolmogorov crystal).

  - "do not write version bridges": .kcr is the only crystal format. Old .iapx crystals are not read; sets are re-ingested.
  - The crystal's krep kind is "KCR\0", version 1.
  - Layout, read FIFO:

    1. the head (13 words) with its CRC-64;
    2. the offsets and limbs with their CRC-64;
    3. the deflated side bytes with a CRC-64 of the inflated bytes;
    4. the member tables and names with their CRC-64;
    5. EOF.

  - engine_iapx_load takes an optional EngineSideBytes *side (the headers are handed back, and lane_offset rides in it); engine_side_release frees it.
  - Names for the other apx files (flattened, OAPX history, BAPX bodies, IMP key) are still open with Doug. .kfc/.kst/.kgr/.ksd were offered; .ans is for the answer file.

  - The code identifiers still say iapx (engine_iapx_*, iapx_path); rename them to crystal.

- Deflate encoder module built (engine/base/representation/deflate). It tries every LZ77 match over the 32 KiB window with one-step lazy matching, package-merge length-limited codes, and picks the smallest of dynamic, fixed and stored. It round-trips 13 inputs through our inflate and zlib, landing within about 1% of zlib-9 on DICOM (no block splitting yet).

- Doug on output, answering "printf in the hot loop":

  - Output goes through our own simple string module, `scriptura`.
  - "zero print in any loop unless we raise our hand, else silently update your functionals returns, which get dumped into each print format and called on demand unless an error, so sorta like verbosity but never trap warn or error."
  - So: functionals fill result structs; one print format per result renders it through the string module on demand; errors and warnings always print.

- "we should be able to output iapx as any valid format, we compress it we can decompress it too." The export runs .kcr → any source format (dicom zip byte-exact from the side bytes, npy, nifti, tiff, zarr...). Build it after v2.

Still to wire:

- in the engine: tower, compression, entropy_history, stack, golden_bands, link_objects, run_cfg, run_log, cfg_json, relate_frames, double_fields, and cycle.c's host run;

- the entry_* ingest (source read, zarr and its codecs). Cell-specific modules (bodies, score_sample, answer_key, the measure/*) are due to move to cell_tracking first.

1. Ruled: CRC every schedule, cfg and key at imprint and verify right before each cycle; CRC every disk read and write, with a flush read-back. Size the buffers generously, by experiment, to match ingest and r/w throughput (Doug suspects a hardware queuing "snap io desert"). Not started.

2. Ruled: `decimal_double` never rounds. `decimal_double_expand` gives the exact leading bits of digits × 10^ex in an `OriorExactInteger` candidate, plus a binary exponent e2. It also reports `fits` (the candidate holds the whole value), `terminates` (the binary expansion ends; a tenth does not) and `needed_bits` (the width to grow to). Checked against Python fractions on 22 cases at 128 and 4096 bits: 1e2200 needs 5109 bits, and a 4096-bit build gets its exact top 4096 with fits 0.

3. Ruled ("split it, go"): the slide and the overlap are engine math and are back in max_tree, wired for errors. The slide's key voxels became generic probe voxels, and the engine returns each recorded level's probe partition. The present/alone tally left the engine for cell_tracking, and a slide count that disagrees with the tree's is now a logic error. The overlap keeps probes (points) and probe links (point pairs) as its only interface. The key-cell grading that goes with this item is the cell program's (the cell workbook's build plan, item 24).

4. Ruled, integrity: “the only acceptable crc is a merkle dag”. Every CRC-64 is replaced by BLAKE3 Merkle roots. The answers:

- root: the full 32 bytes;
- shape: dimensional, leaf = one x-row of LE u16 lanes (its own width, BLAKE3 chunks it), then y, z, t, sample, set;
- stored levels: “every floor of the tower”, and every level’s nodes are stored, rows included (about 6% of a knee crystal at 512 columns);
- scope: “every crc64, we need 100% idempotent determinism against cosmic flips” (head, side bytes, members, stream, parts, sets, schedule, keys, flattened, history, bodies);
- the universal root: “literally every file the engine touches plus one on itself” (inputs read, outputs written, and the engine’s own binary);
- placement: “Measure both, report”: the hash section after the head against a footer, write cost and one-plane check cost on the knee set;
- repair: fail closed and report the path (sample, t, z, y, row); no parity unless Doug asks.
- Module name (Latin, to avoid colliding with a user’s libblake3): “obsignatio aeterna, then you can use the signum signator nota notarius sigillum sigillator and comprobare officiale or variation”. The module is `obsignatio` (engine/base/representation/obsignatio, ENGINE_MODULE_OBSIGNATIO 17).
- Built: the BLAKE3 port. `obsignatio_signum` (host, any length, extended output) and `obsignatio_many` (device, many equal-length messages at a stride; chunk kernel, then level kernels pairing nodes with the odd one carried up, which is BLAKE3’s left-heavy tree). One `__host__ __device__` compression core serves both. Modes: hash, keyed, derive-key context and material.
- “the hashing function needs a full test/ suite for validation ... we just make it happen and prove it.” Suite at obsignatio/test (run.sh, obsignatio_test.cu, the official BLAKE3 test_vectors.json). All pass, 0 failures: host vectors 210 (131-byte extended and 32-byte, three modes); device vectors 1,260 (offsets 0/1, strides L/L+1, three messages, three modes); device context 1; every length 0..102,400 host against device in three modes, 307,203; every single-bit flip of every vector input, 1,807,226 (each differs from the original, all distinct); determinism 2; fail closed 20 (refusal, error kind and module, output untouched).
- DAG node format, ruled: each level is separated by BLAKE3 derive-key, one context string a level, of the form “obsignatio aeterna 2026-09-23 <level>“ (a later format change takes a new date); every node at a level is hashed in keyed mode under that level’s derived key. The sample root seals every section: H(head words ‖ lane-DAG root ‖ stored-stream root ‖ side-bytes root ‖ members root), where the stored stream has per-chunk leaves over the compressed limbs, where a flip on disk localizes before decoding.

- Mutation proof: five mutant ports (a block's last byte dropped, the host stack merged on the wrong count, the device carry zeroed, no ROOT flag, the chunk counter fixed at 0) all fail the suite, each where it should. The every-length and flip checks cannot see a core bug shared by host and device; the official vectors catch those.

- Built, the lane DAG: `obsignatio_level_context`/`obsignatio_level_key` (12 levels: row, plane, volume, lanes, chunk, stream, side, members, sample, set, file, universal) and `obsignatio_lanes` (rows → planes → volumes → lanes root, each level one `obsignatio_many` over the level below; nodes laid out rows, planes, volumes, root, `obsignatio_lanes_nodes` = tzy + tz + t + 1; OBSIGNATIO_EXTENT_AXES 4 in the header). Tests: level keys 16, lanes reference 7 shapes against a host build node by node, lanes locality 9,234 single-lane flips each changing exactly its row, plane, volume and the root, lanes closed 10. All 0 failed.

- CRC-64 sites to replace (crc.h, crc_key.h): krep, engine.cu, tower (key and schedule), entropy_history, flatten, score_sample, the driver's prints. Format-intrinsic checks stay (zip CRC-32, PNG, inflate/zarr).

- Ruled: a stored-stream chunk leaf hashes the chunk's exact bits repacked to start at bit 0 (LSB-first, as compression writes them), prefixed by its bit length; leaves are disjoint. The side root covers both: one leaf over the stored deflated bytes and one over the inflated bytes. `member_crc` is the zip entry's own CRC-32 (a zip field the byte-exact export needs) and stays. krep may not reach obsignatio: engine.cu computes and checks every root and krep only frames the node bytes.

- Doug on tamper protection (the Cython dependency he will have this compiled to): "the redundancy check happens in one swept cycle with every other process simultaneously". The caveat given: an unkeyed root can be recomputed by whoever can edit the files; tamper resistance needs a trust anchor for the universal root (an Ed25519 signature with the public key compiled in, or a published root). Open for when the universal root and the Cython build come; keep every hash level a batched device pass so it can fold into the swept cycle.

- The flow: "ingest->seal kcr shards until we accumulate the grains to a lattice and the same goes for the output and any scheduling, we coalesce a universal hash for the state of the program, so this works when users input their own data, they seal it themselves by running." Each .kcr is a shard sealed at ingest; shard roots fold into the set (lattice) root; outputs and schedules seal the same way; one universal root is the program's state. The engine is deterministic. Rerunning the same input on the same engine must give the same universal root (a reproducibility proof). Doug on its origin: a one-bit flip anywhere in the monolith already stops the program ("on ACCIDENT"), because every stage is exact and chained; the seal makes that stop name its place.

- Built: `obsignatio_bits` (chunk leaves: bit-range messages over a limb stream, LSB-first, each hashed as an 8-byte LE bit length then the bits repacked from bit 0; a check kernel refuses malformed offsets before any write; one thread a message runs the shared stack digest). The core reads message bytes through a source (plain bytes or a bit range), and the host `obsignatio_signum` and the device share `obsignatio_digest`. Tests: bits reference 32

(boundary lengths 0, 1, 7–9, 31–33, 63–65, a BLAKE3 chunk ±1, two chunks, first message at bit 5; hash and keyed), bits locality 57,216 (every bit of the stream, padding included), bits closed 11. Whole suite 0 failed.

- "our specific engine signature gets hashed and stops the program if its different". The engine hashes its own binary at start and refuses to run on a mismatch. Open: where the expected hash lives (it cannot sit inside the binary it hashes unless that placeholder is zeroed while hashing; or a sealed manifest beside it, which brings back the trust anchor question).
- "we do compile time injection for the engine hash". Open detail: the injected value is either the linked binary's hash with the injection placeholder zeroed (patched in after the link) or the hash of the engine's sources/objects computed by the build as a define.
- Found: the tower transform spans all four axes, and no plane decodes alone. A partial check without a full decode is possible only on the stored bytes (chunk leaves); checking decoded rows costs a full decode, though a mismatch still names its row. Matters for the placement measurement.
- Stage A: the .kcr seal. engine_config.h has EngineSignum (32 bytes) and EngineSeal (roots sample, stream, side, members, side stored, side inflated; lane nodes; chunk leaves); EngineStream.crc and EngineSideSection.bytes_crc are gone; EngineHistory.sample_crc became EngineSignum sample. Stage B: history payload/cloud, bodies, tower key/schedule CRCs.
- Stage A done: the .kcr seal built, the suite rerun with 14 levels (2,182,450 cases, 0 failed), tamper 9/9, the theory (obsignatio_seal.md) and the table's M13 and A10 written.
- Placement measured and ruled: both alike on the knee (25,218,496 bytes each, prove median 76 ms each over 15 alternating runs). Ruled: after the head. The at-end placement, KrepSealPlace and –seal-at-end are removed.
- Witness ruled. "we ingest seal and then witness pixel:pixel 1:1 once. then we have our hash and we know our input is verified and witnessed." Ingest witnesses twice (device voxels against the in-memory source, then a second independent source read pixel for pixel on the host); keep both.
- EngineSampleRecord.crystal_read added: a file that krep refuses structurally now prints as not read, not as "0 differ". Verified on the rebuilt engine: a re-ingest wrote a byte-identical crystal, prove 76 ms, tamper 0 cases wrong, the old at-end file refused as not read.

1. Ruled, device-job scheduling ("just a normal job scheduling function, super common for stuff like this"). Module name `tessera`. The answers:

- jobs are packed by memory, several at once while they fit (not one owner);
- the device share is a request field: the caller names the byte budget, the engine never chooses one (schedule.cu's free/3·2 is a chosen constant to remove, listed in the scale audit);

- one arbiter per host/device: “daemon or systemd on linux that self tears down/knocks up, single win pid, single docker hook”, to be deployed onto a SAN;
- clients talk to it over a local socket (named pipe on Windows, Unix domain socket on Linux);
- measurement: NVML per-process device memory by pid, “with periodic dead sweeps ... otherwise itll leak over time and lock down memory that is never used”;
- stale tickets “get dumped to lostandfound”;
- budget: “caller declares, we measure, they overbudget it could be for a reason we ask they can override, they underbudget we grow and report via warn so accounting isnt perturbed”;
- over-budget is asked at admission, “but dont throw it away, we have a complete precalc tossing would be a waste, it goes into holding before stalling and dumping to lostandfound”.
- The projects built on the engine (RSNA, CASMI) use pinned git dependencies; see dependent_projects.md.
- Over budget is judged against the job’s signum’s last measured peak: a job is keyed by the BLAKE3 signum of its request (the engine is deterministic); tessera keeps each signum’s measured peak; a never-seen job is admitted on its declaration and measured.
- Times are per job fields (the holding time before lost and found, and the sweep and idle times travel in the requests; the engine chooses none).
- First build: all of it (“all of it, use websearch”): the daemon and client over the local socket (Windows and Linux), the driver wired through it, the service files (systemd unit, Windows single instance, docker hook), and a tessera/test suite.
- Queue order: backfill with the head kept (a smaller job goes ahead only if the head’s admission is not delayed).
- Times: “what I want is the shortest time to change of anything to be the root because that is most likely event, in order”. Every deadline (each job’s sweep, each withheld job’s holding expiry, the idle teardown) goes into one min-heap keyed by time to change; the daemon waits on its socket with the root’s remaining time, handles the root, re-inserts, in order.
- Lost and found: on a stall the daemon tells the client, and the client writes its precalc there, sealed, beside the ticket, for it to be resumed.
- “see how some of our ingest operations are a tower themselves, and can get crammed into one prep cycle, that will save an enormous amount of time”. Ingest reads one lattice five times with different operators (tower lift, source lane seal, decode, rebuilt lane seal, voxel compare); fuse them into two sweeps. Measure the ingest’s stage times first.
- “you should fix any defect found”.

1. Ruled, the engine reorg: engine/base/ (never sees a body: lattice, representation, codecs, the tower, obsignatio, exact arithmetic, scriptura, tessera), engine/nbody/ (defined over n ≥ 1 bodies, n = 1 the base case; no single/ folder), engine/sims/ (programs composing them: run_cfg, cfg, tsv, nbody_program, the oracle). The test: does the module's request carry a body count. The engine README lays out the six steps ("yep"): 1 ingest (source → lanes), 2 seal (lanes → signa, the witness), 3 lift (lanes ↔ crystal: tower, compression, krep), 4 measure (lattice → fields: residual, moments, entropy), 5 partition (fields → bodies: max_tree, grow, bodies), 6 relate (bodies × frames → links). Each file names the step(s) it belongs to. Move by paste then delete; git index untouched.

- Then: the orior port. Doug, after the table was done: "have them read our engine, and discuss the move together, did we miss something? if so it gets added to the table and implemented before moving so as to not disrupt or disrupt them as little as possible, they do not know how to use this engine it is foreign to them completely it has evolved". Messaged "DMA plan reconciliation (2)" with the reading list (engine/README.md, engine_table.md, obsignatio_seal.md, tessera_scheduler.md) and three asks: their use, the gaps, where it lands. Nothing moves into their tree until agreed; every gap goes into the table and is built here first.

- Reorg done 66 modules moved (copy, diff -r, then delete), build scripts and docs repointed, engine/README.md gained the six steps and every file's step(s) (in the README, since files carry no comments beyond SPDX and cast lines). Every defect in the "Found" lines is to be fixed, not just listed.

- Still to do: stage B (history payload/cloud, bodies, tower key/schedule crc_key.h, flattened, the driver's fingerprint print), then the universal root and its file, the engine self-hash, and the anchor.

- Found on the way: a prove where no sample held still prints "the set's root, sealing every sample's in the order named" over zero samples; the root is only meaningful when every sample held.

- Found in the CASMI project: host cycle.c is compiled at the library's 16-limb width and refuses a 28-limb step that the device accepts (within ENGINE_RECORD_LIMBS_MOST); host and device widths disagree.

1. Built: engine/sims ("engine simulation files, just like in orior engine"; categories noise-floor, synthetic n-body lattice, period, orior art). `bash src/sims/run.sh <sim>`; shared sim.h (tally, keyed draws, exact rationals via the exact integer) and sim_camera.h (camera law: Binomial($4S$,½) − S shot, Binomial($4r^2$,½) − $2r^2$ read, fixed pattern, gain, moving/dividing ellipsoids with truth). Results: nbody_lattice 9/0; noise_floor 11/0 (BM recovers every seeded stream, planes 0–8 random, truth PTC slope 2.0046 vs 2, data-only routes biased 7.0 and 4.5 vs 1 by moving bodies); period_power 599/1 (a deliberate fail, see 29); fixed_pattern 10/0; classify_reject_recover 7/0. Facts sent to the TeX writer. The engine has not been run on the sim's lattice.npy yet.

2. Ruled ("fix all of them"): both period findings fixed and proved. (a) The period is now the smallest candidate whose height clears the band top (the fundamental), not the strongest

peak: from amp 64 up, 32/32 volumes read 5, 0 multiples (was 8/20 at amp 32). (b) The null is a per-axis line shuffle (each line along the axis permuted alone, keyed Fisher–Yates), not a global Feistel: z/y false periods at amp 1024 fell from 23/64 to 5/64 at 8 draws, 2/64 at 19. period_test 700/0 (host reference rewritten to the same rule), period_power 599/0 (the deliberate-fail check now passes). Old global-shuffle and strongest-peak behavior superseded. Timed (scratch program, RTX 3070, 34×960×960 smooth ramp + noise): period_draw 455.9 ms a call, marginal draw in period_read 450.1 ms, fixed 133.7 ms; 2.96× the global shuffle's 152 ms. At 73,158 draws a series that is ~9.1 h a series, ~223,000 GPU-hours for the set (was ~75,000). The set-wide band is even more necessary. period_draw allocates per call (not separated). Sent to the TeX writer.

3. Ruled ("increase the scheduler steps to n" + "implement the lut"): done. (a) ENGINE_RECORD_STEPS_MAX decoupled from ENGINE_RECORD_LIMBS_MOST, raised to 1024 (the sign array's local-memory cost is the bound; scale audit). (b) key_schedule gained opt-in register reuse (KeyScheduleRecordRequest.reuse / EngineRecordRequest.reuse): liveness free-list frees a register once its last reader has run, and a long chain fits the 256-limb file (a 21-step chain: 3 limbs reused vs 21 kept). Default off → proven programs byte-identical. (c) ENGINE_RECORD_TABLE (op 10): a lookup table: the low index_bits of a register select an entry; the engine's nonlinear edge (kolmogorov_arnold.md). Threaded config→keymath→key_schedule→cycle (device + host). Tables compose by reading one through another (in order; "collapsed in order onto" the 2D floor). Test test/engine/record_table_test.{cu,sh}: 15 checks, 0 failed, using the ordinary ops as oracles (x^2, $|x-30000|+100$, composition $g \circ f$, reuse, refusals). engine.h EngineRecordRequest gained tables/table_count/reuse.

4. Port survey: every orior engine C/CUDA file present here is code-identical or ours is larger (max_tree, climb_machine). Missing by name: the n_body basin modules (removed in the v2 split, superseded by the cycle/keymath residual), the sha256 tree (the miner's), and the MATLAB/R/Python ports. Nothing ported back without Doug; the art detector (orior's periodic_energy) now exists as sim code.

5. Ruled ("the tower is its own object, the lut is its own object, if we integrate them both we get a root universal"; chose "bijective edges in stream"): done. TowerEdge {floor, index_bits ≤ TOWER_EDGE_INDEX_BITS_MOST = 20, forward} in tower.h; TowerLiftRequest/TowerLowerRequest gained edges/edge_count (callers memset → no edges → the tower as before). An edge permutes the low index_bits of every approximation coefficient at floor k (the region after k halvings; floor == floors is the collapsed floor), high bits pass through, and it is a bijection on the word; tower_lift lays Eo,Lo,E1,…,L(N-1),EN and tower_lower undoes them as EN^{-1},$L(N-1)^{-1}$,…,Lo^{-1},Eo^{-1} (a floor's edges unwind as a stack). Validated up front (a permutation, width 1..20, floor ≤ floors), refused before any device work. test/tower_edge_test.{cu,sh}: 20/0. Sim engine/sims/root_universal: 1268/0. Exact through lift→code→wipe→decode→lower on 48 camera volumes (4×12×48×48, 3×13×37×41, 171 edges incl. 20-bit). Fold 8/8: a,b on one floor = b(a); b,a differs; a floor between them blocks the fold. Price (coded bits/voxel, plain 6.802): floor 0 at widths 2/4/8/12 → 6.810/6.908/8.719/12.608; middle floor (72 coefficients) ≤ 6.817; collapsed floor

6.802 (free). An unfitted edge is therefore exact but costs the magnitude coder unless it sits high in the tower or is fitted, which feeds the open "which functions to tabulate". Engine rebuilt and published. Not yet: the edges are not wired into engine.cu's crystal/seal path (the .kcr would have to record the edge program); Doug's call.

6. Doug ("reaching a boundary here means we need a dimensional expansion to realize the transform"; approved): the Bennett embedding (x, y) → (x, y ⊕ f(x)) makes any f a permutation of the widened field. A lossy table then rides the tower's reversible stream at the price of width (A14's "only permutations" bound becomes a width statement). No engine change; tables built on the host. tower_edge_test now 25/0: |x| and compare as 16-bit Bennett edges round-trip on 64 lattices, f(x) reads out of the carrier on the collapsed floor, the edge touches only its coefficient, mid-tower Bennett edges round-trip, |x| laid bare is refused. Naimark/Stinespring/purification are the quantum forms of the same expansion, which generally bring square roots.

7. Terms approved (ask, state, answer distribution, magnitude, phase, complete ask, valid set, crossing rule; "represent square roots as their real number with their square operation"; "why would you use 64 bit?"): sim engine/sims/ask_state, 33/0. Exact rationals on the exact integer (word-gcd reduced when they fit, exact unreduced otherwise), Q(√3) numbers as rational + root·√3 with (√3)² = 3 and exact signs. The rational tetrahedral ask (|v|² = 3/4, r = 4Σpv) and the true SIC (|v|² = 1, r = 3Σpv): 25 states (21 pure, 4 mixed) return exactly; pure on the boundary, mixed inside; crossings 75/75; the phase: 14 equator states all ½,½ on the 0/1 ask, all 91 pairs told apart; the weights onto +x are 3/2, −1/2 and ½ ± ½√3 (negative, not classical); (1,0,0,0) refused, crosses to 3/2 and ½ + ½√3; grid 455 at denominator 12: 55 and 87 states, 104 and 72 non-states that still cross to probabilities on x, y, z; the two asks cross into each other 25/25. Host exact arithmetic, no GPU work. Next on Doug's word: KA ψ with symbolic exponents ("the exp is symbolic after the second nth exp"; a sparse (digit, exponent) form, costs polynomial in the scale); the tower_lower hat-function limit; camera-parameter recovery sim.

8. Doug ("Yes" to the ψ sim; "the exp is symbolic"; "our boundary is truthy falsy ... with mp-prec"): sim engine/sims/ka_psi, 114/0, 28 s. Values at depth are sparse sums c·γ^−e with e = β(L) stored as an integer and never expanded. The exact sign comes from the head plus a proven tail bound, decided by bit lengths when the gap is wide. Results:

   - Sprecher (2.5) is exact: 2207/4000 > 11/20. It descends on 10 of the 99,999 pairs of D_5, the first at 0.09000.
   - Köppen, both readings of the carried midpoint: (2.9) adds (γ−2)/2 units (symmetric) and (2.7) as printed adds (γ−1)/2. For n = 2 and n = 3 at γ = 10, on every point of D_1..D_5: the pair step equals the level recursion, ψ is strictly increasing, the least gap is exactly γ^−β(L), and the widest gap equals the recursion G_L = ½G_{L−1} − (s/2)u_L, with s = γ−2 for (2.9) and γ−3 for (2.7).
   - The chained step, checked symbolically to L = 60 (β = 2^60−1) and L = 38 (β ≈ 6.75e17): order kept (u_{L−1} > (γ−1)u_L), least gap kept (u_{L−1} > (γ+1)u_L), widest gap > least, ≤ 2^−(L−1)/γ (continuity) and within Lemma 2.3. The symbolic

G_L equals the measured one on D_2..D_5. A keyed depth-L point's gap lies between the least and the widest.

- Both readings give a continuous, strictly increasing ψ; the discrepancy between (2.7) and (2.9) changes only the widest gap.
- Separation (Doug "Ready go"), now 152/0. ξ = Σα_pψ(x_p) was computed exactly on D_k^n (n = 2: k ≤ 3, 10^6 points; n = 3: k ≤ 2), with the α tails bounded.
  - LEMMA 3.4 IS FALSE AT k = 1. ξ(0.1, 0) − ξ(0, 0.9) = 1/10 − (9/10)α_2 = 0.0090999… < γ^ − 2, and the n = 3 analogue gives 9.0998 units. Lemma 3.8's supports (ramp γ^ − β(k+1)) overlap at k = 1.
  - The proof's break: (3.11), the α cut at r ≤ k, does hold. It measures exactly 10, 50 and 500050 units for n = 2 at k = 1, 2, 3, and 10 and 5000 units for n = 3 at k = 1, 2. But "the tails cannot annihilate it" does not mean "the tails keep it at or above the bound".
  - Correct: |μ_k| ≥ γ^ − nβ(k) − Σ_{p ≥ 2}ε_{k,p}, about (γ − 1)u' with u' = γ^ − β(k+1). That beats the image width (γ − 2)b_k ≈ 8.81u' (n = 2) and 8.88u' (n = 3). With ramp ρ = γ^ − (β(k+1)+2), the supports are disjoint at every k: checked symbolically for k = 1..57 (n = 2) and k = 1..35 (n = 3). The paper's ramp is covered at no k by this bound.
  - Measured from k = 2 the true gaps are far wider (49.5, 500049.5 and 5000 units). The shifts check: m + 1 = 2n + 1 shifts never share a gap; with m + 1 = γ they do.
- Doug ("You have division", "2s compliment", "Mulmask exact", "add that to the exact arithmetic"): engine/base/no_rounding/exact_integer gained three functions, and test/exact_divide_test passes 8/0.
  - orior_exact_divide: Knuth [26] D in base 2^32. It returns the quotient toward zero and a remainder with the numerator's sign. The add-back branch was confirmed to run, using an instrumented scratch copy.
  - orior_exact_divide_exact: the mulmask exact division. Twos shift out; the result is the odd part's inverse mod 2^(32·limbs), by Newton x(2 − dx) with a two's-complement negate, then a multiply and a mask. It is multiplied back, and a remainder is refused.
  - orior_exact_gcd: Stein's binary gcd.
  - New statuses ORIOR_EXACT_BY_ZERO and ORIOR_EXACT_NOT_EXACT.
  - sim_rational.h now keeps every value in lowest terms at any width. ka_psi (152/0) and ask_state (33/0) are unchanged in their verdicts and now print exact fractions (2207/4000, 11/20).
  - Then: Schönhage–Strassen, "We use karatsuba", "Expand the header to accept any n bits power of 2", "Static assert".
    * The header takes ORIOR_EXACT_BITS or ORIOR_EXACT_LIMBS: any power of two ≥ 32 bits, no ceiling. The limbs/bits pair is derived and must agree; DIGITS was briefly min(1024, what fits) and is back to orior's fixed

1024 floor (see the integration below). All checked by static asserts, and the ladder rungs are static-asserted too.

  * Width-sized working copies are compile-time rooms: stack up to ORIOR_EXACT_STACK_LIMBS = 4096, heap beyond, WILL_NOT_FIT if it does not fit.
  * Ladder: long multiplication < ORIOR_EXACT_KARATSUBA_LIMBS = 32, then Karatsuba, then the SSA transform (ring 2^n+1, shifts as twiddles, ψ weighting for the negacyclic product, depth by an integer cost model) from ORIOR_EXACT_TRANSFORM_LIMBS = 8192 (the measured crossover; the transform is 1.44× ahead at 16K, 1.55× at 32K, 1.85× at 65K).
  * gcd is now Lehmer (Knuth L) and returns a status. The exact division's Newton inverse doubles its precision and runs on the ladder.
  * test/exact_transform_test passes 3/0 at 128 limbs, 4096 limbs and 4,194,304 bits (operands to 65,536 limbs).
  * exact_divide_test passes 9/0 (it adds gcd = Euclid on the long division). ka_psi passes 152/0 and ask_state 33/0.
  * Newton division BUILT (Doug "Add the newton port"): a half-precision recursive reciprocal, one Newton step, exact correction. ORIOR_EXACT_NEWTON_LIMBS = 8192, measured 2n/n on the Karatsuba ladder: Knuth leads to 4096; Newton is 1.21× at 8192, 1.06× at 16384, 1.81× at 32768 and 2.44× at 65536. exact_transform_test passes 4/0 at 128 and 4096 limbs.

– Doug ("write these exact limb functions as computer key math primitives ... projected into a bit array of the same dimensions"): record ops ENGINE_RECORD_QUOTIENT = 11, REMAINDER = 12, GCD = 13, EXACT_QUOTIENT = 14.
  * Wired through engine_config.h, keymath imprint (bits: quotient = left, remainder = min, gcd = max), key_schedule refs, cycle.c (host, the library itself) and cycle.cu (device Knuth D, Euclid, and multiply-and-mask on the register arrays).
  * The kernel is templated <WIDE, DIVIDES>: only a dividing program carries 5·WIDE+4 limbs of scratch.
  * test/engine/record_divide_test passes 15/0: device = host word for word in the 64- and 256-limb files, with identities, exact factors, refusals and imprint order. record_table_test still passes.

– Doug ("integrate orior's engine so we can copy push ours to orior"): THIS TREE IS NOW THE SOURCE.
  * build_engine.sh and build_driver.sh default ORIOR_EXACT_ROOT to $TOP/engine/base/no_rounding.
  * 18 files (no_rounding arms, nbody/orior, render) were code-identical to orior's; ours had lost doc text to a sloppy prose pass, and orior's versions were copied in.

    * exact_integer.h/.c were merged by hand: our code with orior's contract, and stale notes fixed. ORIOR_EXACT_DIGITS is back to the fixed 1024 floor (a narrow width must declare it; checked 1..64 limbs, and undeclared at 8 limbs is refused by name). check_exact_limbs.py's regexes still find the literal #define lines.
    * The engine DLL builds. Nothing has been written into orior's repository yet.

– Doug ("Compute chaitins constant"): sim engine/sims/chaitin_omega, Omega for the BLC normalization machine (a closed term halts when it has a normal form).
    * Closed terms are unranked from exact counts (checked against a parse of every code up to 22 bits) and run by normal order on 16 threads, with Brent watching for loops.
    * The tail past L is counted in directed fixed point: parse mass 1 = the smaller root of $T^2 - 3T + 2$. Normal forms past L add to the lower bound.
    * L = 30: $0.122543 \le \Omega \le 0.125993$, 2 bits. (0.12597 was a hand-conversion error, since corrected.)
    * Doug ("everything is truthy falsy steer", "loop with no answer after information space has been searched is an exit condition"): the exits are now halts / loops / grows forever / out of steps / outgrew the space.
        · Bounded space ⇒ halt or repeat (pigeonhole): at 1e6 steps, "out of steps" was 0 through 30 bits.
        · Growth proof (omega_grows_forever): a spine part S, reduced only by head steps at spine position ≥ its own, returns deeper on the spine as $S\,\vec{B} \Rightarrow$ no head normal form ⇒ no normal form. It closes 213 of the 587 unresolved terms through 30 bits (200k steps, 16384 tokens). 64× more space moved none of the rest into halts or loops. The BB table (most steps; BB lambda = the largest normal form) matches BusyBeaverWiki through 22. 11/0.
    * L = 40 (283,817,255 terms; 488,151 loops, 474,630 open): $0.12407 \le \Omega \le 0.12599$, bit 3 NOT settled. The lower bound needs +0.00093 to pass 1/8, about $L \approx 48$, roughly 250× the work: a GPU port or a larger counted halting class. Published: Calude–Dinneen 43/40 bits on their register machine; Tromp's own Ω_λ = .0001… (a different machine); a cryptid at 74 bits.
    * GPU PORT BUILT (Doug "Do it"): omega_device / omega_kernel in chaitin_omega.cu, one term a thread, the host's decisions in the same order; device budgets ≤ 256/256 (steer: anything reaching them, or outgrowing the 2×tokens room, is run by the host under the full budgets, exact since budgets only stop a run). Host handback settled while the next launch runs. Host cross-check through 30 bits (fates + BB + champions) passes; `cpu` arg = host only.
        · Speed: budgets barely matter for Ω (36 bits: 256/256 vs 2048/2048 exact per writer: lower −3.19e-9, open +2.50e-8, upper +2.18e-8 ≈ 2^-25.5; gap 0.00240 ≈ 2^-8.7; the earlier "2^-22" was a misread)

  - LEDGER BUG (reproduced): a line cut inside field 16 then ended by the writer's '\n' parses whole and wins as first match → wrong record holder past 30 (fates/bracket safe). Fix pending Doug's go (he said Stop): end marker/checksum, or drop lines the writer had to end. Doug wants the lambda Ω run BY THE ENGINE (record machine), no step/token/width limits; awaiting his go. Until then Ω runs use 256 256. RTX 3070, 384 threads/SM (cache-bound; more is slower): 44 bits 3.39G terms in 28 s (121M/s, ~7.5× the host at 256/256).
  - L = 44 (256/256): 0.124438573589 ≤ Ω ≤ 0.125987962504; lower short of 1/8 by 0.000561; first typed halt (1). Upper barely moves with L, and bit 3 settles only from below (needs Ω ≥ 1/8); lower's per-bit gain shrinks ~0.93/bit ⇒ crossing ≈ L 54 (~3.7 h device) if the trend holds.
  - BB check split: "≤ published, = where settled" always; strict equality only at budgets ≥ 2048/2048.
  - JOBS ("why aren't you planning these as jobs for the engine and chunking"): `--ledger <path>` plans each length in 2^28-term jobs; each finished job's exact tally is one flushed line; reruns keep whole lines under the same budgets and run the rest (resume; longer L costs only new lengths). Cut lines are ended before appending; a line must parse whole and its fates sum to its count. Tested: fresh 31 → extend 33 (2 run, 27 kept) → rerun (29 kept) identical; cut last line → 1 rerun, identical.
  - RUNNING: `omega_jobs.exe 54 256 256 --ledger build/omega_ledger.tsv` (~3.7 h est.). Read a shorter L's bracket from the ledger any time by running that L with the same ledger (all kept).
- Next: the outer-function contraction (Theorem 3.3) with the narrow ramp; ψ at the shifted points x + qa (non-grid, a = 0.0qqq… in base ɣ).

- Found on the way: a series name matching no DICOM member falls through to the npy reader (error module 15) instead of a "no such member" refusal; a failed ingest leaves the empty sample directory it made (engine_directories_make must report what it made, and its walk[1024] is a chosen bound, with the long-path work).

1. Tessera daemon. Built and proved; moved to engine/daemon ("Put it in engine/daemon"); README on starting it and submitting jobs. Seal done ("Go"):
   - The history is records plus an obsignatio_seal (host keyed BLAKE3, file level). A load whose length breaks, that has no seal, or whose seal fails is refused and the daemon exits.
   - Failed saves are reported on stderr.
   - Tickets end with `seal <hex>`; the precalc note needs a seal that holds and seals the ticket again.
   - The job test runs in a scratch TESSERA_STATE: 24/0, including damaged, cut and unsealed histories refused and the kept peak surviving a restart.

- The old unsealed real history was moved to %LOCALAPPDATA%\tessera<uuid>\history.unsealed.
- The history now loads before the pipe or socket exists, and the note uses s_daemon_precalc. Rerun at 24/0.
- "The engines own runs need to go through tessera". The driver's jobs are the cell program's (the cell workbook's build plan, item 36).
- Sims submit, each 0 failed with its tessera checks:
  - sim_job.cu with sim_job_submit/sim_job_release; sim_close releases; run.sh builds the client and the daemon beside the sim.
  - Declared and peak: period_power 262,144/145,915,904; nbody 26,542,080/173,178,880; noise_floor 9,142,272/187,858,944; root_universal 339,510/152,207,360; fixed_pattern 141,056/143,818,752; classify 118,016/145,915,904.
  - Ω now builds (the capture fix was in) and runs on the engine at L 16 as a job: 16/0, peak 286,425,088.
  - Nothing loads the DLL. cell_shift is orior's engine.
- Linux run (WSL 2, gcc 13.3, CUDA 13.3). Transport: a tree from a scratch index, then git archive piped to ~/tessera_linux; /mnt is banned by the hook.
  - Fixed: PATH_MAX via linux/limits.h; _GNU_SOURCE in the measure; the timer's return; ENGINE_NOINLINE_HELPER; platform strings guarded; the daemon uses its string table, and the ticket names its signum.
  - WSL's NVML lists no process's memory; tessera_measure_paravirtual refuses, and the daemon exits. The suite passes with 0 warnings.
  - Native Linux is not available here.
- The crystal's growth is the compression workbook's (its build plan).
- Service files: engine/daemon/service/tessera@.socket and tessera@.service (systemd user units). Activation was proven in WSL: systemd started the daemon on a client's connection, then the WSL measure refusal ended it. Windows needs no service. The docker mount is documented and has never been run.
- Reservation rule done (by the stop hook's "finish all open items"): tessera_ledger_wants = max(declared, kept peak) is used for admission, the shadow, the spare and the reservation at start. The ledger budget test is 22/0; the job test is 24/0. The real prove reserved 3,958,566,912, its kept peak.
- Docker mount run: Docker Engine 29.1.3 in WSL; the static test/tessera_socket_probe.c in a container, socket mounted, TESSERA_RUNTIME=/run/tessera. systemd started the daemon on the container's connection; the WSL refusal ended it.
- Open: native Linux (no native NVIDIA Linux driver on this machine); the is not run; no service files; engine runs don't submit through tessera (M14); the Ω engine-run path (lambda capture at about chaitin_omega.cu) is unbuilt.
- WSL self-report ("Use wsl"): under WSL each client reports its own device bytes (dx-core, `tessera_self.c`, TESSERA_ASK_MEASURED); the WSL job test is 24/0. The

systemd job test failed on its first pass because a daemon left over from the suite unlinked systemd's socket file at idle. Fixed in tessera_daemon.c: a daemon probes the path before binding and refuses if something answers, and it unlinks only the file it bound (dev/inode). After the fix: 3 of 3 passes 24/0, the Windows suite passes, and a container's job was admitted through the mounted socket.

- Reservation rule: "We reserve what they ask for and then if it cost less we remember that for next time". tessera_ledger_wants is still in the working tree, waiting on his clarification. Not reverted.

2. **The exact-sum programs for C14 to C20.** Each of C14 to C20 in the compression workbook's compression_table.md becomes one program in the noise detector (`src/engine/base/noise_detector/`). Built on orior main, each graded in `noise_terms` by a tessera job (the runs, below). A program takes integer sums over voxels and frames and reads its quantities by cross-multiplying over one denominator, as `noise_static_readings` does (noise_detector.cu). It forms no quotient, root or float before its last division, and it fills the rows of noise_vector_integration_table.md its section names. A sim grades each program: it plants the term in the camera law (`sim_camera.h`) and reads it back through the same code, as `noise_terms` does.

   - **The line over the level (C15, C17 and C19).** A reading's slope and intercept against the level, the least-squares line through each level bin's exact sums, stored as three exact integers over one denominator. Built: the form moved out of `floor_fit` in the `noise_floor` sim into the noise detector as `noise_line_fit` (noise_detector.h). `floor_fit` calls it (noise_floor.cu), and noise_floor's report is identical before and after. The ladder calls it for C15 and C19 (noise_detector.cu). C17's fit has two regressors, q and the pair total M, and does not call it. The fits the noise table's readings came from (`transfer_residual.py`, `moments_ladder.py`, `ladder_against_neighbour.py`, `neighbours_background.py`, `transfer_curvature.py`) are tracked in neither biohub nor orior.
   - **C14, charge before the gain (rows 2 to 6).** Takes a bias series (no light, no exposure) and a dark series at a named Δt, and per voxel Σ I over the frames and Σ d^2 over the frame differences. Reads O as the bias mean and the level-free sum $R^2 + \sigma^2_J + \sigma^2_kTC$ as the bias $E[d^2]/2$. The dark mean less the bias mean is $g{\cdot}D{\cdot}\Delta t$, and the dark $E[d^2]/2$ less the bias's is $g^2{\cdot}D{\cdot}\Delta t$; g is their ratio, compared by cross-multiplying. Rows 2 and 3 are read; row 4's intercept loses $-g{\cdot}O$; rows 4, 5 and 6 stay one sum at one temperature. Built: `noise_charge_series` reads O, $R^2 + \sigma^2_kTC$, $g{\cdot}D{\cdot}\Delta t$, $g^2{\cdot}D{\cdot}\Delta t$ and g exactly, and the camera law gains dark electrons and the reset variance (`dark`, `reset_square`). The sim plants gain 2, read variance 4, reset variance 4 and dark 16 and 32, with a Poisson dark shot: O 19.9997; $R^2 + \sigma^2_kTC$ 8.0038; $g{\cdot}D{\cdot}\Delta t$ 32.0002 and 63.9988; $g^2{\cdot}D{\cdot}\Delta t$ 64.0040 and 127.9703; g 2.0001 and 1.9995. The reaches are 1/500 of O, 1/100 of $R^2 + \sigma^2_kTC$ and of $g{\cdot}D{\cdot}\Delta t$, 1/5 of $g^2{\cdot}D{\cdot}\Delta t$ and 1/300 of g, each at least 5 standard errors. The kcr set holds no bias or dark frame (the noise table's open item 4).
   - **C15, the cumulant ladder (rows 1, 12 and 15).** Takes the moment pass's sums, built: each level bin's blocks, values and S_1 to S_4 products (`NoiseMomentCell`, written

whole to noise_moments.tsv). Reads each bin's k_2, k_3 and k_4 as exact rationals, then the slopes s_2, s_3 and s_4 against the level by the line above. Checks $s_3 \geq {s_2}^2$ and $s_2 \cdot s_4 \geq {s_3}^2$, each cross-multiplied over the slopes' denominators; the rungs g, g^2 and g^3 take g from the neighbor route (row 15). `noise_moments_volume` pools the bins and rounds each k-statistic toward zero in thousandths, and the ladder reads the unrounded rationals per bin instead. Built: `noise_ladder_volume` reads each quiet static block's k_2, k_3 and k_4 against its voxel's level, the three slopes and the two checks. The sim's shot is Poisson (`SIM_SHOT_POISSON`), with gain 2, offset 20 and read variance 4: s_2 1.9852, s_3 3.9933, s_4 9.4410, within 1/50 of g, 1/4 of g^2 and 3 of g^3, each reach about 5 standard errors.

- **C16, thinning (rows 13, 14 and 16).** Sim only; the kcr set holds nothing it reads (the noise table's open item 4). The sim plants a count with Fano factor f, thins it binomially by q = a/2^b, and checks $f' - 1$ against $q \cdot (f - 1)$ by cross-multiplying. In DN f is the count's variance over $g \cdot (L - O)$, and it needs g apart (row 12) and O (C14 or C19). A Poisson count (f = 1) must read $f' = 1$ at every q; that is the null that leaves rows 13 and 14 unread on Poisson light. For row 16 the sim blurs S by an integer kernel over a power of 2 before the draw, and the neighbor pass (built) must read 0 at every reach. Built: the camera law gains two count laws, Fano ½ (`SIM_SHOT_HALF`) and Fano 2 (`SIM_SHOT_PAIRED`), a binomial thinning, the excess $F^2 = 2$ and a blur along x (`blur_eighths`). $g \cdot f'$ at q = ¼, ½, ¾ and 1 reads 2.0007, 1.9995, 1.9996 and 1.9999 on Poisson; 1.7500, 1.5002, 1.2500 and 1.0000 on the half law; 2.4997, 2.9995, 3.5005 and 4.0004 on the paired law; the excess after the thinning reads 3.9993 and 3.9981. Under the blur (1, 2, 1)/4 every column's mean is within 0.0482 of the blurred light, and the neighbor and crosstalk passes read 0.
- **C17, the structure function (rows 7 and 10).** Takes the flicker pass's sums, built: at each lag and level bin, the pairs and Σ d_k², and the same less the x neighbor (`NOISE_FLICKER_*`). Adds, per lag and bin, the sums that fit $(d_k - d_\{k,x\})^2$ against $q = (T \cdot (L - L_x))^2 = (\Sigma_t\, I - \Sigma_t\, I_x)^2$, an integer for T frames: $\Sigma\, q$, $\Sigma\, q^2$ and $\Sigma\, q \cdot (d_k - d_\{k,x\})^2$. By the line above, the intercept is $2 \cdot D(k)$ of each voxel's own noise and the slope is $D_a(k)/T^2$, the shared scale's structure function (row 10). Built: `noise_structure_volume` fits $y = (d_k - d_\{k,x\})^2$ against q and the pair total M within level bins, exactly, with 128-bit device sums, and the camera law gains a shared scale with memory (`scale_square`, `scale_octaves`, `scale_bits`). D_a at lags 1, 2, 4, 8, 16, 32 and 64 reads 3731, 5783, 7744, 11596, 11592, 11750 and 8758 millionths, against the camera's own path 3692, 5737, 7653, 11471, 11442, 11726 and 8690, within 500 millionths. The pairs' own noise, $2 \cdot D(k)$, reads 486.06 to 477.74 against 485.83 to 477.68, within 1.
- **C18, the row and column sums (rows 8, 9 and 10).** Built. `noise_static_readings` (noise_detector.cu) takes T, A_row, A_column and A_plane and the pair counts K over the static voxels, and reads var_row, var_column, var_plane and s over the common denominator $M \cdot K_row \cdot K_column \cdot (K_plane - K_row - K_column)$. `track_driver --run lines` runs it on the kcr set, and `noise_terms` grades it. Nothing owed.
- **C19, the single-electron tail (row 11, and O for row 4).** Takes the moment pass's sums (built) and the line above. With c_2, s_2 from the κ_2 line and c_3, s_3 from the κ_3 line, it reads $O \cdot s_3 = -c_3$ and $R^2 \cdot s_3 = c_2 \cdot s_3 - c_3 \cdot s_2$, cross-multiplied, from frames at one

exposure. It reads O only where C15's two checks hold, and on the kcr set row 1's tail is open (in Ω). For spikes against dips it takes the clip pass's counts (built, noise_spikes.tsv) and needs a null that carries the shot's third cumulant (row 11). Built: the lines and the two readings, in `noise_ladder_volume`; the sim reads O 19.9533 within 3 and R^2 4.1981 within 6. In the sim, a Poisson shot's null holds more spikes than dips by more than 5 standard errors at 16 and 32, and a symmetric shot's holds them equal within 5. The detector carries no null of its own, and the kcr set's spike count against a Poisson null is not read.

- **C20, crosstalk in the neighbor correlation (rows 16 and 17).** Takes $\Sigma\, d'^2$, $C_1 = \Sigma\, d'(v)\cdot d'(v+\delta)$ and $C_2 = \Sigma\, d'(v)\cdot d'(v+2\delta)$ along y and along x, over the same voxels. From C20's three lines, $V' - 2C_2 = V$, $\alpha = C_1/(2\cdot(V' - 2C_2))$ and $\alpha^2 = C_2/(V' - 2C_2)$, all exact rationals. The two agree only where $C_1^2 = 4\cdot C_2\cdot(V' - 2C_2)$, and at three steps the sum must be 0; a pair that fails either is not crosstalk of this form. The neighbor route's σ^2 is its reading over $(1 - 2\alpha + 2\alpha^2)$. The sim plants α as an integer over a power of 2 after the draw. Built: `noise_crosstalk_volume` returns the pairs, $\Sigma\, d'^2$, C_1, C_2, C_3 and $V = \Sigma\, d'^2 - 2C_2$ along y and along x, exactly; how near to agreement and to 0 a reading must come is not set in the detector (noise_detector.h). With $\alpha = 1/8$ planted along x the sim reads α 0.1250, α^2 0.0158 and C_3/V 0.0000; along y and in the null every reading is within 0.0002 of 0. The neighbor route's σ^2 over $(1 - 2\alpha + 2\alpha^2)$ is not built.
- **The runs.** `noise terms: 318 checks, 0 failed`, a tessera job. Every other sim that includes the camera law, rerun there the same day: noise_floor 13 checks, 0 failed (its report identical to before), noise_root 72/0, knf_identity 23/0, floor_match 11/0, period_power 601/0, nbody_lattice 11/0, root_universal 1270/0 and floor_track 36/0.

15

LEDGER

Purpose: Every measurement, in the order it was taken, with the samples it ran on, the number, and what it settled. No result is taken twice, and none is quoted without its run.

Scope: runs of the engine: its modules, its tests and its sims. Samples are named by their id; "the 25" means the first 25 44b6 training samples by name.

15.1 2026-09-21

15.1.1 *The cycle against the passes it replaced*

what	samples	result	settles
the residual from the imprinted key, lane for lane against the smoothing passes and the transform	the 25, 2,500 frames	0 of 10,485,760,000 lanes differ	the key is the program (proved)
time a frame	the 25	31 ms, against 24.5 ms for the passes	the key is not yet faster; factor the binomial as [1 1]^n next

15.1.2 *Bit flips on this machine*

what	result	settles
the tower undone in the spiral and ratio runs	180, then 80 voxels differ, each one bit (0x0100, 0x0400)	the code was cleared first: a rerun holding the coefficients found 0 moved
44b6_0b24845f and 44b6_341df25f, re-proved from their .kcr	one voxel each wrong in the file; the disk, read past the cache, holds the right value	the machine flips single bits in memory under load; both files had been written from a bad cached copy
fix	every .stack read for ingestion and proof now bypasses the file cache; both samples re-ingested and proved	a cached copy can no longer vouch for itself

15.1.3 *The CRC-64 fold*

what	samples	result	settles
CRC-64/XZ folded into the tower's widen, against the plain byte by byte CRC	44b6_0113de3b	both 363bf8bdffac8f29	the fold is the CRC (proved)
every sample ingested with its CRC, then proved from the file alone	the 25	25 of 25 hold; 8,809,343,524 of 20,971,520,000 bytes, 42.0%; set CRC 091daa41e1aceb7e; the proof at will in 19 s	the per file check at will (proved)

15.1.4 *The anchor bits*

Per voxel and per bit, the frames of the 100 that carry the bit; the counts were written as .knf and read back equal.

what	result	settles
bits 0 to 4	set in about 46% of frames at nearly every voxel, in every sample	the floor: these planes carry no anchor
bits 6 to 11, set in every frame	from 0 voxels (5 samples) to 76,646 at bit 10 (44b6_5f15d135)	anchors are real and differ per sample
any voxel anchored in every sample	none	the floor is per sample, not per set

15.2 2026-09-22

15.2.1 *The compression, keymath and key_schedule splits*

what	samples	result	settles
compression split out of the tower: 44b6_0113de3b re-encoded	44b6_0113de3b	the .kcr byte identical to the one before the split	the coder moved unchanged (proved)
every .kcr decoded by the split coder, from the file alone	the 25	25 of 25 hold; set CRC 091daa41e1aceb7e	the decoder moved unchanged (proved)
keymath and key_schedule split out of the cycle: the tracker run	44b6_0113de3b, 44b6_0b24845f	edges identical; .vbo/.ibo files identical but for the embedded .cfg	the imprint and the layout moved unchanged (proved)
the residual's key, sized		weights 1,130 words, 4,520 bytes, plus a 256 byte term table, standing for 268 unit steps a voxel: 112,407,347,200 step applications a sample	a key's size is the program's reach, not its use

15.2.2 *The entropy history*

what	samples	result	settles
the counts against the .stack, by hand in Python	44b6_0113de3b, 2,000 voxels × 9 windows	0 words differ	the history is exact (proved)
the projection	the 25	25 of 25 written and read back whole, about 3.4 s a sample	
the floor	the 25	bits 0 to 3 at 499 to 500 flips per thousand transitions in every window of 19 samples; 350 to 499 in 6	the floor is half the transitions, where every voxel moves
a constant region	44b6_0db75fae, 1574802b, 267148e4, 5740d24b, 587a1e22, 668e0cc7	bits 0 to 3 drop together, identically within a window (e.g. 350, 350, 350, 350): voxels whose value never changes. 5740d24b holds it from the start; 267148e4 grows it from window 3 to about 30% of transitions	a fixed pattern term read exactly, with no flips in any bit; its cause is not established
direction, bits 5 to 10	the 25	12 samples move toward order (last window below the first), 13 toward the floor; peaks fall in different windows; 44b6_0db75fae falls 1,293 to 532 per mille, 44b6_668e0cc7 rises 684 to 1,421	the direction is each sample's own; the window 4 bump on 44b6_0113de3b is that sample's, not the set's

15.2.3 *The codec split: no module reaches another*

what	samples	result	settles
`python maint/audit_reaching.py`		18 modules reach nothing, among them `keymath`, `key_schedule`, `cycle`, `tower`, `compression`, `apxrep`, `entropy_history` (`tower`, `apxrep` and `entropy_history` include only `crc` beyond their own, and `crc` is a root); 19 still reach, `score_sample` reaching 14 modules and `binomial_basins`, `flatten` and `score_sample` reaching up into `entry`	the key chain and the codec hold the rule (proved); the tracker's modules do not yet (not so yet, being split) (keys_explained.md §9)
`bash maint/prove_codec.sh`, run by the session that split the codec	the 25; 44b6_0113de3b re-ingested	every .kcr proved from the file alone, set CRC 091daa41e1aceb7e; the re-ingested .kcr byte identical to the cached one	the split changed nothing in the codec (proved)

15.2.4 *The codec, to the byte*

Read from the code and the files in `cache/iapx`; the claims are in compression_tower.md.

what	samples	result	settles
the .kcr of 44b6_0113de3b, part by part	44b6_0113de3b	head 16 + seven words 56 + 102,400 chunk first bits × 8 = 819,200 + stream 340,189,016 = 341,008,288 bytes, of 838,860,800 raw	the file and the stream are separate numbers: 406 per mille of raw for the file, 405 for the stream, both rounded down
the driver's share of raw		printed as (1000 × bytes / raw) rounded down, shown as a percent with one decimal: "40.6%" is 406 per mille rounded down	the 25 at "42.0%" are 420 per mille: 8,809,343,524 of 20,971,520,000
floors printed on ingestion	the 25 (`logs/iapx_all.log`)	8 floors, 25 of 25	a 100 × 64 × 256 × 256 sample reaches one coefficient in 8 floors (proved)
the floors' blocks	a 44b6 extent	419,430,400 + 26,214,400 + 1,638,400 + 106,496 + 7,168 + 512 + 32 + 4 = 447,397,412 lanes, 1,066 per mille of the sample	depth is paid in shrinking blocks, not in passes over the whole
the CRC join	a 44b6 extent	6,553,600 segments of 64 voxels, joined in 23 levels by operators 2^7 to 2^29; the key's last operator, 2^47, is reached at level 40. The join takes any message below 2^48 bytes	the 24,576 byte key covers 2^48 byte steps, as keys_explained §4 states
the Rice cost bound		24 + 32 = 56 bits a value at most; 5 + 64 × 56 = 3,589 bits a block at most	the escape bounds every value
whether the 5/3 lift is linear		the high-pass coefficient of (0, 0, 1) is 0, of (1, 0, 0) is 0, of (1, 0, 1) is −1	not so: the tower is exact and reversible, but it cannot be imprinted as one key by its impulse

15.3 2026-09-23

15.3.1 *The period reading*

`period_read` (M18 and A12 of engine_table.md), graded by `test/period_test.cu` through `test/period_test.sh`. The random set is 64 uniform u16 volumes of 32 × 64 × 64, three axes each, 192 axes in all.

what	samples	result	settles
the chance floor: P kept when its margin reaches $\Sigma c^2/N^2$	the test's planted lattices; the random set	431 checks, 0 failed; a period on 63 of 192 random axes (33%)	Σc^2 is unchanged by a rearrangement, and the floor passes noise; superseded by the drawn null
the drawn null: P kept only when its margin stands above every margin of d keyed shuffles of the same values (A12)	the same, at d = 8	502 checks, 0 failed; a period on 21 of 192 random axes (10.9%), against 1/(d + 1) = 1/9	the false-period rate is the caller's to set with d (99 draws, 1%); the floor Doug ruled; its margin rule superseded by the local peak
the local peak: P and 2P each above both neighbor lags, the height the smaller rise, judged against the drawn null or a set-wide top from `period_draw` (A12)	the same at d = 8, the planted z grown from 20 to 24 slices; 16 smooth ramps with noise of 24 × 64 × 64 (48 axes)	700 checks, 0 failed; a period on 6 of 192 random axes (3.1%), below 1/(d + 1) = 1/9; on 2 of 48 smooth-ramp axes (4.2%), where the margin rule read P = 2 or 3 on every axis of the 34 × 960 × 960 timing run	smoothness is no longer read as a period; one band serves the set, since 73,158 draws a series cost 152 ms each on 34 × 960 × 960 (RTX 3070), about 75,000 GPU-hours for the set; the rule Doug ruled, and whether one band is fair across shapes is Doug's call; its strongest-peak choice and global shuffle superseded by the fundamental and the per-axis null
the fundamental above a per-axis null: P the smallest local peak whose height clears the band top, the band drawn per axis by shuffling each line along that axis alone (A12)	the same at d = 8; `period_power` (M19) at 8 and 19 draws	700 checks, 0 failed; the planted periods read exactly; a period on 5 of 192 random axes (2.6%) and on 3 of 48 smooth-ramp axes (6.3%); `period_power` 599 checks, 0 failed: from 64 e up x reads 5 on 32 of 32 at 19 draws (at 8 draws, 31 with 1 multiple at 256 e), and with the plant at 1,024 e the unplanted z and y read a period on 5 of 64 at 8 draws and 2 of 64 at 19	the fundamental is read instead of a multiple; the per-axis null holds the false-period rate when another axis carries a strong period; the rule Doug ruled; the per-axis null's cost is timed in the next row
the per-axis null's cost	34 × 960 × 960 (31,334,400 voxels), a smooth ramp x + y + 2z plus uniform noise 0 to 63, RTX 3070; a scratch timing program, one run	`period_draw` 455.9 ms a call (mean of 8 after a warm-up; one count of the original, then a shuffle and a count on each of 3 axes); `period_read` 1,934.2 ms at 4 draws and 9,136.5 ms at 20: 450.1 ms a marginal draw and 133.7 ms fixed (as the run printed them from the unrounded times); 2.96 times the global shuffle's 152 ms	one band serves the set the more: about 9.1 hours a series at 73,158 draws and about 223,000 GPU-hours for the 24,386 series (3.1 and 75,000 before), arithmetic from the measured draw instead of a set run; host wall-clock with the syncs, and `period_draw` allocates on every call, and part may be allocation (not separated)

15.3.2 *The sims*

The first five sims of `engine/sims/` (M19 of engine_table.md; the sixth, `root_universal`, is under "The root universal" below, the seventh, `ask_state`, under "The ask and the state", and the eighth, `ka_psi`, under "Kolmogorov's inner function"), each an exact-integer GPU program run by `bash src/sims/run.sh <sim>`, all under the one camera law of `sim_camera.h`. Every count below is the sim's own output; the ratios are measured, and a lane-for-lane equality is proved.

sim	samples	result	settles
`nbody_lattice`	24 frames of 20 × 96 × 96, 16 bodies (10 founders; divisions at frames 8, 12 and 16); gain 1, read variance 3, offset 100, fixed pattern 0 to 8	9 checks, 0 failed (11 with `--out`); the device truth equals the host's walk of each box, and the signal equals the host's on 4,423,680 of 4,423,680 lanes; the residuals sum to 3,271 against a spread of the square root of 636,863,096; their squares sum to 635,890,703 against 636,863,096 (0.99847)	the camera law renders exactly and holds its two moments; `lattice.npy` and `truth.tsv` are written, and the engine has not been run on them
`noise_floor`, the Kolmogorov mock (C11)	4,096 streams of 512 bits from linear generators of degree 8 to 64	Berlekamp–Massey within the degree on 4,096 of 4,096, each stream regenerated from its first L bits on 4,096 of 4,096; 286,744 of 2,097,152 bits (13.67%); keyed draws read 252 to 262 against 256	a generated stream is found and described by its generator; the keyed draws read as random
`noise_floor`, the bit planes	8,640 streams of 512 lanes a plane of the camera lattice	planes 0 to 8 random on 8,640 of 8,640 (mean L/n 0.5004); plane 9 on 850 (0.0638); plane 10 on 29 (0.0029); planes 11 to 15 on none	the camera lattice's planes 0 to 8 read as random; planes 11 to 15 have mean L/n below 0.0001 (printed 0.0000, truncated), and none of their streams reads as random; exactly zero is not shown
`noise_floor`, the photon transfer curve (C3, C7, C5)	4,239,360 frame differences; planted g = 1, r^2 = 3	truth route (4,226,574 pairs): slope 2.0046 against 2, intercept 5.643 against 6; data alone: slope 7.0460 against 1; neighbor coherence (4,195,200 pairs): slope 4.5292 against 1	removing the motion by the truth recovers the planted gain (slope within 2%); the read-noise intercept, 5.643 against 6, is inside the ±6 check but not pinned by it; the data-only routes do not recover the gain, since sharp-edged moving bodies bias them
`noise_floor`, the neighbor correlation (C6)	the same differences, x against x + 1	$\Sigma dd'/\Sigma d^2$: −71,727/1,161,503,631 on truth-static pairs; 403,193,327/2,288,244,270 (0.1762) on every pair	the noise is uncorrelated between neighbors; the moving bodies correlate them

sim	samples	result	settles
`period_power` (M18, A12)	a period 5 planted along x of 32 × 64 × 64, 32 volumes an amplitude, at 8 and 19 draws	under the strongest peak and the global shuffle, 599 checks, 1 failed by design; under the fundamental and the per-axis null, 599 checks, 0 failed (the tables below)	the strongest peak read a multiple and the global shuffle read periods on the unplanted axes; Doug ruled both fixed, and the fixes hold
`period_power`, an earlier check	the strongest plant then, 32 e, at 8 draws	exactly 5 on 15 of 32 volumes, and the check failed	replaced by "5 or a multiple", which holds at 1,024 e on 32 of 32
`fixed_pattern` (ART-4-005 ported)	frame 8 × 8, 48 frames, swing 20, pattern 40, 8 draws, key 0xF17A	10 checks, 0 failed; period 64 at 179.565 against its shuffle's 0.972 and a band top of 5.241; the residual equals the scene on 3,072 of 3,072 (100%); the wrong period 63 removes 0.2298%; no pattern (1.477) and impulses (2.064) declined; a static feature at depth 0, 5, 15, 30 leaves 100.0000%, 99.9263%, 99.3370%, 97.3480%	the fixed pattern is removed to the bit and only when it is there; a static scene feature goes with it
`classify_reject_recover` (ART-4-007 ported)	the same frame and draws, four cases	7 checks, 0 failed; the fixed pattern found and removed exactly (3,072 of 3,072, 0 of 64 flagged); incoherent noise of variance 1,600 declined though its 3.523 at period 3 clears the band top of 3.294; impulses on a repeat exact on 64 of 64; incoherent noise of variance 144 on a repeat, 55 of 64 flagged and exact on 4 of the 9 where count and median agree	agreement of the two routes does not make a pixel exact: the mode and the median can agree on a wrong value

The period reading's power, `period_power`: x reading 5, a multiple (10 or 15) or another lag, of 32 volumes, and the unplanted z and y reading a period, of 64 axes. Now, the fundamental above the per-axis null (A12):

amplitude (e)	8 draws: x 5 / multiple / other	8 draws: z, y	19 draws: x 5 / multiple / other	19 draws: z, y
0	1 / 0 / 4	7	0 / 0 / 2	4
2	0 / 2 / 2	9	0 / 1 / 0	2
8	0 / 2 / 1	7	1 / 0 / 0	3
16	10 / 6 / 0	8	8 / 5 / 1	2
32	26 / 3 / 0	10	25 / 4 / 0	4
64	32 / 0 / 0	4	32 / 0 / 0	3
128	32 / 0 / 0	3	32 / 0 / 0	4
256	31 / 1 / 0	5	32 / 0 / 0	3
1,024	32 / 0 / 0	5	32 / 0 / 0	2

Before, the strongest peak above the global shuffle (superseded):

amplitude (e)	8 draws: x 5 / multiple / other	8 draws: z, y	19 draws: x 5 / multiple / other	19 draws: z, y
0	0 / 1 / 3	6	0 / 0 / 2	3
2	0 / 0 / 1	7	0 / 1 / 0	2
8	0 / 5 / 2	7	0 / 0 / 1	3
16	2 / 14 / 0	7	4 / 8 / 0	2
32	8 / 20 / 0	7	11 / 17 / 0	3
64	10 / 22 / 0	7	12 / 20 / 0	6
128	13 / 19 / 0	8	14 / 18 / 0	9
256	11 / 21 / 0	13	10 / 22 / 0	9
1,024	11 / 21 / 0	23	14 / 18 / 0	10

The check that failed before is the unplanted axes' false periods within 5σ of 1/(draws + 1): at 8 draws the plant of 1,024 e put 23 of 64 against an expected 64/9. Under the per-axis null it is 5 of 64, and the check holds.

15.3.3 *The scheduler's steps and the lookup table*

Doug's "increase the scheduler steps to n" and "implement the lut", built into the record machine (M10 and A13 of engine_table.md) and graded by `test/engine/record_table_test.cu` through `test/engine/record_table_test.sh`, run as `build/20260923_204007_record_table_test`. The alphabet is the 16-bit one, and every case runs 4,096 random lanes.

what	samples	result	settles
a table filled by running the ops over the whole alphabet, against the ops	x^2; $\lvert x - 30000\rvert + 100$	the table and the ops agree word for word, device and host	a pointwise step is exactly its table (proved on these two)
a table read through a table	$f(x) = 65535 - x$, $g(y) = \lvert y - 30000\rvert$, against the ops for $\lvert 35535 - x\rvert$	the two tables, one table composed on the host, and the ops all agree; f read through g differs	composition keeps program order: it regroups, it does not reorder
register reuse, on against off	a 21-step additive chain (a field, then ten constant-and-sum stages)	the same output, from 3 limbs reused against 21 kept	a chain needs only its live registers, and the step count n (1,024) is stored apart from the 256-limb file
the refusals	a 17-bit index on a 16-bit field; a table step naming a table that is not there	both refused at layout	a table cannot read past its source or outside the program's tables
all of it	the above	15 checks, 0 failed	the table step and register reuse are proved; which functions to tabulate is open (item 3 of kolmogorov_arnold.md)

15.3.4 *The root universal*

The tower with reversible lookup edges between its floors (M20 and A14 of engine_table.md; bijective edges in the stream, not read-only taps). Graded by `test/tower_edge_test.cu` through `test/tower_edge_test.sh`, run as `build/20260923_210432_tower_edge_test`, and by the sixth sim, `root_universal`, run as `build/20260923_211054_sim_root_universal` on camera-law volumes (signal 200 e plus a ramp of 1 e a column, 6 moving bodies of 150 to 400 e, read variance 3, fixed pattern 0 to 8). The edges are not in the crystal path: `engine.cu` passes none.

what	samples	result	settles
the edge test	one edge; stacked edges on one floor; an edge on every floor with the collapsed one; four bad edges; a clean edge after them	20 checks, 0 failed: every edge program rebuilds the exact lanes, an edge changes the crystal, a non-permutation, a width of 0, a width of 21 and floor 50 are each refused, and the clean edge still round-trips	an edge is a bijection the lower undoes, and a bad one is refused before any device work (proved)
the Bennett edges ("a dimensional expansion"), `build/20260923_213400_tower_edge_test`	$\|x\|$ of an 8-bit two's complement field and $x \geq 100$, each a 16-bit edge (8 bits of x, 8 of carrier), $(x, y) \to (x, y \oplus f(x))$; 64 lattices of $1 \times 8 \times 8 \times 8$ alternating the two on the collapsed floor; the two on floors 1 and 2; $\|x\|$ laid bare on 16 bits	25 checks, 0 failed (the 20 above and 5 new): round-trip on 64 of 64; f(x) read out of the carrier, x kept and the high bits passed, on 64 of 64; only the one coefficient touched on 64 of 64; floors 1 and 2 round-trip; $\|x\|$ laid bare refused, since x and $-x$ collide	a function that is not a bijection rides the reversible stream at the price of width, with no engine change (proved)
the root stays exact	48 volumes, 24 of $4 \times 12 \times 48 \times 48$ (81 edges) and 24 of $3 \times 13 \times 37 \times 41$ (90 edges), 6 floors each; 1 to 6 edges a volume, widths 1 to 12 bits, one 20-bit edge per extent; lift, code, wipe, decode, lower	plain exact 48 of 48, edged exact 48 of 48, crystals changed 48 of 48	the edges survive the whole crystal path (proved)
the fold preserves order	8 volumes of $4 \times 12 \times 48 \times 48$, 8-bit edges a and b on floor 1	a then b lays the crystal of the single table b(a(i)) on 8 of 8; b then a differs on 8 of 8; a on floor 1 and b on floor 2 differ on 8 of 8	edges on one floor fold into one table in their order; a floor between them blocks the fold (proved)
the price of an unfitted edge	8 volumes of $4 \times 12 \times 48 \times 48$, one keyed random permutation a floor, coded bits a voxel	below	exact is not free: the coder pays on the low floors, and on the collapsed floor the cost does not show at three decimals (measured); a fitted edge not measured

what	samples	result	settles
all of it	the above	`root_universal`: 1,268 checks, 0 failed	the root universal is built and proved in `tower`; its place in the `.kcr` is Doug's call

The price, coded bits a voxel (6.802 with no edge; the coefficients each floor holds in brackets):

width (bits)	floor 0 (110,592)	floor 3 (72)	floor 6, collapsed (1)	every floor
2	6.810	6.802	6.802	6.808
4	6.908	6.802	6.802	6.911
8	8.719	6.804	6.802	8.742
12	12.608	6.817	6.802	12.714

15.3.5 *The ask and the state*

The seventh sim, `ask_state` (M21 and A15 of engine_table.md; the terms are Doug's), run as `build/20260923_213902_sim_ask_state`: a qubit carried exactly as the answers of a complete ask, E_i = (I + v_i·σ)/4, on the host with no GPU work. Rationals are on the exact integer; the SIC's numbers are in Q(√3), with (√3)² = 3.

what	samples	result	settles
the two asks are complete	the rational tetrahedral ask, v = (±1, ±1, ±1)/2 with an even number of minus signs; the SIC, v = (±1, ±1, ±1)/√3	each sums to the identity, is positive, and has a frame c·I: |v|² = 3/4 and frame I (r = 4 Σ p_i v_i); |v|² = 1 and frame (4/3)I (r = 3 Σ p_i v_i)	the state is read back from the answers alone (proved)
state → answers → state	25 states: 21 pure (14 equator states from rational t, and rational points of the sphere), 4 mixed	25 of 25 exact for each ask; the pure on the boundary 21 of 21, the mixed inside 4 of 4	the complete ask is one to one on states (proved)
the crossings	the +x, +y and +z asks	(1 + r_a)/2 on 75 of 75 for each ask; weights onto +x 3/2, 3/2, −1/2, −1/2 and ½ ± ½√3	the crossing is linear with a negative weight: it is not classical total probability (proved)
the phase	the 14 equator states	all answer ½, ½ to the 0/1 ask; the complete ask tells apart 91 of their 91 pairs	the phase falls out of the ask (proved)
outside the valid set	(1, 0, 0, 0)	refused, no state; crosses onto +x at 3/2 and at ½ + ½√3	no probabilities come of a distribution no state gives (proved)
the asks cross into each other	the SIC's answers into the rational ask's	25 of 25 exact	(proved)
the valid set	the 455 distributions with denominator 12	55 states for the rational ask and 87 for the SIC; of the others, 104 and 72 still cross to probabilities on the x, y and z asks; recounted independently in exact fractions, the same	the valid set is stricter than the three axis crossings (measured)

what	samples	result	settles
all of it	the above	33 checks, 0 failed	the ask and the state are exact on the host; Doug's countable numbers are theory, not built, and his sparse ψ is built in `ka_psi` (below) (A15)

15.3.6 *Kolmogorov's inner function*

The eighth sim, `ka_psi` (A15 of engine_table.md; the full reading is in "Kolmogorov's inner function, exactly" of kolmogorov_arnold.md), run as `build/20260923_221011_sim_ka_psi` in about 3 s here (the first three rows' lines are unchanged from `build/20260923_215902_sim_ka_psi`, 114 checks): the inner function ψ of Braun and Griebel's Theorem 2.1 at γ = 10, exact on the grids and, at depth, a sparse sum Σ c_j γ^ − e_j with each e_j = β(L) an integer never expanded.

what	samples	result	settles
Sprecher's ψ (2.4), n = 2	ψ(0.58999), ψ(0.59)	2207/4000 = 0.55175 and 11/20 = 0.55, the paper's (2.5); m_r read with the empty product as 1 (the literal typeset form gives 0.501)	Sprecher's ψ is not increasing (proved)
its descents	the 99,999 neighboring pairs of D_5	10 descend, the first between 0.08999 and 0.09000; recounted independently in exact fractions, the same	(measured)
Köppen's ψ, (2.9) and (2.7) as printed, n = 2 and 3	every point of D_1 to D_5	the pair recursion equals the level recursion; strictly increasing; least gap γ^ − β(L); widest gap G_L of G_1 = 1/γ, G_L = ½G_{L − 1} − (s/2)γ^ − β(L), s = 8 for (2.9) and 7 for (2.7); recounted independently, the same	both readings are strictly increasing on the grids (proved)
every scale	symbolic to L = 60 (n = 2, β = $2^{60} - 1$) and L = 38 (n = 3, β = $(3^{38} - 1)/2$), both readings	the step keeps the order and the least gap stays γ^ − β(L) on 59 of 59 and 37 of 37 levels; the widest gap stays above the least, at most 2^ − (L − 1)/γ and within Lemma 2.3; the symbolic G_L equals the measured gap on 4 of 4	ψ extends to a continuous, strictly increasing function (proved)
a keyed deep point	depth 60 (n = 2), depth 38 (n = 3)	ψ of 55 and 57 terms (n = 2) and 36 (n = 3), its gap between the least and the widest	(proved)

what	samples	result	settles
the separation, Lemma 3.4	$\xi = \Sigma\ \alpha_p\ \psi(d_p)$ on every point of D_k^n: n = 2 at k = 1, 2, 3; n = 3 at k = 1, 2 (up to 10^6 points)	least gap at k = 1, in units of $\gamma^{\wedge} - \beta(k+1)$, of 9.0999099999991… (n = 2, between (0.1, 0) and (0, 0.9)) and 9.0999099991… (n = 3), against the margin of 10; from k = 2, 49.50499999505… and 500,049.5000005… (n = 2) and 5,000 + 5×10^{-15} (n = 3); recounted independently in exact fractions, the same pairs	the lemma is false at k = 1 (proved)
the proof's step (3.11)	α cut at $r \leq k$, the same grids	least gap 10, 50, 500,050 (n = 2) and 10, 5,000 (n = 3) units; recounted, the same	(3.11) holds, on the margin at k = 1; the tails, which do not annihilate it, still pull μ below (proved)
the corrected bound and the narrow ramp	$\lvert\mu_k\rvert \geq \gamma^{\wedge} - n\beta(k) - \Sigma_{p \geq 2} \varepsilon_{k,p}$, ramp $\gamma^{\wedge} - (\beta(k+1)+2)$; symbolically k = 1 to 57 (n = 2) and 1 to 35 (n = 3)	supports disjoint on 57 of 57 and 35 of 35 levels; with the paper's ramp $\gamma^{\wedge} - \beta(k+1)$, on 0 of 57 and 0 of 35; the narrow ramp separates every grid tested, k = 1 included	Lemmas 3.7 and 3.8 hold at every k with the repair (proved, given (3.11))
the shifts	m + 1 = 5 (n = 2) and 7 (n = 3)	a point sits in at most one gap a coordinate, and at least 3 and 4 shifts put it in a cube; with m + 1 = γ, two shifts share a gap	why $\gamma \geq m + 2$ (the sim's check)
all of it	the above	152 checks, 0 failed	the outer functions' contraction (Theorem 3.3) and ψ at the off-grid shifted points are not built

15.3.7 *The exact integer: width, ladder and division*

This tree's exact integer (`engine/base/no_rounding/exact_integer.{c,h}`, M1 of engine_table.md) has an open width, a multiplication ladder, division, Newton's division and a Lehmer gcd, and the key machine has the division as four record operations (M10). Doug, in order: "You have division", "2s compliment", "Mulmask exact", "add that to the exact arithmetic", "It makes no sense to not have infinite division with infinite mul", a link to Schönhage–Strassen, "We use karstsuba", "Expand the header to accept any n bits power of 2", "Static assert". The work is orior's. The engine build compiles the exact integer from this tree by default, and this tree is the source orior's engine is to be copied from. Every row below is run here on the merged source.

what	samples	result	settles
the width	1, 8 and 64 limbs compiled with declared floors of 9, 76 and 616 digits; 8 limbs with no floor; the tests at 128 limbs, 4,096 limbs and 4,194,304 bits	the three compile; the undeclared floor is refused at compile time (`exact_integer.h`); every test below passes at all three widths	any power of two from 32 bits up, with no ceiling, and a floor declared, never assumed (proved at those widths)
the ladder: long multiplication, Karatsuba from 32 limbs, the Schönhage–Strassen transform from 8,192	`test/exact_transform_test` (`build/20260923_234836_exact_transform_test`, 4 checks, 0 failed at each width, 165 s): 24, 54 and 63 products, balanced and unbalanced, keyed and all ones, across every rung boundary up to half the width	the ladder, and the transform alone, equal a long multiplication kept in the test on every product	the product is exact on every rung (proved on the trials)
products divide back	the same products	each product divides back to its factors, exactly and with remainder, and the gcd holds a factor	(proved on the trials)
Newton's division: the reciprocal grown at half precision, one Newton step, corrected exactly	24, 54 and 63 divisions, divisors to half the width	numerator = quotient · divisor + remainder, the remainder below the divisor and signed as the numerator, equal to the dispatched division	Newton's rung gives what long division gives (proved on the trials)
the rungs kept apart	the same test with the transform, and then also Newton, raised past every size (`build/20260923_235121_exact_transform_test` and `build/20260923_235548_exact_transform_test`)	4 checks, 0 failed at each width, both builds	Karatsuba alone and long division alone are exact too (proved on the trials)
long division, Knuth's algorithm D in base 2^32	`test/exact_divide_test` (`build/20260923_234731_exact_divide_test`, 9 checks, 0 failed, at 128 limbs, 7 s): 200,000 trials of edge-shaped limbs (0, 1, 2^31 − 1, 2^31, 2^32 − 1 and 2^32 − 2 three times in four, a keyed draw otherwise), numerators up to 8 limbs; 4,000 trials at widths drawn up to 127 limbs	numerator = quotient · divisor + remainder, |remainder| < |divisor|, the remainder signed as the numerator, on every trial	the division is exact, rounding toward zero (proved on the trials)
the add-back step	a numerator and divisor built so the top-limb estimate runs one high	holds; an instrumented copy confirmed the branch runs	the rare branch is right (proved on the case)
exact division: the odd part's inverse by Newton's step x(2 − dx), a multiply and a mask	20,000 products of drawn quotients and divisors up to 64 limbs, half the divisors with at least eight twos	the quotient returned on every product; one past each product refused as `ORIOR_EXACT_NOT_EXACT`	(proved on the trials)

what	samples	result	settles
Lehmer's gcd (Knuth's algorithm L), which replaced Stein's binary gcd after its bit-at-a-time shifts stalled the 4,194,304-bit test	4,000 pairs with a drawn shared factor, each part up to 32 limbs; 4,000 more against Euclid's gcd run on the long division	the gcd divides both, the shared factor divides it, and the cofactors' gcd is 1; it equals Euclid's on every pair; gcd(0, x) = |x| and gcd(0, 0) = 0	(proved on the trials)
a zero divisor	both divisions	refused as `ORIOR_EXACT_BY_ZERO`	(proved)
the record operations: quotient, remainder, gcd, exact quotient (ops 11 to 14)	`test/engine/record_divide_test` (`build/20260923_234739_record_divide_test`, 15 checks, 0 failed, 58 s): 4,096 lanes of signed 160-bit numerators in the 64-limb register file, 512 lanes of 2,048-bit numerators in the 256-limb file	the device equals the host word for word; the division identities, the gcd and the exact quotient hold on every lane; a zero divisor and an inexact division refuse the lane on both; 3^40 divides exactly by 3^20 to 3^20; a quotient reading a later step is refused at imprint	the key machine divides exactly, on the device (proved on the lanes)
the sims on lowest terms	`sim_rational.h` reduced at every width by the gcd and exact division; `ka_psi` (`build/20260923_235412_sim_ka_psi`) and `ask_state` (`build/20260923_235440_sim_ask_state`)	152 of 152 and 33 of 33, unchanged; `ka_psi` prints its values as fractions, 2207/4000 and 11/20	the verdicts do not depend on the reduction (measured)

The rungs' timings at 4,194,304 bits, seconds per call on balanced operands, on this host (x86-64, MSVC -O2), one reading each (**measured**). "A" is `build/20260923_235121_exact_transform_test`, with the transform kept off the ladder so the ladder is Karatsuba alone. "B" is `build/20260923_235548_exact_transform_test`, with the transform and Newton both kept off. At this width a call on small operands costs about 1e-4 s for a product and 4e-4 s for a division, because each call touches the whole width.

limbs	long (B)	Karatsuba (A)	Karatsuba (B)	transform (A)	transform (B)
2048	3.778e-3	9.537e-4	1.053e-3	1.634e-3	1.219e-3
4096	1.481e-2	3.771e-3	2.826e-3	3.845e-3	3.267e-3
8192	6.199e-2	8.055e-3	8.731e-3	9.175e-3	9.351e-3
16384		2.435e-2	2.466e-2	2.387e-2	1.875e-2
32768		9.734e-2	7.744e-2	6.453e-2	5.058e-2
65536		2.301e-1	2.306e-1	1.244e-1	9.928e-2

limbs, a 2n-limb numerator by an n-limb divisor, on Karatsuba alone (B)	long division	Newton's
2048	4.757e-3	7.614e-3
4096	1.833e-2	2.311e-2
8192	7.256e-2	6.645e-2
16384	2.841e-1	1.966e-1
32768	1.116	5.746e-1
65536	4.302	1.719

The shape holds on both runs. Karatsuba leads the transform at 8,192 limbs, and the transform leads from 16,384. Long division leads Newton's from 32 to 4,096 limbs, and Newton's leads from 8,192. Below 32 limbs both sit at the width's floor. That is where the header puts `ORIOR_EXACT_TRANSFORM_LIMBS` and `ORIOR_EXACT_NEWTON_LIMBS`, both 8,192. The ratios move between readings. Karatsuba over the transform reads 1.020 and 1.315 at 16,384, 1.508 and 1.531 at 32,768, and 1.850 and 2.323 at 65,536. Long division over Newton's reads 1.092 at 8,192, 1.445 at 16,384, 1.942 at 32,768 and 2.503 at 65,536. orior's run, in the header, reads 1.44, 1.55 and 1.85 for the transform and 1.21, 1.06, 1.81 and 2.44 for Newton's. Two readings of one code path here differ by up to 1.198× (the default build's dispatched division and Newton's at 65,536, the same code), and one column read twice differs by 1.257× (Karatsuba at 32,768; the transform at 65,536, 1.253×). A ratio within about 1.3× of 1 is therefore not settled by one reading, and neither is the second decimal of any ratio here.

15.3.8 *Chaitin's Ω*

`engine/sims/chaitin_omega` (M19 of engine_table.md) brackets Chaitin's halting probability for Tromp's binary lambda calculus. The machine reads a closed term, self-delimited in de Bruijn form, and halts where the term has a normal form. The codes are prefix free, and Ω, the sum of 2^−|t| over the halting terms, is therefore a probability. Its first n bits settle the halting of every program of n bits or fewer. For that reason no machine computes them all.

It runs on the device, one term a thread (`omega_kernel`), on 16 host threads with the argument `cpu`, and, given L alone, on the engine's record machine (`omega_engine`). The runs here:

run	budgets	build	checks
L = 30, the host	2,048 steps and 2,048 tokens	`build/20260924_004819_sim_chaitin_omega`	13, 0 failed
L = 33, the device	2,048 and 2,048	`build/20260924_004740_sim_chaitin_omega`	14, 0 failed
L = 36, the device	2,048 and 2,048	`build/20260924_004859_sim_chaitin_omega`	14, 0 failed
L = 36, the device	256 and 256	`build/20260924_004941_sim_chaitin_omega`	13, 0 failed
L = 44, the device	256 and 256	`build/20260924_005020_sim_chaitin_omega`	13, 0 failed
L = 16, the engine, a tessera job	2,048 and 2,048 as printed	`build/20260924_024217_sim_chaitin_omega`	16, 0 failed
L = 16, the host	2,048 and 2,048	`build/20260924_024434_sim_chaitin_omega`	13, 0 failed

The engine run at L = 16 settles all 226 closed terms as halting, and it gives the host's L = 16 table line for line: 0.11664483… ≤ Ω ≤ 0.12600284…, with the lower bound equal to the halted mass plus the normal forms past L, and the upper to the lower less those normal forms plus the closed mass past L, both exactly (recomputed here from the printed binaries). Its checks are the host's 13, the job's admission and release, and one of its own: every term the engine settled, run again by `omega_run`, gives the same fate at the same step and the same normal form. No longer engine run is quoted here; every longer bracket is the device's or the host's.

The host path has no device to compare against and lacks the cross-check. Budgets under 2,048 steps and tokens drop the strict busy beaver check, and both points are described below. The host runs of the day before give the same fates and bracket at L = 30 and 33. They are `build/20260924_001129_sim_chaitin_omega` (L = 30, 12 checks, 0 failed), `build/20260924_001207_sim_chaitin_omega` (L = 30 at 200,000 steps and 16,384 tokens, 12 checks, 0 failed) and `build/20260924_001712_sim_chaitin_omega` (L = 33, 12 checks, 0 failed). The device's L = 33 run gives the host's L = 33 table line for line, every fate, bound and busy beaver, record holders included. These device runs shared the device with orior's 54-bit run, and their times are not quoted. L = 44 at 256 steps and tokens was run again on the idle device (`build/20260924_005724_sim_chaitin_omega`, an RTX 3070, 17,664 threads, 13 checks, 0 failed): the same fates in 28,080 ms, 120.8 million terms a second, against orior's 28,088 ms (**measured**). A reading of the threads (384 a multiprocessor ahead of 512 and 768 at 38 bits, the kernel bound by the cache) is not run here. The runs before the typed fate (`build/20260924_000606_sim_chaitin_omega` and `build/20260924_000643_sim_chaitin_omega`, 11 checks, 0 failed) gave the same fates and the same bracket at L = 30. A run before the growth proof (`build/20260923_235507_sim_chaitin_omega`) had 587 terms open and an upper bound of 0.12599323…. The hand conversion of that bound, 0.12597, was wrong and is corrected in the README.

A term is proven to grow forever (`omega_grows_forever`) as follows. Since the watcher's last checkpoint, every step was a head step whose redex sat at spine position p or deeper. Then the checkpoint's spine part S at any position up to p was the only part reduced, and it never became a lambda eating an argument outside it. If S now stands deeper on the spine, S head-reduced to S B, and the same steps take S B to S B′ B and on without end. So S has no head normal form, neither has the term, and normal order never halts. The sim checks it on a term the earlier run left open: 01000101101010000101101010 grows forever.

A term left open, or past the space, is also tried for a simple type (`omega_simply_typed`, `OMEGA_TYPED`): Hindley's unification with an occurs check. A simply typed term is strongly normalizing (Tait 1967): it has a normal form and halts. A typed halt adds its mass to the lower bound. It does not write the normal form, and the busy beaver at its length stays unsettled. The same fact gives a check: no term proven to loop or to grow forever may type. At L = 30, 33 and 36, 0 terms are proven to halt by a type, and no loop or growth term types. So none of the open terms is simply typed, and the bracket is unchanged. At L = 44 one term is: the first typed halt.

The device takes every decision the host's run takes, in the same order, under budgets of at most 256 steps and 256 tokens, in a room of twice the token budget. A run is deterministic, and its budgets only stop it. A halt, loop or growth found inside the device's budgets is therefore found at the same step inside larger ones. A term that reaches a device budget smaller than the run's, outgrows the room, or halts in a normal form too large to key, is handed back. The host runs it under the full budgets. The fates are therefore the host's fates. A check holds it on every run: the host's own run of every term through 30 bits (or through L, when L is less) must give the device's fates and busy beavers, record holders included. At L = 36, 2,048 steps and tokens handed 24,501 terms to the host, and 256 handed 75.

`--ledger <path>` plans the device's run as jobs, each length cut every `2^28` terms in rank order. Each finished job's exact tally is one flushed line: the budgets, the length, the job's first rank and count, the six fates, the contradictions, and the two busy beavers with their champions' ranks. A rerun under the same budgets keeps every whole line and runs only the missing jobs. A line

with no line end, or one whose fates do not sum to its count, is not kept. Before a job's line is appended, a cut last line is given its line end. A test here at L = 31 (`build/omega_ledger_cut_verify_20260924_005246.tsv`) finds a gap in that (**measured**):

1. A first run writes 27 jobs.
2. The last line is cut inside its last field, the normal form record holder's rank, 478256 cut to 47825.
3. The rerun runs that one job again, keeps 26, and appends the whole line after the cut one, now ended.
4. A third run keeps all 27, but the cut line comes first and is taken: the busy beaver at 31 still reads 267 bits, but its record holder is the term of rank 47,825, not 478,256, and every check passes (14, 0 failed), since the cross-check stops at 30.

A cut anywhere before the last field leaves too few fields and is not kept. The fates and the bracket are safe. Only a record holder can be wrong. This is a known defect of the ledger, and it is not fixed: Doug has asked for Ω to be run by the engine itself, the record machine, with no step, token or width limit. The sim's ledger may then be replaced instead of patched.

what	samples	result	settles
the enumeration	every closed term through L bits, unranked from exact counts	the counts equal a parse of every code through 22 bits; every unranked term through 16 bits is one term of its length	the terms run are all the closed terms (proved against the parse through 22 bits; from there to L, the counts' recurrence)
the device against the host	every term through 30 bits run by both on each device run	the same fates and busy beavers, record holders included, on every device run here	the device's fates are the host's (proved through 30 bits by the check; past 30, theory: a deterministic run that the budgets only stop, and every budget reached handed back)
the fates	647,463 closed terms, run by normal-order reduction	645,942 reach a normal form (a halt, proved); 934 return to a state they were in before under Brent's watcher (a loop, proved); 213 grow forever (proved); 374 stay open, 86 out of steps and 288 past the space at 2,048 steps and tokens, 47 and 327 at 200,000 steps and 16,384 tokens	647,089 of 647,463 settled, 99.94% (measured); the larger budget settles no more
the typed fate	every open and outgrown term, and every loop and growth term, tried for a simple type	0 typed at L = 30, 33 and 36; 1 at L = 44; no loop or growth term types	the typing adds nothing to the bracket through 36 bits (proved: Hindley's algorithm finds a type whenever one exists)

what	samples	result	settles
the fates at L = 33	3,910,730 closed terms, 2,048 steps and tokens	3,900,966 halt; 5,663 loop; 1,180 grow forever; 2,921 stay open, 663 out of steps and 2,258 past the space; the device and the host alike	3,907,809 of 3,910,730 settled, 99.93% (measured)
the fates at L = 36	24,325,850 closed terms, 2,048 steps and tokens, then 256 and 256	at 2,048: 24,253,187 halt, 39,216 loop, 9,594 grow forever, 23,853 open (5,112 out of steps, 18,741 past the space); at 256: 24,253,078, 39,201, 9,070 and 24,501 (4,767 and 19,734)	99.90% settled at 2,048 (measured); the smaller budgets leave 648 more open
the fates at L = 44	3,392,860,908 closed terms, 256 steps and 256 tokens	3,379,796,187 halt; 1 halts by a type; 6,161,966 loop; 1,223,662 grow forever; 5,679,092 open (888,546 out of steps, 4,790,546 past the space)	99.83% settled (measured)
the budgets	L = 36 at 2,048 and at 256 steps and tokens	the smaller budgets lower the lower bound by 3.19×10^{-9} ($2^{\wedge} - 28.2$) and raise the upper by 2.18×10^{-8} ($2^{\wedge} - 25.5$), against a gap of 0.00240096… ($2^{\wedge} - 8.7$)	the budgets barely move the bracket at 36 bits (measured, from the exact dyadics)
the lower bound	the halted mass, plus the normal forms past L, which halt unrun	$\Omega \geq$ 0.12254295… at L = 30, 0.12312353… at 33, 0.12358892… at 36 and 0.12443857… at 44 (256 and 256)	(proved)
the upper bound	the halted mass, the open mass and the closed mass past L, bounded by the parse's tail with directed rounding	$\Omega \leq$ 0.12599214… at L = 30 (both budgets), 0.12599095… at 33, 0.12598988… at 36 and 0.12598796… at 44 (256 and 256), each the exact sum of its three parts	(proved)
the bits	the two bounds, exact 64-bit dyadics	lower < 1/8 $\leq$ upper at every L run, which puts Ω at 0.00… in binary	2 bits of Ω proved; the third is not settled at L = 44
BB λ	the busiest normal form at each length	22, 24, 26, 30, 42, 52, 44, 58, 223, 160, 267, 298 and 1812 at 21 through 33 bits, and n at every length from 4 through 20 with a closed term: BusyBeaverWiki's values (OEIS A333479) at every length through 33	under any budgets, a check at every length through the smaller of L and 33 that the run's value is at most the published one, and equal to it where every term halted; at 2,048 steps and tokens or more, equal at every such length. A length with an open term holds only a lower bound, and the published value is the true maximum: meeting it means the run reached the record holder (proved against the published values)

From L = 30 to L = 44 the bracket narrows from 0.00344919… to 0.00154938… wide, with 1/8 still inside it. At 44 bits the lower bound is 0.00056142… below 1/8 and the upper 0.00098796… above it. The upper bound falls only by proven non-halting mass, and from 30 to 44 bits it fell by 0.00000418…, against the 0.00098796… it stands above 1/8. The lower bound rose by 0.00189562… over the same lengths (measured). orior reports a 40-bit run, made before the growth proof, that still leaves 1/8 inside the bracket. It is not run here, and its numbers are not quoted. A 54-bit ledger run was stopped at length 45 and gives no bracket.

The sources. The papers' titles, venues and pages were checked here against Crossref, and the table and the program at their pages; Barendregt's book was not checked there. Tait, "Intensional interpretations of functionals of finite type I", Journal of Symbolic Logic 32(2), 198–212, 1967: a simply typed term is strongly normalizing, the typed fate's basis. Wadsworth, "The relation between computational and denotational properties for Scott's D∞-models of the lambda-calculus", SIAM Journal on Computing 5(3), 488–521, 1976, and Barendregt, *The Lambda Calculus: Its Syntax and Semantics* (North-Holland, 1984): a term has a head normal form exactly when head reduction ends, the growth proof's basis. Thiemann and Sternagel, "Loops under Strategies", Rewriting Techniques and Applications 2009, 17–31: nontermination proved by a loop under a fixed reduction strategy, the kind of argument the growth proof makes for normal order. BusyBeaverWiki's BB λ table and OEIS A333479: the exact values 22 through 1812 at 21 through 33 bits, and 327,686 at 34. Tromp's `BB.lhs` (github tromp/AIT, `BB/`): it proves loops by a repeated redex in a term's history and by two self-replication shapes, `W W → H[W W]` and `W _ W → H[W _ W]`, reduces within 42,000,000 tokens, and stops at 36 bits.

15.4 2026-09-24

15.4.1 *π's hex digits on the engine*

`pi_tower`'s BBP run on the record machine (M19 of engine_table.md): each term a lane, a sweep one `cycle_record_run` of up to 70,656 lanes (the RTX 3070's resident threads), the sum reduced by a pair program on the same machine, and only the bits the error decides printed. Run from the source in a scratch build directory (`$TEMP/orior_carry/build_pi`), not under `build/`. Neither tower, T or T^{-1}, is in the program (A13).

what	samples	result	settles
the default request, resolutions 2^1 to 2^100, then the engine	hex positions 0, 999,999, 1,000,000, 9,999,999 and 99,999,999, and 9 for the cell at 2^100	41 checks, 0 failed; 243F6A8885A308D3…, 26C65E52CB4593…, 6C65E52CB459350050E4BB1…, 17AF5863EFED8D… and ECB840E21926EC…, each Bailey's published digits ("The BBP Algorithm for Pi", 2006, table 1 and its text), with 117 to 135 bits certified; the cell at 2^100, 0x5a308d313198a2e0, the exact turn's	the engine computes π's hex digits at a position without the digits before it (proved at those positions)

what	samples	result	settles
the rate	10^7 and 10^8 terms	142 sweeps in 0.371 s; 1,416 sweeps in 3.887 s, 340 steps a term lane on 49 register limbs	about 2.6e7 terms a second at 27-bit positions (measured)
2^(2^30) cells, the deep path	hex position 268,435,440, 268,435,441 terms	3,800 sweeps in 10.638 s, 2.52e7 terms a second; 127 bits certified; the cell's low 64 bits 0x8293097a8232c37f	the depth-2^30 turn read on the engine alone (measured; no second source checks it)
2^googol cells, the deep path	n = 10^100, hex position 2.5e99; P = 3 × 10^100 + 64 bits, and a host turn would need SIM_EXACT_LIMBS = 2^331	46,844,928 of 2.5e99 terms in 663 sweeps and 64.043 s, 7.31e5 terms a second; stopped, no digit printed	at the measured rate the sum takes about 1.08e86 years; the cost is linear in the position (measured)
the tessera job	the default request and 2^(2^30); 2^googol	declared 3,413,540 bytes, peak 221,413,376 on both; 2^googol declared 11,542,512, its peak not read, since the run was killed	the declaration counts the sim's buffers and not the record kernel's per-thread file (measured)

Note. D. H. Bailey, "The BBP Algorithm for Pi", 17 September 2006 (davidhbailey.com, `dhbpapers/bbp-alg.pdf`), read. Its Table 1 (p. 6) prints 26C65E52CB4593 at 10^6, 17AF5863EFED8D at 10^7 and ECB840E21926EC at 10^8, the digits in the first row above. Its text (p. 4) prints 6C65E52CB459350050E4BB1 from position 1,000,001 and calls it the first 24 hex digits there. The string holds 23. The decimal list printed beside it reads 12 in its nineteenth place, where the string reads E, which is 14. The engine's run gives E there with the string's other 22 digits, and its 117 to 135 certified bits cover the 92 bits of 23 hex digits (Derived). The row above keeps the string.

15.4.2 *The Gaussian step on the record machine*

The Gaussian step is BBP's ×16 taken as eight floors of (a, b) ↦ (a − b, a + b), z = a + bi times 1 + i (A16 of engine_table.md; "2 then 1").

- Graded by `test/record_gaussian_test.cu` through `test/record_gaussian_test.sh`, run from the working tree, uncommitted, as `build/20260924_222402_record_gaussian_test`.
- The program: two signed 24-bit fields, each floor one DIFFERENCE and one SUM, 18 steps, and every floor an output.
- The lanes: 4,096, every pair of the five edge values (0, −1, 1, −2^23 and 2^23 − 1), and keyed draws for the rest.

what	samples	result	settles
the program	18 steps over 2 fields	imprinted, laid and loaded; floor k is 24 + k bits wide, 25 to 32	keymath widens each floor by one bit (proved)

what	samples	result	settles
device against host	4,096 lanes	equal word for word	(proved on the lanes)
every floor against z(1 + i)^k	4,096 lanes × 8 floors, the powers taken from their polar form	equal on every lane and floor	the floor is the Gaussian step (proved on the lanes)
four and eight floors	the same	−4z and 16z on every lane	eight steps are ×16, one hex digit (proved on the lanes)
parity	every floor	the two registers share their parity on every lane	each floor's image lies in the ideal (1 + i): one bit dropped a floor (proved on the lanes)
the width at floor 8	the same	values in [−2^27, 2^27), 28 bits, with −2^27 reached; keymath carries 32	the values grow half a bit a floor and the imprint a whole bit; the per-floor widths are derived in A16 (proved on the lanes at floor 8)
all of it	the above	8 checks, 0 failed	the step runs exactly on the engine's record machine; it is in no engine module and not in `pi_tower`
A16's pole algebra, in floating point	a scratch script (`gaussian_check.py` in a temporary directory, not in the tree); 1,001 points on [0, 1]; Simpson's rule at 200,000 panels	partial fractions and residues within 2.2e-15 of BBP's y-form, and the x-form within 3.6e-15 of its reduced form; the numerator below 5.4e-15 at ±i and e^{±3iπ/4}, and 16 and 22.6 at the other four roots; the integral within 3.3e-14 of the float π; the three pieces −2 ln 2, +2 ln 2 and π, each within 1.3e-14; the corners' widths at w = 24 are 25, 26, 27, 27, 28, 28, 28 and 28 bits	the hand algebra holds in floating point (measured; derived in A16, not run on the engine)

15.5 2026-09-25

15.5.1 *keymath's widths by linear forms, and the record tests on tessera*

keymath now carries every register as a linear form over atoms and takes the fewer of the operation's own width and the form's bound (A16 of engine_table.md; "do the keymath rule"). The eight record tests were moved onto tessera (the tests go through the scheduler).

- Each test is one job, submitted before its first device work and released at the end, its daemon built beside it by `maint/engine/tessera_build.sh`.

- The eight were built and run one at a time at below-normal priority, from the working tree, uncommitted.
- An earlier run the same morning, under a narrower rule for a SUM and a DIFFERENCE over the same two registers, gave the Gaussian test the same widths.

what	samples	result	settles
Gaussian	`build/20260925_012737_record_gaussian_test`; 4,096 lanes, 8 floors	11 checks, 0 failed; widths 25, 25, 26, 26, 27, 27, 28 and 28, and some lane fills each one	the linear forms give 24 + ⌈k/2⌉, half a bit a floor, as the values grow (proved on the lanes)
guide	`build/20260925_012900_record_guide_test`	12 checks, 0 failed	the worked example's widths hold
table	`build/20260925_013022_record_table_test`	16 checks, 0 failed	(proved on the lanes)
divide	`build/20260925_013145_record_divide_test`	20 checks, 0 failed	its pinned widths hold under the forms
bitwise	`build/20260925_013306_record_bitwise_test`	25 checks, 0 failed	the narrowed widths 32, 20, 20, 32, 41, 41, 32 and 32 hold; the 700-floor stack and the lifting run
boundary	`build/20260925_013426_record_boundary_test`	49 checks, 1 failed: the ring law ring_0 + 6n(1 − 2^{−ℓ}). The ring by floor is 1,536, 1,664, 1,696, 1,712, 1,720, 1,720, 1,712, 1,696 and 1,600. 4,096 of 4,096 lanes are rebuilt exactly, their heap mirrored at every floor and inside the ring	the ring is now ring_0 + n + 2n(1 − 2^{−ℓ}) (E4); the mirror law and the widest-at-the-crystal law hold
boundary, the law set	`build/20260925_020556_record_boundary_test`	the forward ring law set to ring_0 + n + 2(n − n/2^ℓ) for ℓ ≥ 1; the same ring by floor; 49 checks, 0 failed	every record test passes under the linear forms (proved)
coherence	`build/20260925_013651_record_coherence_test`	15 checks, 0 failed; widest exact register 73 bits; 393,216 of 393,216 lane-widths agree three ways, and 73,728 of 73,728 for the odd divisors	(proved on the lanes)
order	`build/20260925_013814_record_order_test`	18 checks, 0 failed	(proved on the lanes)

what	samples	result	settles
tessera	the eight jobs	declared or reserved: 148,013,056 (Gaussian, its peak kept from the earlier run), 24,368, 606,208, 720,896, 1,048,576, 33,554,432, 524,288 and 8,658,944 bytes; peaks 148,013,056, 143,818,752, 148,013,056, 613,584,896, 250,773,504, 647,139,328, 219,316,224 and 158,498,816	every job is admitted and released; each peak passes its declaration, which counts only the test's own buffers (measured)

15.5.2 *Finding x on floor 2*

Finding x in a for less than half of a ("say we want to find x and x is on floor 2, we have the knf, we only need to &&"; E4 of engine_table.md).

- Graded by the sim `floor_match` (engine/sims), run from the working tree, uncommitted, as `build/20260925_015942_sim_floor_match`, a job on tessera.
- a is one 64^3 camera-law frame, n = 262,144 samples. Floor 2 is the engine's: the tower lifts the frame, and the crystal's 16^3 corner is lowered as its own tower.
- The index is floor 2 laid once as 16 bit planes of 128 words. A query ands the planes, lowest bit first, and a word stops once its mask is empty.

what	samples	result	settles
floor 2	m = 4,096 = n/64 values, 147 distinct	equal to the host's two-level lifting at 4,096 of 4,096; every value inside the 16-bit lane	the engine's floor 2 is the lifting's (proved)
the planes	16 × 128 words	every bit of every floor-2 value stored	(proved)
values on floor 2	1,024 drawn at keyed positions	1,024 of 1,024 match sets equal the host's scan; 56.78 positions found a query on average, 83 at most; 1,173.85 plane words read on average, 1,340 at most	every occurrence found and no other (proved on the queries)
values not on floor 2	1,024 drawn over the lane	1,024 of 1,024 found nowhere; 806.20 plane words read on average, 1,184 at most	(proved on the queries)
the reads	the 2,048 queries	at most 1,340 plane words, 5,360 bytes, against half of a, 131,072 samples, 262,144 bytes; without the early stop 2,048 words, n/128	a query reads about 1% of half of a (measured); the lift reads all of a once to build the index
all of it	the above	11 checks, 0 failed; tessera declared 3,956,736 bytes, peak 148,013,056	the knf is not used: the planes are the index

15.6 2026-09-27

15.6.1 *The energy detector's period lands on a divisor*

The energy detector, `sims/art/periodic_energy.h`, which `fixed_pattern` and `classify_reject_recover` call, returns the period of the highest dispersion ratio. The classes are the period's phases, and the ratio is the energy between them per degree of freedom over the energy within them per its own.

Measured. The detector's Python route (`measure/periodic_energy.py` on orior's `python_engine` branch) agrees with the header line for line on 12 cases, 0 failed. On one of them the highest ratio is a divisor of the addend's period.

- The case: 3,072 values, each a scene value drawn from 0 to 199, plus a hum over 64 phases drawn from −40 to 40, plus noise drawn from −20 to 20.
- `energy_recover` returns period 2 at 14.503. The ratios, equal on both sides, rank 2 at 14.503, 8 at 9.048, 4 at 8.243, 32 at 7.657 and 64 at 7.156. The four periods above 64 each divide 64.

Derived. With independent Gaussian values of one variance and no structure, the ratio at period p is Fisher's F with $p - 1$ and $L - p$ degrees of freedom, and its spread narrows as p grows. The ratios of different periods are therefore not on one scale, and the period of the highest ratio need not be the period whose ratio is least likely without structure.

Computed, a pure Python run from the F distribution's mean and variance over independent Gaussians at L = 3,072: the mean is 1.0007 at periods 2 and 64, and the standard deviation is 1.4158 at period 2 and 0.1802 at period 64. The 14.503 at period 2 stands 9.5 standard deviations above the mean, and the 7.156 at period 64 stands 34.2.

The sims' own readings hold where the addend dominates: `fixed_pattern` reads period 64 at 179.565 against a band top of 5.241 (the sims). The band is drawn for the highest ratio over every period, and it tests the pick without changing which period is picked.

A candidate for testing (not built): a band for each period, drawn from the same shuffles, and the period whose ratio stands highest over its own band top, compared as exact ratios by cross multiplying as the period reading's margin is. It changes what the sims recover and waits on a posit and Doug's word.

Prior art, cited from knowledge. A. Schwarzenberg-Czerny, "On the advantage of using analysis of variance for period search", Monthly Notices of the Royal Astronomical Society 241 (1989), 153–165. It folds a series at a trial period into a fixed number r of phase bins and scores the fold by the analysis of variance statistic, which follows F with $r - 1$ and $n - r$ degrees of freedom when the series holds no signal. With r fixed, the degrees of freedom stay the same across trial periods. The detector here takes r as the period itself.

16

THE ENGINE'S BOUNDARY

Purpose: Two rulings on what the engine may hold, with the dates they were made and what prompted them. No later build crosses either.

Scope: everything under `engine/`.

16.1 THE ENGINE CARRIES NO CELL TRACKING (23 SEPTEMBER)

Never put cell-specific tracking into the engine. Doug had said it at length before this ruling.

Restated "absolutely anything to do with tracking microorganisms goes into cell tracking, not into the engine." Basins (binomial_basins' steepest-ascent cut, basin_overlap) are tracking, a concept only: they live in cell_tracking/src and drive the engine's generic calls (engine_residual, grow_leaves, linking). They were deleted from the engine on his order, and they are not to be put back there; the question is closed.

- The engine holds math only: trees, cuts, overlaps, fields, exact integers. Nothing in engine/ may name or know about cells, divisions, daughters, key cells, moves, merges, appear/disappear, or grading against the answer key.
- Cell semantics and oracle grading go in cell_tracking/. They compose the engine's generic outputs there.
- Before adding any field, kernel or request member under engine/, check it names a mathematical object instead of a biological one. If unsure, ask Doug first.

What prompted it: in the 9c/9d overlap work, max_tree got

- the division_voxels and key_voxels requests;
- the key_parted, key_caught, key_present, key_apart, key_cells_present and key_cells_alone fields, and the disappear, appear, divisions and merges fields;
- the division and key-cell kernels.

engine/base/oracle/score_sample also got the key-division and key-cell grading.

16.2 THE ENGINE IS OPTIMIZED FOR NO SCALE (23 SEPTEMBER)

"what we want to avoid is optimizing the engine for any particular scale, which is what you have been consistently doing that I have been having you rip out over and over, it's dangerous".

- The engine is optimized for no scale. Never write a size, spacing, binomial order, entropy window, jitter width, cell size, voxel size, chunk size tuned to data, or memory fraction into engine/. Each comes in the request or is read from the data.
- A word's width or a file format's specification is not a scale. Where a word is too narrow, raise a request error or report `needed_bits`. Never round, clip, or cap to fit.
- The engine table's "scale audit" (engine_table.md) lists every scale still in the machine, as debts. Check it before adding any constant under engine/. A new constant is added to the audit, never hidden.
- The cell table and the engine table stay separate documents. They are optimized against each other only through their interface sections, never merged.

17

SCRIPTURA'S BLOCKS

Purpose: The one rule on what scriptura's SWAR memory scans may be handed, and why it settles the over-read question.

Scope: `scriptura_compare`, `scriptura_length`, `scriptura_find`, `scriptura_copy`, `scriptura_fill` (M17 of engine_table.md).

Ruled : "never pass unaligned memory to this, ever"; "the solution is to always nul term memory blocks you hand it".

- Every block handed to the scriptura word scans is 8-byte aligned and NUL-terminated. A string literal is not guaranteed aligned: copy it into an aligned block first.
- Used this way, `scriptura_compare(a, b, length of b + 1)` returns strcmp's sign. The bytes read past a short block's NUL never change the result (the first differing lane decides it, bitwise). An aligned word never crosses a page.
- The over-read question is settled by this rule and is not reopened.

18

PROJECTS BUILT ON THE ENGINE

Purpose: How the projects that build on the engine take it. No edit here waits on them, and no build of theirs reads a tree in motion.

Scope: the RSNA knee project and the CASMI chemistry project, .

Ruled : the chemistry engine does not take precedence over the engine's own work, and a project built on the engine takes it as a git dependency.

- Engine source edits are never paused or blocked while one of them is building. A build that needs the tree to hold still uses a pinned checkout.
- A record-API shape change is announced before it lands in a commit, and GPU runs are scheduled by hand until tessera (the device-job scheduler) exists.
- Commits happen only when Doug decides, and the projects move their pins then.

19

BUILD TIME

Purpose: What the time a build takes is spent on.

Scope: every build and long run of the engine.

"you wait a long time for builds you could be using to update the table during that time remember to background those runs and update paperwork during the build time, dont build just do maths".

- Every build and long run is started in the background.
- While it runs, the engine table, the theory documents and the build plan are brought up to date and the algebra is worked. Nothing sits in a wait loop.
- Documents are not compiled, and editing them during a build is safe. Engine sources are still not edited until "driver exit".

20

ENGINE FILE TYPES

Every file extension the engine and its projects write or read. A new extension is Doug's to name. Before one is proposed, it is checked against this table and against the whole biohub tree, orior_python included.

20.1 IN USE

Extension	What it is	Named by
`.kcr`	Kolmogorov information crystal: its own compression format. It reads every source format, takes each format's own compression out, and can rebuild the data into any format (Doug). krep kind "KCR\o", version 1. The only crystal format.	Doug
`.krs`	Kolmogorov information ruleset: one language's forms (`ptx.krs`, `c.krs`, `vhdl.krs`).	Doug
`.kcs`	Kolmogorov information construction set: what reconstructs information. A target's forms and their costs are its construction set. krep kind "KCS\o".	Doug
`.knf`	Kolmogorov noise floor. A sample's floor is its entropy history. krep kind "KNF\o".	Doug
`.kdm`	Kolmogorov device map: the hardware map.	Doug
`.ksc`	Kolmogorov system classification: the language map.	Doug
`.kqr`	Kolmogorov query record: every ask put to a member and what came back, then the paths read off those asks (`src/cu/includes/file_defs/kqr`).	Doug
`.khw`	Kolmogorov hardware: the part's instructions as the part answered them, each form, its encoding and its operands' fields (`src/cu/includes/file_defs/khw`).	Doug

20.2 RETIRED

Extension	What it was
`.iapx`	The crystal before `.kcr`. Not read; sets are re-ingested.

20.3 OFFERED, NOT ADOPTED

Extension	Offered for
`.kfc .kst .kgr .ksd`	The other apx files (flattened, OAPX history, BAPX bodies, IMP key); still open with Doug (build_plan.md).
`.khw`	A separate hardware constraints file. Withdrawn (Doug: file creep); the constraints are entries of the language's `.krs`.

BIBLIOGRAPHY

[1] S. Aaronson. *On black holes, holography, the Quantum Extended Church-Turing Thesis, fully homomorphic encryption, and brain uploading*. Shtetl-Optimized, 27 July 2022, https://scottaaronson.blog/?p=6599.

[2] A. Almheiri, X. Dong, D. Harlow. *Bulk locality and quantum error correction in AdS/CFT*. JHEP 04, 163, doi:10.1007/jhep04(2015)163.

[3] V. I. Arnold. *On functions of three variables*. Doklady Akademii Nauk SSSR 114(4) 679-681. 1957.

[4] F. Barsi, P. Maestrini. *Error correcting properties of redundant residue number systems*. IEEE Transactions on Computers C-22(3), 307-315, doi:10.1109/t-c.1973.223711.

[5] C. H. Bennett. *Logical reversibility of computation*. IBM Journal of Research and Development 17(6) 525-532, doi:10.1147/rd.176.0525. 1973.

[6] L. Borsten, H. Kim. *Limits to Computational Acceleration Imposed by Quantum Field Theory and Quantum Gravity*. arXiv:2604.00182. 2026.

[7] D. H. Bailey, P. B. Borwein, S. Plouffe. *On the Rapid Computation of Various Polylogarithmic Constants*. Mathematics of Computation 66(218), 903-913, doi:10.1090/s0025-5718-97-00856-9. 1997.

[8] G. J. Chaitin. *A theory of program size formally identical to information theory*. Journal of the ACM 22, doi:10.1145/321892.321894. 1975.

[9] G. J. Chaitin. *Algorithmic Information Theory*. Cambridge University Press. 1987.

[10] C. C. Chang. *On unions of chains of models*. Proceedings of the American Mathematical Society 10, doi:10.1090/s0002-9939-1959-0103812-3.

[11] A. Bouland, B. Fefferman, U. Vazirani. *Computational pseudorandomness, the wormhole growth paradox, and constraints on the AdS/CFT duality*. arXiv:1910.14646, doi:10.48550/arXiv.1910.14646. 2019.

[12] R. A. Fisher, F. Yates. *Statistical Tables for Biological, Agricultural and Medical Research*. Oliver and Boyd, Edinburgh. 1938.

[13] H. L. Garner. *The residue number system*. IRE Transactions on Electronic Computers EC-8, 140-147, doi:10.1109/tec.1959.5219515.

[14] G. Gentzen. *Die Widerspruchsfreiheit der reinen Zahlentheorie*. Mathematische Annalen 112, doi:10.1007/bf01565428. 1936.

[15] R. L. Goodstein. *On the restricted ordinal theorem.* Journal of Symbolic Logic 9, doi:10.2307/2268019. 1944.

[16] F. Q. Gouvêa. *p-adic Numbers: An Introduction, second edition.* Springer. 1997.

[17] J. D. Hamkins, A. Lewis. *Infinite time Turing machines.* Journal of Symbolic Logic 65, doi:10.2307/2586556. 2000.

[18] E. Hewitt, K. A. Ross. *Abstract Harmonic Analysis I.* Springer. 1963.

[19] W. Hodges. *Model Theory.* Cambridge University Press. 1993.

[20] A. C. A. Hope. *A simplified Monte Carlo significance test procedure.* Journal of the Royal Statistical Society, Series B 30, doi:10.1111/j.2517-6161.1968.tb00759.x.

[21] A. Hurwitz. *Ueber die angenäherte Darstellung der Irrationalzahlen durch rationale Brüche.* Mathematische Annalen 39, doi:10.1007/bf01206656. 1891.

[22] W. Jones. *Synopsis Palmariorum Matheseos.* London. 1706.

[23] M. Kac. *On the notion of recurrence in discrete stochastic processes.* Bulletin of the American Mathematical Society 53(10) 1002-1010, doi:10.1090/s0002-9904-1947-08927-8.

[24] A. Ya. Khinchin. *Continued Fractions.* University of Chicago Press. 1964.

[25] L. Kirby, J. Paris. *Accessible independence results for Peano arithmetic.* Bulletin of the London Mathematical Society 14, doi:10.1112/blms/14.4.285. 1982.

[26] D. E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms, third edition.* Addison-Wesley. 1997.

[27] N. Koblitz. *p-adic Numbers, p-adic Analysis, and Zeta-Functions, second edition.* Springer. 1984.

[28] P. Koepke. *Turing computations on ordinals.* Bulletin of Symbolic Logic 11, doi:10.2178/bsl/1122038993. 2005.

[29] A. N. Kolmogorov. *A new metric invariant of transitive dynamical systems and automorphisms of Lebesgue spaces.* Dokl. Akad. Nauk SSSR 119(5):861-864, Math-Net.Ru dan22922. 1958.

[30] A. N. Kolmogorov. *On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition.* Doklady Akademii Nauk SSSR 114(5) 953-956. 1957.

[31] A. N. Kolmogorov. *Three approaches to the quantitative definition of information.* Problemy Peredachi Informatsii 1(1) 3-11; in English, International Journal of Computer Mathematics 2(1-4) 157-168, 1968, doi:10.1080/00207166808803030. 1965.

[32] J. H. Lambert. *Mémoire sur quelques propriétés remarquables des quantités transcendantes circulaires et logarithmiques.* Mémoires de l'Académie royale des sciences de Berlin 17, 1761, published 1768.

[33] M. A. Lancret. *Mémoire sur les courbes à double courbure.* Mémoires présentés à l'Institut 1. 1806.

[34] P. Lévy. *Sur le développement en fraction continue d'un nombre choisi au hasard.* Compositio Mathematica 3. 1936.

[35] F. Lindemann. *Über die Zahl π.* Mathematische Annalen 20, doi:10.1007/bf01446522. 1882.

[36] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T. Y. Hou, M. Tegmark. *KAN: Kolmogorov-Arnold Networks.* arXiv:2404.19756. 2024.

[37] J. Łoś, R. Suszko. *On the extending of models (IV).* Fundamenta Mathematicae 44, doi:10.4064/fm-44-1-52-60. 1957.

[38] P. Martin-Löf. *The definition of random sequences.* Information and Control 9, doi:10.1016/s0019-9958(66)80018-9. 1966.

[39] A. Ostrowski. *Über einige Lösungen der Funktionalgleichung φ(x)·φ(y) = φ(xy).* Acta Mathematica 41, doi:10.1007/bf02422947. 1916.

[40] F. Pastawski, B. Yoshida, D. Harlow, J. Preskill. *Holographic quantum error-correcting codes: toy models for the bulk/boundary correspondence.* JHEP 06, 149, doi:10.1007/jhep06(2015)149.

[41] W. W. Peterson, E. J. Weldon. *Error-Correcting Codes, second edition.* MIT Press. 1972.

[42] E. L. Post. *Recursively enumerable sets of positive integers and their decision problems.* Bulletin of the American Mathematical Society 50(5) 284-316, doi:10.1090/S0002-9904-1944-08111-1. 1944.

[43] G. Rauzy. *Échanges d'intervalles et transformations induites.* Acta Arithmetica 34, doi:10.4064/aa-34-4-315-328. 1979.

[44] S. Reiger. *Codes for the correction of 'clustered' errors.* IRE Transactions on Information Theory 6, doi:10.1109/tit.1960.1057558.

[45] I. J. Schoenberg. *Contributions to the problem of approximation of equidistant data by analytic functions. Part A.* Quarterly of Applied Mathematics 4(1) 45-99, doi:10.1090/qam/15914. 1946.

[46] C. E. Shannon. *A Mathematical Theory of Communication.* Bell System Technical Journal 27, 379-423 and 623-656, July and October 1948.

[47] J. R. Shoenfield. *On degrees of unsolvability.* Annals of Mathematics 69, doi:10.2307/1970028. 1959.

[48] N. B. Slater. *The distribution of the integers N for which {θN} < φ.* Proceedings of the Cambridge Philosophical Society 46, doi:10.1017/s0305004100026086. 1950.

[49] N. B. Slater. *Gaps and steps for the sequence nθ mod 1.* Proceedings of the Cambridge Philosophical Society 63, 1115-1123, doi:10.1017/s0305004100042195. 1967.

[50] R. D. Sorkin. *Quantum mechanics as quantum measure theory*. Modern Physics Letters A 9, doi:10.1142/s021773239400294x. 1994.

[51] V. T. Sós. *On the distribution mod 1 of the sequence nα*. Annales Universitatis Scientiarum Budapestinensis de Rolando Eötvös Nominatae, Sectio Mathematica 1. 1958.

[52] W. Sweldens. *The lifting scheme: a custom-design construction of biorthogonal wavelets*. Applied and Computational Harmonic Analysis 3, doi:10.1006/acha.1996.0015.

[53] N. S. Szabo, R. I. Tanaka. *Residue Arithmetic and Its Applications to Computer Technology*. McGraw-Hill. 1967.

[54] J. Tate. *Fourier analysis in number fields and Hecke's zeta-functions*. thesis, Princeton, 1950; in J. W. S. Cassels and A. Fröhlich (eds.), Algebraic Number Theory, Academic Press, 1967.

[55] D. van Dantzig. *Über topologisch homogene Kontinua*. Fundamenta Mathematicae 15. 1930.

[56] T. van Ravenstein. *The three gap theorem (Steinhaus conjecture)*. Journal of the Australian Mathematical Society, Series A 45, doi:10.1017/s1446788700031062. 1988.

[57] L. Vietoris. *Über den höheren Zusammenhang kompakter Räume und eine Klasse von zusammenhangstreuen Abbildungen*. Mathematische Annalen 97. 1927.

[58] P. Walters. *An Introduction to Ergodic Theory*. Springer. 1982.

[59] R. W. Watson, C. W. Hastings. *Self-checked computation using residue arithmetic*. Proceedings of the IEEE 54(12), 1920-1931, doi:10.1109/proc.1966.5275.

[60] H. Weyl. *Über die Gleichverteilung von Zahlen mod. Eins*. Mathematische Annalen 77, doi:10.1007/bf01475864. 1916.

[61] W. K. Wootters, W. H. Zurek. *A single quantum cannot be cloned*. Nature 299, doi:10.1038/299802a0. 1982.